

The
Pragmatic
Programmers

Programming Clojure

Fourth Edition



Alex Miller
with Stuart Halloway
and Aaron Bedra
edited by Jacquelyn Carter

Early Praise for *Programming Clojure, Fourth Edition*

This book packs more core wisdom per paragraph than any other Clojure book on the market, and it's the only guide to the latest language features. Developers who internalize *Programming Clojure*'s fusion of functional programming powered by data-oriented programming will significantly level up their skills.

► **Michael Fogus**

Principal Software Engineer, Nubank

If you've always wanted to learn a modern Lisp like Clojure and use it in production, this is a fantastic companion—not only to teach you the basics in a profound way but also to give you a glimpse into the more intricate structures, patterns, and idioms of programming with Clojure.

► **Stefan Böhringer**

Chair of Educational Data Science, University of Regensburg

This book gives you the mental models needed to become an effective Clojure programmer. The chapter on functional programming is alone worth the price for it!

► **Benoît Fleury**

Clojure Programmer

If you are new to Clojure or want a refresh of current best practices, read this book. Written by members of the Clojure Core development team, *Programming Clojure* provides the foundational concepts and practical guidance that will most certainly level up your Clojure skills.

► **Lee Read**

Clojure Enthusiast



We've left this page blank to make the page numbers the same in the electronic and paper books.

We tried just leaving it out, but then people wrote us to ask about the missing pages.

Anyway, Eddy the Gerbil wanted to say "hello."

Programming Clojure, Fourth Edition

Alex Miller
with Stuart Halloway
and Aaron Bedra

The Pragmatic Bookshelf

Dallas, Texas



See our complete catalog of hands-on, practical,
and Pragmatic content for software developers:

<https://pragprog.com>

Sales, volume licensing, and support:

support@pragprog.com

Derivative works, AI training and testing,
international translations, and other rights:

rights@pragprog.com

The team that produced this book includes:

Publisher: Dave Thomas

COO: Janet Furlow

Executive Editor: Susannah Davidson

Development Editor: Jacquelyn Carter

Copyright © 2026 The Pragmatic Programmers, LLC.

All rights reserved. No part of this publication may be reproduced by any means, nor may any derivative works be made from this publication, nor may this content be used to train or test an artificial intelligence system, without the prior consent of the publisher.

When we are aware that a term used in this book is claimed as a trademark, the designation is printed with an initial capital letter or in all capitals.

The Pragmatic Starter Kit, The Pragmatic Programmer, Pragmatic Programming, Pragmatic Bookshelf, PragProg, and the linking *g* device are trademarks of The Pragmatic Programmers, LLC.

Every precaution was taken in the preparation of this book. However, the publisher assumes no responsibility for errors or omissions or for damages that may result from the use of information (including program listings) contained herein.

ISBN-13: 979-8-88865-191-9

Encoded using recycled binary digits.

Book version: P1.0—May, 2026

Contents

Acknowledgments	ix
Foreword	xi
Introduction	xiii

Part I — Welcome to Clojure

1.	Getting Started	3
	Simplicity and Power in Action	4
	Clojure Coding Quick Start	10
	Navigating Clojure Namespaces	13
	Wrapping Up	16
2.	Exploring Clojure	17
	Reading Clojure	17
	Functions	24
	Vars, Bindings, and Namespaces	29
	Metadata	35
	Calling Java	37
	Comments	40
	Flow Control	40
	Where's My for Loop?	43
	Wrapping Up	46

3.	Developing Interactively	49
	The REPL (Read-Eval-Print-Loop)	49
	Editors	53
	Structural Editing	55
	Loading and Evaluating Code	59
	Dependencies in the REPL	60
	Wrapping Up	62

Part II — Data and Functions

4.	Unifying Data with Sequences	65
	Everything Is Seq-able	66
	Using the Sequence Functions	70
	Lazy and Infinite Sequences	80
	Clojure Makes Java Seq-able	82
	Calling Structure-Specific Functions	87
	Wrapping Up	95
5.	Functional Programming	97
	Functional Programming Concepts	97
	How to Be Lazy	101
	Lazier Than Lazy	109
	Recursion Revisited	114
	Eager Transformations	122
	Wrapping Up	126
6.	Describing Your Data with Specs	129
	Defining Specs	131
	Validating Data	132
	Validating Functions	139
	Generative Function Testing	146
	Wrapping Up	150

Part III — Intermediate Topics

7.	State and Concurrency	155
	Concurrency, Parallelism, and Locking	156
	Use Atoms for Uncoordinated, Synchronous Updates	157
	Refs and Software Transactional Memory	159
	Use Agents for Asynchronous Updates	165
	Managing Per-Thread State with Vars	170
	A Clojure Snake	174
	Wrapping Up	183
8.	Protocols and Datatypes	185
	Programming to Abstractions	185
	Interfaces	187
	Protocols	188
	Datatypes	190
	Records	192
	reify	197
	Metadata Extension	198
	Wrapping Up	199
9.	Multimethods	201
	Living Without Multimethods	201
	Defining Multimethods	203
	Moving Beyond Simple Dispatch	206
	Creating Ad Hoc Taxonomies	208
	When Should I Use Multimethods?	211
	Wrapping Up	214
10.	Java Interop	215
	Creating Java Objects in Clojure	215
	Calling Clojure from Java	221
	Exception Handling	222
	Optimizing for Performance	226
	Handling Java Streams	233
	A Real-World Example	234
	Wrapping Up	237

11. Macros	239
When to Use Macros	239
Writing a Control Flow Macro	240
Making Macros Simpler	246
Taxonomy of Macros	251
Wrapping Up	259

Part IV — Clojure in Practice

12. Project Tooling	263
Project Sources and Dependencies	263
Running a REPL	268
Defining Tests	268
Running Tests	270
Building and Releasing Libraries	271
Compiling Applications	276
Wrapping Up	277
13. Building an Application	279
Getting Started	279
Developing the Game Loop	280
Representing Progress	282
Implementing Players	284
Interactive Play	286
Documenting and Testing Your Game	290
Farewell	294
A1. Developer Tools	295
Testing	295
Code Style and Linting	296
Data Browsing and Visualization	296
Debugging	296
Profiling	297
Dependency Management	297
Bibliography	299

Acknowledgments

Many people have contributed to what is good in this book. The problems and errors that remain are ours alone.

Thanks to Rich Hickey for creating the Clojure language and nurturing the wonderful community around it. Thanks to the Clojure community for embracing the ideas of Clojure and building a functioning ecosystem around it.

Thanks to all of the readers and technical reviewers who have suggested improvements across all four editions of the book. A special thank you to Michael Fogus, Sean Corfield, Lee Read, Larry Jones, Bobbi Towers, Dan Sutton, Eugene Pakhomov, Benoît Fleury, Juan Monetta, and Stefan Böhringer for their careful review and thoughtful suggestions for this edition. Thanks also to Jeff Brown, who suggested the coin toss problem in [Lazier Than Lazy, on page 109](#), and David Liebke, who wrote the original content for [Chapter 8, Protocols and Datatypes, on page 185](#).

Thanks to everyone at Pragmatic Bookshelf. Thanks especially to our editor, Jacquelyn Carter, and former editors Michael Swaine and Susannah Pfalzer for always keeping us focused on the reader. Thanks also to the publisher, Dave Thomas, for creating such a productive home for technical authors.

Thanks to my wife and family for their love, support, and the precious gift of time to create.

Alex

Foreword

I first discovered Clojure in 2008. I was working in a boutique consultancy and looking for a programming language that would give us an edge. I found that and quite a bit more. By 2009, I had written the first book on Clojure. I soon became a Clojure committer and am proud of my small role in guiding the language through every major release since version 1.0.

The book you are reading is the fourth edition descendant of that 2009 book, so I am a bit biased about it. But sticking to something for 17 years is a rare privilege in software, and my reasons for doing so are relevant to your choice to read this book.

Clojure is a language for both *thinking* and *shipping*. It was created by and for people who want to live and think inside their programs in the tightest possible feedback loop (the REPL). They can then ship that same code in mission-critical production systems, where Clojure's commitment to stability is becoming legendary. Almost all of the code I have written in the last 17 years runs today without modification.

As a functional Lisp, Clojure grows by small, careful additions and, more often, by libraries. Your investment in the language itself is durable. You can see this in the chapters of this book. [Chapter 6, Describing Your Data with Specs, on page 129](#) describes a powerful specification system that lives in a separate library. [Chapter 8, Protocols and Datatypes, on page 185](#) describes protocols and datatypes, a small addition to the language that builds on and extends the polymorphic capabilities of the JVM. These are very cool capabilities, but represent no change to the core of the language. People can and do write large libraries and applications without ever touching them.

If you want a production language that is built to last, you have come to the right place. We look forward to welcoming you to the community.

Stuart Halloway

Clojure and Datomic Programmer

Introduction

Clojure is a dynamic programming language for the Java Virtual Machine (JVM) with a compelling combination of features:

- *Clojure is elegant.* Clojure's clean, careful design lets you write programs that get right to the essence of a problem without a lot of clutter and ceremony.
- *Clojure is interactive.* Clojure builds on the tradition of Lisp so that you are always working with running code and immediate feedback.
- *Clojure is a functional language.* Data structures are immutable, and most functions are free from side effects. This makes it easier to write correct programs and to compose large programs from smaller ones.
- *Clojure simplifies concurrent programming.* Many languages build a concurrency model around locking, which is difficult to use correctly. Clojure provides several alternatives to locking: software transactional memory, agents, atoms, and dynamic variables.
- *Clojure embraces Java.* Calling from Clojure to Java is direct and fast, with no translation layer.
- *Unlike many popular dynamic languages, Clojure is fast.* Clojure is written to take advantage of the optimizations possible on modern JVMs.

Many other languages cover *some* of the features described in the previous list. Of all these languages, Clojure stands out. These features are not only interesting in themselves, but they also reinforce each other. For example, Clojure's use of immutable data and functions without side effects is essential for safe concurrency. We will cover all these features and more in [Chapter 1, Getting Started, on page 3](#).

Who This Book Is For

Clojure is a powerful, general-purpose programming language. As such, this book is for programmers with experience in a programming language like Java, JavaScript, C#, Python, or Ruby, but who are new to Clojure and looking for a powerful, elegant language.

Clojure is built on top of the Java Virtual Machine, and it is *fast*. This book will be of particular interest to Java programmers who want the expressiveness of a dynamic language without compromising on performance.

Clojure is helping to redefine what features belong in a general-purpose language. Clojure combines ideas from Lisp, functional programming, and concurrent programming and makes them more approachable to programmers seeing these ideas for the first time. Since Clojure was created, many languages like Java, JavaScript, Elixir, and Swift have adopted features inspired by Clojure.

Clojure is part of a larger phenomenon. Languages such as Erlang, F#, Haskell, and Scala have garnered attention recently for their support of functional programming or their concurrency model. Enthusiasts of these languages will find much common ground with Clojure.

What's in This Book

[Chapter 1, Getting Started, on page 3](#) demonstrates Clojure's elegance as a general-purpose language, plus the functional style and concurrency model that make Clojure unique. It also walks you through installing Clojure and developing code interactively at the REPL (Read Eval Print Loop).

[Chapter 2, Exploring Clojure, on page 17](#) is a breadth-first overview of all of Clojure's core constructs. After this chapter, you'll be able to read most day-to-day Clojure code.

[Chapter 3, Developing Interactively, on page 49](#) explores a crucial part of the Clojure experience—interactive development at the REPL, which is constantly loading and evaluating code as you write it.

The next two chapters cover functional programming. [Chapter 4, Unifying Data with Sequences, on page 65](#) shows how all data can be unified under the powerful *sequence* metaphor.

[Chapter 5, Functional Programming, on page 97](#) shows you how to write functional code in the same style used by the sequence functions and introduces the concepts of transducers.

[Chapter 6, Describing Your Data with Specs, on page 129](#) demonstrates how to write specifications for your data structures and functions and use them to aid in development and testing.

[Chapter 7, State and Concurrency, on page 155](#) delves into Clojure’s concurrency model. Clojure provides four powerful constructs for dealing with concurrency, plus all of the tools in Java’s concurrency libraries.

[Chapter 8, Protocols and Datatypes, on page 185](#) walks through records, types, and protocols in Clojure. Records and types allow you to create your own custom data types, and protocols provide functions that dispatch to implementations based on types.

[Chapter 9, Multimethods, on page 201](#) covers one of Clojure’s answers to polymorphism. Polymorphism usually means “take the *class* of the *first* argument and dispatch a method based on that.” Clojure’s multimethods let you choose *any function* of *all* the arguments and dispatch based on that.

[Chapter 10, Java Interop, on page 215](#) shows you how to call Java from Clojure and call Clojure from Java. You’ll see how to take Clojure straight to the metal and get Java-level performance.

[Chapter 11, Macros, on page 239](#) shows off Lisp’s signature feature. Macros take advantage of the fact that Clojure code is data to provide metaprogramming abilities that are difficult or impossible in anything but a Lisp.

[Chapter 12, Project Tooling, on page 263](#) covers the tools available to manage dependencies, run tests, and debug your code.

Finally, [Chapter 13, Building an Application, on page 279](#) provides a view into a complete Clojure workflow. You will build an application from scratch, working through solving the various parts of a problem and thinking about simplicity and quality.

How to Read This Book

All readers should begin by reading the first three chapters in order. Pay particular attention to [Simplicity and Power in Action, on page 4](#), which provides an overview of Clojure’s advantages.

Experiment continuously. Clojure provides an interactive environment where you can get immediate feedback; see [Using the REPL, on page 11](#) for more information.

After you read the first three chapters, skip around as you like. But read [Chapter 4, Unifying Data with Sequences, on page 65](#) before you read [Chapter 7, State and Concurrency, on page 155](#). These chapters lead you from Clojure's immutable data structures to a powerful model for writing correct concurrency programs.

For Functional Programmers

- Clojure's approach to FP strikes a balance between academic purity and the realities of execution on the current generation of JVMs. Read [Chapter 5, Functional Programming, on page 97](#) carefully to understand how Clojure idioms differ from languages such as Haskell.
- The concurrency model of Clojure ([Chapter 7, State and Concurrency, on page 155](#)) provides several explicit ways to deal with side effects and state and will make FP appealing to a broader audience.

For Java/C# Programmers

- Read [Chapter 2, Exploring Clojure, on page 17](#) carefully. Clojure has very little syntax (compared to Java or C#), and we cover the ground rules fairly quickly.
- If you wish to make use of Java libraries you already have, read [Chapter 10, Java Interop, on page 215](#) to learn how to call or extend Java from Clojure, and pass compatible data between them.
- Pay close attention to macros in [Chapter 11, Macros, on page 239](#). These are the most alien parts of Clojure when viewed from a Java or C# perspective.

For Lisp Programmers

- Some of [Chapter 2, Exploring Clojure, on page 17](#) will be review, but read it anyway. Clojure preserves the key features of Lisp, but it breaks with Lisp tradition in several places, and they are covered here.
- Pay close attention to the lazy sequences in [Chapter 5, Functional Programming, on page 97](#).
- If you like Emacs, it has several modes that provide extensive support for Clojure, including REPL support, paredit, static analysis, debugging, and much more.

For Perl/Python/Ruby Programmers

- Read [Chapter 7, State and Concurrency, on page 155](#) carefully. Intraprocess concurrency is very important in Clojure.
- Embrace macros ([Chapter 11, Macros, on page 239](#)). But do not expect to easily translate metaprogramming idioms from your language into macros. Remember always that macros execute at compile time, not runtime.

Notation Conventions

The following notation conventions are used throughout the book.

Literal code examples use the following font:

```
(+ 2 2)
```

The result of executing a code example is preceded by `->`.

```
(+ 2 2)
-> 4
```

Where console output cannot easily be distinguished from code and results, it's preceded by a pipe character (`()`).

```
(println "hello")
| hello
-> nil
```

When introducing a Clojure function for the first time, we'll show the grammar for the function like this:

```
(example-fn required-arg)
(example-fn optional-arg?)
(example-fn zero-or-more-arg*)
(example-fn one-or-more-arg+)
(example-fn & collection-of-variable-args)
```

The grammar is informal, using `?` for optional, `*` for 0 or more, `+` for 1 or more, and `&` as a marker before a collection of variable arguments.

Clojure code is organized into namespaces. Where examples in the book depend on a namespace that's not part of the Clojure core, we document that dependency with a `require` form that loads the namespace:

```
(require '[clojure.java.io :as io])
(io/file "hello.txt")
-> #object[java.io.File 0x11leadcba "hello.txt"]
```

Here, the `clojure.java.io` namespace is required and aliased as `io`. Clojure returns `nil` from a successful call to `require`—for brevity, this is omitted from the example listings.

While reading the book, you'll enter code in an interactive environment called the REPL. The REPL prompt looks like this:

```
user=>
```

The `user` in the prompt indicates the namespace you're currently working in. For most of the examples, the current namespace is irrelevant. Where the namespace is irrelevant, we use the following syntax for interaction with the REPL:

```
(+ 2 2)      ; input line without namespace prompt
-> 4         ; return value
```

In those instances where the current namespace is important, we use this:

```
user=> (+ 2 2) ; input line with namespace prompt
-> 4         ; return value
```

Online Resources

Programming Clojure's official home on the web is the *Programming Clojure* home page¹ at the Pragmatic Bookshelf website. From there, you can order electronic copies of the book or offer feedback by submitting errata entries or posting in the forums.²

The sample code for the book is also available from this page and is updated to match each release of the book. Most code examples will be accessible if you start your REPL from the `code/` directory. The hangman project from the final chapter is in the `code/hangman/` directory. *Author: added note to resolve an errata*

Throughout the book, listings begin with their filename and a link if reading the book in PDF form. For example, the following listing comes from `src/examples/preface.clj`:

```
src/examples/preface.clj
(println "hello")
```

With the sample code in hand, you are ready to get started. We'll begin by meeting the combination of features that make Clojure unique.

-
1. <https://www.pragprog.com/titles/shcloj4/>
 2. <https://devtalk.com/books/programming-clojure-fourth-edition/>

Part I

Welcome to Clojure

Learn Clojure's syntax, features, and interactive REPL and begin solving your programming problems in a fresh, expressive way.

Getting Started

Clojure is a functional programming language on the JVM with great support for managing state and concurrency. Two key concepts drive everything in Clojure: simplicity and power.

Simplicity has several meanings that are relevant in software, but the definition we care about is the original one: a thing is simple if it is not compound (see “Simple Made Easy”¹ by Rich Hickey). Simple components allow systems to do what their designers intend, without also doing other things irrelevant to the task at hand.

Power also has many meanings. The one we care about is whether the capabilities are adequate for the tasks we want to undertake. To feel powerful as a programmer, you need to build on a platform that’s itself capable and widely deployed, such as the JVM. Then, your tools must give you full, unrestricted access to that power. Power is an essential requirement for projects that must get the most out of their platform.

As programmers, we’ve spent years choosing between power and simplicity in our tools. Some trade-offs are fundamental, but power vs. simplicity is not one of them. Clojure shows that they can instead go hand in hand.

We’re going to start by diving into some examples to see how Clojure differentiates itself from other languages. Clojure puts simple and powerful tools in your hands by encouraging interactive development with the REPL. We’ll see how to work efficiently at the REPL and also how to use the REPL to explore the environment and other libraries.

1. <https://www.youtube.com/watch?v=LKtk3HCgTa8>

Simplicity and Power in Action

All of the distinctive features in Clojure are there to provide simplicity, power, or both. Some of these features include interactive programming at the REPL, concise and expressive programs, the expressivity of macros, an immutable-first approach to state and concurrency, and an embrace of the JVM host and its ecosystem. Let's look at a few examples that demonstrate these high-level features.

Clojure Is Elegant

Clojure is high signal, low noise. As a result, Clojure programs are short programs. Short programs are cheaper to build, cheaper to deploy, and cheaper to maintain. This is particularly true when the programs are concise rather than merely terse. As an example, consider writing a function to check whether a string is blank:

```
src/examples/introduction.clj
(defn blank? [str]
  (every? Character/isWhitespace str))
```

This function is short, but more importantly, it's *simple*: it has no variables, no mutable state, and no branches. This is possible thanks to *higher-order functions*. A higher-order function is a function that takes functions as arguments and/or returns functions as results. The `every?` function takes a function and a collection as its arguments and returns true if that function returns true for every item in the collection.

Note also that this definition also works correctly for special cases like `nil` and the empty string without requiring explicit checks. Both `nil` and the empty string case are simply treated as sequences of 0 characters and `every?` is defined to return true for empty sequences.

Because the Clojure version has no branches, it's easy to read and test. These benefits are magnified in larger programs. Also, while the code is concise, it's still readable. In fact, the code reads like a *definition* of blank: a string is blank if every character in it is whitespace.

Clojure has a lot of elegance baked in, but if you find something missing, you can add it yourself, thanks to the power of Lisp.

Clojure Is Lisp Reloaded

Clojure is a Lisp. Lisps have a tiny language core, almost no syntax, and a powerful macro facility. With these features, you can bend Lisp to meet your

design, instead of the other way around. Clojure takes a new approach to Lisp by keeping the essential ideas while embracing a set of syntax enhancements that make Clojure friendlier to non-Lisp programmers.

Consider the following snippet of Java code:

```
public class Person {
  private String firstName;
  public String getFirstName() {
    // continues
  }
}
```

In this code, `getFirstName()` is a method. Methods are polymorphic and give the programmer control over meaning, but the interpretation of *every other word* in the example is *fixed by the language*. Sometimes you really need to change what these words mean. So, for example, you might do the following:

- Redefine `private` to mean “private for production code but public for serialization and unit tests.”
- Redefine `class` to automatically generate getters and setters for private fields, unless otherwise directed.
- Create a subclass of `class` that provides callback hooks for life-cycle events. For example, a life-cycle-aware class could fire an event whenever an instance of the class is created.

These kinds of needs are commonplace. In most languages, you would have to petition the language implementer to add the kinds of features mentioned here. In Clojure, you can add your own language features with *macros* ([Chapter 11, Macros, on page 239](#)). Clojure itself is built out of macros such as `defrecord`:

```
(defrecord name [field1 field2 field3])
```

If you need different semantics, write your own macro. If you want a variant of records with strong typing and configurable nil-checking for all fields, you can create your own `defrecord` macro, to be used like this:

```
(defrecord name [Type :field1 Type :field2 Type :field3]
  :allow-nils false)
```

This ability to reprogram the language from within the language is the unique advantage of Lisp. You will see facets of this idea described in various ways:

- Lisp is homoiconic.² That is, Lisp code is also Lisp data. This makes it easy for programs to write and transform other programs.
- The whole language is there, all the time. Paul Graham's essay "Revenge of the Nerds"³ explains why this is so powerful.

Clojure offers a combination of features that make Lisp more approachable:

- Clojure generalizes Lisp's physical list into an abstraction called a *sequence*. This preserves the power of lists while extending that power to a variety of other data structures, including ones you make yourself.
- Clojure's hosted approach includes access to the broad and portable Java standard library and a deployment platform with great reach.
- Clojure's symbol resolution and syntax quoting makes it easier to write many common macros.

Many Clojure programmers will be new to Lisp, and they've probably heard bad things about all those parentheses. Clojure keeps the parentheses (and the power of Lisp!) but improves on traditional Lisp syntax in several ways:

- Clojure provides a convenient literal syntax for a variety of data structures besides just lists: regular expressions, maps, sets, vectors, and metadata. These features make Clojure code less *list-y* than other Lisps. For example, function parameters are specified in a vector [] instead of a list ().

```
src/examples/introduction.clj
(defn hello-world [username]
  (println (format "Hello, %s" username)))
```

The vector makes the argument list jump out visually and makes Clojure function definitions easy to read.

- In Clojure, unlike most Lisps, using commas to separate elements is optional—this provides concise literal collections. In fact, Clojure literally treats commas as whitespace and ignores them.

```
; make vectors look like arrays in other languages
[1, 2, 3, 4]
-> [1 2 3 4]
```

2. <https://en.wikipedia.org/wiki/Homoiconicity>

3. <https://www.paulgraham.com/icad.html>

- Idiomatic Clojure primarily uses parentheses to signal *function invocation*, not grouping, whereas other Lisps use them for both. Consider the `cond` macro, present in both Common Lisp and Clojure. `cond` evaluates a set of test/result pairs, returning the first result for which a test form yields true. Each test/result pair is grouped with parentheses:

```
; Common Lisp cond
(cond ((= x 10) "equal")
      ((> x 10) "more"))
```

Common Lisp uses parentheses here sometimes to mean invocation and sometimes to group the parts of a case. Clojure uses parentheses in its `cond` only for invocation, and uses position to imply grouping instead:

```
; Clojure cond
(cond (= x 10) "equal"
      (> x 10) "more")
```

This is an aesthetic decision, and both approaches have their supporters. The important thing is that Clojure takes the opportunity to improve on Lisp traditions when it can do so without compromising Lisp's power.

Clojure is an excellent Lisp, for both Lisp experts and Lisp beginners.

Clojure Is a Functional Language

Clojure is a functional language, but not a pure functional language like Haskell. Functional languages have the following properties:

- Functions are *first-class objects*. That is, functions can be created at runtime, passed around, returned and, in general, used like any other datatype.
- Data is immutable.
- Functions are *pure*; that is, they have no side effects.

For many tasks, functional programs are easier to understand, less error prone, and *much* easier to reuse. For example, the following short program searches a database of compositions for every composer who has written a composition named “Requiem”:

```
(for [c compositions :when (= (:name c) "Requiem")] (:composer c))
-> ("W. A. Mozart" "Giuseppe Verdi")
```

The name `for` does not introduce a loop but a *list comprehension*. Read the code as, “For each `c` in `compositions`, where the name of `c` is “Requiem”, yield the composer of `c`.” List comprehension is covered more fully in [Transforming Sequences, on page 76](#).

This example has four desirable properties:

- It is *simple*; it has no loops, variables, or mutable state.
- It is *thread safe*; no locking is needed.
- It is *parallelizable*; you could farm out individual steps to multiple threads without changing the code for each step.
- It is *generic*; compositions could be a plain set, XML data, or a database result set.

Contrast functional programs with *imperative* programs, where explicit statements alter program state. Most object-oriented programs are written in an imperative style and have *none* of the advantages listed here; they are unnecessarily complex, not thread safe, not parallelizable, and difficult to generalize. (For a head-to-head comparison of functional and imperative styles, skip forward to [Where's My for Loop?, on page 43.](#))

Clojure's approach to changing state enables concurrency without explicit locking and complements Clojure's functional core.

Clojure Simplifies Concurrent Programming

Clojure's support for functional programming makes it easy to write thread-safe code. Since immutable data structures cannot *ever* change, there's no danger of data corruption based on another thread's activity.

However, Clojure's support for concurrency goes beyond just functional programming. When you need references to mutable data, Clojure provides both atoms and software transactional memory (STM). Atoms provide safe updates to a single value in memory via an update function. STM is a higher-level approach to thread safety than the locking mechanisms that Java provides. Coordinated updates are described in transactions, similar to database transactions.

For example, the following code creates a working, thread-safe, in-memory database of accounts:

```
(def accounts (atom #{}))
```

The atom function creates a protected reference to the set of accounts. Account updates are described as functions that update the state. For example, adding a new account is just using conj to update the set:

```
(swap! accounts conj {:id "CLJ" :balance 1000.0})
```

The swap! ensures that the set is safely updated even if multiple threads add an account simultaneously. The primary limitation of swap! is that only one

stateful value can be updated at a time. Clojurists have been surprised to discover that relying on immutable data and atomic values is sufficient to satisfy most applications. When you really need it, refs and the STM are available to coordinate updates across multiple stateful values.

Although the example here is trivial, the technique is general, and it works on real-world problems. See [Chapter 7, State and Concurrency, on page 155](#) for more on concurrency and state in Clojure.

Clojure Embraces the Java Virtual Machine

Clojure gives you clean, simple, direct access to Java. You can call any Java API directly:

```
(System/getProperties)
-> {"java.specification.version" "21",
    ... many more ...}
```

Clojure adds a lot of syntactic sugar for calling Java. We won't get into the details here (see [Calling Java, on page 37](#)), but notice that in the following code, the Clojure version has *fewer parentheses* than the Java version:

```
// Java
"hello".getClass().getName()

; Clojure
(.. "hello" getClass getName)
```

Clojure provides simple functions for implementing Java interfaces and subclassing Java classes. Also, all Clojure functions implement Callable and Runnable, so it is trivial to pass an anonymous function to the constructor for a Java Thread:

```
(.start (Thread. (fn [] (println "Hello" (Thread/currentThread)))))
-> Hello #object[java.lang.Thread 0x2057ff1f Thread[Thread-0,5,main]]
```

The funny output here is Clojure's way of printing a Java instance. `java.lang.Thread` is the class name of the instance, `0x2057ff1f` is the hash code of the instance, and `Thread[Thread-0,5,main]` is the instance's `toString` representation. (Note that in the preceding example, the new thread will run to completion, but its output may interleave in some strange way with the REPL prompt. This is not a problem with Clojure but simply the result of having more than one thread writing to an output stream.)

Because the Java invocation syntax in Clojure is clean and simple, it's idiomatic to use Java directly, rather than to hide Java behind Lispy wrappers.

Now that you've seen a few of the reasons to use Clojure, it's time to start writing some code.

Clojure Coding Quick Start

Clojure is built and distributed in a Java archive (a jar file) that can be run on the Java Virtual Machine (JVM). Running programs on the JVM requires assembling a *classpath* that contains all of the code required to run your program, which includes Clojure itself, your code, and any Java or Clojure dependencies needed by your code.

To run Clojure and the code in this book, you need two things:

- A Java runtime—to run the host environment
- Clojure command line tool—to manage dependencies, create classpaths, and run Clojure programs

The “Install Clojure” page⁴ contains the instructions on how to install the latest version of both of these.

You will use the command line tools (specifically `clj`) to install Clojure and all of the dependencies for the sample code in this book. For more details on basic usage of `clj`, see the guide⁵ and reference⁶ pages. Don’t worry about learning everything now, though, because this book will guide you through the commands necessary to follow along.

You can test your install by navigating to the directory where you placed the sample code and running a Clojure *read-eval-print loop* (REPL) using the `clj` tool:

```
clj
```

The `clj` tool uses an optional configuration file, `deps.edn`, to declare dependencies (we’ll explore this in depth in [Chapter 12, Project Tooling, on page 263](#)). When `clj` starts, it will download any necessary dependencies (and their transitive dependencies). If you have just installed the `clj` tool, you may see statements about Clojure and other dependencies being downloaded. Once the REPL starts, it prints some helpful version information, then prompts you with:

```
user=>
```

Now, you are ready for “Hello World.”

4. https://clojure.org/guides/install_clojure

5. https://clojure.org/guides/deps_and_cli

6. https://clojure.org/reference/clojure_cli

Using the REPL

To see how to use the REPL, let's create a few variants of "Hello World." First, type `(println "hello world")` at the REPL prompt:

```
user=> (println "hello world")
| hello world
-> nil
```

The second line, `hello world`, is the console output you requested. The third line is the return value from the `println` expression.

Next, encapsulate your "Hello World" into a function that can address a person by name:

```
(defn hello [name] (str "Hello, " name))
-> #'user/hello
```

Let's break this down:

- `defn`, `hello`, `name`, and `str` are all *symbols*, which are names that refer to things. Legal symbols are defined in [Symbols, on page 19](#).
- `defn` defines a function and stores it in a *var*.
- `hello` is the function name.
- `hello` takes one argument, `name`.
- `str` is a function call that concatenates an arbitrary list of arguments into a string.

Look at the return value, `#'user/hello`. The prefix `#'` is the syntax for a Clojure var, and `user` is the *namespace* of the function. (The `user` namespace is the REPL default, like the default package in Java.) You don't need to worry about vars and namespaces yet; they're covered in [Vars, Bindings, and Namespaces, on page 29](#).

Now, you can call `hello`, passing in your name:

```
user=> (hello "Stu")
-> "Hello, Stu"
```

If you get your REPL into a state that confuses you, the simplest fix is to kill the REPL with `Ctrl+C` on Windows or `Ctrl+D` on *nix and then start another one.

Special Variables

The REPL includes several useful, special variables. When you're working in the REPL, the results of evaluating the three most recent expressions are stored in the special variables `*1`, `*2`, and `*3`, respectively. This makes it easy to work iteratively. Say hello to a few different names:

```
user=> (hello "Stu")
-> "Hello, Stu"

user=> (hello "Clojure")
-> "Hello, Clojure"
```

You can use the special variables to combine the results of your recent work:

```
(str *1 " and " *2)
-> "Hello, Clojure and Hello, Stu"
```

If you make a mistake in the REPL, you'll see a Java exception. The details are often omitted for brevity. For example, dividing by zero is a no-no:

```
user=> (/ 1 0)
-> Execution error (ArithmeticException) at user/eval1 (REPL:1).
|   Divide by zero
```

Here, the problem is obvious, but sometimes the problem is more subtle, and you want the detailed stack trace. The `*e` special variable holds the last exception. Because Clojure exceptions are Java exceptions, you can ask for the stack trace by calling `pst` (print stack trace).

```
user=> (pst)
-> ArithmeticException Divide by zero
|   clojure.lang.Numbers.divide (Numbers.java:190)
|   clojure.lang.Numbers.divide (Numbers.java:3915)
|   user/eval1 (NO_SOURCE_FILE:1)
|   user/eval1 (NO_SOURCE_FILE:1)
|   ...
```

Java interop is covered in [Chapter 10, Java Interop, on page 215](#).

If you have a block of code that's too large to conveniently type at the REPL, save the code into a file, and then load that file from the REPL. You can use an absolute path or a path relative to where you launched the REPL:

```
; save some work in temp.clj, and then ...
user=> (load-file "temp.clj")
```

The REPL is a terrific environment for trying ideas and getting immediate feedback. Keep a REPL open at all times while you read this book. You can

find lots more about working with the REPL in [Chapter 3, Developing Interactively, on page 49](#).

At this point, you should feel comfortable entering small bits of code at the REPL. Larger units of code aren't that different; you can load and run Clojure namespaces from the REPL as well. Let's explore that next.

Navigating Clojure Namespaces

Clojure code is packaged in *namespaces*, which we cover in more depth in [Namespaces, on page 33](#). For now, know that namespaces are used for packaging and name resolution. They must be loaded for use with `require`:

```
(require quoted-namespace-symbol)
```

When you `require` a namespace named `clojure.java.io`, Clojure looks for a file named `clojure/java/io.clj` on the CLASSPATH. Any dashes in the namespace name are converted to underscores in the file name.

```
user=> (require 'clojure.java.io)
-> nil
```

The leading single quote (`'`) is required, and it *quotes* the namespace name. The `nil` returned indicates success. While you're at it, test that you can load the sample code for this chapter, `examples.introduction`:

```
user=> (require 'examples.introduction)
-> nil
```

Library as an Overloaded Term

This book uses *library* to refer to a group of Clojure files packaged and released as an artifact for reuse and *namespace* to refer to a single file of Clojure code loaded from the classpath. Beware, however, that many older Clojure core docstrings use the word *library* or *lib* to refer to single file namespaces, especially in the context of loading code. This meaning has largely fallen out of use in the community since the early days of Clojure.

The `examples.introduction` namespace includes an implementation of the Fibonacci numbers, the traditional “Hello World” program for functional languages. We'll explore the Fibonacci numbers in more detail in [How to Be Lazy, on page 101](#). For now, just make sure you can execute the sample function `fibs`. Enter the following line of code at the REPL to take the first 10 Fibonacci numbers:

```
(take 10 examples.introduction/fibs)
-> (0 1 1 2 3 5 8 13 21 34)
```

If you see the first 10 Fibonacci numbers as listed here, you have successfully installed the book samples.

The book samples are all unit tested, with tests located in the `examples/test` directory. The tests for the samples themselves are not explicitly covered in the book, but you may find them useful for reference.

Often, you can find the documentation you need right at the REPL. The most basic helper function (a macro) is `doc`:

```
(doc name)
```

Use `doc` to print the documentation for `str`:

```
user=> (doc str)
-----
clojure.core/str
([] [x] [x & ys])
With no args, returns the empty string. With one arg x, returns
x.toString(). (str nil) returns the empty string. With more than
one arg, returns the concatenation of the str values of the args.
```

The first line of `doc`'s output contains the fully qualified name of the function. The next line contains the possible argument lists, generated directly from the code. (Some common argument names and their uses are explained in [Conventions for Parameter Names, on page 16](#).) Finally, the remaining lines contain the function's *doc string*, if the function definition included one.

You can add a doc string to your own functions by placing it immediately after the function name:

```
src/examples/introduction.clj
(defn hello
  "Writes hello message to *out*. Calls you by username"
  [username]
  (println (str "Hello, " username)))
```

Sometimes, you won't know the exact name you want documentation for. The `find-doc` function will search for anything whose doc output matches a regular expression or string you pass in:

```
(find-doc s)
```

Use find-doc to explore how Clojure does reduce:

```
user=> (find-doc "reduce")
-----
clojure.core/areduce
([a idx ret init expr])
Macro
... details elided ...
-----
clojure.core/reduce
([f coll] [f val coll])
... details elided ...
```

reduce reduces Clojure collections and is covered in [Transforming Sequences, on page 76](#). areduce is for interoperability with Java arrays and is covered in [Using Java Arrays, on page 230](#).

Much of Clojure is written in Clojure, and it is instructive to read the source code. You can view the source code of a Clojure function using the source function.

```
(source a-symbol)
```

Try viewing the source of the simple identity function:

```
(source identity)
-> (defn identity
    "Returns its argument."
    {:added "1.0"
     :static true}
    [x] x)
```

Of course, you can also use Java's Reflection API. You can use methods, such as class, ancestors, and instance?, to reflect against the underlying Java object model and tell, for example, that Clojure's collections are also Java collections:

```
(instance? java.util.Collection [1 2 3])
-> true
```

Clojure's complete API is documented online.⁷ The right sidebar links to all functions and macros by name. Or you can find a helpful categorization of functions in the Clojure Cheatsheet.⁸

7. <https://clojure.github.io/clojure>

8. <https://clojure.org/api/cheatsheet>

Conventions for Parameter Names

The documentation strings for `reduce` and `areduce` show several terse parameter names. Here are some parameter names and how they are normally used:

Parameter	Usage	Parameter	Usage	Parameter	Usage
<code>a</code>	A Java array	<code>agt</code>	An agent	<code>coll</code>	A collection
<code>expr</code>	An expression	<code>f</code>	A function	<code>idx</code>	An index
<code>r</code>	A ref	<code>v</code>	A vector	<code>val</code>	A value

These names may seem a little terse, but there's a good reason for them: the “good names” are often taken by Clojure functions! Naming a parameter that collides with a function name is legal but considered bad style; the parameter will shadow the function while the parameter is in scope. So, don't call your refs `ref`, your agents `agent`, or your counts `count`. Those names refer to functions.

Wrapping Up

You have just experienced the whirlwind tour of Clojure. You've seen Clojure's expressive syntax, learned about Clojure's approach to Lisp, and seen how easy it is to call Java code from Clojure.

You have Clojure running in your own environment, and you've written short programs at the REPL to demonstrate functional programming and the reference model for dealing with state. Now, it's time to explore the entire language.

Exploring Clojure

Clojure offers great power through functional style, concurrency support, and clean Java interop. But before you can appreciate all of these features, you have to start with the language basics. Clojure is very expressive, and this chapter covers many concepts quite quickly. Don't worry if you don't understand every detail; we'll revisit these topics in more detail in later chapters. If possible, bring up a REPL and follow along with the examples as you read.

We'll start by looking at how to read and understand Clojure code, introducing the key parts of Clojure syntax. If your background is primarily in imperative languages, this tour may seem to be missing key language constructs, such as variables and for loops. Don't worry, you'll soon learn how to work in new ways that don't require them.

Reading Clojure

In this section, we'll cover many of the key parts of Clojure syntax and how they're used to form Clojure programs. In Clojure, there are no statements, only expressions that can be nested in mostly arbitrary ways. When evaluated, every expression returns a value that's used in the parent expression. This is a simple model, yet sufficient to cover everything in Clojure.

Numbers

To begin our exploration, let's consider a simple arithmetic expression as expressed in Clojure:

```
(+ 2 3)  
-> 5
```

All Clojure code is made up of expressions, and every expression, when evaluated, returns a value. In Clojure, parentheses are used to indicate a list, in this example, a list containing the symbol `+` and the numbers 2 and 3.

Clojure's runtime evaluates lists as function calls. The first element (+ in this example) is always treated as the operation with the remaining elements treated as arguments. The style of placing the function first is called *prefix notation*, as opposed to the more familiar *infix notation*, $2 + 3$.

A practical advantage of prefix notation is that you can easily extend it for arbitrary numbers of arguments:

```
(+ 1 2 3 4)
-> 10
```

Even the degenerate case of no arguments works as you'd expect, returning zero. This helps to eliminate special-case logic for boundary conditions:

```
(+)
-> 0
```

Many mathematical and comparison operators have the names and semantics that you'd expect from other programming languages. Addition, subtraction, multiplication, comparison, and equality all work as you would expect:

```
(- 10 5)
-> 5

(* 3 10 10)
-> 300

(> 5 2)
-> true

(>= 5 5)
-> true

(< 5 2)
-> false

(= 5 2)
-> false
```

Division may surprise you:

```
(/ 22 7)
-> 22/7
```

As you can see, Clojure has a built-in ratio type.

If what you actually want is decimal division, use a floating-point literal for the dividend:

```
(/ 22.0 7)
-> 3.142857142857143
```


Keywords

Keywords like `:title` or `:author` follow the same parsing rules as symbols, but always start with a `:`. While symbols are used to refer to other values, keywords always evaluate to themselves. They are ideal to use as identifiers or enumerated values. Like symbols, keywords may be namespaced (see [Namespaces, on page 33](#)). Keywords that start with `::` are called *auto-resolved keywords* and we'll discuss those in [Defining Specs, on page 131](#).

Because keywords evaluate to themselves, the Clojure runtime *interns* them (all instances of a keyword refer to the same cached value), so they are memory efficient and provide fast equality comparison and hash computation.

Collections

Clojure provides four primary collection types—lists, vectors, sets, and maps. All Clojure collections are heterogeneous (can hold any type of value) and are compared for equality based on their contents. The four Clojure collection types are used in combination to create larger composite data structures.

First, let's consider vectors, which are sequential, indexed collections. We can create a vector of the numbers 1, 2, and 3 using the following:

```
[1 2 3]
-> [1 2 3]
```

Vectors are preferred for sequential and/or indexed data.

Lists are sequential collections stored as a linked list. A list is printed as `(1 2 3)`, but we can't create a literal list at the REPL like we can with vectors. As we discussed in the previous section, Clojure function calls are represented as lists and evaluated by invoking the first element as the function. Thus `(1 2 3)` would be interpreted as invoking the function 1 with the arguments 2 and 3.

If we want a list to be read and interpreted as data (not evaluated like a function call), we can use the quote special form:

```
(quote (1 2 3))
-> (1 2 3)
```

Lists are used to represent Clojure code or for the (relatively rare) cases where inserting and deleting values at the front of the collection is preferred.

Quoting also has a *reader macro* form (`'`) understood by the reader. Reader macros are abbreviations of longer list forms and are used as shortcuts to improve readability. We'll see more of these as we explore. Here's how the quote looks in the shorter form:

```
'(1 2 3)
-> (1 2 3)
```

Sets are unordered collections that do not contain duplicates:

```
#{1 2 3 5}
-> #{1 3 2 5}
```

Because sets are unordered, you may see the elements printed in a different order than the original literal, and you should not expect any particular order. Sets are a good choice when you want fast addition and removal of elements and the ability to quickly check for whether a set contains a value.

Finally, Clojure maps are collections of key/value pairs. You can use a map literal to represent a programming language:

```
{:language "Lisp" :author "McCarthy"}
-> {:language "Lisp", :author "McCarthy"}
```

Keywords are commonly used as map keys because they evaluate to themselves, have fast equality and hash computations, and are memory efficient. Here, the map has two entries. The key `:language` is associated with the value `"Lisp"` and the key `:author` is associated with the value `"McCarthy"`. Like sets, maps are unordered, and the key/value pairs may be printed in an order different than the original map literal.

You may have noticed that the printed version lists a comma between the two key/value pairs. In Clojure, commas are whitespace and you're free to use them as an optional delimiter if you find it improves readability.

Any Clojure value can be a key in a map. However, the most common key type is the Clojure keyword, especially when representing information.

When you have maps with standard keys, and it would be useful to dispatch on map type, you can create a record with `defrecord`:

```
(defrecord name [field*])
```

For example, consider using the `defrecord` to create a `Book` record:

```
(defrecord Book [title author])
-> user.Book
```

Then, you can instantiate that record with the `->Book` constructor function:

```
(->Book "Lord of the Rings" "J.R.R. Tolkien")
-> #user.Book{:title "Lord of the Rings", :author "J.R.R. Tolkien"}
```

Once you instantiate a Book, it behaves like any other map. We will learn more about when and how to use records in [Chapter 8, Protocols and Datatypes](#), on page 185.

Strings and Characters

Strings are another kind of form. They are delimited by double quotes and are allowed to span multiple lines. Clojure strings reuse the Java String implementation.

```
"This is a\nmultiline string"
-> "This is a\nmultiline string"

"This is also
a multiline string"
-> "This is also\na multiline string"
```

As you can see, the REPL always shows string literals with escaped newlines. If you actually print a multiline string, it will print on multiple lines:

```
(println "another\nmultiline\nstring")
| another
| multiline
| string
-> nil
```

Perhaps the most common string function you'll use is `str`, which takes any number of objects, converts them to strings, and concatenates the results into a single string. Any nils passed to `str` are ignored:

```
(str 1 2 nil 3)
-> "123"
```

Clojure characters are also Java characters (16 bits, capable of representing UTF-16 code units). Their literal syntax is `\{letter}`, where `letter` can be a letter, a UTF-16 code point prefixed with `u`, or in a few special cases, the name of a character: backspace, formfeed, newline, return, space, or tab:

```
(str \g \o \o \d \space \j \o \b \space \u2605 \u2605 \u2605 \u2605 \u2605)
-> "good job ★★★★★"
```

Unicode code points beyond the UTF-16 range (including most emojis) are represented in Java and Clojure strings as a surrogate pair—two character values that together represent a single code point. For example, Unicode character U+1F60D ("Smiling Face with Heart-Eyes") can be represented in a string by the surrogate pair `"\ud83d\ude0d"`.

Regular Expressions

Clojure includes literal syntax for regular expressions similar to string syntax but prefixed with #:

```
#"a?(b+c)*"
```

The regular expression syntax compiles to an instance of `java.util.regex.Pattern`, Java's regular expression implementation. All syntax available in `Pattern` is also available in Clojure regular expressions. In Java, regular expression patterns are declared via strings, and thus, special characters must be escaped. However, in Clojure, you do not escape special characters in regular expressions, making them a bit easier to write and read.

Clojure provides a family of functions for matching strings with regular expressions, and we'll meet some of those later on in [Seq-ing Regular Expressions, on page 84](#).

Booleans and nil

Clojure's rules for Booleans are easy to understand:

- `true` is true, and `false` is false.
- In addition to `false`, `nil` evaluates to false when used in a Boolean context.
- Other than `false` and `nil`, *everything else* evaluates to true in a Boolean context.

Note that `true`, `false`, and `nil` follow the rules for symbols but are read as special values (either a Boolean or `nil`). These are the only special-case tokens like this in Clojure—anything else symbol-like is read as a symbol.

The empty list is not false in Clojure:

```
; condition then      else
(if ()          "() is true" "() is false")
-> "() is true"
```

Zero is not false in Clojure, either:

```
; condition then      else
(if 0          "Zero is true" "Zero is false")
-> "Zero is true"
```

Next, we'll examine functions more broadly, both how to invoke and how to define them.

Predicates

A *predicate* is a function that returns either true or false. In Clojure, it's common to name predicates with a trailing question mark, for example, `true?`, `false?`, `nil?`, and `zero?`:

```
(true? expr)
(false? expr)
(nil? expr)
(zero? expr)
```

`true?` tests whether a value is exactly the true value, *not* whether the value evaluates to true in a Boolean context. The only thing that's true? is true itself:

```
(true? true)
-> true

(true? "foo")
-> false
```

`nil?` and `false?` work the same way. Only nil is nil?, and only false is false?. `zero?` works with any numeric type, returning true if it's zero:

```
(zero? 0.0)
-> true

(zero? (/ 22 7))
-> false
```

Watch for more predicates as you continue learning!

Functions

In Clojure, a function call is simply a list whose first element is a function. For example, this call to `str` concatenates its arguments to create a string:

```
(str "hello" " " "world")
-> "hello world"
```

To define your own functions, use `defn`:

```
(defn name doc-string? attr-map? [params*] prepost-map? body)
```

The name is a symbol naming the function (implicitly defined within the current namespace). The doc-string is an optional string describing the function. The attr-map associates metadata with the function's var. It's covered separately in [Metadata, on page 35](#). The prepost-map? can be used to define preconditions and postconditions that are automatically checked on invocation, and the body contains any number of expressions. The result of the final expression is the return value of the function.

Let's create a greeting function:

```
src/examples/exploring.clj
(defn greeting
  "Returns a greeting of the form 'Hello, username.'"
  [username]
  (str "Hello, " username))
```

You can call greeting:

```
(greeting "world")
-> "Hello, world"
```

You can also consult the documentation for greeting:

```
user=> (doc greeting)
-----
exploring/greeting
([username])
Returns a greeting of the form 'Hello, username.'
```

What does greeting do if the caller omits username?

```
(greeting)
-> ArityException Wrong number of args (0) passed to: user/greeting
clojure.lang.AFn.throwArity (AFn.java:429)
```

Clojure functions enforce their *arity*, that is, their expected number of arguments. If you call a function with an incorrect number of arguments, Clojure throws an `ArityException`. If you want to make greeting issue a generic greeting when the caller omits username, you can use this form of `defn`, which takes multiple argument lists and method bodies:

```
(defn name doc-string? attr-map?
  ([params*] body)+)
```

Different arities of the same function can call one another, so you can easily create a zero-argument greeting that delegates to the one-argument greeting, passing in a default username:

```
src/examples/exploring.clj
(defn greeting
  "Returns a greeting of the form 'Hello, username.'"
  "Default username is 'world'."
  ([] (greeting "world"))
  ([username] (str "Hello, " username)))
```

You can verify that the new greeting works as expected:

```
(greeting)
-> "Hello, world"
```

You can create a function with variable arity by including an ampersand in the parameter list. Clojure binds the name after the ampersand to a sequence of all the remaining parameters. There may be only one variable arity parameter, and it must be last in the parameter list.

The following function allows two people to go on a date with a variable number of chaperones:

```
src/examples/exploring.clj
(defn date [person-1 person-2 & chaperones]
  (println person-1 "and" person-2
           "went out with" (count chaperones) "chaperones."))

(date "Romeo" "Juliet" "Friar Lawrence" "Nurse")
| Romeo and Juliet went out with 2 chaperones.
```

Writing function implementations differing by arity is useful. But if you come from an object-oriented background, you'll want *polymorphism*, that is, different implementations that are selected by *type*. Clojure can do this and a whole lot more. See [Chapter 9, Multimethods, on page 201](#) and [Chapter 8, Protocols and Datatypes, on page 185](#) for details.

`defn` is intended for defining functions at the top level. If you want to create a function from within another function, you should use an anonymous function form instead.

Anonymous Functions

In addition to named functions with `defn`, you can also create anonymous functions with `fn`. At least three reasons exist to create an anonymous function:

- The function is so brief and self-explanatory that giving it a name makes the code harder to read, not easier.
- The function is being used only from inside another function and needs a local name, not a top-level binding.
- The function is created inside another function for the purpose of capturing the values of parameters or local bindings.

Functions used as predicates when filtering data are often brief and self-explanatory. For example, imagine that you want to create an index for a sequence of words, and you don't care about words shorter than three characters. You can write an `indexable-word?` function like this:

```
src/examples/exploring.clj
(defn indexable-word? [word]
  (> (count word) 2))
```

Then, you can use `indexable-word?` to extract indexable words from a sentence:

```
(require '[clojure.string :as str])
(filter indexable-word? (str/split "A fine day it is" #"\\W+"))
-> ("fine" "day")
```

The call to `split` breaks the sentence into words and then `filter` calls `indexable-word?` once for each word, returning those for which `indexable-word?` returns true.

Anonymous functions let you do the same thing in a single line. The simplest anonymous fn form is the following:

```
(fn [params*] body)
```

With this form, you can plug the implementation of `indexable-word?` directly into the call to `filter`:

```
(filter (fn [w] (> (count w) 2)) (str/split "A fine day" #"\\W+"))
-> ("fine" "day")
```

There's an even shorter reader macro syntax for anonymous functions, using implicit parameter names. The parameters are named `%1`, `%2`, and optionally, a final `%&` to collect the rest of a variable number of arguments. You can also use just `%` for the first parameter, preferred for single-argument functions. This syntax looks like this:

```
#{body}
```

You can rewrite the call to `filter` with the shorter anonymous form:

```
(filter #(> (count %) 2) (str/split "A fine day it is" #"\\W+"))
-> ("fine" "day")
```

A second motivation for anonymous functions is when you want to use a named function but only inside the scope of another function. Continuing with the `indexable-word?` example, you could write this:

```
src/examples/exploring.clj
```

```
(defn indexable-words [text]
  (let [indexable-word? (fn [w] (> (count w) 2))]
    (filter indexable-word? (str/split text #"\\W+"))))
```

The `let` binds the name `indexable-word?` to the same anonymous function you wrote earlier, this time inside the lexical scope of `indexable-words`. (`let` is covered in more detail under [Vars, Bindings, and Namespaces, on page 29](#).) You can verify that `indexable-words` works as expected:

```
(indexable-words "a fine day it is")
-> ("fine" "day")
```

The combination of `let` and an anonymous function says the following to readers of your code: “The function `indexable-word?` is interesting enough to have a name but is relevant only inside `indexable-words`.”

A third reason to use anonymous functions is when you create a function dynamically at runtime. Earlier, you implemented a simple greeting function. Extending this idea, you can create a `make-greeter` function that creates greeting functions. `make-greeter` will take a `greeting-prefix` and return a new function that composes greetings from the `greeting-prefix` and a name.

```
src/examples/exploring.clj
```

```
(defn make-greeter [greeting-prefix]
  (fn [username] (str greeting-prefix " " username)))
```

It is not necessary to name the `fn`, because it’s creating a *different* function each time `make-greeter` is called. However, you may want to name the results of specific calls to `make-greeter`. You can use `def` to name functions created by `make-greeter`, as shown here:

```
(def hello-greeting (make-greeter "Hello"))
-> #'user/hello-greeting

(def aloha-greeting (make-greeter "Aloha"))
-> #'user/aloha-greeting
```

Now, you can call these functions, just like any other functions:

```
(hello-greeting "world")
-> "Hello, world"

(aloha-greeting "world")
-> "Aloha, world"
```

Moreover, there’s no need to give each greeter a name. You can simply create a greeter and place it in the first (function) slot of a form:

```
((make-greeter "Howdy") "pardner")
-> "Howdy, pardner"
```

As you can see, the different greeter functions remember the value of `greeting-prefix` at the time they were created. More formally, the greeter functions are *closures* over the value of `greeting-prefix`.

When to Use Anonymous Functions

Anonymous functions have a terse syntax—sometimes too terse. You may actually prefer to be explicit, creating named functions such as `indexable-word?`. That’s perfectly fine and will certainly be the right choice if `indexable-word?` needs to be called from more than one place.

Anonymous functions are an option, not a requirement. Use the anonymous forms only when you find that they make your code more readable. They take a little getting used to, so don't be surprised if you gradually use them more and more.

Vars, Bindings, and Namespaces

One of the most important tools in a programming language is the ability to name and remember values or functions for later use. In Clojure, a *namespace* is a collection of names (symbols) that refer to *vars*. Each var is bound to a value. Let's consider vars more closely.

Vars

`def` and `defn` create vars in the current namespace. For example, the following `def` creates a var named `user/foo`:

```
(def foo 10)
-> #'user/foo
```

The symbol `user/foo` refers to the var `foo` in the `user` namespace whose value is *bound* to 10. If you ask Clojure to evaluate the symbol `foo`, it will return the value of the associated var:

```
foo
-> 10
```

The initial value of a var is called its *root binding*. Sometimes it's useful to have per-thread bindings for a var—these are called *dynamic vars* and covered in [Managing Per-Thread State with Vars, on page 170](#).

You can refer to a var directly. The `var` special form returns a var itself, not the var's value:

```
(var a-symbol)
```

You can use `var` to return the var bound to `user/foo`:

```
(var foo)
-> #'user/foo
```

You will almost never see the `var` form directly in Clojure code. Instead, you'll see the equivalent reader macro `#'`, which also returns the var for a symbol:

```
#'foo
-> #'user/foo
```

The `#'` syntax is read as “var-quote”.

Why would you want to refer to a var directly? Most of the time, you won't, and you can often simply ignore the distinction between symbols and vars.

But keep in the back of your mind that vars have many abilities other than just storing a value:

- The same var can be aliased into more than one namespace ([Namespaces, on page 33](#)). This allows you to use convenient short names.
- Vars can have metadata ([Metadata, on page 35](#)). Var metadata includes documentation ([Navigating Clojure Namespaces, on page 13](#)), type hints for optimization, and unit tests.
- Vars can be dynamically rebound on a per-thread basis ([Managing Per-Thread State with Vars, on page 170](#)).

Bindings

Vars are bound to names, but there are other kinds of bindings as well. For example, in a function call, argument values bind to parameter names. In the following call, the name `number` is locally bound to the value 10 inside the `triple` function when it's called:

```
(defn triple [number] (* 3 number))
-> #'user/triple

(triple 10)
-> 30
```

A function's parameter bindings have a *lexical* scope: they're visible only inside the text of the function body. Functions are not the only way to create a lexical binding. The special form `let` evaluates `exprs` in the context of a set of lexical bindings:

```
(let [bindings*] exprs*)
```

The bindings are then in effect for `exprs`, and the value of the `let` is the value of the last expression in `exprs`.

Imagine that you want coordinates for the four corners of a square, given the bottom, left, and size. You can let the top and right coordinates, based on the values given:

```
src/examples/exploring.clj
(defn square-corners [bottom left size]
  (let [top (+ bottom size)
        right (+ left size)]
    [[bottom left] [top left] [top right] [bottom right]]))
```

The `let` binds `top` and `right`. This saves you the trouble of calculating `top` and `right` more than once. (Both are needed twice to generate the return value.) The `let` then returns its last form, which in this example becomes the return value of `square-corners`.

If you need bindings inside a `defn`, do not use `def` (vars are global); instead, use `let` to create local bindings.

Destructuring

In many programming languages, you bind a variable to an *entire* collection when you need to access only *part* of the collection.

Imagine that you're working with a database of book authors. You track both first and last names, but some functions need to use only the first name:

```
src/examples/exploring.clj
(defn greet-author-1 [author]
  (println "Hello," (:first-name author)))
```

The `greet-author-1` function works fine:

```
(greet-author-1 {:last-name "Vinge" :first-name "Vernor"})
| Hello, Vernor
```

Having to bind `author` is unsatisfying. You don't need the `author`; all you need is the `first-name`. Clojure solves this with *destructuring*. Any place that you bind names, you can nest a vector or a map in the binding to reach into a collection and bind only the part you want. Here is a variant of `greet-author` that binds only the first name:

```
src/examples/exploring.clj
(defn greet-author-2 [{:fname :first-name}]
  (println "Hello," fname))
```

The binding form `{:fname :first-name}` tells Clojure to bind `fname` to the `:first-name` of the function argument. `greet-author-2` behaves just like `greet-author-1`:

```
(greet-author-2 {:last-name "Vinge" :first-name "Vernor"})
| Hello, Vernor
```

You can also use the `:keys` clause to destructure many keys at once:

```
src/examples/exploring.clj
(defn greet-author-3 [{:keys [first-name last-name]}]
  (println "Hello," first-name last-name))
```

Just as you can use a map to destructure any associative collection, you can use a vector to destructure any sequential collection. For example, you could bind only the first two coordinates in a three-dimensional coordinate space:


```
(let [[x y] [1 2 3]]
  [x y])
-> [1 2]
```

The expression `[x y]` destructures the vector `[1 2 3]`, binding `x` to 1 and `y` to 2. Since no symbol lines up with the final element 3, it's not bound to anything.

Sometimes you want to skip elements at the start of a collection. Here's how you could bind only the `z` coordinate:

```
(let [[_ _ z] [1 2 3]]
  z)
-> 3
```

The underscore (`_`) is a legal symbol and is often used to indicate, "I don't care about this binding."

It's also possible to simultaneously bind both a collection and elements within the collection. Inside a destructuring expression, an `:as` clause gives you a binding for the entire enclosing structure. For example, you could capture the `x` and `y` coordinates individually, plus the entire collection as `coords`, to report the total number of dimensions:

```
(let [[x y :as coords] [1 2 3 4 5 6]]
  (str "x: " x ", y: " y ", total dimensions " (count coords)))
-> "x: 1, y: 2, total dimensions 6"
```

Let's use destructuring to create an `ellipse` function. `ellipse` should take a string and return the first three words followed by an ellipsis (`...`).

```
src/examples/exploring.clj
(require '[clojure.string :as str])
(defn ellipse [words]
  (let [[w1 w2 w3] (str/split words #"\s+")]
    (str/join " " [w1 w2 w3 "..."])))

(ellipse "The quick brown fox jumps over the lazy dog.")
-> "The quick brown ..."
```

`split` splits the string around the regex matches (for whitespace), and then the destructuring form `[w1 w2 w3]` grabs the first three words. The destructuring ignores any extra items, which is exactly what we want. Finally, `join` reassembles the three words, adding the ellipsis at the end.

Destructuring has several other features not shown here and is a mini-language in itself. The Snake game in [A Clojure Snake, on page 174](#) makes heavy

use of destructuring. For a complete list of destructuring options, see the online documentation for binding forms² and the destructuring guide.³

Namespaces

Root bindings live in a namespace. You can see evidence of this when you start the Clojure REPL and create a binding:

```
user=> (def foo 10)
-> #'user/foo
```

The `user=>` prompt tells you that you're currently working in the `user` namespace. Most of the REPL session listings in this book omit the REPL prompt for brevity. In this section, the REPL prompt will be included whenever the current namespace is important. You should treat `user` as a scratch namespace for exploratory development.

When Clojure resolves the name `foo`, it namespace-qualifies `foo` in the current namespace `user`. You can verify this by calling `resolve`:

```
(resolve sym)
```

`resolve` returns the var a symbol is bound to in the current namespace:

```
(resolve 'foo)
-> #'user/foo
```

`resolve` also resolves class names to classes according to the imports of the current namespace:

```
(resolve 'String)
-> java.lang.String
```

You can switch namespaces, creating a new one if needed, with `ns`:

```
(ns name docstring? attr-map? references*)
```

Namespaces, like vars, can have a docstring and a metadata map. Additionally, namespaces can have *references* that configure the namespace as it's created. Each reference starts with a keyword like `:require` or `:import` and maps to a function of the same name in `clojure.core`, like `require` or `import`, which can also be used independently from `ns`. We will see more on those soon.

2. https://clojure.org/reference/special_forms#binding-forms

3. <https://clojure.org/guides/destructuring>

Try creating a myapp namespace:

```
user=> (ns myapp)
-> nil
myapp=>
```

Now, you're in the myapp namespace, and anything you def or defn will belong to myapp.

Every new namespace automatically imports the Java classes in java.lang:

```
myapp=> String
-> java.lang.String
```

By default, the class names outside java.lang must be fully qualified. You can't just say File:

```
myapp=> File/separator
-> java.lang.Exception: No such namespace: File
```

Instead, you must specify the fully qualified java.io.File. Note that your file separator character may be different from that shown here:

```
myapp=> java.io.File/separator
-> "/"
```

If you want to use a short name, rather than a fully qualified class name, you can import classes from a Java package into the current namespace using import.

```
(import [package Class+])
```

Once you import a class, you can use its short name:

```
(import [java.io InputStream File])
-> java.io.File

(.exists (File. "/tmp"))
-> true
```

import is one of the references that can be used directly inside ns as :import:

```
(ns myapp
  (:import [java.io InputStream File]))
```

import is only for Java classes. To use a Clojure var from another namespace without the namespace qualified, you must refer the external vars into the current namespace. For example, take Clojure's split function that resides in clojure.string:

```
(require '[clojure.string :refer [split]])
(split "Something,separated,by,commas" #"",")
-> ["Something" "separated" "by" "commas"]
```

If you wish to refer to `split` with a namespace alias, call `require` on the `clojure.string` namespace and give it the alias `str`:

```
(require '[clojure.string :as str])
(str/split "Something, separated, by, commas" #"",")
-> ["Something" "separated" "by" "commas"]
```

This simple form of `require` causes the current namespace to reference *all* public vars in `clojure.string` via the alias `str`.

The following call to `ns` appears at the top of the sample code for this chapter and includes both `:import` and `:require` references:

```
src/examples/exploring.clj
(ns examples.exploring
  (:require [clojure.string :as str])
  (:import [java.io File]))
```

The namespace functions can do quite a bit more than we've shown here.

You can examine namespaces and add or remove mappings at any time. To find out more, issue this command at the REPL. Since we've moved around a bit in the REPL, we'll also ensure that we're in the user namespace so that our REPL utilities are available to us:

```
(ns user)
(find-doc "ns-")
```

Alternatively, browse the reference documentation.⁴

Metadata

Metadata is data that describes other data. In Clojure, metadata is data that is *independent of the logical value of an object*. For example, a person's first and last names are plain old data. The fact that a person object can be serialized to XML has nothing to do with the person and is therefore metadata. Likewise, the fact that a person object is dirty and needs to be flushed to the database is metadata. Importantly, metadata never affects equality between objects.

The Clojure language itself uses metadata in several places. For example, vars have a metadata map containing documentation, type information, and source information. Here is the metadata for the `str` var:

```
(meta #'str)
{:ns #object[clojure.lang.Namespace 0x62ccf439 "clojure.core"],
 :name str,
 :added "1.0",
```

4. <https://clojure.org/reference/namespaces>

```
:file "clojure/core.clj",
:line 544,
:column 1,
:tag java.lang.String,
:arglists ([] [x] [x & ys]),
:doc
  "With no args, ...[etc]"}
```

Some common metadata keys and their uses are shown in the following table.

Key	Used For	Key	Used For
:ns	Namespace	:column	Source column number
:name	Local name	:tag	Expected argument or return type
:added	Version this function was added	:arglists	Parameter info used by doc
:file	Source file	:doc	Documentation used by doc
:line	Source line number	:macro	True for macros

Much of the metadata on a var is added automatically by the Clojure compiler. To add your own key/value pairs to a var, use the metadata reader macro:

```
^metadata form
```

For example, you could create a simple shout function that upcases a string, and then document it using the :doc key:

```
; see also shorter form below
(defn ^{:doc "Make a shouty string"} shout
  [s] (clojure.string/upper-case s))
-> #'user/shout
```

You can inspect shout's metadata to see that Clojure added the :doc:

```
(meta #'shout)
-> {:arglists ([s]),
   :ns #object[clojure.lang.Namespace 0x284c1da6 "user"],
   :name shout,
   :line 32,
   :column 1,
   :file "NO_SOURCE_PATH",
   :doc "Make a shouty string"}
```

You provided the :doc, and Clojure provided the other keys. The :file value NO_SOURCE_PATH indicates that the code was entered at the REPL.

Because `:doc` metadata is so common, `defn` accepts the string directly after the var name:

```
(defn shout
  "Make a shouty string"
  [s] (clojure.string/upper-case s))
-> #'user/shout
```

In this case, `defn` will attach the `:doc` metadata to `#'shout` on your behalf so the result is the same.

Next, we'll examine how we can take advantage of Java libraries, including the Java Development Kit (Java's standard library), which is always available.

Calling Java

Clojure provides simple, direct syntax for calling Java code: creating objects, invoking methods, and accessing static methods and fields.

Not all types in Java are created equal; the primitives and arrays work differently. Where Java has special cases, Clojure gives you direct access to these as well. Finally, Clojure provides a set of convenience functions for common tasks that would be unwieldy in Java.

The first step in many Java interop scenarios is creating a Java object. Clojure provides the new special form for this purpose:

```
(new classname)
```

Try creating a new `Random`:

```
(new java.util.Random)
-> #object[java.util.Random 0x30dae81 "java.util.Random@30dae81"]
```

A frequently used shortcut for creating a new instance of a class is to append a trailing `.` to the class name:

```
(java.util.Random.)
-> #object[java.util.Random 0x133314b "java.util.Random@133314b"]
```

The REPL simply prints out the new `Random` instance indicating its class, hash code, and the result of calling its `toString()` method. To use a `Random` instance, you need to save it away somewhere. For now, simply use `def` to save the `Random` into a var:

```
(def rnd (new java.util.Random))
-> #'user/rnd
```

You can also use the qualified constructor syntax to invoke `new` like a method, particularly useful when invoking a constructor as a higher-order function:

```
(map java.io.File/new ["a.txt" "b.txt"])
-> (#object[java.io.File 0x18025ced "a.txt"]
    #object[java.io.File 0x13cf7d52 "b.txt"])
```

This code maps the `File` constructor over a vector of strings to create a sequence of `File` objects.

For almost everything else in Java, you will be invoking members of a class or instance. The special form `dot` (`.`) supports all member access:

```
(. class-or-instance member-symbol & args)
(. class-or-instance (member-symbol & args))
```

However, the `dot` form is rarely used directly—instead, Clojure provides a more concise syntax for both instance and static method invocation that is preferred:

```
;; Instance method invocation
(.method instance & args)
(Class/.method instance & args)

;; Instance field access
(.field instance)
(.-field instance)

;; Static method invocation
(Class/method & args)

;; Static field access
Class/field
```

For example, `Random` has a method `nextInt()` which you can invoke like this:

```
(.nextInt rnd 10) ;; or (Random/.nextInt rnd 10)
-> 8
```

The `Point` class demonstrates public instance field access (not common in Java as most fields are private):

```
(.x p) ;; or (.-x p) for disambiguation with 0-arity methods
-> 10
```

Static methods always use a class qualifier and no `..`:

```
(System/lineSeparator)
-> "\n"
```

Static fields are never invoked with parens, the qualified field name simply resolves to the value:

```
Math/PI
-> 3.141592653589793
```

Notice in the previous examples that `Math` is not fully qualified, because Clojure imports `java.lang` classes automatically. To avoid typing `java.util.Random` everywhere, you can import it:

```
(import [package-symbol & class-name-symbols]*)
```

`import` takes a variable number of lists, with the first part of each list being a package name and the rest being names to import from that package. The following import allows unqualified access to `Random`, `Locale`, and `MessageFormat`:

```
(import [java.util Random Locale]
        [java.text MessageFormat])
-> java.text.MessageFormat

Random
-> java.util.Random

Locale
-> java.util.Locale

MessageFormat
-> java.text.MessageFormat
```

You can now do the following:

- Import class names
- Create instances
- Access fields
- Invoke methods

Due to not needing to reduce syntax and types, calling Java from Clojure can be easier than calling Java from Java!

Although reaching into Java from Clojure is easy, remembering how all of the Java bits work underneath can be daunting. Clojure provides a `javadoc` function that can open the relevant Java documentation in your browser while exploring.

```
(javadoc java.net.URL)
```


Comments

There are several ways to create comments in Clojure. The most common is to use `;;`, which creates a comment to the end of the line. While everything after the first `;` is ignored, you'll often see multiple semicolons to make a greater visual impact:

```
;; this is a comment
```

Clojure also contains a comment macro that ignores its body and returns `nil`. This is useful to wrap around a block of existing code. However, because it's still read by the Clojure reader, it must be valid code.

```
(comment
  (defn ignore-me []
    ;; not done yet
  ))
```

One common use of the comment macro is to save a chunk of utility or test code in a comment block at the end of the file, which is useful in combination with REPL-based development.

Clojure also contains a reader macro `#_` to tell the reader to read the next form but ignore it.

```
(defn triple [number]
  #_(println "debug triple" number)
  (* 3 number))
```

In this example, the `println` expression is being read but ignored due to the `#_` reader macro. This is commonly used in debugging or testing.

At this point, we've seen a broad overview of the basics of Clojure syntax. Next, we'll dive into the constructs that Clojure provides for flow control.

Flow Control

Clojure has very few flow control forms. In this section, you'll meet `if` (and some derivatives), `do`, and `loop/recur`. As it turns out, this is almost all you'll ever need. Clojure provides many additional functions, but they're largely built from these primitives.

Branch with `if`

Clojure's `if` evaluates its first argument. If the argument is logically true, it returns the result of evaluating its second argument:

```
src/examples/exploring.clj
(defn is-small [number]
  (if (< number 100) "yes"))

(is-small 50)
-> "yes"
```

If the first argument to `if` is logically false, it returns `nil`:

```
(is-small 50000)
-> nil
```

If you want to define a result for the “else” part of `if`, add it as a third argument:

```
src/examples/exploring.clj
(defn is-small [number]
  (if (< number 100) "yes" "no"))

(is-small 50000)
-> "no"
```

The `when` and `when-not` control flow macros are built on top of `if` and are described in [when and when-not, on page 245](#).

When you want to expand to multiple conditions, you can use `cond`, which takes any number of alternating tests and expressions:

```
(cond
  (> x 1000) "big"
  (> x 100)  "medium"
  :else      "small")
```

`cond` has no default fallback expression; a final test is required. By convention, Clojure programmers use `:else` as the trailing, always logically true test in the `cond`, although any always true value could serve the same role.

Other options for multiple conditions include `condp` (a variant of `cond` that applies a binary predicate), or the special form `case`, designed for fast matching to one of a set of values.

Introduce Side Effects with `do`

Clojure’s `if` allows only one form for each branch. What if you want to do more than one thing on a branch? For example, you might want to log that a certain branch was chosen. `do` takes any number of forms, evaluates them all, and returns the last.

You can use a `do` to print a logging statement from within an `if`:

```
src/examples/exploring.clj
(defn is-small [number]
  (if (< number 100)
    "yes"
    (do
      (println "Saw a big number" number)
      "no")))
```

which results in:

```
(is-small 200)
| Saw a big number 200
-> "no"
```

This is an example of a *side effect*. The `println` doesn't contribute to the return value of `is-small` at all. Instead, it reaches into the world outside the function and actually *does something*.

Many programming languages mix pure functions and side effects in a completely ad hoc fashion. Not Clojure. In Clojure, side effects are explicit and unusual. `do` is one way to say “side effects to follow.” Since `do` ignores the return values of all its forms except the last, those forms must have side effects to be of any use at all.

Recur with loop/recur

The Swiss Army knife of flow control in Clojure is `loop`:

```
(loop [bindings*] exprs*)
```

The `loop` special form works like `let`, establishing bindings and then evaluating `exprs`. The difference is that `loop` sets a *recursion point*, which can then be targeted by the `recur` special form:

```
(recur exprs*)
```

`recur` binds new values for `loop`'s bindings and returns control to the top of the loop. For example, the following `loop/recur` returns a countdown:

```
src/examples/exploring.clj
(loop [result [], x 5]
  (if (zero? x)
    result
    (recur (conj result x) (dec x))))
```

```
-> [5 4 3 2 1]
```

The first time through, loop binds result to an empty vector and binds x to 5. Since x is not zero, recur then rebinds the names result and x:

- result binds to the previous result conjoined with the previous x.
- x binds to the decrement of the previous x.

Control then returns to the top of the loop. Since x is again not zero, the loop continues, accumulating the result and decrementing x. Eventually, x reaches zero, and the if terminates the recurrence, returning result.

Instead of using a loop, you can recur back to the top of a function. This makes it simple to write a function whose entire body acts as an implicit loop:

```
src/examples/exploring.clj
(defn countdown [result x]
  (if (zero? x)
      result
      (recur (conj result x) (dec x))))

(countdown [] 5)
-> [5 4 3 2 1]
```

recur is a powerful building block. But you may not use it very often, because many common recursions are provided by Clojure's sequence functions—see [Using the Sequence Functions, on page 70](#).

At this point, you've seen quite a few language features but still no mutable local variables. Most variables in traditional languages are unnecessary and downright dangerous. Let's see how Clojure gets rid of them.

Where's My for Loop?

Clojure has no for loop and no direct mutable variables. Clojure provides *indirect* mutable references, but these must be explicitly called out in your code. See [Chapter 7, State and Concurrency, on page 155](#) for details. So how do you write all that code you're accustomed to writing with for loops?

Rather than create a hypothetical example, we decided to grab a piece of open source Java code (sort of) randomly, find a method with some for loops and variables, and port it to Clojure. We opened the Apache Commons project, which is very widely used. We selected the StringUtils class in Commons Lang, (assuming that such a class would require little domain knowledge to understand), searched for a suitable method, and found indexOfAny:

```
data/snippets/StringUtils.java
// From Apache Commons Lang, http://commons.apache.org/lang/
public static int indexOfAny(String str, char[] searchChars) {
    if (isEmpty(str) || ArrayUtils.isEmpty(searchChars)) {
        return -1;
    }
    for (int i = 0; i < str.length(); i++) {
        char ch = str.charAt(i);
        for (int j = 0; j < searchChars.length; j++) {
            if (searchChars[j] == ch) {
                return i;
            }
        }
    }
    return -1;
}
```

`indexOfAny` walks `str` and reports the index of the first char that matches any char in `searchChars`, returning -1 if no match is found.

Here are some example results from the documentation for `indexOfAny` (note * represents *any*, these are not literal calls):

```
StringUtils.indexOfAny(null, *)           = -1
StringUtils.indexOfAny("", *)             = -1
StringUtils.indexOfAny(*, null)           = -1
StringUtils.indexOfAny(*, [])             = -1
StringUtils.indexOfAny("zzabyycdxx", ['z', 'a']) = 0
StringUtils.indexOfAny("zzabyycdxx", ['b', 'y']) = 3
StringUtils.indexOfAny("aba", ['z'])      = -1
```

Two ifs, two fors, three possible points of return, and three mutable local variables are in `indexOfAny`, and the method is 14 lines long, as counted by David A. Wheeler's SLOCCount.⁵

Now, let's build a Clojure index-of-any, step by step. If we just wanted to find the matches, we could use a Clojure filter. But we want to find the *index* of a match. So, we create `indexed`, a function that takes a collection and returns an indexed collection:

```
src/examples/exploring.clj
(defn indexed [coll] (map-indexed vector coll))
```

Fortunately, Clojure can do most of the work for you with `map-indexed`, a sequence function that applies a two-parameter function (here, `vector`) to each value and its index (passed first) in the sequence. Thus, `indexed` returns a sequence of pairs of the form `[idx val]`. Try indexing a string:

5. <https://www.dwheeler.com/sloccount/>

```
(indexed "abcde")
-> ([0 |a] [1 |b] [2 |c] [3 |d] [4 |e])
```

Next, we want to find the indices of all the characters in the string that match the search set. Create an index-filter function that is similar to Clojure's filter but that returns the indices instead of the matches themselves:

```
src/examples/exploring.clj
```

```
(defn index-filter [pred coll]
  (for [[idx val] (indexed coll) :when (pred val)] idx))
```

Clojure's for is *not* a loop but a sequence comprehension (see [Transforming Sequences, on page 76](#)). The index/value pairs of (indexed coll) are bound to the names idx and val. The comprehension yields the value of idx for each matching pair, for only those pairs where (pred val) is true.

Clojure sets are also functions that test membership in the set. So, you can pass a set of characters and a string to index-filter and get back the indices of all characters in the string that belong to the set. Try it with a few different strings and character sets:

```
(index-filter #{|a |b} "abcdbbb")
-> (0 1 4 5 6)

(index-filter #{|a |b} "xyz")
-> ()
```

At this point, we've accomplished *more* than the stated objective. index-filter returns the indices of all of the matches, and we need only the first index. So, index-of-any simply takes the first result from index-filter:

```
src/examples/exploring.clj
```

```
(defn index-of-any [pred coll]
  (first (index-filter pred coll)))
```

Test that index-of-any works correctly with a few different inputs:

```
(index-of-any #{|z |a} "zzabyycdxx")
-> 0

(index-of-any #{|b |y} "zzabyycdxx")
-> 3
```

As the following table shows, the Clojure version is simpler than the imperative version by every metric.

Metric	LOC	Branches	Exits/Method	Variables
Imperative version	14	4	3	3
Functional version	5	1	1	0

What accounts for the difference?

- The imperative `indexOfAny` must deal with several special cases: null or empty strings, a null or empty set of search characters, and the absence of a match. These special cases add branches and exits to the method. With a functional approach, most of these kinds of special cases just work without any explicit code.
- The imperative `indexOfAny` introduces local variables to traverse collections (both the string and the character set). By using higher-order functions such as `map` and sequence comprehensions such as `for`, the functional `index-of-any` avoids all need for variables.

Unnecessary complexity tends to snowball. For example, the special case branches in the imperative `indexOfAny` use the magic number `-1` to indicate a non-match. Should the magic number be a symbolic constant? Whatever you think the right answer is, *the question itself disappears* in the functional version. While shorter and simpler, the functional `index-of-any` is also *vastly more general*:

- `indexOfAny` searches a string, while `index-of-any` can search any sequence.
- `indexOfAny` matches against a set of characters, while `index-of-any` can match against any predicate.
- `indexOfAny` returns the first match, while `index-filter` returns all of the matches and can be further composed with other filters.

As an example of how much more general the functional `index-of-any` is, you could use one of the individual functions we wrote to find the third occurrence of “heads” in a series of coin flips:

```
(nth
  (index-filter #{:h} [:t :t :h :t :h :t :t :t :h :h])
  2)
-> 8
```

So, writing `index-of-any` in a functional style, without loops or variables, is simpler, less error prone, and more general than the imperative `indexOfAny`. On larger units of code, these advantages become even more telling.

Wrapping Up

This has been a long chapter. But think about how much ground you’ve covered: you can work with basic Clojure data and collections, define and call functions, work with Java APIs, manage namespaces, and read and write metadata. You can write purely functional code, and yet you can easily introduce side effects when you need to. You’ve also met Lisp concepts, including reader macros, special forms, and destructuring.

While we'll still introduce many features of Clojure in the remainder of the book, virtually all of it (macros are a notable counter-example) is built upon the syntax and structures introduced to this point.

So far, we've focused on the language itself, but in the next chapter, we'll focus instead on the tools you use to interactively develop Clojure, a unique part of the Clojure experience.

Developing Interactively

One of Clojure's most defining characteristics is its dynamic and interactive approach to the development process. In Clojure, the focus is not on writing a program that you subsequently run, but on interactively composing and evaluating code as you write it, almost like a conversation.

In Clojure, most aspects of the language are designed to be inspected, modified, and dynamically loaded as you work, allowing you to stay in the flow. In this chapter, we'll see how to arrange your development environment to make the most of this approach and some of the tools that build on top of these ideas to give you superpowers other languages can only dream about.

The REPL (Read-Eval-Print-Loop)

In Clojure, the REPL is not merely an interpreter to try snippets of code; it is actually how the Clojure runtime works. The **ONLY** thing the Clojure runtime does is read and evaluate code in a loop. The REPL just makes this interactive by accepting your input at the beginning of the loop and printing the result at the end of each loop. The individual steps (read, evaluate, and print) are also available to you individually for direct use in your programs—an amazing superpower!

First, let's dissect how the REPL actually works by examining the parts in greater detail.

Read

The *read* phase invokes the Clojure reader component on a text expression, reading it into Clojure data—you can think of this as Clojure's parser component. The reader component is available for use in your own programs via functions in `clojure.core`: `read` (the most generic form that reads from a character

stream) and `read-string` (a simplified interface for reading expressions from a string). When reading Clojure code, you should use these.

Additionally, there are variants in the `clojure.edn` namespace, which specifically read edn data (Extended Data Notation, a portable subset of Clojure data with fewer features). If you are reading data (not code), use the `clojure.edn` versions.

One important note is that the read functions read the “next” value from the character stream or string and leave or ignore any value after that. The `read` and `read-string` functions have special options that allow you to ensure the input is fully read if needed.

Here’s an example of using the `read-string` function at the REPL:

```
;; Does not evaluate, just returns as data
(read-string "(+ 1 1)")
-> (+ 1 1)
```

This example reads the string `"(+ 1 1)"`, parses it as Clojure data, and returns a list of three values: the symbol `+`, the number `1`, and the number `1`.

In the REPL, the reader reads using the `read` function from `*in*`, a dynamic var bound to a Java character stream reader. By default, `*in*` is bound to the standard input reader (aka “standard in” or “stdin”).

Once the reader creates Clojure data, the REPL moves on to evaluate that data.

Eval

The Clojure evaluator takes Clojure data (not Clojure as text), macroexpands it, compiles it, and evaluates the compiled form, returning new Clojure data (not text).

The Clojure evaluator is available to you as the `clojure.core` function `eval`:

```
(eval 100)
-> 100

;; Lists and sequences invoke the fn in operator position
(eval '(+ 1 1))
-> 2

;; Symbols evaluate in the context of the current namespace (per *ns*),
;; here to a function
(eval 'eval)
-> #object[clojure.core$eval 0x2cc44ad "clojure.core$eval@2cc44ad"]
```

The result of evaluation is always a single Clojure data value, but it may be a composite collection like a vector or map. Once the expression has been evaluated, the result is passed on to the next phase, print.

Print

The print phase uses the Clojure printer to print the result of evaluation to the dynamic var `*out*`, which must be bound to a Java character stream writer. By default, `*out*` is bound to the standard output writer (aka “standard out” or “stdout”).

The Clojure printer is available in `clojure.core` as the `print` function or variants like `println`, which prints a new line after the value. There are other variants of printing available as well: `pr` (and `prn`) for printing data and `clojure.pprint/pprint` for pretty-printing data.

Here are some examples of round-tripping from text through `read-string` to Clojure data, then back to text via `println`. Note that `print` is a function with the side-effect of printing the value to `*out*` and it always returns `nil`. Depending on your REPL environment, you may see the printed value and the `nil` in different colors or even different panes of your editor.

```
(println (eval (read-string "100")))
| 100
-> nil

(println (eval (read-string "(+ 1 1)")))
| 2
-> nil
```

At this point, we’ve accomplished one spin of the REPL wheel, reading Clojure expressions as text into Clojure data, evaluating that Clojure data, and printing it back to text. Time to do it again!

Loop

There’s no mystery here—we just loop back to the top, ready to read again. You may have noticed that `clojure.core` holds all of the functions you need to write the REPL and, in fact, we’ll do just that.

Building your own REPL

Now that you’ve seen all of the components inside a REPL, you might imagine ways you could use those individual pieces in your own program. For example, you might store business rules as Clojure code in a database, then read and eval those rules at runtime, allowing you to dynamically modify your business logic.

Another way you can use these components is to write your very own REPL, which we will do next. You could, for example, save a transcript of the REPL session to a file as it happens, to recall what you've done later, or to use as an input to an AI model of how a REPL session looks. The important point is that Clojure provides the individual components, and you have full control of your development experience.

First, let's put the read-eval-print steps together into a helper method without the loop:

```
(defn rep []
  (try
    (print (str "(" *ns* "> ") ) ;; prompt
    (flush)
    (let [form (read-string (read-line)) ;; READ
          result (eval form)           ;; EVAL
          (println result)              ;; PRINT
          result)
        (catch Throwable e
          (println (str "Error: " (ex-message e))))))
```

In this example, we print a prompt for the user (one that looks visually different from our typical REPL prompt), then read a string from standard input, evaluate it, and print it back to standard output. When an exception is thrown, we print its message.

You can test this out at the REPL (you'll type the part after the prompts):

```
(rep)
| (user)> (+ 1 1)
| 2
-> 2

(rep)
| (user)> (/ 1 0)
| Error: Divide by zero
-> nil
```

Then, we need to wrap `rep` into a loop. We'll use the special keyword `:exit` to signal when to terminate the loop:

```
(defn repl []
  (loop [] ;; LOOP
    (when-not (= :exit (rep))
      (recur))))
```

Try running `(repl)` and using it as normal, exiting with `:exit`. Did it feel the same as your “normal” REPL?

We've gone through this exercise not because you'll want to design your own custom REPL (although sometimes that is useful!), but to demystify what the REPL really is and how you can use these components yourself.

Developing with a Connected REPL

Every Clojure editing environment has the ability to use a *connected REPL*. The connected REPL is nothing more than a REPL as we've just built, running in the context of your project, which the editor can “send” expressions to on your behalf. When you have a connected REPL, you typically don't need to type expressions directly into the REPL. Instead, you press buttons or key bindings in your editor, requesting to evaluate an expression, change namespaces, or load a file. Depending on the editor and the way you evaluate the command, the result may be displayed inline or in a separate pane.

Next, we'll dive into some of the most common Clojure editor environments. Which one you use is up to you, but we recommend choosing an environment that is similar to a tool you're already familiar with while you are learning Clojure, then re-evaluating once you know what features are important to you.

Editors

This section discusses tools that are under active development, so details are likely to change and become out of date—check the tool websites for the latest details on how to install and use these editors. In general, all of the tools mentioned here are free (with the exception of Cursive), work on multiple operating systems, and have excellent Clojure support via one or more plugins or tools. They are all sufficient for both getting started with Clojure and using in professional development, and are in active development at the time of this writing.

Visual Studio Code and the Calva plugin

Visual Studio Code¹ (VS Code) is rapidly becoming one of the most popular coding environments across many languages. VS Code was created by Microsoft and is freely available with support for a wide range of languages. For Clojure, use the Calva plugin,² which is free, open source, and frequently updated.

1. <https://code.visualstudio.com/>

2. <https://calva.io/>

To get started, download and install VS Code, then search for Calva in the VS Code Extension pane and install it. Once you have it installed, Calva provides a getting-started experience that is accessible via the VS Code command palette: “Calva: Create a Getting Started REPL Project.” This creates a template project and some instructions on how to use Calva to edit and evaluate Clojure code.

Some good reasons to choose Calva include familiarity with VS Code and friendliness to beginners. Clojure’s developers are very responsive and constantly improving this excellent tool. If you’re not sure which editor to use, start here.

Emacs and CIDER

Emacs³ is one of the oldest and most storied editors in existence, dating from the 1970’s. Some say it is more of a lifestyle than a mere editor. If you are coming from another Lisp language like Common Lisp or Racket, you’re likely already familiar with Emacs. Emacs is highly customizable and uses Emacs Lisp (its own variant of Lisp) for configuration and extension. Not surprisingly, there is a great deal of resonance between the Lisp and Emacs communities, although Emacs has support for many languages.

Configuring Emacs is a topic big enough that multiple books have been written about that alone, but for Clojure, we recommend you use the CIDER⁴ environment. To get started, install Emacs using the platform-specific installer and then install CIDER via the package.el package manager. On CIDER, you’ll start a connected REPL with the `cider-jack-in` command.

CIDER and Emacs has been the most popular Clojure editing environment in the Clojure annual user survey for the last decade. You should consider it if you are already familiar with Emacs or are intrigued by getting to know this legendary editor and highly customizable environment.

IntelliJ IDEA and Cursive

IntelliJ IDEA⁵ is a commercial editor produced by JetBrains and originally targeted at Java developers. Since then, it has grown into an integrated development environment (IDE) with plugin support for many languages, including Clojure. The commercial Cursive⁶ plugin provides Clojure support.

3. <https://www.gnu.org/software/emacs/>

4. <https://cider.mx/>

5. <https://www.jetbrains.com/idea/>

6. <https://cursive-ide.com/>

Both IntelliJ (via the Community edition) and Cursive (via the non-commercial license) can be used for free for learning and non-commercial work.

One of the areas where Cursive excels is integration with the Java and JVM ecosystem, due to its long history in that space. IntelliJ and Cursive are a particularly good choice if you are already familiar with IntelliJ or Java, or you plan to do both Clojure and Java, or extensive Java interop. This support extends to the debugger, which can walk through both Clojure and Java code.

Others

There are a number of other environments available; the list above is not exhaustive. In particular, there are multiple good Clojure environments for Vim and Neovim. Vim is an editor with almost as storied a tradition as Emacs, but focused on efficient text editing. If you come to Clojure with Vim experience, you should search for the latest options to see if they will meet your needs.

Additionally, many lesser-used options are not listed here, and more appear all the time.

Next, we'll refocus our attention away from the REPL for a moment and, instead, consider how we'll edit our Clojure code.

Structural Editing

Text editors primarily consider input and editing as a line-oriented activity. Editing commands for selection, cut/copy/paste, deletion/replacement, and movement work on characters, words, or lines. For programming languages that consist of statements, this is a good match. Languages with “blocks”, often identified with braces or other delimiters, also have support for folding or manipulating code blocks.

Clojure (and other Lisp family languages) are somewhat different in that there are no statements—everything is an expression and all expressions can be nested arbitrarily. Every top-level expression can thus be seen as a tree of expressions where nesting is indicated by delimiter pairs `( )` for lists, `[ ]` for vectors, and `{ }` for maps. Given this tree-like structure, it is often more useful to modify code by manipulating the expression tree. For example, reordering arguments in a function call means moving adjacent expression nodes inside a list delimited by `( )`, which may span many lines. Clojure's treatment of commas as whitespace helps a bit here as well, removing yet another piece of punctuation that doesn't need to be updated.

Structural editing is a set of operations for manipulating code (selection, navigation, copy/paste, delete, and so forth) by altering the nodes in the expression tree rather than lines in a file. This shift in perspective aligns the editor model with the language structure and is more efficient once you’ve learned the most common commands.

The two best-known systems for structural editing are known as *paredit* and *parinfer*. Both of these terms referred first to a specific implementation, but are now also used to refer to their general approach (implemented in many editors). Before we look at either, though, let’s consider the importance of balanced delimiters in editing Clojure code.

Balanced Delimiters

One of the most important principles in structural editing is that delimiters are always maintained in balanced and properly nested pairs. That is, you will never see a single (or) or ([)], only properly balanced pairs of delimiters. To ensure this, when you type a single opening delimiter, the closing delimiter will also be inserted, and your cursor will be between them.

For examples in this section only, we’ll use | as a special symbol to indicate the location of your cursor (not an actual part of the code). We’ll break down the changes as “before” (with cursor mark), “type” (what you type), and “after” (the resulting text in the editor with cursor mark).

Delimiters are added in pairs:

```
Before: |
Type:  (
After:  (|)
```

After you’ve entered the left parenthesis, the right parenthesis is automatically added, and your cursor will be between them. If you want to remove the balanced pair, hit the backspace key to remove the left parenthesis, and the right parenthesis will also be removed, maintaining balance.

When you get to the point where you want to end a delimited expression, the ending delimiter is already there. You could use a keyboard or mouse movement command to move past it, but it is usually simpler to just type the ending delimiter again. *Paredit* understands that it does not need to emit a new closing parenthesis and only moves your cursor:

```
Before: (+ 1 2|)
Type:   )
After:  (+ 1 2)|
```


This behavior is the same for all delimiter pairs. Notice that there are no special commands here—you just type (and) when they’re needed, so there is no practical effect on your typing, but delimiter pairs are maintained at all times.

Often, the hardest part of adopting paredit to your typing workflow is breaking key command habits that work in line-oriented ways, the most common of which is “delete line.” Because these commands are not part of paredit, they often result in unbalanced parentheses, which prevent paredit from working properly. Instead, you should use structural editing commands to remove one or more expressions at a time.

Next, we’ll look at paredit and some of the commands you can use to manipulate Clojure code per-expression rather than per-line.

Paredit Commands

Paredit originated as an Emacs mode, but its ideas and key bindings are available on every Clojure editor, usually enabled by default. We will not provide specific key bindings here because the defaults vary between editors and operating systems—consult the appropriate manual for your environment. Balanced delimiters are perhaps the fundamental constraint of structural editing, and we’ll start there.

The “slurp” and “barf” commands modify an inner delimited list of expressions by pushing or pulling adjacent expressions of the outer delimited list. “Barf” pushes an element of the current list into the outer list, and “slurp” pulls an element of the outer list into the inner list. Both commands can be done “forward” (to the right side) or “backward” (to the left side).

So “forward-barf” pushes an element of the “current” delimited expression (your cursor can be anywhere inside the inner list) to the right of the inner expression in the outer list:

```
Before: (+ 5 (|* 4 3 2) 1)
Type:   forward-barf command
After:  (+ 5 (|* 4 3) 2 1)
```

Conversely, “forward-slurp” pulls an element of the outer list into the right side of the inner list:

```
Before: (+ 5 (|* 4 3) 2 1)
Type:   forward-slurp command
After:  (+ 5 (|* 4 3 2) 1)
```

The backward variants do the same operation, but to the left of the inner expression.

The “raise” command replaces the parent expression with the current expression:

```
Before: (+ 5 |(* 4 3) 2 1)
Type:   raise-sexp command
After:  |(* 4 3)
```

The “kill” commands delete whole expressions (not characters or words), and there is a whole family of commands to delete expressions to the right, to the left, or to the end of the current list. For example, the “kill-right” command kills all expressions to the end of the list or newline:

```
Before: (+ 5 4 |(* 3 2) 1)
Type:   kill-right command (may kill just one expression, depends on editor)
After:  (+ 5 4)
```

This was a whirlwind tour of paredit—we have still barely scratched the surface of all of its functionality. The big picture idea is thinking in the structure of nested expressions, not in characters and lines. Paredit is not the only game in town; next, we’ll also cover an alternative structural editing paradigm.

Parinfer

Parinfer⁷ was created in 2017 by Shaun Lebron and is a new take on structural editing ala paredit. Shaun realized that paredit uses parentheses structure to auto-update indentation, and it was possible to drive this relationship in either direction. That is, parentheses affect indentation, but also, indentation can affect parentheses. In parinfer, these two modes are called paren mode and indent mode.

Some users (particularly those familiar with whitespace-significant languages like Python) prefer indent mode so that parens are automatically inferred from structure. Rather than learn new structural editing commands like “slurp” and “barf,” you merely change code indentation, and parinfer infers changes to the tree-like structure of code based on indentation. Another benefit is that line editing commands like “delete line” are allowed (parinfer infers the current parentheses from the indent structure), which makes this a gentler introduction to structural editing.

Parinfer is available in every Clojure editor and is worth investigating if you are new to structural editing, particularly if you are coming from a non-Lisp language.

7. <https://shaunlebron.github.io/parinfer/>

Loading and Evaluating Code

Clojure is, by design, a live runtime where you interactively build the program while running it. New code is added and changed in small increments and immediately takes effect via evaluation or loading. This workflow is extremely responsive, providing immediate feedback and a sense of flow that is hard to match in most other languages.

Throughout this book, we have given examples of typing code at the REPL and viewing the results, and indeed, this is one mode of interaction with the Clojure runtime. However, many experienced Clojure developers rarely type expressions directly at a REPL. Instead, they use an editor with a connected REPL and evaluate parts of their source code as they write it. This mode of interaction is one essential part of the Clojure development experience (the other is structural editing).

Every Clojure editor has support for running and connecting to a REPL for the project under development (the details vary considerably, so we will not attempt to cover them here). Once you have a connected REPL, you need to load the namespace in your open editor. To work in the context of a namespace at the REPL, you could use `require` (to load) and `in-ns` to change the current namespace:

```
(require 'my.namespace)
-> nil
(in-ns 'my.namespace)
-> #object[clojure.lang.Namespace 0x7a560583 "my.namespace"]
```

Clojure editors typically have a bound command that performs these actions for you based on the currently open editor. For example, Calva has “Load/Evaluate Current File and its Requires/Dependencies,” Cursive has “Load file in REPL,” and CIDER has “cider-load-buffer-and-switch-to-repl-buffer.” These differ slightly in their details, but in general, they all cause the REPL to have the running context of the code being edited.

Once you have loaded the current namespace in your REPL, you can directly evaluate code in that context. All of the editor environments have multiple commands to evaluate code, letting you choose between the expression immediately prior to the cursor, the expression around the cursor, or the top-level expression around the cursor. The choices here are largely a matter of personal preference—if you are just starting, try a few of the options in your editor to see what feels right to you.

At this point, you can now load (or reload) the current namespace and evaluate individual expressions (like the function you just defined). Another common need is creating “scratch” code to exercise your functions. You could type this code directly into a REPL, but often you will want to modify and re-run that code, if not immediately, then the next time you return to this namespace. To facilitate that, a common practice is to create “Rich Comment Forms.” The name is somewhat of a pun—they are executable “rich” comments and, also, this is a practice established early by Rich Hickey, the creator of Clojure.

There is nothing magical about Rich Comment Forms; it is simply using the comment macro to inline scratch code at the top-level of your namespace, often at the end of the `.clj` file. You can then load the namespace and evaluate some or all of the code in the RCF via your connected REPL while working in the namespace. Because the comment macro evaluates to `nil`, this has no effect on the loading of the namespace.

Dependencies in the REPL

Sometimes, while working in the REPL, you may discover that you need to add one or more libraries or development tools to continue working. Clojure includes the ability to download and integrate libraries without restarting your REPL and losing the state of your program. There are really two slightly different cases: either you are adding libraries solely to work with in the REPL, or you are adding them to the project for future and continued use. We'll examine these separately.

Adding New Libraries to the REPL

In the cases of speculative libraries or dev tools, we are often adding these tools to our runtime environment, but don't necessarily expect to use them in the future. In this case, you'll want to use the `add-lib` (for one) or `add-libs` (for many) functions. These are included in the `clojure.repl.deps` namespace and auto-referred in the user namespace by the REPL.

We'll focus mostly on `add-lib`, as you are most often adding one library at a time. `add-lib` takes the library and, optionally, a coordinate in the same format as the `deps.edn` deps map (see [Chapter 12, Project Tooling, on page 263](#) for details). When that lib is not already included in the REPL classpath, it (and all of its transitive deps) is downloaded and dynamically added to the classpath. If the optional coordinate is omitted, a query is made to find the newest version (Maven or git) corresponding to the specified lib and download that.

For example:

```
(add-lib 'org.clojure/core.cache {:mvn/version "1.1.234"})
-> [org.clojure/core.cache org.clojure/data.priority-map]
```

Here, we asked for the `core.cache` library version 1.1.234. The return value indicates all of the libraries that were added to the REPL classpath—this may be none (if the library is already on the REPL classpath or the version does not exist), one, or many (if the library has transitive deps like `org.clojure/data.priority-map` here). Note that if some of the transitive deps are already on the classpath, they will not be modified or listed in the return value.

Because the newest version is almost always the one you want, you can also omit the version, and `add-lib` will look up the newest version. In this case, this would have had the same effect:

```
(add-lib 'org.clojure/core.cache)
-> [org.clojure/core.cache org.clojure/data.priority-map]
```

Sometimes, the libraries we add are not speculative. We may be adding a new library, but we know for sure that we will continue with that library as a dependency.

Adding New Libraries to the Project

If we know this dependency will be used in the future, it's better to add the library to our `deps.edn` file as a new dependency and call `sync-deps`. In this case, our first step is modify the `deps.edn` to add the new dep:

```
{:deps {org.clojure/core.cache {:mvn/version "1.1.234"}}
```

Now, let's sync the modified `deps.edn` desired deps into the running REPL:

```
(sync-deps)
-> [org.clojure/core.cache org.clojure/data.priority-map]
```

Clojure reads the `deps.edn`, finds any new dependencies or transitive dependencies, and calls `add-libs` on them. Remember that `add-libs`, `add-lib`, and `sync-deps` are best effort and cannot delete or modify the version of a library that is already on the classpath. In this case, you must restart your REPL.

Once a new library has been loaded via any of these functions, it can be immediately required and used:

```
(require '[clojure.core.cache :as cache])
-> nil
(dir cache)
| ->BasicCache
| ->FIFOCache
...
```

Being able to add new dependencies to your project and REPL on the fly and keep working without breaking flow or losing your REPL state is a powerful tool in your toolbox. You can even add debugging or introspection tools to a production REPL to debug a problem on the fly.

More Tools



Because Clojure is such an open environment with access to low-level components, it's easy to build tools to help developers, and the Clojure community has built a wide range of them! You can find a good starting list at [Appendix 1, Developer Tools, on page 295](#).

Wrapping Up

In this chapter, we saw a whirlwind tour of how to use Clojure for interactive development, starting with a thorough breakdown of what the REPL actually is and how to customize it if desired. Effective REPL-based development relies on having a high-quality editor, using a connected REPL, and learning to leverage structural editing for efficient development. The REPL also provides built-in tools for loading and reloading code, dynamically adding dependencies, and using introspection on namespaces, function docs, and source.

Next, we'll dive deeper into the most fundamental parts of Clojure. At its core, working with Clojure primarily involves representing your domain as data, then writing functions that manipulate that data. We'll start with sequences, the unifying abstraction over how we traverse and transform data in Clojure.

Part II

Data and Functions

At the heart of every Clojure program is your domain, represented as data, and a set of functions that manipulate that data.

Unifying Data with Sequences

Programs manipulate data. At the lowest level, programs work with structures such as strings, lists, vectors, maps, sets, and trees. At a higher level, these same data structure abstractions crop up again and again. For example:

- XML data is a tree.
- Database result sets can be viewed as lists or vectors.
- Directory hierarchies are trees.
- Files are often viewed as one big string or as a vector of lines.

In Clojure, all these data structures can be accessed through a single abstraction: the sequence (or *seq*).

A *seq* (pronounced “seek”) is a *logical* list. It’s logical because Clojure does not tie sequences to the concrete implementation details of the list data structure. Instead, the *seq* is an abstraction that can be used everywhere.

Collections that can be viewed as *seqs* are called *seq-able* (pronounced “SEEK-a-bull”). In this chapter, you’ll meet a variety of *seq-able* collections:

- All Clojure collections
- All Java collections
- Strings and Java arrays
- Regular expression matches
- Directory structures
- I/O streams
- XML trees

You’ll also meet the sequence functions, which work on any *seq-able* data. Because so many things are sequences, the sequence functions are much more powerful and general than the collection API in most languages. These functions act as the collection API for Clojure, and they also replace many of the loops you would write in an imperative language.

In this chapter, you will become a power user of Clojure sequences. You'll see how to use a common set of very expressive functions with a wide range of datatypes. Then, in the next chapter ([Chapter 5, Functional Programming, on page 97](#)), you'll learn the functional style in which the sequence functions are written.

Everything Is Seq-able

Every aggregate data structure in Clojure can be viewed as a sequence. A sequence has three core capabilities:

- You can get the first item in a sequence:

```
(first aseq)
```

`first` returns `nil` if its argument is empty or `nil`.

- You can get everything after the first item—the rest of a sequence:

```
(rest aseq)
```

`rest` returns an empty seq (not `nil`) if there are no more items.

- You can construct a new sequence by adding an item to the front of an existing sequence. This is called `consing`:

```
(cons elem aseq)
```

While collections are seq-able, they are often not themselves seqs. The `seq` function is used to obtain the seq view of any seq-able collection:

```
(seq coll)
```

Sequence functions expect to take a seq-able collection and will call `seq` on it (at some point) to work with the seq view of the collection. Often, that call is delayed for the sake of laziness.

`seq` will return `nil` if its `coll` is empty or `nil`. This function, in combination with the rules for logical truth (see [Booleans and nil, on page 23](#)), lets us build conditionals based on the result of `seq`. If the seq has one or more elements, the return value is logically true. If the seq is empty, `seq` returns false. It is idiomatic to use `(seq coll)` as a predicate, and this is known as *nil-punning*.

Looking back at `rest`, we see that it always returns a seq (never `nil`). The benefit of this behavior is laziness—the next element of the sequence does not need to be forced or computed to see if it is at the end of the sequence. The trade-

off in this approach is the lack of nil-punning (the result of rest can't be used as a logical test). To get the other side of this trade-off (eager, but useful for conditional decisions), use next:

```
(next aseq)
```

(next aseq) is equivalent to (seq (rest aseq)).

The seq functions work on lists:

```
(first '(1 2 3))
```

```
-> 1
```

```
(rest '(1 2 3))
```

```
-> (2 3)
```

```
(cons 0 '(1 2 3))
```

```
-> (0 1 2 3)
```

The seq functions work on all other Clojure data structures as well. For example, you can use the seq functions on vectors:

```
(first [1 2 3])
```

```
-> 1
```

```
(rest [1 2 3])
```

```
-> (2 3)
```

```
(cons 0 [1 2 3])
```

```
-> (0 1 2 3)
```

The Origin of cons

Clojure's sequence is an abstraction based on Lisp's concrete lists. In the original implementation of Lisp, the three fundamental list operations were named `car`, `cdr`, and `cons`. `car` and `cdr` are acronyms that refer to implementation details of Lisp on the original IBM 704 platform. Many Lisps, including Clojure, replace these esoteric names with the more meaningful names `first` and `rest`.

The third function, `cons`, is short for construct. Lisp programmers use `cons` as a noun, verb, and adjective. You use `cons` to create a data structure called a *cons cell*, or just a *cons* for short.

Most Lisps, including Clojure, retain the original `cons` name, since “construct” is a pretty good mnemonic for what `cons` does. It also helps remind you that sequences are immutable. For convenience, you might say that `cons` adds an element to a sequence, but it's more accurate to say that `cons` *constructs* a new sequence, which is like the original sequence but with one element added.

When you apply `rest` or `cons` to a vector, the result is a `seq`, not a vector. In the REPL, `seqs` print just like lists, as you can see in the earlier output. You can check the actual returned type by using the `seq?` predicate:

```
(seq? (rest [1 2 3]))
-> true
```

The generality of `seqs` is very powerful, but sometimes you want to produce a specific implementation type. This is covered in [Calling Structure-Specific Functions, on page 87](#).

You can treat maps as `seqs`, if you think of a map as a sequence of map entries (where each entry is a key/value pair):

```
(first {:fname "Aaron" :lname "Bedra"})
-> [:lname "Bedra"]

(rest {:fname "Aaron" :lname "Bedra"})
-> ([:fname "Aaron"])

(cons [:mname "James"] {:fname "Aaron" :lname "Bedra"})
-> ([:mname "James"] [:lname "Bedra"] [:fname "Aaron"])
```

You can also treat sets as `seqs`:

```
(first #{:the :quick :brown :fox})
-> :brown

(rest #{:the :quick :brown :fox})
-> (:quick :fox :the)

(cons :jumped #{:the :quick :brown :fox})
-> (:jumped :brown :quick :fox :the)
```

Maps and sets have a stable traversal order, but that order depends on implementation details, and you shouldn't rely on it. Elements of a set will not necessarily come back in the order that you put them in:

```
#{:the :quick :brown :fox}
-> #{:fox :the :quick :brown}
```

If you want a reliable order, you can use this:

```
(sorted-set & elements)
```

`sorted-set` will sort the values by their natural sort order:

```
(sorted-set :the :quick :brown :fox)
-> #{:brown :fox :quick :the}
```

Likewise, key/value pairs in maps won't necessarily come back in the order you put them in:

```
{:a 1 :b 2 :c 3}
-> {:a 1, :c 3, :b 2}
```

You can create a sorted map with `sorted-map`:

```
(sorted-map & elements)
```

`sorted-maps` won't come back in the order you put them in either, but they *will* come back sorted by key:

```
(sorted-map :c 3 :b 2 :a 1)
-> {:a 1, :b 2, :c 3}
```

In addition to the core capabilities of `seq`, two other capabilities are worth meeting immediately: `conj` and `into`.

```
(conj coll element & elements)
(into to-coll [xform] from-coll)
```

`conj` adds one or more elements to a collection, and `into` adds all of the items from one collection to another. Both `conj` and `into` add items at an efficient insertion spot for the underlying data structure. For lists, `conj` and `into` add to the front:

```
(conj '(1 2 3) :a)
-> (:a 1 2 3)

(into '(1 2 3) '(:a :b :c))
-> (:c :b :a 1 2 3)
```

For vectors, `conj` and `into` add elements to the back:

```
(conj [1 2 3] :a)
-> [1 2 3 :a]

(into [1 2 3] [ :a :b :c ])
-> [1 2 3 :a :b :c]
```

Because `conj` (and related functions) does the efficient operation for the underlying data structure, you can often write code that is both efficient and completely decoupled from a specific underlying implementation.

The Clojure sequence functions are particularly suited for large (or even infinite) sequences. Most Clojure sequences are *lazy*: they generate elements only when they are actually needed. Thus, Clojure's sequence functions can process sequences too large to fit in memory.

Clojure sequences are *immutable*: they never change. This makes it easier to reason about programs and means that Clojure sequences are safe for concurrent access. It does, however, create a small problem for human language.

Why Do Functions on Vectors Return Lists?

When you try examples at the REPL, the results of `rest` and `cons` print as lists, even when the inputs are vectors, maps, or sets. Does this mean that Clojure is converting everything to a list internally? No! The sequence functions always return a seq regardless of their inputs. You can verify this by using the `seq?` predicate:

```
(list? (rest [1 2 3]))
-> false

(seq? (rest [1 2 3]))
-> true
```

As you can see, the result of `(rest [1 2 3])` is not a list but a sequence. Sequences are logical lists (but not concrete lists). Both lists and sequences print in the same way.

English-language descriptions flow much more smoothly when describing mutable things. Consider the following two descriptions for a hypothetical sequence function `triple`:

- `triple` triples each element of a sequence.
- `triple` takes a sequence and returns a new sequence with each element of the original sequence tripled.

The latter version is specific and accurate. The former is much easier to read, but it might lead to the mistaken impression that a sequence is actually changing. Don't be fooled: *sequences never change*. If you see the phrase “foo changes x,” mentally substitute “foo returns a changed copy of x.”

Using the Sequence Functions

The Clojure sequence functions provide a rich set of functionality that can work with any sequence. If you come from an object-oriented background where nouns rule, the sequence functions are truly “Revenge of the Verbs.” The functions provide a rich backbone of functionality that can take advantage of any data structure that obeys the basic `first/rest/cons` contract.

The following functions are grouped into four broad categories:

- Functions that create sequences
- Functions that filter sequences
- Sequence predicates
- Functions that transform sequences

These divisions are somewhat arbitrary. Since sequences are immutable, *most* of the sequence functions create new sequences. Some of the sequence

functions both filter and transform. Nevertheless, these divisions provide a rough road map through a large collection of functions.

Creating Sequences

Clojure provides a number of functions that create sequences. `range` produces a sequence from a start to an end, incrementing by step each time.

```
(range end)
(range start end)
(range start end step)
```

Ranges include their start but not their end. If you do not specify them, start defaults to zero, end defaults to positive infinity, and step defaults to 1. Try creating some ranges at the REPL:

```
(range 10)                ;; end only
-> (0 1 2 3 4 5 6 7 8 9)

(range 10 20)             ;; start + end
-> (10 11 12 13 14 15 16 17 18 19)

(range 1 25 2)            ;; step by 2
-> (1 3 5 7 9 11 13 15 17 19 21 23)

(range 0 -1 -0.25)        ;; negative step
-> (0 -0.25 -0.5 -0.75)

(range 1/2 4 1)           ;; ratios
-> (1/2 3/2 5/2 7/2)
```

The `repeat` function repeats an element x n times:

```
(repeat n? x)
```

Try to repeat some items from the REPL:

```
(repeat 5 1)
-> (1 1 1 1 1)

(repeat 10 "x")
-> ("x" "x" "x" "x" "x" "x" "x" "x" "x" "x")
```

If n is omitted, `repeat` creates an infinite series of x 's. Be careful experimenting with this at the REPL! Use `(set! *print-length* 20)` to protect yourself against printing infinite sequences.

`iterate` begins with a value x and continues forever, applying a function f to each value to calculate the next.

```
(iterate f x)
```

If you begin with 1 and iterate with inc, you can generate the whole numbers:

```
(take 10 (iterate inc 1))
-> (1 2 3 4 5 6 7 8 9 10)
```

Since the sequence is infinite, you need another new function to help you view the sequence from the REPL.

```
(take n sequence)
```

take returns a lazy sequence of the first n items from a collection and provides one way to create a finite view onto an infinite collection.

The whole numbers are a pretty useful sequence to have around, so let's def them for future use:

```
(def whole-numbers (iterate inc 1))
-> #'user/whole-numbers
```

When called with a single argument, repeat returns a lazy, infinite sequence:

```
(repeat x)
```

Try repeating at the REPL. Don't forget to wrap the result in a take:

```
(take 20 (repeat 1))
-> (1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
```

The cycle function takes a collection and cycles it infinitely:

```
(cycle coll)
```

Try cycling some collections at the REPL:

```
(take 10 (cycle (range 3)))
-> (0 1 2 0 1 2 0 1 2 0)
```

The interleave function takes multiple collections and produces a new collection that cycles through each collection, taking the next value until one of the collections is exhausted.

```
(interleave & colls)
```

When one of the collections is exhausted, the interleave stops. So, you can mix finite and infinite collections:

```
(interleave whole-numbers ["A" "B" "C" "D" "E"])
-> (1 "A" 2 "B" 3 "C" 4 "D" 5 "E")
```

Closely related to `interleave` is `interpose`, which returns a sequence with each of the elements of the input collection segregated by a separator:

```
(interpose separator coll)
```

You can use `interpose` to build delimited strings:

```
(interpose "," ["apples" "bananas" "grapes"])
-> ("apples" "," "bananas" "," "grapes")
```

`interpose` works nicely with `(apply str ...)` to produce output strings:

```
(apply str (interpose "," ["apples" "bananas" "grapes"]))
-> "apples,bananas,grapes"
```

The `(apply str (interpose separator sequence))` idiom is common enough that Clojure provides a performance-optimized version as `clojure.string/join`:

```
(join separator sequence)
```

Use `clojure.string/join` to comma-delimit a list of words:

```
(require '[clojure.string :refer [join]])
(join \, ["apples" "bananas" "grapes"])
-> "apples,bananas,grapes"
```

For each collection type in Clojure, there is a function that takes an arbitrary number of arguments and creates a collection of that type:

```
(list & elements)
(vector & elements)
(hash-set & elements)
(hash-map key-1 val-1 ...)
```

`hash-set` has a cousin `set` that works a little differently; `set` expects a collection as its first argument:

```
(set [1 2 3])
-> #{1 2 3}
```

`hash-set` takes a variable list of arguments:

```
(hash-set 1 2 3)
-> #{1 2 3}
```

`vector` also has a cousin, `vec`, which takes a single collection argument instead of a variable argument list:

```
(vec (range 3))
-> [0 1 2]
```


Now that you have the basics of creating sequences, you can use other Clojure functions to filter and transform them.

Filtering Sequences

Clojure provides a number of functions that filter a sequence, returning a subsequence of the original sequence. The most basic of these is `filter`:

```
(filter pred coll)
```

`filter` takes a predicate and a collection and returns a sequence of objects for which the filter returns true (when interpreted in a Boolean context). You can filter the whole-numbers from the previous section to get the odd numbers or the even numbers:

```
(take 10 (filter even? whole-numbers))
-> (2 4 6 8 10 12 14 16 18 20)

(take 10 (filter odd? whole-numbers))
-> (1 3 5 7 9 11 13 15 17 19)
```

You can take from a sequence while a predicate remains true with `take-while`:

```
(take-while pred coll)
```

For example, to take all of the characters in a string up to the first vowel, we can define some useful helper functions:

```
(def vowel? #{\a\e|i|o|u})
(def consonant? (complement vowel?))
```

Then, use those predicates to take the characters from the string up to the first vowel:

```
(take-while consonant? "the-quick-brown-fox")
-> (\t \h)
```

A couple of interesting things are happening here:

- Sets act as functions that look up a value in the set and return either the value or `nil` if not found. So, you can read `#{\a\e|i|o|u}` as “the function that tests to see whether its argument is a vowel.”
- `complement` takes a function and returns a function with the same behavior but the logical not of its return value. Here, we create `consonant?` by defining it as the function that is the complement of `vowel?`.

The opposite of take-while is drop-while:

```
(drop-while pred coll)
```

drop-while drops elements from the beginning of a sequence while a predicate is true and then returns the rest. You could use drop-while to drop all leading non-vowels from a string:

```
(drop-while consonant? "the-quick-brown-fox")
-> (|e | - |q |u |i |c |k | - |b |r |o |w |n | - |f |o |x)
```

split-at and split-with will split a collection into two collections:

```
(split-at index coll)
(split-with pred coll)
```

split-at takes an index, and split-with takes a predicate:

```
(split-at 5 (range 10))
->[(0 1 2 3 4) (5 6 7 8 9)]

(split-with #(<= % 10) (range 0 20 2))
->[(0 2 4 6 8 10) (12 14 16 18)]
```

Sequence Predicates

Filter functions take a predicate and return a sequence. Closely related are the sequence predicates. A sequence predicate asks how some other predicate applies to every item in a sequence. For example, the every? predicate asks whether some other predicate is true for every element of a sequence.

```
(every? pred coll)
```

```
(every? odd? [1 3 5])
-> true
```

```
(every? odd? [1 3 5 8])
-> false
```

A lower bar is set by some:

```
(some pred coll)
```

some returns the first non-false value for its predicate or returns nil if no element matched:

```
(some even? [1 2 3])
-> true
```

```
(some even? [1 3 5])
-> nil
```

Notice that `some` does not end with a question mark. `some` is not a predicate, although it's often used like one. `some` returns the *actual value* of the first match instead of `true`. The distinction is invisible when you pair `some` with `even?`, since `even?` is itself a predicate. To see a non-true match, try using `some` with `identity` to find the first logically true value in a sequence:

```
(some identity [nil false 1 nil 2])
-> 1
```

A common use of `some` is to perform a linear search to see if a sequence contains a matching element, which is typically written as a set of a single element (treating the set as a predicate for containment). Here's an example to see if a sequence contains the value 3:

```
(some #{3} (range 20))
-> 3
```

Note that the value returned is the value in the sequence, which would act as a truthy value if this were used as a conditional test.

The behavior of the other predicates is obvious from their names:

```
(not-every? pred coll)
(not-any? pred coll)
```

Not every whole number is even:

```
(not-every? even? whole-numbers)
-> true
```

But it would be a lie to claim that not any whole number is even:

```
(not-any? even? whole-numbers)
-> false
```

Note that we picked questions to which we already knew the answer. In general, you have to be careful when applying predicates to infinite collections. They might run forever.

Transforming Sequences

Transformation functions transform the values in the sequence. The simplest transformation is `map`:

```
(map f coll)
```

map takes a source collection coll and a function f, and it returns a new sequence by invoking f on each element in the coll. You could use map to wrap every element in a collection with an HTML tag.

```
(map #(format "<p>%s</p>" %) ["the" "quick" "brown" "fox"])
-> ("<p>the</p>" "<p>quick</p>" "<p>brown</p>" "<p>fox</p>")
```

map can also take more than one collection argument. f must then be a function of multiple arguments. map will call f with one argument from each collection, stopping whenever the smallest collection is exhausted:

```
(map #(format "<%s>%s</%s>" %1 %2 %1)
 ["h1" "h2" "h3" "h1"] ["the" "quick" "brown" "fox"])
-> ("<h1>the</h1>" "<h2>quick</h2>" "<h3>brown</h3>"
 "<h1>fox</h1>")
```

Another common transformation is reduce:

```
(reduce f val? coll)
```

f is a function of two arguments. reduce applies f on val and the first element in coll and then applies f to the result and the second element, and so on. reduce is useful for functions that “total up” a sequence in some way. You can use reduce to add items:

```
(reduce + 0 (range 1 11))
-> 55
```

or to multiply them:

```
(reduce * 1 (range 1 11))
-> 3628800
```

If val is not provided, reduce starts by applying f to the first two values of coll.

You can sort a collection with sort or sort-by:

```
(sort comp? coll)
(sort-by a-fn comp? coll)
```

sort sorts a collection by the natural order of its elements, where sort-by sorts a sequence by the result of calling a-fn on each element:

```
(sort [42 1 7 11])
-> (1 7 11 42)

(sort-by str [42 1 7 11])
-> (1 11 42 7)
```

If you don't want to sort by natural order, you can specify an optional comparison function `comp` for either `sort` or `sort-by`:

```
(sort > [42 1 7 11])
-> (42 11 7 1)

(sort-by :grade > [{:grade 83} {:grade 90} {:grade 77}])
-> ({:grade 90} {:grade 83} {:grade 77})
```

The granddaddy of all filters and transformations is the *list comprehension*. A list comprehension creates a list based on an existing list, using set notation. In other words, a comprehension states the properties that the result list must satisfy. In general, a list comprehension will consist of the following:

- Input list(s)
- Placeholder bindings for elements in the input lists
- Predicates on the elements
- An output form that produces output from the elements of the input lists that satisfy the predicates

Of course, Clojure generalizes the notion of list comprehension to *sequence* comprehension. Clojure comprehensions use the `for` macro. Note that the list comprehension `for` has nothing to do with the `for` loop found in imperative languages.

```
(for [binding-form coll-expr filter-expr? ...] expr)
```

`for` takes a vector of binding-form/coll-exprs, plus optional filter-exprs, and then yields a sequence of exprs.

List comprehension is more general than functions such as `map` and `filter` and can, in fact, emulate most of the filtering and transformation functions described earlier.

You can rewrite the previous `map` example as a list comprehension:

```
(for [word ["the" "quick" "brown" "fox"]]
  (format "<p>%s</p>" word))
-> ("<p>the</p>" "<p>quick</p>" "<p>brown</p>" "<p>fox</p>")
```

This reads almost like English: “For [each] word in [a sequence of words], format [according to format instructions].”

Comprehensions can emulate `filter` using a `:when` clause. You can pass `even?` to `:when` to filter the even numbers:

```
(take 10 (for [n whole-numbers :when (even? n)] n))
-> (2 4 6 8 10 12 14 16 18 20)
```

A `:while` clause continues the evaluation only while its expression holds true:

```
(for [n whole-numbers :while (even? n)] n)
-> ()
```

The real power of `for` comes when you work with more than one binding expression. For example, you can express all possible positions on a chess-board in algebraic notation by binding both rank and file:

```
(for [file "ABCDEFGH"
      rank (range 1 9)]
  (format "%c%d" file rank))
-> ("A1" "A2" ... elided ... "H7" "H8")
```

Clojure iterates over the rightmost binding expression in a sequence comprehension first, and then works its way left. Because `rank` is listed to the right of `file` in the binding form, `rank` iterates faster. If you want files to iterate faster, you can reverse the binding order and list `rank` first:

```
(for [rank (range 1 9)
      file "ABCDEFGH"]
  (format "%c%d" file rank))
-> ("A1" "B1" ... elided ... "G8" "H8")
```

In many languages, transformations, filters, and comprehensions do their work immediately. Do not assume this in Clojure. Most sequence functions do not traverse elements until you actually try to use them.

```
(run! proc coll)
```

The `run!` function runs a `proc` (a function that takes a single input) for side effects on every item in a collection via `reduce`. `nil` is returned.

```
(run! println (range 3))
| 0
| 1
| 2
-> nil
```

In this example, the `println` function is run for every item in the range, printing to the console.

Threading Functions

It's common in Clojure to combine a series of sequence functions together as a pipeline:

```
(reduce + (map #(* % %) (filter odd? (range 100))))
-> 166650
```

This works great, but is difficult to read because data flows from the right (the range) to the reduce at the left. Clojure provides the *thread-last* macro `->>` to rewrite that expression so it flows cleanly from left to right instead:

```
(->> (range 100) (filter odd?) (map #(* % %)) (reduce +))
```

The reason `->>` is called *thread-last* is that the result from the first expression (the range) is threaded into the last argument of the next expression, the filter, and so on down the line. All of the sequence functions take the source sequence as the last argument. The `->>` macro merely rewrites this expression into the original expression, a fun preview of what macros can do (see [Chapter 11, Macros, on page 239](#)).

In contrast, all of the collection functions that operate on lists, vectors, maps, and sets take the collection as the first argument, and Clojure also provides a *thread-first* macro `->` that works well with pipelines of collection functions.

Lazy and Infinite Sequences

Most Clojure sequences are *lazy*; in other words, elements are not calculated until they're needed. Using lazy sequences has many benefits:

- You can postpone expensive computations that may not, in fact, be needed.
- You can work with huge data sets that don't fit into memory.
- You can delay I/O until it's absolutely needed.
- You can decouple the generation and consumption of values.

Consider the code and following expression that produces (mostly) prime numbers using wheel factorization:¹

```
src/examples/primes.clj
(ns examples.primes)
;; Taken from clojure.contrib.lazy-seqs
;; primes cannot be written efficiently as a function, because
;; it needs to look back on the whole sequence. contrast with
;; fibs and powers-of-2 which only need a fixed buffer of 1 or 2
;; previous values.
(def primes
  (concat
    [2 3 5 7]
    (lazy-seq
      (let [primes-from
            (fn primes-from [n [f & r]]
              (if (some #(zero? (rem n %))
```

1. https://en.wikipedia.org/wiki/Wheel_factorization

```

        (take-while #(<= (* % %) n) primes))
      (recur (+ n f) r)
      (lazy-seq (cons n (primes-from (+ n f) r))))))
wheel (cycle [2 4 2 4 6 2 6 4 2 4 6 6 2 6 4 2
              6 4 6 8 4 2 4 2 4 8 6 4 6 2 4 6
              2 6 6 4 2 4 6 2 6 4 2 4 2 10 2 10]))
(primes-from 11 wheel))))

(require '[examples.primes :refer :all])
(def ordinals-and-primes (map vector (iterate inc 1) primes))
-> #'user/ordinals-and-primes

```

ordinals-and-primes includes pairs like [5, 11] (11 is the fifth prime number). Both ordinals and primes are infinite, but ordinals-and-primes fits into memory just fine, because it's lazy. Just take what you need from it:

```

(take 5 (drop 1000 ordinals-and-primes))
-> ([1001 7927] [1002 7933] [1003 7937] [1004 7949] [1005 7951])

```

When should you prefer lazy sequences? Most of the time. Most sequence functions return lazy sequences, so you “pay” only for what you use. More importantly, lazy sequences do not require any special effort on your part. In the previous example, `iterate`, `primes`, and `map` return lazy sequences, so `ordinals-and-primes` gets laziness “for free.”

Lazy sequences are critical to functional programming in Clojure. [How to Be Lazy, on page 101](#) explores creating and using lazy sequences in much greater detail. Additionally, [Eager Transformations, on page 122](#) talks about those cases when you should prefer non-lazy approaches.

When you're viewing a large sequence from the REPL, you may want to use `take` to prevent the REPL from evaluating the entire sequence. In other contexts, you may have the opposite problem. You've created a lazy sequence, and you want to force the sequence to evaluate fully. The problem usually arises when the code generating the sequence has side effects. Consider the following sequence, which embeds side effects via `println`:

```

(def x (for [i (range 1 3)] (do (println i) i)))
-> #'user/x

```

Newcomers to Clojure are surprised that the previous code prints nothing. Since the definition of `x` doesn't actually use the elements, Clojure does not evaluate the comprehension to get them. You can force evaluation with `doall`.


```
(doall coll)
```

`doall` forces Clojure to walk the elements of a sequence and returns the elements as a result:

```
(doall x)
| 1
| 2
-> (1 2)
```

You can also use `dorun`:

```
(dorun coll)
```

`dorun` walks the elements of a sequence without keeping past elements in memory. As a result, `dorun` can walk collections too large to fit in memory.

```
(def x (for [i (range 1 3)] (do (println i) i)))
-> #'user/x

(dorun x)
| 1
| 2
-> nil
```

The `nil` return value is a telltale reminder that `dorun` does not hold a reference to the entire sequence. Most Clojure code either avoids side effects or expects them to happen lazily. Use `dorun` and `doall` when you are explicitly trying to force side effects to happen.

Clojure Makes Java Seq-able

The `seq` abstraction of `first/rest` applies to anything that there can be more than one of. In the Java world, that includes the following:

- The Collections API
- Regular expressions
- File system traversal
- XML processing
- Relational database results

Clojure wraps these Java APIs, making the sequence functions available for almost everything you do.

Seq-ing Java Collections

If you try to apply the sequence functions to Java collections, you'll find that they behave as sequences. Collections that can act as sequences are called *seq-able*. For example, arrays are seq-able:

```
; String.getBytes returns a byte array
(first (.getBytes "hello"))
-> 104

(rest (.getBytes "hello"))
-> (101 108 108 111)

(cons (int \h) (.getBytes "ello"))
-> (104 101 108 108 111)
```

Hashtables and Maps are also seq-able:

```
; System.getProperties returns a Hashtable
(first (System/getProperties))
-> #object[java.util.Hashtable$Entry 0x12468a38
    "java.runtime.name=Java(TM) SE Runtime Environment"]

(rest (System/getProperties))
-> (#object[java.util.Hashtable$Entry 0x5b239d7d
    "sun.boot.library.path=/Library/... etc. ...
```

The helpful predicate function `seqable?` can be used to check whether something can produce a sequence.

Remember that sequences are immutable, even when the underlying Java collection is mutable. So, you can't update the system properties by consing a new item onto `(System/getProperties)`. `cons` will return a new sequence; the existing properties are unchanged. Similarly, you cannot see updated system properties if your sequence already walked over them.

Since strings are sequences of characters, they are also seq-able:

```
(first "Hello")
-> \H

(rest "Hello")
-> (\e \l \l \o)

(cons \H "ello")
-> (\H \e \l \l \o)
```

Clojure will automatically obtain a sequence from a collection, but it won't automatically convert a sequence back to the original collection type. With most collection types, this behavior is intuitive, but with strings, you'll often want to convert the result to a string. Consider reversing a string. Clojure provides `reverse`.

```
; probably not what you want
(reverse "hello")
-> (l o l l l e l h)
```

To convert a sequence back to a string, use (apply str seq):

```
(apply str (reverse "hello"))
-> "olleh"
```

The Java collections are seq-able, but for most scenarios, they don't offer advantages over Clojure's built-in collections. Prefer the Java collections only in interop scenarios where you're working with legacy Java APIs.

Seq-ing Regular Expressions

Clojure's regular expressions use the `java.util.regex` package under the hood. At the lowest level, this exposes the mutable nature of Java's `Matcher`. You can use `re-matcher` to create a `Matcher` for a regular expression and a string and then loop on `re-find` to iterate over the matches.

```
(re-matcher regexp string)
```

```
src/examples/sequences.clj
```

```
; don't do this!
```

```
(let [m (re-matcher #"|w+" "the quick brown fox")]
  (loop [match (re-find m)]
    (when match
      (println match)
      (recur (re-find m)))))
```

```
| the
| quick
| brown
| fox
-> nil
```

A much better option is to use the higher-level `re-seq`.

```
(re-seq regexp string)
```

`re-seq` exposes an immutable seq over the matches. This gives you the power of all of Clojure's sequence functions. Try these expressions at the REPL:

```
(re-seq #"|w+" "the quick brown fox")
-> ("the" "quick" "brown" "fox")

(sort (re-seq #"|w+" "the quick brown fox"))
-> ("brown" "fox" "quick" "the")

(drop 2 (re-seq #"|w+" "the quick brown fox"))
-> ("brown" "fox")
```

```
(map clojure.string/upper-case (re-seq #"\\w+" "the quick brown fox"))
-> ("THE" "QUICK" "BROWN" "FOX")
```

re-seq is an example of how good abstractions reduce code bloat. Regular expression matches are not a special thing requiring special methods to deal with them. They are sequences, just like everything else. Thanks to the number of sequence functions, you get more functionality “for free” than you would likely end up with after a misguided foray into writing regexp-specific functions.

Seq-ing the File System

You can seq over the file system. For starters, you can call java.io.File directly:

```
(import java.io.File)
(seq (.listFiles (File. ".")))
-> #object["Ljava.io.File;" 0x5a7005d "Ljava.io.File;@5a7005d"]
```

The return value is an array of java.io.File objects. A better way to see the results is to map them to a string form with the File.getName() method:

```
(defn file-names [files]
  (map File/.getName files))
-> #'user/file-names

(file-names (.listFiles (File. ".")))
-> ("src" "data" "test" ...)
```

Often, you want to recursively traverse the entire directory tree. Clojure provides a depth-first walk via file-seq. If you file-seq from the sample code directory for this book, you will see a lot of files:

```
(count (file-seq (File. ".")))
-> 169
```

What if you want to see only the files that have been changed recently? Write a predicate recently-modified? that checks to see whether a file was touched in the last half hour:

```
src/examples/sequences.clj
```

```
(defn minutes-to-millis [mins] (* mins 1000 60))

(defn recently-modified? [file]
  (> (.lastModified file)
    (- (System/currentTimeMillis) (minutes-to-millis 30))))
```

Give it a try, combining the recently-modified? filter predicate with the file-names function we defined earlier. (Note that your results will vary from those shown here.)

```
(file-names (filter recently-modified? (file-seq (File. "."))))
-> ("." "src" "test" ...)
```

Seq-ing a Character Stream

In Java, a Reader provides a stream of characters. You can seq over the lines of any Java Reader using line-seq. To get a Reader, you can always use Clojure's clojure.java.io namespace. The clojure.java.io namespace provides a reader function that returns a reader on a stream, file, URL, or URI.

```
(require '[clojure.java.io :refer [reader]])
; leaves reader open...
(take 2 (line-seq (reader "src/examples/utils.clj")))
-> ("(ns examples.utils"
    "  (:import [java.io BufferedReader InputStreamReader]))")
```

Since readers can represent non-memory resources that need to be closed, you should wrap reader creation in a with-open. Create an expression that uses the sequence function count, to count the number of lines in a file, and uses with-open to correctly close the reader when the body is complete:

```
(with-open [rdr (reader "src/examples/utils.clj")]
  (count (line-seq rdr)))
-> 64
```

To make the example more useful, add a filter to count only non-blank lines:

```
(with-open [rdr (reader "src/examples/utils.clj")]
  (count (filter #(re-find #"\S" %) (line-seq rdr))))
-> 55
```

Using seqs both on the file system and on the contents of individual files, you can quickly create interesting utilities. Create a program that defines these three predicates:

- non-blank? detects non-blank lines.
- hidden? detects metadata files to ignore.
- clojure-source? detects Clojure source code files.

Then, create a clojure-loc function that counts the lines of Clojure code in a directory tree, using a combination of sequence functions along the way: reduce, for, count, and filter.

```
src/examples/sequences.clj
(require '[clojure.java.io :refer [reader]])
(require '[clojure.string :refer [blank? starts-with? ends-with?]])

(defn non-blank? [line] (not (blank? line)))

(defn hidden? [file] (starts-with? (str file) "."))

(defn clojure-source? [file] (ends-with? (str file) ".clj"))
```

```
(defn clojure-loc [base-file]
  (reduce
    +
    (for [file (file-seq base-file)]
      :when (and (not (hidden? file)) (clojure-source? file)))
    (with-open [rdr (reader file)]
      (count (filter non-blank? (line-seq rdr)))))))
```

Now, let's use `clojure-loc` to find out how much Clojure code is in Clojure² itself:

```
(clojure-loc (java.io.File. "clojure/src"))
-> 19923
```

The `clojure-loc` function is very task-specific, but because it's built out of sequence functions and simple predicates, you can easily tweak it to very different tasks.

Calling Structure-Specific Functions

Clojure's sequence functions allow you to write very general code. Sometimes you'll want to be more specific and take advantage of the characteristics of a specific data structure. Clojure includes functions that specifically target lists, vectors, maps, and sets.

We'll take a quick tour of some of these structure-specific functions next. For a complete list of structure-specific functions in Clojure, see the Data Structures section of the Clojure website.³

Functions on Lists

Clojure supports the traditional names `peek` and `pop` for retrieving the first element of a list and the remainder, respectively:

```
(peek coll)
(pop coll)
```

Give a simple list a peek and pop:

```
(peek '(1 2 3))
-> 1

(pop '(1 2 3))
-> (2 3)
```

2. <https://github.com/clojure/clojure>

3. https://clojure.org/reference/data_structures

peek is the same as first, but pop is *not* the same as rest. pop will throw an exception if the sequence is empty:

```
(rest ())
-> ()

(pop ())
| Execution error (IllegalStateException) at user/eval149 (REPL:1).
| Can't pop empty list
```

Functions on Vectors

Vectors also support peek and pop, but they work with the element at the end of the vector:

```
(peek [1 2 3])
-> 3

(pop [1 2 3])
-> [1 2]
```

get returns the value at an index or returns nil if the index is outside the vector:

```
(get [:a :b :c] 1)
-> :b

(get [:a :b :c] 5)
-> nil
```

Vectors are themselves functions. They take an index argument and return a value, or they throw an exception if the index is out of bounds:

```
([:a :b :c] 1)
-> :b

(:a :b :c] 5)
| Execution error (IndexOutOfBoundsException) at user/eval1 (REPL:1).
```

assoc associates a new value with a particular index:

```
(assoc [0 1 2 3 4] 2 :two)
-> [0 1 :two 3 4]
```

subvec returns a partial segment of a vector:

```
(subvec avec start end?)
```

If end is not specified, it defaults to the end of the vector:

```
(subvec [1 2 3 4 5] 3)
-> [4 5]

(subvec [1 2 3 4 5] 1 3)
-> [2 3]
```

You could simulate subvec with a combination of drop and take:

```
(take 2 (drop 1 [1 2 3 4 5]))
-> (2 3)
```

The difference is that take and drop are general and can work with any sequence. On the other hand, subvec is *much* faster for vectors because it creates a view into the original vector rather than constructing a new vector. Whenever a structure-specific function like subvec duplicates functionality already available in the sequence functions, it's probably there for performance.

Functions on Maps

Clojure provides several functions for reading the keys and values in a map. keys returns a sequence of the keys, and vals returns a sequence of the values:

```
(keys map)
(vals map)
```

Try taking keys and values from a simple map:

```
(keys {:sundance "spaniel", :darwin "beagle"})
-> (:sundance :darwin)

(vals {:sundance "spaniel", :darwin "beagle"})
-> ("spaniel" "beagle")
```

Note that while maps are unordered, both keys and vals are guaranteed to return the keys and values in the same order as a seq on the map.

get returns the value for a key or returns nil.

```
(get map key value-if-not-found?)
```

Test in the REPL that get behaves as expected for present and missing keys:

```
(get {:sundance "spaniel", :darwin "beagle"} :darwin)
-> "beagle"

(get {:sundance "spaniel", :darwin "beagle"} :snoopy)
-> nil
```

There's an approach that's even simpler than get. Maps are functions of their keys. So, you can leave out the get entirely, putting the map in function position at the beginning of a form:

```
({:sundance "spaniel", :darwin "beagle"} :darwin)
-> "beagle"

({:sundance "spaniel", :darwin "beagle"} :snoopy)
-> nil
```


Keywords are also functions. They take a collection as an argument and look themselves up in the collection. Since `:darwin` and `:sundance` are keywords, the earlier forms can be written with their elements in reverse order.

```
(:darwin {:sundance "spaniel", :darwin "beagle"} )
-> "beagle"

(:snoopy {:sundance "spaniel", :darwin "beagle"} )
-> nil
```

If you look up a key in a map and get `nil` back, you can't tell whether the key was missing from the map or present with a value of `nil`. The `contains?` function solves this problem by testing for the mere presence of a key.

```
(contains? map key)
```

Create a map where `nil` is a legal value:

```
(def score {:stu nil :joey 100})
```

`:stu` is present, but if you see the `nil` value, you might not think so:

```
(:stu score)
-> nil
```

If you use `contains?`, you can verify that `:stu` is in the game, although presumably not doing very well:

```
(contains? score :stu)
-> true
```

Another approach is to call `get`, passing in an optional third argument that will be returned if the key is not found:

```
(get score :stu :score-not-found)
-> nil

(get score :aaron :score-not-found)
-> :score-not-found
```

The default return value of `:score-not-found` makes it possible to distinguish that `:aaron` is not in the map, while `:stu` is present with a value of `nil`.

If `nil` is a legal value in map, use `contains?` or the three-argument form of `get` to test the presence of a key.

Clojure also provides several functions for building new maps:

- `assoc` returns a map with a key/value pair added.
- `dissoc` returns a map with a key removed.
- `select-keys` returns a map, keeping only a specified set of keys.

- `update-keys` returns a map with each key modified according to a function.
- `update-vals` returns a map with each value modified according to a function.
- `merge` combines maps. If multiple maps contain a key, the rightmost wins.
- `zipmap` constructs a new map from a sequence of keys and a sequence of values, stopping when one of them runs out.

To test these functions, create some song data:

```
src/examples/sequences.clj
```

```
(def song {:name "Agnus Dei"
           :artist "Krzysztof Penderecki"
           :album "Polish Requiem"
           :genre "Classical"})
```

Next, create various modified versions of the song collection:

```
(assoc song :kind "MPEG Audio File")
-> {:name "Agnus Dei", :album "Polish Requiem",
   :kind "MPEG Audio File", :genre "Classical",
   :artist "Krzysztof Penderecki"}

(dissoc song :genre)
-> {:name "Agnus Dei", :album "Polish Requiem",
   :artist "Krzysztof Penderecki"}

(select-keys song [:name :artist])
-> {:name "Agnus Dei", :artist "Krzysztof Penderecki"}

(merge song {:size 8118166, :time 507245})
-> {:name "Agnus Dei", :album "Polish Requiem",
   :genre "Classical", :size 8118166,
   :artist "Krzysztof Penderecki", :time 507245}
```

Remember that song itself never changes. Each of these functions returns a new collection.

The most interesting map construction function is `merge-with`.

```
(merge-with merge-fn & maps)
```

`merge-with` is like `merge`, except that when two or more maps have the same key, you can specify your own function for combining the values under the key. Use `merge-with` and `concat` to build a sequence of values under each key:

```
(merge-with
 concat
 {:rubble ["Barney"], :flintstone ["Fred"]}
 {:rubble ["Betty"], :flintstone ["Wilma"]}
 {:rubble ["Bam-Bam"], :flintstone ["Pebbles"]})
-> {:rubble ("Barney" "Betty" "Bam-Bam"),
   :flintstone ("Fred" "Wilma" "Pebbles")}
```

Starting with three distinct collections of family members keyed by last name, the previous code combines them into one collection keyed by last name.

Functions on Sets

In addition to the set functions in the `clojure.core` namespace, Clojure provides a group of functions in the `clojure.set` namespace. To use these functions with unqualified names, call `(require '[clojure.set :refer :all])` from the REPL. For the following examples, you'll also need the following vars:

```
src/examples/sequences.clj
(def languages #{"java" "c" "d" "clojure"})
(def beverages #{"java" "chai" "pop"})
```

The first group of `clojure.set` functions performs operations from set theory:

- `union` returns the set of all elements present in either input set.
- `intersection` returns the set of all elements present in *both* input sets.
- `difference` returns the set of all elements present in the first input set minus those in the second.
- `select` returns the set of all elements matching a predicate.

Write an expression that finds the union of all languages and beverages:

```
(union languages beverages)
-> #{ "java" "c" "d" "clojure" "chai" "pop" }
```

Next, try the languages that are not also beverages:

```
(difference languages beverages)
-> #{ "c" "d" "clojure" }
```

If you enjoy terrible puns, you'll like the fact that some things are both languages *and* beverages:

```
(intersection languages beverages)
-> #{ "java" }
```

A number of languages can't afford a name larger than a single character:

```
(select #(= 1 (count %)) languages)
-> #{ "c" "d" }
```

Set union and set difference are part of set theory, but they're also part of *relational algebra*, which is the basis for query languages such as SQL. Relational algebra consists of six primitive operators: set union and set difference (described earlier), plus rename, selection, projection, and cross product.

Clojure sets (and maps) have everything we need to implement a basic relational algebra system. We'll use maps to describe each tuple (like a row in a relational database) and a set to contain all of the tuples in a relation (like a table in a relational database).

The following examples work against an in-memory database of musical compositions. First, we'll define the data in our "database," which are just sets of maps:

```
src/examples/sequences.clj
```

```
(def compositions
  #{{:name "The Art of the Fugue" :composer "J. S. Bach"}
    {:name "Musical Offering" :composer "J. S. Bach"}
    {:name "Requiem" :composer "Giuseppe Verdi"}
    {:name "Requiem" :composer "W. A. Mozart"}})

(def composers
  #{{:composer "J. S. Bach" :country "Germany"}
    {:composer "W. A. Mozart" :country "Austria"}
    {:composer "Giuseppe Verdi" :country "Italy"}})

(def nations
  #{{:nation "Germany" :language "German"}
    {:nation "Austria" :language "German"}
    {:nation "Italy" :language "Italian"}})
```

The `rename` function renames keys (*database columns*) based on a map from original names to new names.

```
(rename relation rename-map)
```

Rename the compositions to use a title key instead of name:

```
(rename compositions {:name :title})
-> #{{:title "Requiem", :composer "Giuseppe Verdi"}
     {:title "Musical Offering", :composer "J.S. Bach"}
     {:title "Requiem", :composer "W. A. Mozart"}
     {:title "The Art of the Fugue", :composer "J.S. Bach"}}
```

The `select` function returns maps, for which a predicate is true, and is analogous to the WHERE portion of a SQL SELECT:

```
(select pred relation)
```

Write a select expression that finds all of the compositions whose title is "Requiem":

```
(select #(= (:name %) "Requiem") compositions)
-> #{{:name "Requiem", :composer "W. A. Mozart"}
     {:name "Requiem", :composer "Giuseppe Verdi"}}
```

The project function returns only the parts of maps that match a set of keys.

```
(project relation keys)
```

project is similar to a SQL SELECT that specifies a subset of columns. Write a projection that returns only the name of the compositions:

```
(project compositions [:name])
-> #{{:name "Musical Offering"}
    {:name "Requiem"}
    {:name "The Art of the Fugue"}}
```

The final relational primitive, which is a cross product, is the foundation for the various kinds of joins in relational databases. The cross product returns every possible combination of rows in the different tables. You can do this easily enough in Clojure with a list comprehension:

```
(for [m compositions c composers] (concat m c))
-> ... 4 x 3 = 12 rows ...
```

Although the cross product is theoretically interesting, you'll typically want some subset of the full cross product. For example, you might want to join sets based on shared keys:

```
(join relation-1 relation-2 keymap?)
```

You can join the composition names and composers on the shared key :composer:

```
(join compositions composers)
-> #{{:name "Requiem", :country "Austria",
    :composer "W. A. Mozart"}
    {:name "Musical Offering", :country "Germany",
    :composer "J. S. Bach"}
    {:name "Requiem", :country "Italy",
    :composer "Giuseppe Verdi"}
    {:name "The Art of the Fugue", :country "Germany",
    :composer "J. S. Bach"}}
```

If the key names in the two relations don't match, you can pass a keymap that maps the key names in relation-1 to their corresponding keys in relation-2. For example, you can join composers, which uses :country, to nations, which uses :nation.

```
(join composers nations {:country :nation})
-> #{{:language "German", :nation "Austria",
    :composer "W. A. Mozart", :country "Austria"}
    {:language "German", :nation "Germany",
    :composer "J. S. Bach", :country "Germany"}
    {:language "Italian", :nation "Italy",
    :composer "Giuseppe Verdi", :country "Italy"}}
```

You can combine the relational primitives. Perhaps you want to know the set of all countries that are home to the composer of a requiem. You can use `select` to find all of the requiems, join them with their composers, and `project` to narrow the results to just the country names:

```
(project
  (join
    (select #(= (:name %) "Requiem") compositions)
    composers)
  [:country])
-> #{{:country "Italy"} {:country "Austria"}}
```

The analogy between Clojure's relational algebra and a relational database is instructive. Remember, though, that Clojure's relational algebra is a general-purpose tool. You can use it on any kind of set-relational data. And while you're using it, you have the entire power of Clojure and Java at your disposal.

Wrapping Up

Clojure unifies all kinds of collections under a single abstraction, the sequence. Clojure also provides a comprehensive set of functions for working with data in the form of sequences. In combination, they provide a great deal of reuse across every Clojure application and library.

Clojure's sequences are implemented using functional programming techniques: immutable data, recursive definition, and lazy realization. In the next chapter, you'll see how to use these techniques directly, further expanding the power of Clojure.

Functional Programming

Functional programming (FP) is a big topic, not to be learned in 21 days¹ or in a single chapter of a book. Nevertheless, you can reach a first level of effectiveness using lazy and recursive techniques in Clojure fairly quickly, and that is what we'll accomplish in this chapter.

We'll start with a quick overview of FP terms and concepts and an introduction to the guidelines of Clojure FP that we'll refer to throughout the chapter. Next, we'll experience the power of lazy sequences by working through a series of implementations of the Fibonacci numbers. As cool as lazy sequences are, you rarely need to construct them yourself, and we'll see better ways to recast problems to solve them directly with the sequence functions.

We'll close with some advanced techniques and see some scenarios where eager transformations have advantages over lazy sequences.

Functional Programming Concepts

Functional programming leads to code that is easier to write, read, test, and reuse. Here's how it works.

Pure Functions

Programs are built out of *pure functions*. A pure function has no *side effects*; that is, it doesn't depend on anything but its arguments, and its only influence on the outside world is through its return value.

Mathematical functions are pure functions. Two plus two is four, no matter where or when you ask. Also, asking doesn't *do* anything other than return the answer.

1. <https://norvig.com/21-days.html>

Program output is decidedly *impure*. For example, when you `println`, you change the outside world by pushing data onto an output stream. Also, the results of `println` depend on state outside the function; the standard output stream might be redirected, closed, or broken.

If you start writing pure functions, you'll quickly realize that pure functions and *immutable* data go hand in hand. Consider the following mystery function:

```
(defn mystery [input]
  (if input data-1 data-2))
```

If `mystery` is a pure function, then regardless of what it does, `data-1` and `data-2` have to be immutable! Otherwise, changes to the data would cause the function to return different values for the same input.

A single piece of mutable data can ruin the game, rendering an entire call chain of functions impure. So, once you make a commitment to writing pure functions, you end up using immutable data in large sections of your application.

Persistent Data Structures

Immutable data is critical to Clojure's approach to both FP and state. On the FP side, pure functions cannot have side effects, such as updating the state of a mutable object. On the state side, Clojure's reference types require immutable data structures to implement their concurrency guarantees.

The fly in the ointment is performance. When all data is immutable, "update" translates into "create a copy of the original data, plus my changes." This will use up memory quickly! Imagine that you have an address book that takes up 5 MB of memory. Then, you make five small updates. With a mutable address book, you are still consuming about 5 MB of memory. But if you have to copy the whole address book for each update, then an immutable version would balloon to 25 MB!

Clojure's data structures don't take this naive "copy everything" approach. Instead, all Clojure data structures are *persistent*. In this context, persistent means that the data structures preserve old copies of themselves by efficiently *sharing structure* between older and newer versions.

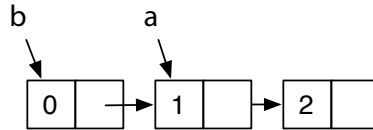
Structural sharing is easiest to visualize with a list. Consider list `a` with two elements:

```
(def a '(1 2))
-> #'user/a
```


Then, from `a`, you can create a `b` with an additional element added:

```
(def b (cons 0 a))  
-> #'user/b
```

`b` can reuse all of `a`'s structure, rather than have its own private copy:



All of Clojure's data structures share structure where possible. For structures other than simple lists, the mechanics are more complex, of course. If you're interested in the details, check out the following articles:

- “Ideal Hash Trees”² by Phil Bagwell
- “Understanding Clojure's Persistent Vectors”³ by Jean Niklas L'orange

Laziness and Recursion

Functional programs make heavy use of *recursion* and *laziness*. A recursion occurs when a function calls itself, either directly or indirectly. With laziness, an expression's evaluation is postponed until it's actually needed. Evaluating a lazy expression is called *realizing* the expression.

In Clojure, functions and expressions are not lazy. However, sequences *are* generally lazy. Because so much Clojure programming is sequence manipulation, you get many of the benefits of a fully lazy language. In particular, you can build complex expressions using lazy sequences and then “pay” only for the elements you actually need.

Lazy techniques imply pure functions. You never have to worry about when to call a pure function, since it always returns the same thing. Impure functions, on the other hand, do not play well with lazy techniques. As a programmer, you must explicitly control when an impure function is called, because if you call it at some other time, it may behave differently!

Referential Transparency

Laziness depends on the ability to replace a function call with its result at any time. Functions that have this ability are called *referentially transparent*, because calls to such functions can be replaced without affecting the behavior

2. <https://lampwww.epfl.ch/papers/idealhashtrees.pdf>

3. <https://hypirion.com/musings/understanding-persistent-vector-pt-1>

of the program. In addition to laziness, referentially transparent functions can also benefit from the following:

- *Memoization*: automatic caching of results
- Automatic *parallelization*: moving function evaluation to another processor or machine

Pure functions are referentially transparent *by definition*. Most other functions are *not* referentially transparent, and those that are must be proven safe by code review.

Benefits of FP

Well, that is a lot of terminology, and we promised it would make your code easier to write, read, test, and compose. Here's how.

You'll find functional code easier to *write* because the relevant information is right in front of you, in a function's argument list. You don't have to worry about global scope, session scope, application scope, or thread scope. Functional code is easier to *read* for exactly the same reason.

Code that is easier to read and write is going to be easier to test, but functional code brings an additional benefit for testing. As projects get large, it often takes a lot of effort to set up the right environment to execute a test. This is much less of a problem with functional code, because there *is no relevant environment* beyond the function's arguments.

Functional code improves *reuse*. To reuse code, you must be able to:

- Find and understand a piece of useful code.
- Compose the reusable code with other code.

The readability of functional code helps you find and understand the functions you need, but the benefit for *composing* code is even more compelling.

Composability is a hard problem. For years, programmers have used *encapsulation* to try to create composable code. Encapsulation creates a firewall, providing access to data only through a public API.

Encapsulation helps, but it's nowhere near enough. Even with encapsulated objects, there are far too many surprising interactions when you try to compose entire systems. The problem is those darn side effects. *Impure functions* violate encapsulation because they let the outside world reach in (invisibly!) and change the behavior of your code. Pure functions, on the other hand, are truly encapsulated and composable. Put them anywhere you want in a system, and they will always behave in the same way.

Guidelines for Use

Clojure takes a unique approach to FP that strikes a balance between academic purity and the reality of running well on the JVM. That means there's a lot to learn all at once. But fear not. If you're new to FP, the following guidelines will help on your initial steps toward FP mastery, Clojure-style:

1. Avoid direct recursion. The JVM can't optimize recursive calls (either nested or self calls), and Clojure programs that recurse may overflow their stack.
2. Use `recur` when you're producing scalar values or small, fixed sequences. Clojure *will* optimize calls that use an explicit `recur`.
3. When producing large or variable-sized sequences, always be lazy. (Do *not* `recur`.) Then, your callers can consume just the part of the sequence they actually need.
4. Be careful not to realize more of a lazy sequence than you need.
5. Know the sequence functions. You can often write code without using `recur` or the lazy APIs at all.
6. Subdivide. Divide even simple-seeming problems into smaller pieces, and you'll often find solutions in the sequence functions that lead to more general, reusable code.
7. If the size of the data or the number of transformations is large, consider using transducers to reduce memory usage and improve performance.

Now, let's get started writing functional code.

How to Be Lazy

Before we get to laziness, we first need to delve into recursion as an approach to enumerating sequences of values.

Functional programs make great use of *recursive definitions*. A recursive definition consists of two parts:

- A *basis*, which explicitly enumerates some members of the sequence
- An *induction*, which provides rules for combining members of the sequence to produce additional members

Our challenge in this section is converting a recursive definition into working code. You might do this in several ways:

- A simple recursion, using a function that calls itself in some way to implement the induction step
- A tail recursion, using a function that calls itself only at the tail end of its execution. Tail recursion enables an important optimization.
- A lazy sequence that eliminates actual recursion and calculates a value later, when it's needed

Choosing the right approach is important. Implementing a recursive definition poorly can lead to code that either performs terribly, consumes all available stack and fails, consumes all available heap and fails, or does all of these. In Clojure, being lazy is often the right approach.

We'll explore all of these approaches by applying them to the Fibonacci numbers. Named for the Italian mathematician Leonardo (Fibonacci) of Pisa (c.1170–c.1250), the Fibonacci numbers were actually known to Indian mathematicians as far back as 200 BC. The Fibonacci numbers have many interesting properties, and they crop up again and again in algorithms, data structures, and even biology.⁴ The Fibonacci numbers have a very simple recursive definition:

- Basis: F_0 , the zeroth Fibonacci number, is zero. F_1 , the first Fibonacci number, is one.
- Induction: For $n > 1$, F_n equals $F_{n-1} + F_{n-2}$.

Using this definition, the first 10 Fibonacci numbers are as follows:

(0 1 1 2 3 5 8 13 21 34)

Let's begin by implementing the Fibonacci numbers using a simple recursion. The following Clojure function will return the n th Fibonacci number:

src/examples/functional.clj

```
Line 1 ; bad idea
2 (defn stack-consuming-fibo [n]
3   (cond
4     (= n 0) 0
5     (= n 1) 1
6     :else (+ (stack-consuming-fibo (- n 1))
7              (stack-consuming-fibo (- n 2)))))
```

4. https://en.wikipedia.org/wiki/Fibonacci_number

Lines 4 and 5 define the basis, and line 6 defines the induction. The implementation is recursive because `stack-consuming-fibo` calls itself on lines 6 and 7.

Test that `stack-consuming-fibo` works correctly for small values of n :

```
(stack-consuming-fibo 9)
-> 34
```

Good so far, but there's a problem calculating larger Fibonacci numbers such as $F_{1000000}$:

```
(stack-consuming-fibo 1000000)
-> Execution error (StackOverflowError) at user/stack-consuming-fibo (REPL:1).
```

Because of the recursion, each call to `stack-consuming-fibo` for $n > 1$ begets two more calls to `stack-consuming-fibo`. At the JVM level, these calls are translated into method calls, each of which allocates a data structure called a *stack frame*.

The `stack-consuming-fibo` creates a depth of stack frames proportional to n , which quickly exhausts the JVM stack and causes the `StackOverflowError` shown earlier. (It also creates a total number of stack frames that's exponential in n , so its performance is poor even when the stack does not overflow.)

Clojure function calls are designated as *stack-consuming* because they allocate stack frames that use up stack space. In Clojure, you should almost always avoid stack-consuming recursion as shown in `stack-consuming-fibo`.

Tail Recursion

Functional programs can solve the stack-usage problem with *tail recursion*. A tail-recursive function is still defined recursively, but the recursion must come at the tail, that is, at an expression that's a return value of the function. Languages can then perform *tail-call optimization* (TCO), converting tail recursions into iterations that don't consume the stack.

The `stack-consuming-fibo` definition of Fibonacci is not tail recursive, because it calls `add (+)` *after* both calls to `stack-consuming-fibo`. To make `fibo` tail recursive, you must create a function whose arguments carry enough information to move the induction forward, without any extra “after” work (like an addition) that would push the recursion out of the tail position. For `fibo`, such a function needs to know two Fibonacci numbers, plus an ordinal n that can count down to zero as new Fibonacci numbers are calculated. You can write `tail-fibo`, shown in the following:

```

src/examples/functional.clj
Line 1 (defn tail-fibo [n]
2     (letfn [(fib
3               [current next n]
4               (if (zero? n)
5                   current
6                   (fib next (+ current next) (dec n))))]
7     (fib 0N 1N n))

```

Line 2 introduces the `letfn` macro:

```
(letfn fnspecs & body) ; fnspecs ==> [(fname [params*] exprs)+]
```

`letfn` is like `let` but is dedicated to creating local functions. Each function declared in a `letfn` can call itself or any other function in the same `letfn` block. Line 3 declares that `fib` has three arguments: the current Fibonacci, the next Fibonacci, and the number `n` of steps remaining.

Line 5 returns `current` when there are no steps remaining, and line 6 continues the calculation, decrementing the remaining steps by one. Finally, line 7 kicks off the recursion with the basis values 0 and 1, plus the ordinal `n` of the Fibonacci we're looking for.

`tail-fibo` works for small values of `n`:

```
(tail-fibo 9)
-> 34N
```

But although it's tail recursive, it still fails for large `n`:

```
(tail-fibo 1000000)
-> Execution error (StackOverflowError) at java.math.BigInteger/add
```

The problem here is the JVM. While functional languages such as Scheme or Haskell perform TCO, the JVM doesn't perform this optimization. The absence of TCO is unfortunate but not a showstopper for functional programs.

Clojure provides several pragmatic workarounds: explicit self-recursion with `recur`, lazy sequences, and explicit mutual recursion with `trampoline`. We'll discuss the first two here and defer the discussion of `trampoline`, which is a more advanced feature, until later in the chapter.

Self-Recursion with `recur`

One special (and common) case of recursion that *can* be optimized away on the JVM is self-recursion. Fortunately, the `tail-fibo` is an example: it calls itself directly, not through some series of intermediate functions.

In Clojure, you can convert a function that tail-calls itself into an explicit self-recursion with `recur`. Using this approach, convert `tail-fibo` into `recur-fibo`:

```
src/examples/functional.clj
Line 1 ; better but not great
2 (defn recur-fibo [n]
3   (letfn [(fib
4             [current next n]
5             (if (zero? n)
6                 current
7                 (recur next (+ current next) (dec n))))]
8     (fib 0N 1N n)))
```

The critical difference between `tail-fibo` and `recur-fibo` is on line 7, where `recur` replaces the call to `fib`.

The `recur-fibo` won't consume stack as it calculates Fibonacci numbers and can calculate F_n for large n if you have the patience:

```
(recur-fibo 9)
-> 34N

(recur-fibo 1000000)
-> 195 ... 208,982 other digits ... 875N
```

The complete value of $F_{1000000}$ is included in the sample code at `output/f-1000000`.

The `recur-fibo` calculates one Fibonacci number. But what if you want several? Calling `recur-fibo` multiple times would be wasteful, since none of the work from any call to `recur-fibo` is cached for the next call. But how many values should be cached? Which ones? These choices should be made by the *caller* of the function, not the implementer.

Ideally, you'd define sequences with an API that makes *no reference* to the specific range that a particular client cares about, and then let clients pull the range they want with `take` and `drop`. This is exactly what lazy sequences provide.

Lazy Sequences

Lazy sequences are constructed using the macro `lazy-seq`:

```
(lazy-seq & body)
```

A `lazy-seq` invokes its `body` only when needed, that is, when `seq` is called directly or indirectly. `lazy-seq` then caches the result for subsequent calls. You can use `lazy-seq` to define a lazy Fibonacci series as follows:

```

src/examples/functional.clj
Line 1 (defn lazy-seq-fibo
2     ([
3       (concat [0 1] (lazy-seq-fibo 0N 1N)))
4     ([a b]
5       (let [n (+ a b)]
6         (lazy-seq
7           (cons n (lazy-seq-fibo b n))))))

```

On line 3, the zero-argument body returns the concatenation of the basis values [0 1] and then calls the two-argument body to calculate the rest of the values. On line 5, the two-argument body calculates the next value *n* in the series, and on line 7 it conses *n* onto the rest of the values.

The key is line 6, which makes its body lazy. Without this, the recursive call to `lazy-seq-fibo` on line 7 would happen immediately, and `lazy-seq-fibo` would recurse until it blew the stack. This illustrates the general pattern: wrap the recursive part of a function body with `lazy-seq` to replace recursion with laziness.

`lazy-seq-fibo` works for small values:

```

(take 10 (lazy-seq-fibo))
-> (0 1 1N 2N 3N 5N 8N 13N 21N 34N)

```

`lazy-seq-fibo` also works for large values. Use `(rem ... 1000)` to print only the last three digits of the one-millionth Fibonacci number:

```

(rem (nth (lazy-seq-fibo) 1000000) 1000)
-> 875N

```

The `lazy-seq-fibo` approach follows [guideline #3 on page 101](#), using laziness to implement an infinite sequence. But as is often the case, you don't need to explicitly call `lazy-seq` yourself. By guideline #5, you can reuse existing sequence functions that return lazy sequences. Consider this use of `iterate`:

```

(take 5 (iterate (fn [[a b]] [b (+ a b)]) [0 1]))
-> ([0 1] [1 1] [1 2] [2 3] [3 5])

```

The `iterate` begins with the first pair of Fibonacci numbers: [0 1]. Working by pairs, it then calculates the Fibonacci numbers by carrying along just enough information (two values) to calculate the next value.

The Fibonacci numbers are simply the first value of each pair. They can be extracted by calling `map first` over the entire sequence, leading to the following definition of `fibo` suggested by Christophe Grand:

```

src/examples/functional.clj
(defn fibo []
  (map first (iterate (fn [[a b]] [b (+ a b)]) [0N 1N])))

```


`fibonacci` returns a lazy sequence because it builds on `map` and `iterate`, which also return lazy sequences. `fibonacci` is also *simple*. `fibonacci` is the shortest implementation we've seen so far. But if you're accustomed to writing imperative, looping code, correctly *choosing* `fibonacci` over other approaches may not seem simple at all! Learning to think recursively, lazily, and within the JVM's limitations on recursion—all at the same time—can be intimidating. Let the [guidelines on page 101](#) help you. The Fibonacci numbers are infinite: guideline #3 correctly predicts that the right approach in Clojure will be a lazy sequence, and guideline #5 lets the existing sequence functions do most of the work.

Lazy definitions consume *some* stack and heap. But they don't consume resources proportional to the size of an entire (possibly infinite!) sequence. Instead, *you choose* how many resources to consume when you traverse the sequence. If you want the one millionth Fibonacci number, you can get it from `fibonacci` without having to consume stack or heap space for all of the previous values.

There's no such thing as a free lunch. But with lazy sequences, you can have an infinite menu and pay only for the menu items you're eating at a given moment. When writing Clojure programs, you should prefer lazy sequences over `loop`/`recur` for any sequence that varies in size and for any large sequence.

Coming to Realization

Lazy sequences consume significant resources only as they are *realized*, that is, when a portion of the sequence is actually instantiated in memory. Clojure works hard to be lazy and avoid realizing sequences until it's absolutely necessary. For example, `take` returns a lazy sequence and does no realization at all. You can see this by creating a var to hold, say, the first billion Fibonacci numbers:

```
(def lots-o-fibs (take 1000000000 (fibonacci)))
-> #'user/lots-o-fibs
```

The creation of `lots-o-fibs` returns almost immediately, because it does *almost nothing*. If you ever call a function that needs to *actually use* some values in `lots-o-fibs`, Clojure will calculate them. Even then, it'll do only what's necessary. For example, the following will return the 100th Fibonacci number from `lots-o-fibs`, without calculating the millions of other numbers that `lots-o-fibs` promises to provide:

```
(nth lots-o-fibs 100)
-> 354224848179261915075N
```

Most sequence functions return lazy sequences. If you aren't sure whether a function returns a lazy sequence, the function's documentation string typically will tell you the answer:

```
(doc take)
-----
clojure.core/take
([n coll])
Returns a lazy seq of the first n items in coll, or all items if
there are fewer than n.
```

The REPL, however, is *not lazy*. The printer used by the REPL will, by default, print the entirety of a collection. That's why we stuffed the first billion Fibonacci numbers into `lots-o-fibs`, instead of evaluating them at the REPL. Don't enter the following at the REPL:

```
; don't do this
(take 1000000000 (fibo))
```

If you enter the previous expression, the printer will attempt to print a billion Fibonacci numbers, realizing the entire collection as it goes. You'll probably get bored and exit the REPL before Clojure runs out of memory.

As a convenience for working with lazy sequences, you can configure how many items the printer will print by setting the value of `*print-length*`:

```
(set! *print-length* 10)
-> 10
```

For collections with more than 10 items, the printer will now print only the first 10, followed by an ellipsis. So, you can safely print a billion fibos:

```
(take 1000000000 (fibo))
-> (0N 1N 1N 2N 3N 5N 8N 13N 21N 34N ...)
```

Or even all of the fibos:

```
(fibo)
-> (0N 1N 1N 2N 3N 5N 8N 13N 21N 34N ...)
```

Lazy sequences are wonderful. They do only what's needed, and for the most part, you don't have to worry about them. If you ever want to force a sequence to be fully realized, you can use either `doall` or `dorun`, discussed in [Lazy and Infinite Sequences, on page 80](#).

Losing Your Head

There's one last thing to consider when working with lazy sequences. Lazy sequences let you define a large (possibly infinite) sequence and then work with a small part of that sequence in memory at a given moment. This clever ploy will fail if you (or some API) unintentionally hold a reference to the part of the sequence you no longer care about.

The most common way this can happen is if you accidentally hold the head (first item) of a sequence. In the examples in previous sections, each variant of the Fibonacci numbers was defined as a function returning a sequence, not the sequence itself.

You could define the sequence directly as a top-level var:

```
src/examples/functional.clj
; holds the head (avoid!)
(def head-fibo (lazy-cat [0N 1N] (map + head-fibo (rest head-fibo))))
```

This definition uses `lazy-cat`, which is like `concat` except that the arguments are evaluated only when needed. This is a very pretty definition, in that it defines the recursion by mapping a sum over (each element of the Fibonacci) and (each element of the *rest* of the Fibonacci).

`head-fibo` works great for small Fibonacci numbers:

```
(take 10 head-fibo)
-> (0N 1N 1N 2N 3N 5N 8N 13N 21N 34N)
```

but not so well for huge ones:

```
(nth head-fibo 10000000)
-> Execution error (OutOfMemoryError) at java.math.BigInteger/add
```

The problem is that the top-level var `head-fibo` *holds the head* of the collection. This prevents the garbage collector from reclaiming elements of the sequence after you've moved past those elements. So, any part of the Fibonacci sequence that you actually use gets cached for the life of the value referenced by `head-fibo`, which is likely to be the life of the program.

Unless you want to cache a sequence as you traverse it, you must be careful not to keep a reference to the head of the sequence. As the `head-fibo` example demonstrates, you should normally expose lazy sequences as a function that *returns* the sequence, not as a var that *contains* the sequence. If a caller of your function wants an explicit cache, the caller can always create its own var. With lazy sequences, losing your head is often a good idea.

Lazier Than Lazy

Clojure's lazy sequences are a great form of laziness at the language level. As a programmer, you can be *even lazier* by finding solutions that don't require explicit sequence manipulation at all. You can often combine existing sequence functions to solve a problem, without having to get your hands dirty at the level of `recur` or lazy sequences.

As an example of this, let's implement several solutions to the following problem. You are given a sequence of coin-toss results, where heads is `:h` and tails is `:t`:

```
[ :h :t :t :h :h :h ]
```

How many times in the sequence does heads come up twice in a row? In the previous example, the answer is two. Toss 4 and toss 5 are both heads, and toss 5 and toss 6 are both heads.

The sequence of coin tosses might be very large, but it'll be finite. Since you're looking for a scalar answer (a count), by rule 2, it's acceptable to use `recur`:

```
src/examples/functional.clj
Line 1 (defn count-heads-pairs [coll]
2   (loop [cnt 0 coll coll]
3     (if (empty? coll)
4       cnt
5       (recur (if (= :h (first coll) (second coll))
6                (inc cnt)
7                cnt)
8                (rest coll)))))
```

Since the purpose of the function is to count something, the loop introduces a `cnt` binding, initially zero on line 2. The loop also introduces its own binding for the `coll`, so that we can shrink the `coll` each time through the `recur`. Line 3 provides the basis for the recurrence. If a sequence of coin tosses is empty, it certainly has zero runs of two heads in a row.

Line 5 is the meat of the function, incrementing the `cnt` by one if the first and second items of `coll` are both heads (`:h`).

Try a few inputs to see that `count-heads-pairs` works as advertised:

```
(count-heads-pairs [:h :h :h :t :h])
-> 2

(count-heads-pairs [:h :t :h :t :h])
-> 0
```

Although `count-heads-pairs` works, it fails as code prose. The key notion of “two in a rowness” is obscured by the boilerplate for `loop/recur`. To fix this, you need to use guidelines 5 and 6, subdividing the problem to take advantage of Clojure's sequence functions.

Transforming the Input Sequence

The first problem you encounter is that almost all of the sequence functions do something to each element in a sequence in turn. This doesn't help us at

all, since we want to look at each element in the context of its immediate neighbors. So, let's transform the sequence. When you see this:

```
[ :h :t :t :h :h :h ]
```

you should mentally translate that into a sequence of every adjacent pair:

```
[ [ :h :t ] [ :t :t ] [ :t :h ] [ :h :h ] [ :h :h ] ]
```

Write a function named `by-pairs` that performs this transformation. Because the output of `by-pairs` varies based on the size of its input, according to rule 3, you should build this sequence lazily:

```
src/examples/functional.clj
Line 1 ; overly complex, better approaches follow...
2 (defn by-pairs [coll]
3   (let [take-pair (fn [c]
4                     (when (next c) (take 2 c)))]
5     (lazy-seq
6       (when-let [pair (seq (take-pair coll))]
7         (cons pair (by-pairs (rest coll)))))))
```

Line 3 defines a function that takes the first pair of elements from the collection. Line 5 ensures that the recursion is evaluated lazily.

Line 6 is a conditional: if the next pair doesn't contain two elements, we must be (almost) at the end of the list, and we implicitly terminate. If we do get two elements, then on line 7, we continue building the sequence by consing our pair onto the pairs to be had from the rest of the collection.

Check that `by-pairs` works:

```
(by-pairs [ :h :t :t :h :h :h ])
-> ([ :h :t ] [ :t :t ] [ :t :h ] [ :h :h ] [ :h :h ])
```

Now that we can think of the coin tosses as a sequence *of pairs* of results, it's easy to describe count-heads-pairs in English:

“Count the pairs of results that are all heads.”

This English description translates directly into existing sequence functions: “Count” is `count`, of course, and “that are all heads” suggests a filter:

```
src/examples/functional.clj
(defn count-heads-pairs [coll]
  (count (filter (fn [pair] (every? #{:h} pair))
                (by-pairs coll))))
```

This is much more expressive than the recur-based implementation, and it makes clear that we're counting all of the adjacent pairs of heads. But we

can make things even simpler. Clojure already has a more general version of by-pairs named partition:

```
(partition size step? coll)
```

partition breaks a collection into chunks of size size. So, you could break a heads/tails vector into a sequence of pairs:

```
(partition 2 [ :h :t :t :h :h :h ])
-> ((:h :t) (:t :h) (:h :h))
```

This isn't quite the same as by-pairs, which yields overlapping pairs. But partition can do overlaps, too. The optional step argument determines how far partition moves down the collection before starting its next chunk. If not specified, step is the same as size. To make partition work like by-pairs, set size to 2 and set step to 1:

```
(partition 2 1 [ :h :t :t :h :h :h ])
-> ((:h :t) (:t :t) (:t :h) (:h :h) (:h :h))

(by-pairs [ :h :t :t :h :h :h ])
-> ((:h :t) (:t :t) (:t :h) (:h :h) (:h :h))
```

Another possible area of improvement is the count/filter idiom used to count the pairs that are both heads. This combination comes up often enough that it's worth encapsulating in a count-if function:

```
src/examples/functional.clj
(def ^{:doc "Count items matching a filter"}
  count-if (comp count filter))
```

comp is used to *compose* two or more functions:

```
(comp f & fs)
```

The composed function is a new function that applies the rightmost function to its arguments, the next-rightmost function to that result, and so on. So, count-if will first filter and then count the results of the filter:

```
(count-if odd? [1 2 3 4 5])
-> 3
```

Finally, you can use count-if and partition to create a count-runs function that's more general than count-heads-pairs:

```
src/examples/functional.clj
(defn count-runs
  "Count runs of length n where pred is true in coll."
  [n pred coll]
  (count-if #(every? pred %) (partition n 1 coll)))
```

count-runs is a winning combination: both simpler and more general than the previous versions of count-heads-pairs. You can use it to count pairs of heads:

```
(count-runs 2 #{:h} [:h :t :t :h :h :h])
-> 2
```

But you can just as easily use it to count pairs of tails:

```
(count-runs 2 #{:t} [:h :t :t :h :h :h])
-> 1
```

Or, instead of pairs, how about runs of three heads in a row?

```
(count-runs 3 #{:h} [:h :t :t :h :h :h])
-> 1
```

Partial Application

If you still want to have a function named count-heads-pairs, you can implement it in terms of count-runs:

```
src/examples/functional.clj
(def ^{:doc "Count runs of length two that are both heads"}
  count-heads-pairs (partial count-runs 2 #{:h}))
```

This version of count-heads-pairs builds a new function using partial:

```
(partial f & partial-args)
```

partial performs a *partial application* of a function. You specify a function *f* and part of the argument list when you perform the partial. You specify the remainder of the argument list later, when you call the function created by partial. So, the following:

```
(partial count-runs 1 #{:h})
```

is a more expressive way of saying this:

```
(fn [coll] (count-runs 1 #{:h} coll))
```

You've seen a lot of new forms in this section. Don't let all of the details obscure the key idea: by combining existing functions on sequences, you were able to create a solution that was both simpler and more general than the direct approach. And, you didn't have to worry about laziness or recursion at all. Instead, you worked at a higher level of abstraction and let Clojure deal with laziness and recursion for you.

Recursion Revisited

Clojure works very hard to balance the power of functional programming with the reality of the Java Virtual Machine. One example of this is the well-motivated choice of explicit TCO through `loop/recur`.

But blending the best of two worlds always runs the risk of unpleasant compromises, and it makes sense to ask the question, “Does Clojure contain hidden design compromises that, while not obvious on day one, will bite me later?”

This question is *never* fully answerable for any language, but let’s consider it by exploring some more complex recursions. First, we’ll look at *mutual recursion*.

A mutual recursion occurs when the recursion bounces between two or more functions. Instead of A calls A calls A, we have A calls B calls A again. As a simple example, you could define `my-odd?` and `my-even?` using mutual recursion:

```
src/examples/functional.clj
(declare my-odd? my-even?)

(defn my-odd? [n]
  (if (= n 0)
      false
      (my-even? (dec n))))

(defn my-even? [n]
  (if (= n 0)
      true
      (my-odd? (dec n))))
```

Because `my-odd?` and `my-even?` each call the other, you need to create both vars before you define the functions. You could do this with `def`, but the `declare` macro lets you create both vars (with no initial binding) in a single line of code.

Verify that `my-odd?` and `my-even?` work for small values:

```
(map my-even? (range 10))
-> (true false true false true false true false true)

(map my-odd? (range 10))
-> (false true false true false true false true false)
```

`my-odd?` and `my-even?` consume stack frames proportional to the size of their argument, so they will fail with large numbers.

```
(my-even? (* 1000 1000 1000))
-> Execution error (StackOverflowError) at user/my-even? (REPL:1).
```

This is similar to the problem that motivated the introduction of `recur`. But you can’t use `recur` to fix it because `recur` works with self-recursion, not mutual recursion. Of course, odd/even can be implemented more efficiently *without*

recursion anyway. Clojure's implementation uses bit-and (bitwise and) to implement odd? and even?:

```
; from core.clj
(defn even? [n] (zero? (bit-and n 1)))
(defn odd? [n] (not (even? n)))
```

We picked odd/even for its simplicity. Other recursive problems are not so simple and don't have an elegant nonrecursive solution. We'll examine four approaches you can use to solve such problems:

- Converting to self-recursion
- Trampolining a mutual recursion
- Replacing recursion with laziness
- Shortcutting recursion with memoization

Converting to Self-Recursion

Mutual recursion is often a nice way to model separate but related concepts. For example, oddness and evenness are separate concepts but clearly related to one another.

You can convert a mutual recursion to a self-recursion by coming up with a single abstraction that deals with multiple concepts simultaneously. For example, you can think of oddness and evenness in terms of a single concept: *parity*. Define a parity function that uses recur and returns 0 for even numbers and 1 for odd numbers:

```
src/examples/functional.clj
(defn parity [n]
  (loop [n n par 0]
    (if (= n 0)
      par
      (recur (dec n) (- 1 par)))))
```

Test that parity works for small values:

```
(map parity (range 10))
-> (0 1 0 1 0 1 0 1 0 1)
```

At this point, you can trivially implement my-odd? and my-even? in terms of parity:

```
src/examples/functional.clj
(defn my-even? [n] (= 0 (parity n)))
(defn my-odd? [n] (= 1 (parity n)))
```

Parity is a straightforward concept. Unfortunately, many mutual recursions will not simplify into an elegant self-recursion. If you try to convert a mutual

recursion into a self-recursion and you find the resulting code to be full of conditional expressions that obfuscate the definition, then don't use this approach.

Trampolining Mutual Recursion

A *trampoline* is a technique for optimizing mutual recursion. A trampoline is like an after-the-fact recur, imposed by the *caller* of a function instead of the *implementer*. Since the caller can call more than one function inside a trampoline, trampolines can optimize mutual recursion.

Clojure's trampoline function invokes one of your mutually recursive functions:

```
(trampoline f & partial-args)
```

If the return value is not a function, then a trampoline works just like calling the function directly. Try trampolining a few simple Clojure functions:

```
(trampoline list)
-> ()
(trampoline + 1 2)
-> 3
```

If the return value is a function, then trampoline assumes you want to call it recursively and calls it for you. trampoline manages its own recur, so it will keep calling your function until it stops returning functions.

Back in [Tail Recursion, on page 103](#), you implemented a tail-fibo function. You saw how the function consumed stack space and replaced the tail recursion with a recur. Now, we have another option. You can take the code of tail-fibo and prepare it for trampolining by wrapping the recursive return case inside a function.

```
src/examples/trampoline.clj
```

```
Line 1 ; Example only. Don't write code like this.
- (defn trampoline-fibo [n]
-   (let [fib (fn fib [f-2 f-1 current]
-               (let [f (+ f-2 f-1)]
-                 (if (= n current)
-                     f
-                     #(fib f-1 f (inc current)))))]
-     (cond
-       (= n 0) 0
-       (= n 1) 1
-       :else (fib 0N 1 2))))
```

The primary difference between this and the original version of tail-fibo is the initial # on line 7 that wraps the call to fib in an anonymous function. Try bouncing trampoline-fibo on a trampoline:

```
(trampoline trampoline-fibo 9)
-> 34N
```

Since trampoline does a recur, it can handle large inputs just fine, without throwing a StackOverflowError:

```
(rem (trampoline trampoline-fibo 1000000) 1000)
-> 875N
```

We've ported tail-fibo to use trampoline to compare and contrast trampoline and recur. For self-recursions like trampoline-fibo, trampoline offers no advantage, and you should prefer recur. But with mutual recursion, trampoline comes into its own.

Consider the mutually recursive definition of my-odd? and my-even?, which we presented at the beginning of [Recursion Revisited, on page 114](#). You can convert these broken, stack-consuming implementations to use trampoline, with the same approach you used to convert tail-fibo: prepend a # to any recursive tail calls so that they return a function to produce the value, rather than the value itself:

```
src/examples/trampoline.cj
Line 1 (declare my-odd? my-even?)
-
- (defn my-odd? [n]
-   (if (= n 0)
5     false
-     #(my-even? (dec n))))
-
- (defn my-even? [n]
-   (if (= n 0)
10    true
-     #(my-odd? (dec n))))
```

The only difference from the original implementation is the function wrappers on lines 6 and 11. With this change in place, you can trampoline large values of n without blowing the stack:

```
(trampoline my-even? 1000000)
-> true
```

A trampoline is a special-purpose solution to a specific problem. It requires doctoring your original functions to return a different type to indicate recursion. If one of the other techniques presented here provides a more elegant implementation for a particular recursion, that's great. If not, you'll be happy

to have trampoline in your box of tools. In practice, many Clojure programmers never encounter a case requiring trampoline at all.

Replacing Recursion with Laziness

Of all the techniques for eliminating or optimizing recursion discussed in this chapter, laziness is the one you'll probably use most often.

For our example, we'll implement the `replace` function developed by Eugene Wallingford to demonstrate mutual recursion.⁵

`replace` works with an *s-list* data structure, which is a list that can contain both symbols and lists of symbols. `replace` takes an *s-list*, an `oldsym`, and a `newsym` and replaces all occurrences of `oldsym` with `newsym`. For example, this call to `replace` replaces all occurrences of `b` with `a`:

```
(replace '((a b) (((b g r) (f r)) c (d e)) b) 'b 'a)
-> (((a a) (((a g r) (f r)) c (d e)) a)
```

The following is a fairly literal translation of the Scheme implementation from Wallingford's paper. We've converted from Scheme functions to Clojure functions, changed the name to `replace-symbol` to avoid collision with Clojure's `replace`, and shortened names to better fit the printed page, but otherwise, we've preserved the structure of the original:

```
src/examples/wallingford.clj
; overly-literal port, do not use
(declare replace-symbol replace-symbol-expression)
(defn replace-symbol [coll oldsym newsym]
  (if (empty? coll)
      ()
      (cons (replace-symbol-expression
              (first coll) oldsym newsym)
            (replace-symbol
             (rest coll) oldsym newsym))))
(defn replace-symbol-expression [symbol-expr oldsym newsym]
  (if (symbol? symbol-expr)
      (if (= symbol-expr oldsym)
          newsym
          symbol-expr)
      (replace-symbol symbol-expr oldsym newsym)))
```

The two functions `replace-symbol` and `replace-symbol-expression` are mutually recursive, so a deeply nested structure could blow the stack. To demonstrate the problem, create a deeply-nested function that builds deeply nested lists containing a single bottom element:

5. <https://www.cs.uni.edu/~wallingf/patterns/recursion.html>

```
src/examples/replace_symbol.clj
```

```
(defn deeply-nested [n]
  (loop [n n
        result '(bottom)]
    (if (= n 0)
        result
        (recur (dec n) (list result)))))
```

Try deeply-nested for a few small values of n:

```
(deeply-nested 5)
-> ((((((bottom))))))

(deeply-nested 25)
-> (((((((((((((((((((((((((((((((((((((((bottom))))))))))))))))))))))))))))))
```

Clojure provides a **print-level** that controls how deeply the Clojure printer will go into a nested data structure. Set the **print-level** to a modest value so that the printer doesn't go crazy trying to print a deeply nested structure. You'll see that when we nest deeper, the printer simply prints a # and stops:

```
(set! *print-level* 25)
-> 25

(deeply-nested 5)
-> ((((((bottom))))))

(deeply-nested 25)
-> (((((((((((((((((((((((((((((((((((((((#))))))))))))))))))))))))))
```

Now, try to use `replace-symbol` to change `bottom` to `deepest` for different levels of nesting. You'll see that large levels blow the stack. Depending on your JVM implementation, you may need a larger value than the 10000 shown here:

```
(replace-symbol (deeply-nested 5) 'bottom 'deepest)
-> ((((((deepest))))))

(replace-symbol (deeply-nested 10000) 'bottom 'deepest)
-> Execution error (StackOverflowError) at user/replace-symbol (REPL:1).
```

All of the recursive calls to `replace-symbol` are inside a `cons`. To break the recursion, all you have to do is wrap the recursion with `lazy-seq`. *It's really that simple*. You can break a sequence-generating recursion by wrapping it with a `lazy-seq`. This works by replacing the body during macroexpansion, rather than doing it in a new stack frame at runtime.

Since the transition to laziness was so simple, we couldn't resist the temptation to make the function more Clojure-ish in another way as well.

```
src/examples/replace_symbol.clj
Line 1 (defn- coll-or-scalar [x & _] (if (coll? x) :collection :scalar))
2 (defmulti replace-symbol coll-or-scalar)
3 (defmethod replace-symbol :collection [coll oldsym newsym]
4   (lazy-seq
5     (when (seq coll)
6       (cons (replace-symbol (first coll) oldsym newsym)
7             (replace-symbol (rest coll) oldsym newsym))))))
8 (defmethod replace-symbol :scalar [obj oldsym newsym]
9   (if (= obj oldsym) newsym obj))
```

On line 4, the lazy-seq breaks the recursion, preventing stack overflow on deeply nested structures. The other improvement is on line 2. Rather than have two different functions to deal with symbols and lists, there's a single multimethod replace-symbol with one method for lists and another for symbols. (Multimethods are covered in detail in [Chapter 9, Multimethods, on page 201](#).) This gets rid of an if form and improves readability.

Make sure the improved replace-symbol can handle deep nesting:

```
(replace-symbol (deeply-nested 10000) 'bottom 'deepest)
-> ((((((((((((((((((((((((((((((((((#))))))))))))))))))))))))))
```

Laziness is a powerful ally. You can often write recursive and even mutually recursive functions, and then break the recursion with laziness.

Shortcutting Recursion with Memoization

To demonstrate a more complex mutual recursion, let's look at the Hofstadter Female and Male sequences. The first Hofstadter sequences were described in [Gödel, Escher, Bach: An Eternal Golden Braid \[Hof99\]](#). The Female and Male sequences are defined as follows:⁶

$$F(0) = 1; M(0) = 0$$

$$F(n) = n - M(F(n-1)), n > 0$$

$$M(n) = n - F(M(n-1)), n > 0$$

This suggests a straightforward definition in Clojure:

```
src/examples/male_female.clj
; do not use these directly
(declare m f)
(defn m [n]
  (if (zero? n)
    0
    (- n (f (m (dec n))))))
```

6. https://en.wikipedia.org/wiki/Hofstadter_sequence

```
(defn f [n]
  (if (zero? n)
      1
      (- n (m (f (dec n))))))
```

The Clojure definition is easy to read and closely parallels the mathematical definition. However, it performs poorly for large values of n . Each value in the sequence requires calculating two other values from scratch, which in turn requires calculating two other values from scratch. On one of our 2024-era MacBook Pro computers, it takes more than 1 second to calculate $(m\ 200)$:

```
(time (m 200))
"Elapsed time: 797.260959 msecs"
-> 124
```

Is it possible to preserve the clean, mutually recursive definition *and* have decent performance? Yes, with a little help from *memoization*. Memoization trades space for time by caching the results of past calculations. When you call a memoized function, it first checks your input against a map of previous inputs and their outputs. If it finds the input in the map, it can return the output immediately, without having to perform the calculation again.

Rebind m and f to memoized versions of themselves, using Clojure’s `memoize` function:

```
src/examples/memoized_male_female.clj
(def m (memoize m))
(def f (memoize f))
```

Now, Clojure needs to calculate F and M only once for each n . The speedup is enormous. Calculating $(m\ 200)$ is thousands of times faster:

```
(time (m 200))
"Elapsed time: 2.515833 msecs"
-> 124
```

And, of course, once the memoization cache is built, “calculation” of a cached value is almost instantaneous:

```
(time (m 200))
"Elapsed time: 0.07875 msecs"
-> 124
```

Memoization alone is not enough, because it shortcuts the recursion only if the memoization cache is already populated. If you start with an empty cache and ask for m of a large number, you’ll blow the stack before the cache can be built:

```
(m 10000)
-> Execution error (StackOverflowError) at user/m (REPL:1).
```

The final trick is to guarantee that the cache is built from the ground up by exposing *sequences*, instead of *functions*. Create `m-seq` and `f-seq` by mapping `m` and `f` over the whole numbers:

```
src/examples/male_female_seq.clj
(def m-seq (map m (iterate inc 0)))
(def f-seq (map f (iterate inc 0)))
```

Now, callers can get $M(n)$ or $F(n)$ by taking the n th value from a sequence:

```
(nth m-seq 200)
-> 124
```

The approach is quite fast, even for larger values of n :

```
(time (nth m-seq 10000))
"Elapsed time: 4.988875 msecs"
-> 6180
```

The approach we used here is as follows:

- Define a mutually recursive function in a natural way.
- Use memoization to shortcut recursion for values that have already been calculated.
- Expose a sequence so that dependent values are cached before they're needed.

This approach is heap-consuming, in that it caches all previously seen values. If this is a problem, in some situations, you can eliminate it by selecting a more complex caching policy. The library `core.memoize`⁷ provides this and many more features.

We've spent much of this chapter exploring different techniques for implementing and leveraging laziness. However, Clojure also provides a range of solutions for *eager* evaluation of transformations over collections. Next, we'll explore when these are preferred and how to perform eager transformations.

Eager Transformations

While lazy sequences are the right solution in many cases, they're not the best answer in every case. An eager approach can be better when we wish to construct an output collection rather than a sequence, optimize memory and performance for transformations of large collections, or control external resources.

7. <https://github.com/clojure/core.memoize>

Producing Output Collections

Sequence transformations produce output only as needed, which allows us to avoid unnecessary work and even to work with infinite sequences. However, it's also common to encounter situations where we want our output to be a concrete collection, not a sequence. In this case, we want all of the transformation to be completed, so we won't get any advantage from laziness.

For example, consider applying a function to a sequence to produce an output vector. In this case, we'd need to apply a final `vec` function to transfer the elements of the sequence back into a collection:

```
src/examples/eager.clj
(defn square [x] (* x x))

(defn squares-seq [n]
  (vec (map square (range n))))
```

Recall that sequences cache the result of their evaluation so that they're safe, immutable values, just like collections. This example will produce an input sequence (from the range), which stores the values in memory, then the output of the map, which is another sequence of values, and then the final output vector.

Instead, we can perform the intermediate transformation on the input collection values and place the result directly into the output vector using `into` with a *transducer*, eliminating the intermediate sequence entirely. Let's see what that looks like before we discuss what it means:

```
src/examples/eager.clj
(defn squares-into
  [n]
  (into [] (map square) (range n)))
```

Here, the call to `into` takes three arguments: the output collection, the transformation, and the input collection. The transformation here is a transducer. Many of the sequence functions (like `map`) can be called without their input collection argument to return a transducer. A transducer is a function that captures the essence of a collection transformation without tying it to the form of either the input collection or the output collection.

The `into` provides the elements from the input range to the `map` transformation and then adds the results directly into the output collection. This happens eagerly and returns the final vector, avoiding the creation of the intermediate lazy sequences, which are just overhead if we know the output target is a collection.

In the next section, we'll see a more complicated example and examine the performance implications more closely.

Optimizing Performance

In the previous example, we saw the difference between a single sequence transformation and a single transducer. We can also compose transducers and apply multiple transformations within into in a single pass over the input.

Imagine you want to find all of the predicate functions in the namespaces we've loaded so far. Predicate functions typically end in `?`, so we can find all of the loaded namespaces, then find their public vars, filter down to just those ending in `?`, and finally convert them to friendly names. Using sequences, we often chain transformations together with `->>`:

```
src/examples/eager.clj
```

```
(defn preds-seq []
  (->> (all-ns)
    (map ns-publics)
    (mapcat vals)
    (filter #(clojure.string/ends-with? % "?"))
    (map #(str (.sym %)))
    vec))
```

Each step of this transformation creates a sequence, which caches the intermediate values of the sequence. Finally, at the end, we collect the result into a vector.

Transducers can be composed in a similarly pipelined fashion using `comp`. Transducers are composed as a stack of function calls, which means that `comp` combines them left-to-right, just like `->>`. Using chained transducers with `into` looks like this equivalent function:

```
src/examples/eager.clj
```

```
(defn preds []
  (into []
    (comp
      (map ns-publics)
      (mapcat vals)
      (filter #(clojure.string/ends-with? % "?"))
      (map #(str (.sym %))))
    (all-ns)))
```

The sequence implementation creates four intermediate sequences. The transducer implementation composes the four transformations into a single combined transformation and applies it during a single traversal of the `(all-ns)` input sequence. It then places the results directly into the output vector. Removing the intermediate sequences can result in a significant reduction in memory usage, particularly if the size of the input collection is large, or the number of transformations is large.

The transducer version can also take advantage of some extra optimizations. First, if the input collection knows how to reduce itself, then the resulting compiled code is often far more efficient. We can't take advantage of this here, but in the squares example, range created a collection that knows how to reduce itself as a series of primitive long values in a tight loop, rather than boxing the values into Long objects and unboxing them to do math.

Second, some Clojure collections can be more efficiently bulk loaded using *transients*. Transients temporarily make an immutable collection mutable while adding values, then switch the collection back to a normal immutable collection when done. This happens in a constrained scope such that users are never exposed to the mutable version. `into` (and some other transducer aware functions) automatically takes advantage of this optimization.

Now that you've seen some of the performance advantages of doing eager transformations with transducers, let's consider how laziness complicates management of external resources and how eager transformation can help.

Managing External Resources

When data is read from things like files, databases, or web services, it's tempting to return a lazy sequence so we can begin processing the data as it's read:

```
src/examples/eager.clj
```

```
(defn non-blank? [s]
  (not (clojure.string/blank? s)))

(defn non-blank-lines-seq [file-name]
  (let [reader (clojure.java.io/reader file-name)]
    (filter non-blank? (line-seq reader))))
```

The problem here is that the reader is not being closed. This resource is being left open and stranded in this code. Beyond that, it's not clear when it could be closed because we need to wait for the processing of the last line in the lazy sequence of lines. Another approach is to eagerly process all of the lines, then close the resource before returning:

```
src/examples/eager.clj
```

```
(defn non-blank-lines [file-name]
  (with-open [reader (clojure.java.io/reader file-name)]
    (into [] (filter non-blank?) (line-seq reader))))
```

This solves the dangling resource problem, but only by eagerly creating the entire vector of non-blank lines and returning it. That's fine for small files, but it's a recipe for an eventual `OutOfMemoryError` with a sufficiently large file. What we really want is to process the file without the caching aspects of lazy sequences and to know when the input has been exhausted so that the reader can be closed.

First, let's refactor the `non-blank-lines` function to take a reader and return an *education*:

```
src/examples/eager.clj
```

```
(defn non-blank-lines-education [reader]
  (education (filter non-blank?) (line-seq reader))))
```

An education is a suspended transformation (like a sequence), but it processes the entire input each time it's asked (usually just once). Because it's processed anew each time, there's no caching as with a sequence. Instead, the transformation is just applied to the input to produce an output in a full pass through the data, usually performed with a `reduce` (when the output is a single computed value) or an `into` (when the output is a collection). For example, we can use `non-blank-lines-education` to count lines:

```
src/examples/eager.clj
```

```
(defn line-count [file-name]
  (with-open [reader (clojure.java.io/reader file-name)]
    (reduce (fn [cnt el] (inc cnt)) 0 (non-blank-lines-education reader))))
```

In `line-count`, we first create the reader in a `with-open` block, which will automatically close the reader before returning the result of the body. Next, we process the education in the context of a `reduce`. The education processes each line and then releases the associated memory, so the education will hold only one line in memory at a time.

Transducers and educations allow us to choose exactly when an input source is processed and thus know when the external resource is done and ready to release. With lazy sequences, it's much harder to know when processing is *done* and it's time to release.

Wrapping Up

In this chapter, you've seen how Clojure's support for functional programming strikes a well-motivated balance between academic purity and effectiveness on the Java Virtual Machine. Clojure provides a wide variety of techniques, including self-recursion with `recur`, mutual recursion with trampoline, lazy sequences, and memoization.

Better still, for a wide variety of everyday programming tasks, you can use the sequence functions without ever having to define your own explicit recursions of lazy sequences. Functions like `partition` create clean, expressive solutions that are much easier to write.

Finally, we saw how Clojure provides eager approaches as a companion to lazy sequences for use cases where eager evaluation is better suited.

Transducers provide fast, composable transformations that can be used in a variety of situations to produce output collections, improve performance, and manage external resources.

At this point, you've seen the core of Clojure—the syntax, immutable collections, the sequence abstraction, functional programming, and recursion. With the next chapter, we'll begin adding additional features. These features all build on top of the core ideas of immutable data and functional transformation. We'll start with *spec*, a Clojure library for describing the structure of data and functions.

Describing Your Data with Specs

Statically typed languages like Java create unique, named data structures (classes) for every unit of data in a program. Clojure is dynamically typed and instead relies on reusing a few generic collection types (vectors, maps, sets, lists, and sequences) to represent a program's data. This approach yields tremendous opportunities for code reuse, simplicity, generality, and extension.

However, one consequence of this approach is that functions in a Clojure program lack the explicit types that programmers in a statically typed language rely on as signposts to understand a piece of code.

For example, a Java program might have a function that takes a recipe ingredient and scales the quantity for a larger recipe. In Java, we would define an `Ingredient` class and `Unit` class, and the method might look like this:

```
public class Ingredient {
    private String name;
    private double quantity;
    private Unit unit;

    // ...

    public static Ingredient scale(Ingredient ingredient, double factor) {
        ingredient.setQuantity(ingredient.getQuantity() * factor);
        return ingredient;
    }
}
```

In a Clojure program, an ingredient might just be a map with some well-known keys. A scale function would then take and return a map without ever mentioning any explicit types:

```
src/examples/spec.clj
(defn scale-ingredient [ingredient factor]
  (update ingredient :quantity * factor))
```

This code is preferable to the Java version in several ways. It takes and returns immutable values, rather than updating mutable state, so it’s easier to reason about and inherently thread-safe by default. Also, the ingredient maps are just collections of attributes that can easily evolve over the life of a program, rather than being trapped behind the custom API of a Java class. However, one thing the Clojure code is lacking is explicit information about the expected structure of ingredient. We need to infer the structure from documentation or other parts of the program.

Clojure includes the *spec* library, which allows us to create “specs” that describe the structure of our data and the inputs and outputs of our functions. The actual code we write is the same—it still uses generic collections and generic operations on those collections. Adding specs to your code can make the implicit structure explicit.

For all code examples in this chapter, we’ll assume we’ve created a namespace with the following requires and changed to that namespace in our REPL:

```
src/examples/spec.clj
(ns examples.spec
  (:require [clojure.spec.alpha :as s]
            [clojure.spec.gen.alpha :as gen]
            [clojure.spec.test.alpha :as stest]))
```

We can start by describing the shape of an ingredient map, its fields, and their contents:

```
src/examples/spec.clj
;; Specs describing an ingredient
(s/def ::ingredient (s/keys :req [::name ::quantity ::unit]))
(s/def ::name      string?)
(s/def ::quantity  number?)
(s/def ::unit      keyword?)

;; Function spec for scale-ingredient
(s/fdef scale-ingredient
  :args (s/cat :ingredient ::ingredient :factor number?)
  :ret  ::ingredient)
```

The function spec gives us an explicit definition of the arguments and return value of `scale-ingredient`. These specs don’t just serve as documentation. The spec library uses them to provide several additional tools that operate on specs—data validation, explanations of invalid data, generation of example data, and even automatically created generative tests for functions with a spec.

We’ll start by considering how to define specs in our code and use them at runtime, followed by how we combine specs and validate data. Finally, we’ll see how to use `s/fdef` to define function specs for argument checking and generative testing.

Defining Specs

Specs are logical compositions of predicates (functions returning a logical true/false value) used to describe a set of data values. The spec library provides operations to create, combine, and register specs.

The `s/def` macro names and registers a spec in the global registry of specs accessible in the current Clojure program. The syntax for `s/def` is:

```
(s/def name spec)
```

Spec names are qualified keywords. For example, a simple spec definition using a predicate spec looks like:

```
(s/def :my.app/company-name string?)
```

Place this spec definition in your Clojure source file at the top level, just like var definitions made with the special form `def`. When creating function specs, most developers place them before the function they describe or sometimes collect all of the function specs into a separate namespace. For example, `clojure.core` itself has specs in a separate namespace, `clojure.core.specs.alpha`.

At runtime, specs are read and evaluated like function definitions. The global spec registry stores the spec, keyed by name.

Spec names must be fully qualified keywords. As a Clojure developer, it's your responsibility to use sufficiently qualified keywords as names so that your code will work with other code in the greater ecosystem. If you're writing a library for public reuse, you should follow rules similar to those used when choosing the project group ID and artifact ID when deploying the project artifact. Namely, the qualified part of the keyword should start with a domain name or a product or project name for which you control the trademark or mindshare in the market.

When these qualified keywords are cumbersome in code, we can use aliases and auto-resolved keywords to simplify their names. Auto-resolved keywords start with `::`. If no qualifier is specified (`::ingredient`), then the current namespace is used as the qualifier and the keyword is `:examples.spec/ingredient`. Since project namespaces are often sufficiently qualified, this is a great way to piggyback on good choices we've already made.

If a qualifier is specified, then it can refer to an alias defined in the current namespace. For example, `::recipe/ingredient` might expand to the namespace aliased to `recipe`, perhaps `:cookingco.recipe/ingredient`. In this chapter, we often use auto-resolved keywords for brevity in the code examples.

Sometimes, particularly with specs, it is useful to alias a namespace solely for naming that does not correspond to a code namespace. The `:as-alias` qualifier on `require` exists for exactly this purpose. The syntax (`require '[some.spec.namespace :as-alias short]`) will create an alias but NOT attempt to load `some.spec.namespace` as code.

Next, we'll look at how to create and combine specs to validate data, which we can choose to do during development or even at runtime.

Validating Data

Any time you receive a value from a user or an external source, that data may contain values that break your expectations. Clojure specs precisely describe those expectations, check whether the data is valid, and determine how it conforms to the specification. In the case where data is invalid, you may want to know what parts are invalid and why.

Predicates are the simplest kind of spec—they check whether a predicate matches a single value. Other specs compose predicates (and other specs) to create more complicated specifications. Some of the tools we'll discuss include range specs, logical connectors like `and` and `or`, and collection specs. These specs combine to cover any data structure you need to describe.

Predicates

Predicate functions that take a single value and return a logical true or false value are valid specs. Clojure provides dozens of predicates (many of these functions end in `?`); you can use any of those or ones in your own project. Some example predicates in `clojure.core` are functions like `boolean?`, `string?`, and `keyword?`. These predicates check for a single underlying type.

Other predicates combine several types together, such as `rational?`, which returns true for a value that is an integer, a decimal, or a ratio. Many predicates verify a property of the value itself, like `pos?`, `zero?`, `empty?`, `any?`, and `some?`.

Consider a simple predicate spec for a company name:

```
(s/def :my.app/company-name string?)
```

We've registered the specification for a company name in our application, so let's use it to validate incoming data:

```
(s/valid? :my.app/company-name "Acme Moving")
-> true

(s/valid? :my.app/company-name 100)
-> false
```

Now that we've created the spec and registered it with a name (`:my.app/company-name`), other parts of our program can use it as well. As we code, we build and record the semantics of our domain.

Next, we'll consider the common case of writing a spec that matches a set of enumerated values.

Enumerated Values

If we're in the business of selling marbles, we might stock three colors—red, green, and blue. When defining a spec for the color, we want to match any of those three values (declared as a keyword). Clojure sets are a perfect match for this—they allow us to store a set of non-duplicate items with a fast check for containment. Even better, sets implement the function interface and are valid specs as well:

```
(s/def :marble/color #{:red :green :blue})

(s/valid? :marble/color :red)
-> true

(s/valid? :marble/color :pink)
-> false
```

Consider writing a spec to match a bowling roll, where 0–10 pins could be knocked down. We can write such a spec like this:

```
(s/def :bowling/roll #{0 1 2 3 4 5 6 7 8 9 10})

(s/valid? :bowling/roll 5)
-> true
```

This works fine, but it seems silly to need to say all those numbers in order. We should make our computer take care of that instead, and range specs provide this functionality.

Range Specs

Clojure spec provides several range specs for the purpose of validating a range of values, like a bowling roll. Let's rewrite our spec using the provided `s/int-in` spec. We provide this spec with beginning (inclusive) and end (exclusive) integer values.

```
(s/def :bowling/ranged-roll (s/int-in 0 11))

(s/valid? :bowling/ranged-roll 10)
-> true
```

In addition to `s/int-in`, `s/double-in` and `s/inst-in` are for ranges of doubles and time instants. All of these cases define value ranges with lower and upper bounds.

The details vary slightly depending on the data type, so check the doc string for each function for proper use using (doc s/double-in) or (doc s/inst-in). Now, let's see how specs handle the special case of nil values.

Handling nil

Most predicates in Clojure will return false for the nil value; however, there are many cases where you'll want to take an existing spec and extend it to also include the nil value. You can use the special s/nilable operation to extend an existing spec.

For example, we could define our company name field as accepting either strings or nil:

```
(s/def :my.app/company-name-2 (s/nilable string?))
(s/valid? :my.app/company-name-2 nil)
-> true
```

The s/nilable predicate provides optimal performance and is preferred over equivalent specs that you might construct with s/or or other operations.

One case you might encounter is spec'ing the set of values true, false, or nil. It's tempting to use an explicit set for this #{true, false, nil}. However, when you ask whether a set contains a value, it returns the matching value—in this case, possibly false or nil. Spec interprets this as the predicate rejecting the value, rather than accepting it. Instead, use s/nilable to add nil as a valid value to the boolean? predicate:

```
(s/def ::nilable-boolean (s/nilable boolean?))
```

This will give you the correct behavior and the best performance.

Now that we know the basics of working with predicates and sets, we can consider writing more interesting compound specs that combine specs using logical operations.

Logical Specs

Logical specs create composite specs from other specs using s/and or s/or. For example, to create a spec for an odd integer, combine the predicates int? and odd?:

```
(s/def ::odd-int (s/and int? odd?))
(s/valid? ::odd-int 5)
-> true
(s/valid? ::odd-int 10)
-> false
(s/valid? ::odd-int 5.2)
-> false
```

In this example, we're combining predicates, but `s/and` can take any kind of spec. Similarly, we use `s/or` to combine multiple alternatives. For instance, to add 42 as a valid value in our last spec:

```
(s/def ::odd-or-42 (s/or :odd ::odd-int :const #{42}))
```

With `s/or`, we see that things are a bit different—each option in the `s/or` contains a keyword tag used to report how a value matches (or doesn't match) the spec.

If we want to know how a value matched a spec, we can use `s/conform`, which returns the value annotated with information about optional choices or the components of the value. We call this a *conformed value*.

For all of the specs we've seen until this point, there were no options or components, and so the conformed value was the same as the original value. The `s/or` contains a choice, and the conformed value tags the choice taken with the key (either `:const` or `:odd`):

```
(s/conform ::odd-or-42 42)
-> [:const 42]
(s/conform ::odd-or-42 19)
-> [:odd 19]
```

The conformed value for an `s/or` is a map entry, and the key and val functions extract the tag and value, respectively. The `s/conform` operation parses a value and describes the parse structure using the tags.

Conversely, the `s/explain` function describes all of the ways an *invalid* value didn't match its spec:

```
(s/explain ::odd-or-42 0)
| 0 - failed: odd? at: [:odd] spec: :examples.spec/odd-int
| 0 - failed: #{42} at: [:const] spec: :examples.spec/odd-or-42
```

Here, spec has found and reported two problems with the value. The first problem is that the value is not odd. Note that it passed the `int?` check inside `::odd-int`, so the report only includes the failing predicates. In each problem line, we see the failing value, the spec being checked, the tag path to the failing spec, and the failing predicate.

The explain messages print to the console, but we can alternatively retrieve this info as a string with `s/explain-str` or as data with `s/explain-data`.

Now that we've started to compose specs, you may be looking at your own data and thinking about how to write specs for it. Most Clojure data is not made of individual values but of collections, and there are a number of ways to write collection specs.

Collection Specs

The two most common collection specs you'll use are `s/coll-of` and `s/map-of`. The `s/coll-of` spec describes lists, vectors, sets, and seqs. You provide a spec that members of the collection must satisfy, and spec checks all members.

```
(s/def ::names (s/coll-of string?))
(s/valid? ::names ["Alex" "Stu"])
-> true
(s/valid? ::names #{"Alex" "Stu"})
-> true
(s/valid? ::names '("Alex" "Stu"))
-> true
```

The `s/coll-of` spec also comes with many additional options supplied as keyword arguments at the end of the spec.

- `:kind`—a predicate checked at the beginning of the spec. Common examples are `vector?` and `set?`.
- `:into`—one of these literal collections: `[]`, `()`, or `#{}`. Conformed values collect into the specified collection.
- `:count`—an exact count for the collection
- `:min-count`—a minimum count for the collection
- `:max-count`—a maximum count for the collection
- `:distinct`—true if the elements of the collection must be unique

As you can see, these options allow for specifying many common collection shapes and constraints. For example, we can choose to match just int sets with at least two values with a spec like this:

```
(s/def ::my-set (s/coll-of int? :kind set? :min-count 2))
(s/valid? ::my-set #{10 20})
```

Similar to `s/coll-of`, `s/map-of` specs a lookup map where the keys and values each follow a spec, such as a mapping from player names to scores:

```
(s/def ::scores (s/map-of string? int?))
(s/valid? ::scores {"Stu" 100, "Alex" 200})
-> true
```

All of the `s/coll-of` options also apply to `s/map-of`, although you won't typically need to use `:into` or `:kind` because they default to map-specific settings.

If you recall, the `s/conform` function tells how a value was parsed according to a spec. `s/map-of` conforms to a map and always conforms values. Keys are not conformed by default, but you can change that using the `:conform-keys` flag.

In cases of large collections or large maps, you might not want to validate or conform all values. For these cases, you can use the sampling specs `s/every` and `s/every-kv` instead.

Collection Sampling

The sampling collection specs are `s/every` and `s/every-kv` for collections and maps, respectively. They are similar in operation to `s/coll-of` and `s/map-of`, except they check up to only `s/*coll-check-limit*` elements (by default, 101).

Because these specs validate only a limited subset of values and conform no elements, large collections and maps have much better validation performance.

Tuples

Tuples are vectors with a known structure of fixed length, where each element has its own spec. For example, a vector of `x` and `y` coordinates can represent a point:

```
(s/def ::point (s/tuple float? float?))
(s/conform ::point [1.3 2.7])
-> [1.3 2.7]
```

Tuples do not name or tag the returned fields. Later in this chapter, we'll see another approach to handling sequential collections with well-known internal structure (using `s/cat`).

Clojure spec provides a number of tools for handling information maps, which we'll look at next.

Information Maps

It's common in Clojure to represent domain objects as maps with well-known fields; for example, we might be managing data related to bands and music albums and representing a particular release like this:

```
{::music/id #uuid "40e30dc1-55ac-33e1-85d3-1f1508140bfc"
 ::music/artist "Rush"
 ::music/title "Moving Pictures"
 ::music/date #inst "1981-02-12"}
```

We start by describing specs for the attributes:

```
src/examples/spec.clj
(require '[domain.music :as-alias music])
(s/def ::music/id uuid?)
(s/def ::music/artist string?)
(s/def ::music/title string?)
(s/def ::music/date inst?)
```

Many validation libraries define the structure of a map of attributes by including definitions of the attributes. This approach introduces important long-term problems. Attributes have independent semantics and may be reused across different structures where they should have the identical meaning without needing to be redefined.

By contrast, Clojure spec requires attributes to be defined independently and then defines map specs as open collections of attributes with no need for restatement of attribute definitions. This approach easily supports subsets of maps, the evolution of maps over time, and the transfer of maps through subsystems, where not all attributes need to be understood by the intermediary.

To specify a map of attributes, we use `s/keys`, which describes both required and optional attributes:

```
src/examples/spec.clj
(s/def ::music/release
  (s/keys :req [::music/id]
          :opt [::music/artist
                ::music/title
                ::music/date]))
```

Here, we define a `::music/release` map to require only a `::music/id` attribute, and all other attributes are optional.

An important additional feature of `s/keys` is that it will validate the values of all registered keys, even if they're not listed as required or optional. For example, if new optional attributes for `::music/release` are added in the future, the `s/keys` spec will automatically validate those as well. This approach encourages the uniform validation of registered attributes and the growth of systems over time.

In the case where you have existing maps that don't use qualified keys, the variant options `:req-un` and `:opt-un` take a fully qualified spec identifier, but check the value for the unqualified name of the key. For example, let's say our map instead looked like this:

```
{:id #uuid "40e30dc1-55ac-33e1-85d3-1f1508140bfc"
 :artist "Rush"
 :title "Moving Pictures"
 :date #inst "1981-02-12"}
```

We could use our existing attribute specs to create an unqualified version of the spec:

```
src/examples/spec.clj
(s/def ::music/release-unqualified
  (s/keys :req-un [::music/id]
          :opt-un [::music/artist
                  ::music/title
                  ::music/date]))
```

This version of the spec will match the spec for `::music/id` with the unqualified attribute `id` and so on for the other attributes as well. Specs for records also use the same approach with unqualified attributes. If needed, you can mix `:req`, `:req-un`, `:opt`, and `:opt-un` in the same spec.

You’ve now seen an overview of the basic tools for specifying the structure of our data. In the next section, we’ll take things to the next level by specifying the inputs and outputs of our functions using specs. When we do that, we’ll need some way to specify the syntax for function calls, and that will bring in one final set of specs we’ve not yet examined—*regex ops*.

Validating Functions

A function spec describes the operation of a function. It thus contains up to three specs: an “args” spec for the arguments of the function, a “ret” spec describing the return value, and a “fn” spec used to relate the arguments to the return value. Creating function specs unlocks many of the most useful capabilities of the spec library, such as instrumentation and generative testing.

The argument spec defines the collection of arguments that you can use to invoke the function being spec’ed. The spec operations we’ve seen so far allowed us to create specs for collections, but we need more power to represent the broad range of variability we see in function arguments—repeated arguments, optional arguments, and other structured collections. For these, spec provides *regex op* specs.

Most languages have support for matching the characters in a string according to a *regular expression*,¹ which defines the structure of the string (including repeated or optional patterns). Similarly, regex op specs apply one or more specs to match the values in a collection.

Let’s see how we can use regex op specs to define the arguments in a function spec and then *instrument* our code to automatically validate the arguments of a function call.

1. https://en.wikipedia.org/wiki/Regular_expression

Sequences with Structure

String regular expressions match the characters in a string. For example, the regular expression `abc*` describes strings like `"ab"`, `"abc"`, and `"abcc"`. In string regular expressions, placing two matching expressions next to each other indicates concatenation. The special operator `*` represents the repetition of 0 or more times of the prior matching expression (in this case, `c`).

Regex op specs provide a similar function, but instead of matching characters in a string, they match any arbitrary value in a collection. Like string regular expressions, regex op specs can match concatenated, repeated, and optional patterns.

Perhaps the most common regex op spec is `s/cat`, which specifies a concatenation (a series of elements, in order) for a variable length collection where each element is simply another spec. `s/cat` specs also name each component for use in conforming valid values or explaining invalid values.

For example, a spec to describe a sequential collection taking a string and an integer looks like this:

```
(s/def ::cat-example (s/cat :s string? :i int?))
-> :examples.spec/cat-example

(s/valid? ::cat-example ["abc" 100])
-> true
```

In the `s/cat`, each component has a keyword tag naming the component in the conformed result:

```
(s/conform ::cat-example ["abc" 100])
-> {:s "abc", :i 100}
```

There is also a regex op spec `s/alt` for indicating alternatives within the sequential structure. The conformed value is an entry with the matched tag and value.

```
(s/def ::alt-example (s/alt :i int? :k keyword?))
-> :examples.spec/alt-example

(s/valid? ::alt-example [100])
-> true

(s/valid? ::alt-example [:foo])
-> true

(s/conform ::alt-example [:foo])
-> [:k :foo]
```

Just like string regular expressions, the spec also contains operators for the repetition of a spec, which we'll dive into next.

Repetition Operators

There are three repetition operators—`s/?` for 0 or 1, `s/*` for 0 or more, and `s/+` for 1 or more.

All of the regex operators can be combined and nested arbitrarily along with predicates, sets, and other specs. The key thing to remember is that all connected regex ops describe the structure of a single sequential collection.

Consider the example of a collection that contains one or more odd numbers and an optional trailing even number:

```
(s/def ::oe (s/cat :odds (s/+ odd?) :even (s/? even?)))
-> :examples.spec/oe

(s/conform ::oe [1 3 5 100])
-> {:odds [1 3 5], :even 100}
```

Note how the nested `s/+` and `s/?` don't describe nested collections like `[[1 3 5] [100]]`. All regex op specs combine to describe the structure of a single top-level collection. This also applies to named regex ops, which allows us to factor regex op specs into smaller reusable pieces:

```
(s/def ::odds (s/+ odd?))
-> :examples.spec/odds

(s/def ::optional-even (s/? even?))
-> :examples.spec/optional-even

(s/def ::oe2 (s/cat :odds ::odds :even ::optional-even))
-> :examples.spec/oe2

(s/conform ::oe2 [1 3 5 100])
-> {:odds [1 3 5], :even 100}
```

Variable Argument Lists

Consider now how we might spec the arguments for a function that takes multiple arguments. For example, `println` takes zero or more objects and prints them as a string with spaces between them. We can use the `any?` predicate to specify each object and `s/*` to indicate the repetition:

```
(s/def ::println-args (s/* any?))
```

We might also have both some fixed arguments and a variable argument at the end. For example, in the `clojure.set` namespace, the `intersection` function takes at least one initial set, followed by any number of sets to intersect:

```
(doc clojure.set/intersection)
| -----
| clojure.set/intersection
| ([s1] [s1 s2] [s1 s2 & sets])
```

```
| Return a set that is the intersection of the input sets
-> nil

(clojure.set/intersection #{1 2} #{2 3} #{2 5})
-> #{2}
```

We can spec the arguments to intersection as follows.

```
(s/def ::intersection-args
  (s/cat :s1 set?
        :sets (s/* set?)))

(s/conform ::intersection-args '([#{1 2} #{2 3} #{2 5}]))
-> {:s1 #{1 2}, :sets [#{3 2} #{2 5}]}
```

To conform the args, we pass them in a vector, just as if we were invoking apply on the function and passing this vector of args. The conformed value returns the map describing each argument.

In this case, because each argument is the same spec, we could also use just s/+:

```
(s/def ::intersection-args-2 (s/+ set?))
-> :examples.spec/intersection-args-2

(s/conform ::intersection-args-2 '([#{1 2} #{2 3} #{2 5}]))
-> [#{1 2} #{3 2} #{2 5}]
```

Another common case in Clojure is the use of optional keyword arguments. For example, looking at the atom function, it has a signature (atom x & options) with options named :meta or :validator. Clojure supports the destructuring of these keyword options as if they were a map. Clojure spec can also create a regex spec as if these were a map using s/keys*, which has the identical structure to s/keys.

You can spec atom's args like this (some of these args are deliberately underspecified for demonstration purposes):

```
(s/def ::meta map?)
-> :examples.spec/meta

(s/def ::validator ifn?)
-> :examples.spec/validator

(s/def ::atom-args
  (s/cat :x any? :options (s/keys* :opt-un [::meta ::validator])))
-> :examples.spec/atom-args

(s/conform ::atom-args [100 :meta {:foo 1} :validator int?])
-> {:x 100,
    :options {:meta {:foo 1},
              :validator #object[clojure.core$int_QMARK_ ...]}}
```

The atom function follows a typical pattern of having two arities—one with options and one without. This is the most common case for multi-arity functions. However, it's also typical to encounter functions with an optional first argument, multiple invocation styles, or an argument that is repeated many times. Regex specs can cover all of these cases, and we'll look at another example in the next section.

Multi-arity Argument Lists

You can see another case of multi-arity argument lists in the repeat function, which has two arities, one with and one without the length `n`, which is the first argument, not the second. The spec can simply declare that first argument as optional:

```
(doc repeat)
| -----
| clojure.core/repeat
| ([x] [n x])
| Returns a lazy (infinite!, or length n if supplied) sequence of xs.
-> nil

(s/def ::repeat-args
  (s/cat :n (s/? int?) :x any?))
-> :examples.spec/repeat-args

(s/conform ::repeat-args [100 "foo"])
-> {:n 100, :x "foo"}

(s/conform ::repeat-args ["foo"])
-> {:x "foo"}
```

In some relatively rare cases, the arities are sufficiently different that it makes more sense to use `s/alt` to fully describe each arity.

Now that we've examined how to spec the arguments of a function, it's time to spec the function itself.

Specifying Functions

Function specs are a combination of three different specs for the arguments, the return value, and the “fn” spec that describes the relationship between the arguments and return.

Let's start with a function spec for `rand`:

```
clojure.core/rand
([] [n])
  Returns a random floating point number between 0 (inclusive) and
  n (default 1) (exclusive).
```

We can first create an argument spec that is either empty or takes an optional number:

```
src/examples/spec.clj
(s/def ::rand-args (s/cat :n (s/? number?)))
```

The docstring states that the function's return value is a floating-point number, but we can more precisely state that it will be a double:

```
src/examples/spec.clj
(s/def ::rand-ret double?)
```

We then need to consider the `:fn` spec, which receives a map containing the conformed args and the conformed return value based on their specs. In this case, the docs state that the random number must be ≥ 0 and $\leq n$. Let's state that as a predicate:

```
src/examples/spec.clj
(s/def ::rand-fn
  (fn [{:keys [args ret]}]
    (let [n (or (:n args) 1)]
      (cond (zero? n) (zero? ret)
            (pos? n) (and (>= ret 0) (< ret n))
            (neg? n) (and (<= ret 0) (> ret n))))))
```

We can now tie all these together using `s/fdef`, which takes a fully qualified function name and one or more of the specs (we'll supply all three):

```
src/examples/spec.clj
(s/fdef clojure.core/rand
  :args ::rand-args
  :ret ::rand-ret
  :fn ::rand-fn)
```

In a moment, we'll see how to use these specs, but first, let's take a slight detour to talk about spec'ing anonymous functions.

Anonymous Functions

Higher-order functions (ones that take and return functions) are common in Clojure. Use `s/fspec` to define the spec of an anonymous function. The syntax is the same as `s/fdef` but omits the function name. For instance, consider a function `opposite`, which takes a predicate function and creates the opposite predicate function:

```
src/examples/spec.clj
(defn opposite [pred]
  (comp not pred))
```

The function `opposite` accepts a predicate function, which we can describe using `s/fspec`. We can use that function spec in both the `:args` and `:ret` spec for `opposite`.

```
src/examples/spec.clj
(s/def ::pred
  (s/fspec :args (s/cat :x any?)
           :ret  boolean?))

(s/fdef opposite
  :args (s/cat :pred ::pred)
  :ret  ::pred)
```

It's also worth considering a simpler spec for anonymous functions—they don't always need to be fully spec'd, and sometimes, simply using `ifn?` as the spec is sufficient. Now that we've seen how to spec functions, let's consider what we can do with them.

Instrumenting Functions

During development and testing, we can use *instrumentation* (`stest/instrument`) to wrap a function with a version that uses spec to verify that the incoming arguments to a function conform to the function's spec.

Instrumentation is for :args Only

Note that instrumentation does not check the `:ret` or `:fn` specs—the purpose of instrumentation is for verifying correct invocation, not correct implementation. We'll see more on testing implementations later with `stest/check`.

Once you've defined a spec for a function with `s/fdef`, call `stest/instrument` on the fully qualified function symbol to enable it:

```
(stest/instrument 'clojure.core/rand)
-> [clojure.core/rand]
```

You can also use `stest/enumerate-namespace` to enumerate a collection of all symbols in a namespace to pass to `stest/instrument`:

```
(stest/instrument (stest/enumerate-namespace 'clojure.core))
-> [clojure.core/rand]
```

Note that `instrument` returns a collection of all symbols that were successfully instrumented. Check the return value to ensure it contains your intended symbols.

Instrumenting a function replaces its var with a new var that will check args and invoke the old function via its var. In the following, an invalid call to `rand` triggers an error:

```
(rand :boom)
| Execution error - invalid arguments to clojure.core/rand at (REPL:1).
| :boom - failed: number? at: [:n] spec: :examples.spec/rand-args
```

Here, we see an explain failure for the `rand` args spec. It expected a number? but received the value `:boom`, clearly not a number.

Instrumentation is not designed for production-time usage (there is an overhead in validating the spec and invoking a secondary var), but instrumentation is a valuable tool for catching errors faster at development or test time.

Let's move from catching invalid calls to a function to instead using a function's specs to test the function itself with `stest/check`.

Generative Function Testing

Classic *example-based unit testing* relies on the programmer to test a function by writing a series of example inputs, then writing assertions about the return value when the function is invoked with each input.

In comparison, *generative testing* is a technique that produces thousands of random inputs, runs a procedure, and verifies a set of properties for each output. Generative testing is a great technique for getting broader test coverage of your code.

Spec can automatically perform generative testing for functions that have function specs. Let's see how that's done and then explore ways to influence those tests for more accurate coverage.

Checking Functions

Spec implements automated generative testing with the function `check` in the namespace `clojure.spec.test.alpha` (commonly aliased as `stest`). You can run `stest/check` on any symbol or symbols that have been spec'ed with `s/def`.

All invocations of spec generators (either directly or within `stest/instrument`, `stest/check`, and so on) require including the `test.check` library as a dependency, typically as either a test or dev profile dependency, so that it's not included at production runtime. For our continued evaluation here, let's load it dynamically with `add-lib`:

```
(clojure.repl.deps/add-lib 'org.clojure/test.check)
-> [org.clojure/test.check]
```

When you invoke `check`, it generates 1000 sets of arguments that are valid according to the function's `:args` spec. For each argument set, `check` invokes

the function, then checks that the return value is valid according to the `:ret` spec and that the `:fn` spec is valid for the arguments and return value.

Let's see how it works with a spec for the Clojure core function `symbol`, which takes a name and an optional namespace:

```
(doc symbol)
| -----
| clojure.core/symbol
| ([name] [ns name])
| Returns a Symbol with the given namespace and name.
```

First, we need to define the function spec:

```
src/examples/spec.clj
(s/fdef clojure.core/symbol
  :args (s/cat :ns (s/? string?) :name string?)
  :ret symbol?
  :fn (fn [{:keys [args ret]]
        (and (= (name ret) (:name args))
              (= (namespace ret) (:ns args)))))
```

Then, we can run the test as follows:

```
(stest/check 'clojure.core/symbol)
-> ({:sym clojure.core/symbol
     :spec #object[clojure.spec.alpha$fspec_impl$reify__14282 ...],
     :clojure.spec.test.check/ret {
       :result true,
       :pass? true,
       :time-elapsed-ms 167,
       :num-tests 1000,
       :seed 1485241441400}})
```

You can see from the output that `stest/check` ran 1000 tests on `clojure.core/symbol` and found no problems. When an error occurs, the `test.check` library goes the extra step of “shrinking” the error to the least complicated possible input that still fails. In complex tests, this is a crucial step to produce tractable input for reproduction and fixing.

One step that we glossed over is *how* spec generated 1000 random input arguments. While we haven't mentioned it before, every spec is both a validator and a data generator for values that match the spec. Once we created a spec for the function arguments, `check` was able to use it to generate random arguments.

Generating Examples

The argument spec we used above was `(s/cat :ns (s/? string?) :name string?)`. To simulate how check generates random arguments from that spec, we can use the `s/exercise` function, which produces pairs of examples and their conformed values:

```
(s/exercise (s/cat :ns (s/? string?) :name string?))
-> ([{" " " "} {:ns " ", :name " "}]
    [{"F" " "} {:ns "F", :name " "}]
    [{"s" "73"} {:ns "s", :name "73"}]
    [{"u"} {:name "u"}]
    [{" " " "} {:name " "}]
    [{" " "3y"} {:ns " ", :name "3y"}]
    [{"t" "9pudu"} {:ns "t", :name "9pudu"}]
    [{"Xhw25" "nPR7C9C"} {:ns "Xhw25", :name "nPR7C9C"}]
    [{"FXs3E" "N"} {:ns "FXs3E", :name "N"}]
    [{"UhUN5dZK1" "le8"} {:ns "UhUN5dZK1", :name "le8"}])
```

This example works, but sometimes spec can't automatically create a valid generator, or you need to create a custom generator for related arguments. In the following sections, we'll see how to address these issues. First, let's consider a case where an `s/and` spec doesn't produce any values.

Combining Generators with `s/and`

One of the most common ways to compose specs is with `s/and`. The `s/and` operation uses the generator of its first component spec, then filters the values by each subsequent component spec.

For example, consider the following spec for an odd number greater than 100:

```
(defn big? [x] (> x 100))
(s/def ::big-odd (s/and odd? big?))
```

This would work as a spec, but its automatic generator doesn't work:

```
(s/exercise ::big-odd)
-> Unable to construct gen at: [] for: odd?
```

The problem is that while many common Clojure predicates have automatically mapped generators, the predicates we're using here do not. The `odd?` predicate works on more than one numeric type and so is not mapped to a generator. The `big?` predicate is a custom predicate that will never have mappings.

To fix this, we need to add an initial predicate that has a mapped generator—the type-oriented predicates are all good choices for that. Let's insert `int?` at the beginning:

```
(s/def ::big-odd-int (s/and int? odd? big?))
-> ::big-odd-int

(s/exercise ::big-odd-int)
-> ([1367 1367]
    [7669 7669]
    [171130765 171130765]
    ... )
```

When you debug generators for `s/and` specs, remember that only the first component spec’s generator is used. Another related problem with `s/and` generators is when the component specs after the first one are too “sparse,” filtering so many values that the generator would have to work for a long time to generate a valid one. The best way to solve this problem is with a custom generator.

Creating Custom Generators

There are many cases where the automatic generator is either inefficient or will not produce related values that are useful. For example, a function might take two arguments—a collection and an element from that collection. An automatic generator is unlikely to independently produce valid combinations of values. Instead, you need to supply your own custom generator that satisfies this constraint.

You have a number of opportunities in spec to replace the automatically created generator with your own implementation. Some specs like `s/coll-of`, `s/map-of`, `s/every`, `s/every-kv`, and `s/keys` accept a custom generator option.

Also, you can explicitly add a replacement generator to any existing spec with `s/with-gen`. And you can temporarily override a generator by name or path with generator overrides in some functions like `s/exercise`, `stest/instrument`, and `test/check`—those overrides take effect only for the duration of the call.

We can create generators in several ways. The simplest way is to first create a different spec, then use `s/gen` to retrieve its generator. Alternatively, the `clojure.spec.gen.alpha` namespace, typically aliased as `gen`, contains other generators and functions to combine generators. The generators in this namespace are wrappers around the `test.check` library, so you need that library on your classpath to do any work with generators.

Let’s look at how we can supply a replacement generator for `:marble/color` to hard-code exactly the color to return. Occasionally, this is useful to reduce the randomness of your inputs or to directly supply a complex input that would be difficult to generate.

Here, we use `s/with-gen` to override the default generator for `:marble/color` (which produces marbles of all colors) and replace it with a generator that only produces red marbles:

```
(s/def :marble/color-red
  (s/with-gen :marble/color #(s/gen #{:red})))
-> :marble/color-red

(s/exercise :marble/color-red)
-> ([:red :red] [:red :red] [:red :red] ...)
```

The `clojure.spec.gen.alpha` namespace contains many functions to generate all of the standard Clojure types, collections, and more. One function it provides (`gen/fmap`) allows you to start from a source generator, then modify each generated value by applying another function.

For example, to generate strings that start with a standard prefix, you can generate the random suffix, then concatenate the prefix. The generator itself might look like this:

```
(require '[clojure.string :as str])

(s/def ::sku
  (s/with-gen (s/and string? #(str/starts-with? % "SKU-"))
    (fn [] (gen/fmap #(str "SKU-" %) (s/gen string?)))))
```

Here, `gen/fmap` starts with a source generator (the generator for any valid string), then applies a function to the generated values to prefix the random string with "SKU-". That generator is then attached to the spec. Let's try exercising it:

```
(s/exercise ::sku)
-> (["SKU-" "SKU-"] ["SKU-P" "SKU-P"] ["SKU-L56" "SKU-L56"] ...)
```

You now know how to adjust your specs to create better generators, and when necessary, how to replace your generators with your own custom implementations. With these tools, you can create good argument specs for your functions and automatically test your functions with generative testing.

Wrapping Up

In this chapter, we've looked at how to specify the structure of both our data and our functions. You learned how to validate your data using a spec with `s/valid?`, as well as how to discover how it conformed with `s/conform` or how it failed with `s/explain`.

Additionally, you learned how to check for invalid calls with `stest/instrument` when working at the REPL or in tests and how to automatically test a properly spec'd function using `stest/check`.

The spec library requires a fair amount of extra effort in the code you write, particularly if you take the extra step to ensure that the `:args spec` of any function generates high-quality inputs for testing. However, you also gain lots of leverage by doing so, and it's entirely up to you how much of your code to spec.

Next, we change our focus to consider the management of state and how to write concurrent programs using Clojure.

Part III

Intermediate Topics

Now that you've learned the basics, it's time to explore Clojure's tools for managing state, concurrency, and behavioral abstraction. Learn how to access the underlying host via Java interop and extend the language with macros.

State and Concurrency

A *state* is the value of an identity at a point in time.

Quite a lot is packed into the previous sentence. Let's unpack the word *value* first. A value is an immutable data structure. When you can program entirely with values, life is easy, as we saw in [Chapter 5, Functional Programming, on page 97](#).

The flow of time makes things substantially more difficult. Are the New York Yankees the same now as they were last year? In 1927? The roster of the Yankees is an *identity* whose value changes over time.

Updating an identity does not destroy old values. In fact, updating an identity has no impact on existing values whatsoever. The Yankees could trade every player, or disband in a fit of boredom, without in any way altering our ability to think about any past Yankees we happen to care about.

Clojure's reference model clearly separates identities from values. Almost everything in Clojure is a value. For identities, Clojure provides four reference types:

- Refs manage *coordinated, synchronous* changes to shared state.
- Atoms manage *uncoordinated, synchronous* changes to shared state.
- Agents manage *asynchronous* changes to shared state.
- Vars manage *thread-local* state.

Each of these APIs is discussed in this chapter. At the end of the chapter, we'll develop a sample application. The Snake game demonstrates how to divide an application model into immutable and mutable components.

Before we start, let's review the intersection of state with concurrency and parallelism and look at the difficulty with traditional lock-based approaches.

Concurrency, Parallelism, and Locking

A concurrent program models more than one thing happening simultaneously. A parallel program takes an operation that could be sequential and chooses to break it into separate pieces that can execute concurrently to speed overall execution.

There are many reasons to write concurrent or parallel programs:

- For decades, performance improvements have come from packing more power into cores. Now, and for the near future, performance improvements will come from using more cores. Our hardware is itself more concurrent than ever, and systems must be concurrent to take advantage of this power.
- Expensive computations may need to execute in parallel on multiple cores (or multiple boxes) to complete in a timely manner.
- Tasks that are blocked waiting for a resource should stand down and let other tasks use available processors.
- User interfaces need to remain responsive while performing long-running tasks.
- Operations that are logically independent are easier to implement if the platform can recognize and take advantage of their independence.

Concurrency makes it glaringly obvious that more than one observer (for example, thread) may be looking at your data. This is a big problem for languages that complicate¹ value and identity. Such languages treat a piece of data as a bank ledger with only one line. Each new operation erases history, potentially corrupting the work of every other thread on the system.

While concurrency makes the challenges more obvious, it's a mistake to assume that multiple observers come into play only with concurrency. If your program ever has two variables that refer to the same data, those variables are different observers. If your program allows mutability at all, then you must think carefully about state.

Mutable languages tend to tackle the challenge by locking and defensive copying. Continuing the ledger analogy: the bank hires guards (locks) to supervise the activities of anybody using a ledger, and nobody is allowed to modify a ledger while anybody else is using it.

1. <https://www.youtube.com/watch?v=SxdOUGdseq4>

When the performance becomes really bad, the bank may even ask ledger readers to make their own private copies of the ledger so they can get out of the way and let transactions continue. These copies must still be supervised by the guards and also have to be reconciled later!

As irritating as this model sounds, it gets worse at the level of implementation detail. Choosing what and where to lock is a difficult task. If you get it wrong, all sorts of bad things can happen. *Race conditions* between threads can corrupt data. *Deadlocks* can stop an entire program from functioning at all. [Java Concurrency in Practice \[Goe06\]](#) covers these and other problems, plus their solutions, in detail. It's a terrific book, but it's difficult to read it and not ask yourself, "Isn't there another way?"

Clojure's model for state and identity solves these problems. The bulk of program code is functional. The small parts of the codebase that truly benefit from mutability are distinct and must explicitly select one of four reference models. Using these models, you can split your models into two layers:

- A *functional model* that has no mutable state. Most of your code will normally be in this layer, which is easier to read, easier to test, and easier to parallelize.
- *Reference models* for the parts of the application that you find more convenient to deal with using mutable state (despite its disadvantages)

Let's get started working with state in Clojure, using the simplest reference model: atoms.

Use Atoms for Uncoordinated, Synchronous Updates

Atoms allow updates of a single value, uncoordinated with anything else.

You create atoms with `atom`, which takes the initial state (required) and options like a validator function and a metadata map:

```
(atom initial-state options?)
; options include:
;   :validator validate-fn
;   :meta metadata-map
```

For example, you could create a reference to the current song in your music playlist:

```
(def current-track (atom "Mars, the Bringer of War"))
-> #'user/current-track
```


The atom wraps and protects access to its internal state. To read the contents of the reference, you can call `deref`:

```
(deref reference)
```

The `deref` function can be shortened to the `@` reader macro. Try using both `deref` and `@` to dereference `current-track`:

```
(deref current-track)
-> "Mars, the Bringer of War"

@current-track
-> "Mars, the Bringer of War"
```

Notice how in this example the Clojure model fits the real world. A track is an immutable entity. It doesn't change into another track when you're finished listening to it. But the *current* track is a reference to an entity, and it does change.

To set the value of an atom, just call `reset!`.

```
(reset! an-atom newval)
```

For example, you can set `current-track` to "Credo":

```
(reset! current-track "Credo")
-> "Credo"
```

What if you want to coordinate an update of both `current-track` and `current-composer` with an atom? The short answer is, "You can't." That's the difference between refs and atoms. If you need coordinated access, use a ref.

The longer answer is, "You can ... if you're willing to change the way you model the problem." What if you store the track title and composer in a map and then store the whole map in a single atom?

```
(def current-track (atom {:title "Credo" :composer "Byrd"}))
-> #'user/current-track
```

Now, you can update both values in a single `reset!`.

```
(reset! current-track {:title "Spem in Alium" :composer "Tallis"})
-> {:title "Spem in Alium", :composer "Tallis"}
```

Maybe you like to listen to several tracks in a row by the same composer. If so, you want to change the track title but keep the same composer. `swap!` will do the trick:

```
(swap! an-atom f & args)
```

swap! updates an-atom by calling function f on the current value of an-atom, plus any additional args.

To change just the track title, use swap! with assoc to update only the :title:

```
(swap! current-track assoc :title "Sancte Deus")
-> {:title "Sancte Deus", :composer "Tallis"}
```

swap! returns the new value. Calls to swap! might be retried if other threads are attempting to modify the same atom. So, the function you pass to swap! should have no side effects.

The most important caveat with atom is that you can only hold and update a single (possibly compound) value at a time. To coordinate updates of multiple independent state values at the same time, we need refs.

Refs and Software Transactional Memory

You create a ref with the ref function, similar to atom:

```
(ref initial-state options?)
; options include:
;   :validator validate-fn
;   :meta metadata-map
;   :min-history (default 0)
;   :max-history (default 10)
```

Returning to our music player example, we could create a current-track ref instead:

```
(def current-track (ref "Mars, the Bringer of War"))
```

ref-set

You can change where a reference points with ref-set:

```
(ref-set reference new-value)
```

Call ref-set to listen to a different track:

```
(ref-set current-track "Venus, the Bringer of Peace")
| Execution error (IllegalStateException) at user/eval3 (REPL:1).
| No transaction running
```

Oops. Because refs are mutable, you must protect their updates. In many languages, you would use a *lock* for this purpose. In Clojure, you can use a *transaction*. Transactions are wrapped in a dosync:

```
(dosync & exprs)
```

Wrap your ref-set with a dosync, and all is well.

```
(dosync (ref-set current-track "Venus, the Bringer of Peace"))
-> "Venus, the Bringer of Peace"
```

The current-track reference now refers to a different track.

Transactional Properties

Like database transactions, Software Transactional Memory (STM) allows programmers to describe reads and writes to stateful references in the scope of a transaction. These transactions guarantee some important properties:

- Updates are *atomic*. If you update more than one ref in a transaction, the cumulative effect of all of the updates will appear as a single instantaneous event to anyone not inside your transaction.
- Updates are *consistent*. Refs can specify validation functions. If any of these functions fail, the entire transaction will fail.
- Updates are *isolated*. Running transactions can't see partially completed results from other transactions.

Databases provide the additional guarantee that updates are *durable*. Because Clojure's transactions are in-memory transactions, Clojure does not guarantee that updates are durable. If you want a durable transaction in Clojure, you should use a database.

Together, the four transactional properties are called ACID.² Relational databases typically provide ACID; Clojure's STM provides ACI but does not provide durability.

If you change more than one ref in a single transaction, the changes are all coordinated to “happen at the same time” from the perspective of any code outside the transaction. So, you can make sure that updates to current-track and current-composer are *coordinated*:

```
(def current-track (ref "Venus, the Bringer of Peace"))
-> #'user/current-track
(def current-composer (ref "Holst"))
-> #'user/current-composer

(dosync
  (ref-set current-track "Credo")
  (ref-set current-composer "Byrd"))
-> "Byrd"
```

2. <https://en.wikipedia.org/wiki/ACID>

Because the updates are in a transaction, no other thread will ever see an updated track with an out-of-date composer, or vice versa.

alter

The current-track example is deceptively easy, because updates to the ref are totally independent of any previous state. Let's build a more complex example, where transactions need to update existing information. A simple chat application fits the bill. First, create a message record that has a sender and some text:

```
src/examples/chat.clj
(defrecord Message [sender text])
```

Now, you can create messages by instantiating the record:

```
(->Message "Aaron" "Hello")
-> #:user.Message{:sender "Aaron", :text "Hello"}
```

Users of the chat application want to see the most recent message first, so a list is a good choice. Create a messages reference that points to an initially empty list:

```
(def messages (ref []))
```

Now, you need a function to add a new message to the front of messages. You could simply deref to get the list of messages, cons the new message, and then ref-set the updated list back into messages:

```
; bad idea
(defn naive-add-message [msg]
  (dosync (ref-set messages (cons msg @messages))))
```

But there's a better option. Why not perform the read and update in a single step? Clojure's `alter` will apply an update function to a referenced object within a transaction:

```
(alter ref update-fn & args...)
```

`alter` returns the new value of the ref within the transaction. When a transaction successfully completes, the ref will take on its last in-transaction value. Using `alter` instead of `ref-set` makes the code more readable:

```
(defn add-message [msg]
  (dosync (alter messages conj msg)))
```

Notice that the update function is `conj` (short for “conjoin”), not `cons`. This is because `conj` takes arguments in an order suitable for use with `alter`:

```
(cons item sequence)
(conj sequence item)
```

The `alter` function calls its `update-fn` with the current reference value as its first argument, as `conj` expects. If you plan to write your own update functions, they should follow the same structure as `conj`:

```
(your-func thing-that-gets-updated & optional-other-args)
```

Try adding a few messages to see that the code works as expected:

```
(add-message (->Message "user 1" "hello"))
-> (#:user.Message{:sender "user 1", :text "hello"})

(add-message (->Message "user 2" "howdy"))
-> (#:user.Message{:sender "user 2", :text "howdy"}
    #:user.Message{:sender "user 1", :text "hello"})
```

`alter` is the workhorse of Clojure's STM and the primary means of updating refs. But if you know a little about how the STM works, you may be able to optimize your transactions in certain scenarios.

How STM Works: MVCC

Clojure's STM uses a technique called Multiversion Concurrency Control (MVCC), which is also used in several major databases. Here's how MVCC works in Clojure.

Transaction A begins by taking a *point*, which is simply a number that acts as a unique timestamp in the STM world. Transaction A has access to its own *effectively private* copy of any reference it needs, associated with the point. Clojure's persistent data structures ([Persistent Data Structures, on page 98](#)) make it cheap to provide these effectively private copies.

During Transaction A, operations on a ref work against (and return) the transaction's private copy of the ref's data, called the *in-transaction value*.

If at any point the STM detects that another transaction has already set/alterd a ref that Transaction A wants to set/alter, Transaction A is forced to retry. If you throw an exception out of the `dosync` block, then Transaction A aborts *without* a retry.

If and when Transaction A commits, its heretofore private writes will become visible to the world, associated with a single point in the transaction timeline.

Sometimes the approach implied by `alter` is too cautious. What if you *don't care* that another transaction altered a reference in the middle of your transaction? If in such a situation you'd be willing to commit your changes anyway, you can beat `alter`'s performance with `commute`.

commute

commute is a specialized variant of alter allowing for more concurrency:

```
(commute ref update-fn & args...)
```

Of course, there's a trade-off. Commutes are so named because they must be *commutative*. That is, updates must be able to occur in any order. This gives the STM system freedom to reorder commutes.

To use commute, replace alter with commute in your implementation of add-message:

```
(defn add-message-commute [msg]
  (dosync (commute messages conj msg)))
```

commute returns the new value of the ref. However, the last in-transaction value you see from a commute will *not* always match the end-of-transaction value of a ref, because of reordering. If another transaction sneaks in and alters a ref that you're trying to commute, the STM will not restart your transaction. Instead, it will run your commute function again, out of order. Your transaction will *never even see* the ref value that your commute function finally ran against.

Since Clojure's STM can reorder commutes behind your back, you can only use them when you don't care about ordering. Literally speaking, this isn't true for a chat application. The list of messages most certainly has an order, so if two message adds get reversed, the resulting list will not correctly show the order in which the messages arrived.

Practically speaking, chat message updates are *commutative enough*. STM-based reordering of messages will likely happen on time scales of microseconds or less. For users of a chat application, there are already reorderings on much larger time scales due to network and human latency. (Think about times that you have “spoken out of turn” in an online chat because another speaker's message hadn't reached you yet.) Since these larger reorderings are unfixable, it's reasonable for a chat application to ignore the smaller reorderings that might bubble up from Clojure's STM.

Prefer alter

Many updates are not commutative. For example, consider a counter that returns an increasing sequence of numbers. You might use such a counter to build unique IDs in a system. The counter can be a simple reference to a number:

```
src/examples/concurrency.clj
(def counter (ref 0))
```

You should not use `commute` to update the counter. `commute` returns the in-transaction value of the counter at the time of the commute, but reorderings could cause the actual end-of-transaction value to be different. This could lead to more than one caller getting the same counter value. Instead, use `alter`:

```
(defn next-counter [] (dosync (alter counter inc)))
```

Try calling `next-counter` a few times to verify that the counter works as expected:

```
(next-counter)
-> 1

(next-counter)
-> 2
```

In general, you should prefer `alter` over `commute`. Its semantics are easy to understand and error-proof. `commute`, on the other hand, requires that you think carefully about transactional semantics. If you use `alter` when `commute` would suffice, the worst thing that might happen is performance degradation. But if you use `commute` when `alter` is required, you'll introduce a subtle bug that's difficult to detect with automated tests.

Adding Validation to Refs

Database transactions maintain consistency through various integrity checks. You can do something similar with Clojure's transactional memory, by specifying a validation function when you create a ref:

```
(ref initial-state options*)
; options include:
; :validator validate-fn
; :meta metadata-map
```

The options to `ref` include an optional validation function that can throw an exception to prevent a transaction from completing. Note that `options` is *not* a map; it's a sequence of key/value pairs spliced into the function call.

Continuing the chat example, add a validation function to the messages reference that guarantees that all messages have non-nil values for `:sender` and `:text`:

```
src/examples/chat.clj
(defn valid-message? [msg]
  (and (:sender msg) (:text msg)))

(def validate-message-list #(every? valid-message? %))

(def messages (ref () :validator validate-message-list))
```

This validation acts like a key constraint on a table in a database transaction. If the constraint fails, the entire transaction rolls back. Try adding an ill-formed message, such as a simple string:

```
(add-message "not a valid message")
| Execution error (IllegalStateException) at examples.chat/add-message.
| Invalid reference state
@messages
-> ()
```

Messages that match the constraint are no problem:

```
(add-message (->Message "Aaron" "Real Message"))
-> (#:user.Message{:sender "Aaron", :text "Real Message"})
```

Both refs and atoms perform synchronous updates. When the update function returns, the value is already changed. If you don't need this level of control and can tolerate updates happening asynchronously at some later time, prefer an agent.

Use Agents for Asynchronous Updates

Some applications have tasks that can proceed independently with minimal coordination between tasks. Clojure *agents* support this style of task.

Agents have much in common with refs. Like refs, you create an agent by wrapping some piece of initial state:

```
(agent initial-state)
```

Create a counter agent that wraps an initial count of zero:

```
(def counter (agent 0))
-> #'user/counter
```

Once you have an agent, you can send the agent a function to update its state. `send` queues an update-fn to run later, on a thread in a thread pool:

```
(send agent update-fn & args)
```

Sending to an agent is much like commuting a ref. Tell the counter to inc:

```
(send counter inc)
-> #object[clojure.lang.Agent 0x7ae288e1 {:status :ready, :val 1}]
```

Notice that the call to `send` doesn't return the new value of the agent, returning instead the agent itself. That's because `send` *does not know* the new value. After `send` queues the `inc` to run later, it returns immediately.

Although `send` does not know the new value of an agent, the REPL *might* know. Depending on whether the agent thread or the REPL thread runs first, you might see a 1 or a 0 for the `:val` in the previous output.

You can check the current value of an agent with `deref/@`, just as you would a ref. By the time you get around to checking the counter, the `inc` will almost certainly have completed on the thread pool, raising the value to 1:

```
@counter
-> 1
```

If the race condition between the REPL and the agent thread bothers you, there is a solution. If you want to be sure that the agent has completed the actions *you sent* to it, you can call `await` or `await-for`:

```
(await & agents)
```

```
(await-for timeout-millis & agents)
```

These functions cause the current thread to block until all actions sent from the current thread or agent have completed. `await-for` returns `nil` if the timeout expires and returns a non-`nil` value otherwise. `await` has no timeout, so be careful: `await` is willing to wait forever.

Validating Agents and Handling Errors

Agents have other points in common with refs. They also can take a validation function:

```
(agent initial-state options*)
; options include:
; :validator validate-fn
; :meta metadata-map
; :error-handler handler-fn
; :error-mode mode-keyword (:continue or :fail)
```

Recreate the counter with a validator that ensures it's a number:

```
(def counter (agent 0 :validator number?))
-> #'user/counter
```

Try to set the agent to a value that's not a number by passing an update function that ignores the current value and simply returns a string:

```
(send counter (fn [_] "boo"))
-> #object[clojure.lang.Agent 0x3a46c14f {:status :ready, :val 0}]
```

Everything looks fine (so far) because `send` still returns immediately. When the agent tries to update itself on a pooled thread, it encounters an exception while applying the action. Agents have two possible error modes—`:fail` and `:continue`. If no `:error-handler` is supplied when the agent is created, the error mode is set to `:fail`, and any exception that occurs during an action or during validation puts the agent into an exceptional state.

When an agent is in this failed state, it can still be dereferenced and will return the last value from before the failed action. To discover the last error on an agent, call `agent-error` which returns either the failure or `nil` if not in a failed state:

```
(agent-error counter)
-> #error {
  :cause "Invalid reference state"
  :via [{:type java.lang.IllegalStateException
        :message "Invalid reference state"
        :at [clojure.lang.ARef validate "ARef.java" 33]}]
  :trace
    [[clojure.lang.ARef validate "ARef.java" 33]
     ... ]]}
```

All new actions are queued until the agent is restarted using `restart-agent`. Once an agent has errors, all subsequent attempts to query the agent return an error. You can make the agent active again by calling `restart-agent`:

```
(restart-agent counter 0)
-> 0

@counter
-> 0
```

If an `:error-handler` is supplied when the agent is created, the agent will instead be in error mode `:continue`. When an error occurs, the error handler is invoked, and the agent then continues as if no error occurred.

```
(defn handler [agent err]
  (println "ERR!" (.getMessage err)))
-> #'user/handler

(def counter2 (agent 0 :validator number? :error-handler handler))
-> #'user/counter2

(send counter2 (fn [_] "boo"))
| ERR! Invalid reference state
-> #object[clojure.lang.Agent 0x5ba87f7f {:status :ready, :val 0}]

(send counter2 inc)
-> #object[clojure.lang.Agent 0x5ba87f7f {:status :ready, :val 0}]

@counter2
-> 1
```

Now that you know the basics of agents, let's use them in conjunction with refs and transactions.

Including Agents in Transactions

Transactions should not have side effects, because Clojure may retry a transaction an arbitrary number of times. However, sometimes you *want* a side effect when a transaction succeeds. Agents provide a solution. If you send an action to an agent from within a transaction, that action is sent exactly once, if and only if the transaction succeeds.

As an example of where this would be useful, consider an agent that writes to a file when a transaction succeeds. You could combine such an agent with the chat example from [commute, on page 163](#) to automatically back up chat messages. First, create a backup-agent that stores the filename to write to:

```
src/examples/concurrency.clj
```

```
(def backup-agent (agent "output/messages-backup.clj"))
```

Then, create a modified version of add-message. The new function add-message-with-backup should do two additional things:

- Grab the return value of commute, which is the current database of messages, in a let binding.
- While still inside a transaction, send an action to the backup agent that writes the message database to filename. For simplicity, have the action function return filename so that the agent uses the same filename for the next backup.

```
(defn add-message-with-backup [msg]
  (dosync
    (let [snapshot (commute messages conj msg)]
      (send-off backup-agent (fn [filename]
                              (spit filename snapshot)
                              filename))
      snapshot)))
```

The new function has one other critical difference: it calls send-off instead of send to communicate with the agent. send-off is a variant of send for actions that expect to block, as a file write might do. send-off actions get their own expandable thread pool. Never send a blocking function, or you may unnecessarily prevent other agents from making progress.

Try adding some messages using add-message-with-backup:

```
(add-message-with-backup (->Message "John" "Message One"))
-> (:user.Message{:sender "John", :text "Message One"})
```

```
(add-message-with-backup (->Message "Jane" "Message Two"))
-> (#:user.Message{:sender "Jane", :text "Message Two"}
    #:user.Message{:sender "John", :text "Message One"})
```

You can check both the in-memory messages as well as the backup file messages-backup to verify that they contain the same structure.

You could enhance the backup strategy in this example in various ways. You could provide the option to back up less often than on every update, or back up only information that has changed since the last backup.

Since Clojure's STM provides the ACI properties of ACID, and since writing to a file provides the D (“durability”), it's tempting to think that STM plus a backup agent equals a database. This is *not* the case. A Clojure transaction promises only to send/send-off an action to the agent; it does not actually *perform* the action under the ACI umbrella. So, for example, a transaction could complete, and then someone could unplug the power cord before the agent writes to the database. The moral is simple. If your problem calls for a real database, use a real database.

The Unified Update Model

As you've seen, refs, atoms, and agents all provide functions for updating their state by applying a function to their previous state. This unified model for handling shared state is one of the central concepts of Clojure. The unified functions for each reference type are summarized in the following table.

Update Mechanism	Ref Function	Atom Function	Agent Function
Function application	alter	swap!	send-off
Function (commutative)	commute	N/A	N/A
Function (nonblocking)	N/A	N/A	send
Simple setter	ref-set	reset!	N/A

The unified update model is by far the most important way to update refs, atoms, and agents. The ancillary functions, on the other hand, are optimizations and options that stem from the semantics peculiar to each API:

- The opportunity for the commute optimization arises when coordinating updates. Since only refs provide coordinated updates, commute makes sense only for refs.
- Updates to refs and atoms take place on the thread they are called on, so they provide no scheduling options. Agents update later, on a thread pool, making blocking/nonblocking a relevant scheduling option.

Clojure's final reference type, the `var`, is a different beast entirely. Vars do not participate in the unified update model and are instead used to manage thread-local, private state.

Managing Per-Thread State with Vars

When you call `def` or `defn`, you create a *var*. In all of the examples so far in the book, you pass an initial value to `def`, which becomes the *root binding* for the var. For example, the following code creates a var `*foo*` with binding of 10, however we mark it as `^:dynamic` with metadata:

```
(def ^:dynamic *foo* 10)
-> #'user/*foo*
```

While it is not required, dynamic vars are usually named with leading and trailing `*` to alert users that the var is dynamically scoped.

The binding of `*foo*` is shared by all threads. You can check the value of `*foo*` on your own thread:

```
*foo*
-> 10
```

You can also verify the value of `*foo*` from another thread by running it in a future on a background thread (output may interleave):

```
user=> (def f (future (println *foo*)))
-> #'user/f
| 10
```

By default, vars are not dynamic and only have a root binding. However, you can set a *thread-local* binding for a dynamic var with the `binding` macro:

```
(binding [bindings] & body)
```

Bindings have *dynamic scope*. In other words, a binding is visible anywhere a thread's execution takes it, until the thread exits the scope where the binding began. A binding is not visible to any other thread.

Structurally, a binding looks a lot like a let. To observe a dynamic binding across function calls, let's create a simple function that prints the value of `*foo*`:

```
(defn print-foo [] (println *foo*))
-> #'user/print-foo
```

Now, try calling `print-foo` from inside a binding:

```
(binding [*foo* "bound foo"] (print-foo))
| bound foo
```

The binding stays in effect through any nested function invocations, so `print-foo` prints `bound foo`.

Acting at a Distance

One place where Clojure uses dynamic binding to allow thread-specific overrides are the standard I/O streams `*in*`, `*out*`, and `*err*`. Dynamic bindings enable *action at a distance*. When you change a dynamic binding, you can change the behavior of distant functions without changing any function arguments.

One kind of action at a distance is temporarily augmenting the behavior of a function. In some languages, this would be classified as aspect-oriented programming; in Clojure, it's accomplished via dynamic binding.

As an example, imagine that you have a function that performs an expensive calculation. To simulate this, write a function named `slow-double` that sleeps for a 10th of a second and then doubles its input.

```
(defn ^:dynamic slow-double [n]
  (Thread/sleep 100)
  (* n 2))
```

Next, write a function named `calls-slow-double` that calls `slow-double` for each item in `[1 2 1 2 1 2]`:

```
(defn calls-slow-double []
  (map slow-double [1 2 1 2 1 2]))
```

Time a call to `calls-slow-double`. With six internal calls to `slow-double`, it should take a little over six-tenths of a second. Note that you'll have to run through the result with `dorun`; otherwise, Clojure's `map` will outsmart you by immediately returning a lazy sequence.

```
(time (dorun (calls-slow-double)))
"Elapsed time: 601.418 msecs"
```

Reading the code, you can tell that `calls-slow-double` is slow because it does the same work over and over again. One times two is two, no matter how many times you ask.

Calculations such as `slow-double` are good candidates for *memoization*, which we explored in [Shortcutting Recursion with Memoization, on page 120](#). When you memoize a function, it keeps a cache mapping past inputs to past outputs. If subsequent calls hit the cache, they return almost immediately. Thus, you're trading space (the cache) for time (calculating the function again for the same inputs).

Clojure provides `memoize`, which takes a function and returns a memoization of that function:

```
(memoize function)
```

`slow-double` is a great candidate for memoization, but it isn't memoized yet, and clients like `calls-slow-double` already use the slow, unmemoized version. With dynamic binding, this is no problem. Create a binding to a memoized version of `slow-double` and call `calls-slow-double` from within the binding.

```
(defn demo-memoize []
  (time
    (dorun
      (binding [slow-double (memoize slow-double)]
        (calls-slow-double))))))
```

With the memoized version of `slow-double`, `calls-slow-double` runs three times faster, completing in about two-tenths of a second:

```
(demo-memoize)
"Elapsed time: 203.115 msecs"
```

This example demonstrates the power and the danger of action at a distance. By dynamically rebinding a function such as `slow-double`, you change the behavior of *other* functions such as `calls-slow-double` without their knowledge or consent. With lexical binding forms such as `let`, it's easy to see the entire range of your changes. Dynamic binding is not so simple. It can change the behavior of other forms in other files, far from the point in your source where the binding occurs.

Used occasionally, dynamic binding has great power. But it should not become your primary mechanism for extension or reuse. Functions that use dynamic bindings are not pure functions and can quickly lose the benefits of Clojure's functional style.

Working with Java Callback APIs

Several Java APIs depend on callback event handlers. UI frameworks such as Swing use event handlers to respond to user input. XML parsers such as SAX depend on the user implementing a callback handler interface.

These callback handlers are written with mutable objects in mind. Also, they tend to be single-threaded. In Clojure, the best way to meet such APIs halfway is to use dynamic bindings. This involves mutable references that feel almost like variables, but because they're used in a single-threaded setting, they don't present any concurrency problems.

Clojure provides the `set!` special form for setting a thread-local dynamic binding:

```
(set! var-symbol new-value)
```

`set!` should be used rarely. One of the only places in the entire Clojure core that uses `set!` is the Clojure implementation of a SAX `ContentHandler`.

A `ContentHandler` receives callbacks as a parser encounters various bits of an XML stream. In nontrivial scenarios, the `ContentHandler` needs to keep track of *where it is* in the XML stream: the current stack of open elements, current character data, and so on.

In Clojure-speak, you can think of a `ContentHandler`'s current position as a mutable pointer to a specific spot in an immutable XML stream. It's unnecessary to use references in a `ContentHandler`, since everything happens on a single thread. Instead, Clojure's `ContentHandler` uses dynamic variables and `set!`. Here is the relevant detail:

```
; extracted from Clojure's xml.clj to focus on dynamic variable usage
(startElement
  [uri local-name q-name #^Attributes atts]
  ; details omitted
  (set! *stack* (conj *stack* *current*))
  (set! *current* e)
  (set! *state* :element))
nil)

(endElement
  [uri local-name q-name]
  ; details omitted
  (set! *current* (push-content (peek *stack*) *current*))
  (set! *stack* (pop *stack*))
  (set! *state* :between))
nil)
```

A SAX parser calls `startElement` when it encounters an XML start tag. The callback handler updates three thread-local variables. The `*stack*` is a stack of all of the elements the current element is nested inside. The `*current*` is the current element, and the `*state*` keeps track of what kind of content is inside. (This is important primarily when inside character data, which is not shown here.)

`endElement` reverses the work of `startElement` by popping the `*stack*` and placing the top of the `*stack*` in `*current*`.

It's worth noting that this style of coding is the industry norm: objects are mutable, and programs are single-threadedly oblivious to the possibility of concurrency. Clojure permits this style as an explicit special case, and you should use it for interop purposes only.

The `ContentHandler`'s use of `set!` does not leak mutable data into the rest of Clojure. Clojure uses the `ContentHandler` implementation to build an immutable Clojure structure.

You have now seen four different models for dealing with state. And since Clojure is built atop Java, you can also use Java's lock-based model. The models and their uses are summarized in the following table.

Model	Usage	Functions
Atoms	Uncoordinated, synchronous updates	Pure
Refs and STM	Coordinated, synchronous updates	Pure
Agents	Uncoordinated, asynchronous updates	Any
Vars	Thread-local dynamic scopes	Any
Java locks	Coordinated, synchronous updates	Any

Let's put these models to work in designing a small but complete application.

A Clojure Snake

The Snake game features a player-controlled snake that moves around a game grid, hunting for an apple. When your snake eats an apple, it grows longer by a segment, and a new apple appears. If your snake reaches a certain length, you win. But if your snake crosses over its own body, you lose.

Before you start building your own snake, take a minute to try the completed version. Follow the instructions in the README file at the root of the sample code for the book to start a REPL, then enter the following:

```
(require '[examples.snake :refer :all])
(game)
```

Select the Snake window and use the arrow keys to control your snake.

Our design for the snake takes advantage of Clojure's functional nature and its support for explicit mutable state by dividing the game into three layers:

- The *functional model* will use pure functions to model as much of the game as possible.

- The *mutable model* will handle the mutable state of the game. The mutable model will use one or more of the reference models discussed in this chapter. Mutable state is much harder to test, so we'll keep this part small.
- The *GUI* will use Swing to draw the game and to accept input from the user.

These layers will make the Snake easy to build, test, and maintain.

As you work through this example, you'll be recreating the file `src/examples/snake.clj` in the sample code. Use the following imports to give you the Swing classes you'll need:

```
src/examples/snake.clj
(ns examples.snake
  (:import (java.awt Color Dimension Graphics)
           (javax.swing JPanel JFrame Timer JOptionPane)
           (java.awt.event ActionListener KeyListener KeyEvent
            WindowAdapter WindowEvent)))
```

Now, you're ready to build the functional model.

The Functional Model

First, create a set of constants to describe time, space, and motion:

```
(def width 75)           ;; Game board width
(def height 50)          ;; Game board height
(def point-size 10)      ;; Pixels per point
(def turn-millis 50)     ;; Time (ms) between turns
(def win-length 5)       ;; Snake segments to win
(def dirs { KeyEvent/VK_LEFT  [-1 0]
            KeyEvent/VK_RIGHT [ 1 0]
            KeyEvent/VK_UP    [ 0 -1]
            KeyEvent/VK_DOWN  [ 0 1]})
```

The `dirs` maps symbolic constants for the four directions to their vector equivalents. Since Swing already defines the `VK_` constants for different directions, we'll reuse them here rather than define our own.

Next, create some basic math functions for the game:

```
(defn add-points [& pts]
  (vec (apply map + pts)))

(defn point-to-screen-rect [pt]
  (map #(* point-size %)
       [(pt 0) (pt 1) 1 1]))
```

Other Snake Implementations

There's more than one way to skin a snake. You may enjoy comparing the snake presented here with these other snakes:

- David Van Horn's Snake,^a written in Typed Scheme, has no mutable state.
- Dale Vaillancourt's Worm Game^b includes some verifications using the theorem prover ACL2.
- Mark Volkmann wrote a Clojure Snake^c designed for readability.

Each of the snake implementations has its own distinctive style. What would *your* style look like?

- <https://planet.racket-lang.org/package-source/dvanhorn/snake.plt/2/1/main.ss>
- <https://dracula-lang.github.io/worm.html>
- <https://mvolkmann.github.io/clojure/ClojureSnake.html>

The `add-points` function adds points together. You can use `add-points` to calculate the new position of a moving game object. For example, you can move an object at `[10, 10]` left by one:

```
(add-points [10 10] [-1 0])
-> [9 10]
```

`point-to-screen-rect` converts a point in game space to a rectangle on the screen:

```
(point-to-screen-rect [5 10])
-> (50 100 10 10)
```

Next, let's write a function to create a new apple:

```
(defn create-apple []
  {:location [(rand-int width) (rand-int height)]
   :color (Color. 210 50 90)
   :type :apple})
```

Apples occupy a single point, the `:location`, which is guaranteed to be on the game board. Snakes are a bit more complicated:

```
(defn create-snake []
  {:body (list [1 1])
   :dir [1 0]
   :type :snake
   :color (Color. 15 160 70)})
```

Because a snake can occupy multiple points on the board, it has a `:body`, which is a list of points. Also, snakes are always in motion in some direction expressed by `:dir`.

Next, create a function to move a snake. This should be a pure function, returning a new snake. Also, it should take a grow option, allowing the snake to grow after eating an apple.

```
(defn move [{:keys [body dir] :as snake} & grow]
  (assoc snake :body (cons (add-points (first body) dir)
                           (if grow body (butlast body)))))
```

move uses a fairly complex binding expression. The `{:keys [body dir]}` part makes the snake's body and dir available as their own bindings, and the `:as snake` part binds snake to the entire snake. The function proceeds as follows:

1. `add-points` creates a new point, which is the head of the original snake offset by the snake's direction of motion.
2. `cons` adds the new point to the front of the snake. If the snake is growing, the entire original snake is kept. Otherwise, it keeps all of the original snake except the last segment (`butlast`).
3. `assoc` returns a new snake, which is a copy of the old snake but with an updated `:body`.

Test move by moving and growing a snake:

```
(move (create-snake))
-> {:body ([2 1]), ; etc.}

(move (create-snake) :grow)
-> {:body ([2 1] [1 1]), ; etc.}
```

Write a `win?` function to test whether a snake has won the game:

```
(defn win? [{:body :body}]
  (>= (count body) win-length))
```

Test `win?` against different body sizes. Note that `win?` binds only the `:body`, so you don't need a "real" snake, just anything with a `body`:

```
(win? {:body [[1 1]]})
-> false

(win? {:body [[1 1] [1 2] [1 3] [1 4] [1 5]]})
-> true
```

A snake loses if its head ever comes into contact with the rest of its body. Write a `head-overlaps-body?` function to test for this, and use it to define `lose?`:

```
(defn head-overlaps-body? [{[head & body] :body}]
  (contains? (set body) head))

(def lose? head-overlaps-body?)
```

Test `lose?` against overlapping and nonoverlapping snake bodies:

```
(lose? {:body [[1 1] [1 2] [1 3]])
-> false

(lose? {:body [[1 1] [1 2] [1 1]])
-> true
```

A snake eats an apple if its head occupies the apple's location. Define an `eats?` function to test this:

```
(defn eats? [{[snake-head] :body} {apple :location}]
  (= snake-head apple))
```

Notice how clean the body of the `eats?` function is. All the work is done in the bindings: `{[snake-head] :body}` binds `snake-head` to the first element of the snake's `:body`, and `{apple :location}` binds `apple` to the apple's `:location`. Test `eats?` from the REPL:

```
(eats? {:body [[1 1] [1 2]]} {:location [2 2]})
-> false

(eats? {:body [[2 2] [1 2]]} {:location [2 2]})
-> true
```

Finally, you need some way to turn the snake, updating its `:dir`:

```
(defn turn [snake newdir]
  (assoc snake :dir newdir))
```

`turn` returns a new snake, with an updated direction:

```
(turn (create-snake) [0 -1])
-> {:body ([1 1]), :dir [0 -1], ; etc.
```

All of the code you've written so far is part of the functional model of the Snake game. It's easy to understand in part because it has no local variables and no mutable state. As you'll see in the next section, the amount of *mutable* state in the game is quite small. (It's even possible to implement the Snake with *no* mutable state, but that's not the purpose of this demo.)

Building a Mutable Model with STM

The mutable state of the Snake game can change in only three ways:

- A game can be reset to its initial state.
- Every turn, the snake updates its position. If it eats an apple, a new apple is placed.
- A snake can turn.

We'll implement each of these changes as functions that modify Clojure refs inside a transaction. That way, changes to the position of the snake and the apple will be synchronous and coordinated.

reset-game is trivial:

```
(defn reset-game [snake apple]
  (dosync (ref-set apple (create-apple))
          (ref-set snake (create-snake))))
  nil)
```

You can test reset-game by passing in some refs and then checking that they dereference to a snake and an apple:

```
(def test-snake (ref nil))
(def test-apple (ref nil))

(reset-game test-snake test-apple)
-> nil

@test-snake
-> {:body ([1 1]), :dir [1 0], ; etc.

@test-apple
-> {:location [52 8], ; etc.
```

update-direction is even simpler than that; it's just a trivial wrapper around the functional turn:

```
(defn update-direction [snake newdir]
  (when newdir (dosync (alter snake turn newdir))))
```

Try turning your test-snake to move in the “up” direction:

```
(update-direction test-snake [0 -1])
-> {:body ([1 1]), :dir [0 -1], ; etc.
```

The most complicated mutating function is update-positions. If the snake eats the apple, a new apple is created, and the snake grows. Otherwise, the snake simply moves:

```
(defn update-positions [snake apple]
  (dosync
    (if (eats? @snake @apple)
      (do (ref-set apple (create-apple))
          (alter snake move :grow))
      (alter snake move)))
  nil)
```

To test update-positions, reset the game:

```
(reset-game test-snake test-apple)
-> nil
```

Then, move the apple into harm's way, under the snake:

```
(dosync (alter test-apple assoc :location [1 1]))
-> {:location [1 1], ; etc.
```

Now, after you update-positions, you should have a bigger, two-segment snake:

```
(update-positions test-snake test-apple)
-> nil

(:body @test-snake)
-> ([2 1] [1 1])
```

And that is all of the mutable state of the Snake world: three functions, about a dozen lines of code.

The Snake GUI

The Snake GUI consists of functions that paint screen objects, respond to user input, and set up the various Swing components. Since snakes and apples are drawn from simple points, the painting functions are simple. The fill-point function fills in a single point:

```
(defn fill-point [^Graphics g pt color]
  (let [[x y width height] (point-to-screen-rect pt)]
    (.setColor g color)
    (.fillRect g x y width height)))
```

The paint multimethod knows how to paint snakes and apples:

```
Line 1 (defmulti paint (fn [g object & _] (:type object)))
2
3 (defmethod paint :apple [g {:keys [location color]}}
4   (fill-point g location color))
5
6 (defmethod paint :snake [g {:keys [body color]}}
7   (doseq [point body]
8     (fill-point g point color)))
```

paint takes two required arguments: g is a java.awt.Graphics instance, and object is the object to be painted. The defmulti includes an optional rest argument so that future implementations of paint have the option of taking more arguments. (See [Defining Multimethods, on page 203](#) for an in-depth description of defmulti.) On line 3, the :apple method of paint binds the location and color of the apple and uses them to paint a single point on the screen. On line 6, the :snake method binds the snake's body and color and then uses doseq to paint each point in the body.

The meat of the UI is the game-panel function, which creates a Swing JPanel with handlers for painting the game, updating on each timer tick, and responding to user input:

```

Line 1 (defn game-panel [frame snake apple]
-   (proxy [JPanel ActionListener KeyListener] []
-     (paintComponent [<^Graphics g]
-       (proxy-super paintComponent g)
5       (paint g @snake)
-       (paint g @apple))
-     (actionPerformed [e]
-       (update-positions snake apple)
-       (when (lose? @snake)
10         (reset-game snake apple)
-       (JOptionPane/showMessageDialog frame "You lose!"))
-       (when (win? @snake)
-         (reset-game snake apple)
-         (JOptionPane/showMessageDialog frame "You win!"))
15       (.repaint ^JPanel this))
-     (keyPressed [<^KeyEvent e]
-       (update-direction snake (dirs (.getKeyCode e))))
-     (getPreferredSize []
-       (Dimension. (* (inc width) point-size)
20                     (* (inc height) point-size)))
-     (keyReleased [e])
-     (keyTyped [e])))

```

game-panel is long but simple. It uses proxy to create a panel with a set of Swing callback methods:

- Swing calls paintComponent (line 3) to draw the panel. paintComponent calls proxy-super to invoke the normal JPanel behavior, and then it paints the snake and the apple.
- Swing will call actionPerformed (line 7) on every timer tick. actionPerformed updates the positions of the snake and the apple. If the game is over, the program displays a dialog and resets the game. Finally, it triggers a repaint with (.repaint this).
- Swing calls keyPressed (line 16) in response to keyboard input. keyPressed calls update-direction to change the snake's direction. (If the keyboard input is not an arrow key, the dirs function returns nil and update-direction does nothing.)
- The game panel ignores keyReleased and keyTyped.

The game function creates a new game:

```

Line 1 (defn game []
-   (let [snake (ref (create-snake))
-         apple (ref (create-apple))
-         frame (JFrame. "Snake")
5         panel ^JPanel (game-panel frame snake apple)
-         timer (Timer. turn-millis panel)]

```



```

-   (doto panel
-     (.setFocusable true)
-     (.addKeyListener panel))
10  (doto frame
-    (.add panel)
-    (.pack)
-    (.setDefaultCloseOperation JFrame/DISPOSE_ON_CLOSE)
-    (.addWindowListener
15      (proxy [WindowAdapter] []
-        (windowClosing [^WindowEvent e]
-          (.stop timer)
-          (.dispose frame))))
-    (.setVisible true))
20  (.start timer)
-  [snake, apple, timer]))

```

On line 2, game creates all of the necessary game objects: the mutable model objects snake and apple and the UI components frame, panel, and timer. Lines 7 and 10 perform boilerplate initialization of the panel and frame. Line 20 starts the game by kicking off the timer.

Line 21 returns a vector with the snake, apple, and time. This is for convenience when testing at the REPL; you can use these objects to move the snake and apple or to start and stop the game.

To start the game, use the snake namespace at the REPL and run game. If you entered the code yourself, you can use the namespace name you picked (examples.reader in the instructions); otherwise, you can use the completed sample at examples.snake:

```

(require '[examples.snake :refer :all])
(game)

```

The game window may appear behind your REPL window. If this happens, use your operating system commands to locate the game window.

There are many possible improvements to the Snake game. If the snake reaches the edge of the screen, perhaps it should turn to avoid disappearing from view. Or maybe you just lose the game. Sorry! Make the Snake game your own by improving it to suit your personal style.

Snakes Without Refs

We chose to implement the Snake game's mutable model using refs so we could coordinate the updates to the snake and the apple. Other approaches are also valid. For example, you could combine the snake and apple state into a single game object. With only one object, coordination is no longer required, and you can use an atom instead.

The file `examples/atom-snake.clj` demonstrates this approach. Functions like `update-positions` become part of the functional model and return a new game object with updated state:

```
src/examples/atom_snake.clj
```

```
(defn update-positions [{snake :snake, apple :apple, :as game}]
  (if (eats? snake apple)
    (merge game {:apple (create-apple) :snake (move snake :grow)})
    (merge game {:snake (move snake)})))
```

Notice how destructuring makes it easy to get at the internals of the game: both `snake` and `apple` are bound by the argument list.

The actual mutable updates are now all atom swap!s. We found these to be simple enough to leave them in the UI function `game-panel`, as this excerpt shows:

```
(actionPerformed [e]
  (swap! game update-positions)
  (when (lose? (@game :snake))
    (swap! game reset-game)
    (JOptionPane/showMessageDialog frame "You lose!")))
```

There are other possibilities as well. Chris Houser’s fork of the book’s sample code³ demonstrates using an agent that `Thread/sleeps` instead of a Swing timer, as well as using a new agent per game turn to update the game’s state.

Wrapping Up

Clojure’s reference model is the most innovative part of the language. The combination of software transactional memory, agents, atoms, and dynamic binding that you’ve seen in this chapter gives Clojure powerful abstractions for all sorts of stateful systems. It also makes Clojure one of the few languages suited to the coming generation of multicore computer hardware.

Next, we’ll look at one of Clojure’s newer features. Some call it a solution to the “expression problem.”⁴ We call it a protocol.

3. <https://github.com/Chouser/programming-clojure/blob/master/examples/snake.clj>

4. https://en.wikipedia.org/wiki/Expression_problem

Protocols and Datatypes

Abstractions lie at the foundation of reusable code. The Clojure language has abstractions for sequences, collections, and callability. Abstractions are described on the JVM by Java interfaces and are implemented using Java classes.

- Protocols provide an alternative to Java interfaces for high-performance polymorphic method dispatch.
- Datatypes provide an alternative to Java classes for creating implementations of abstractions defined with either protocols or interfaces.

Protocols and datatypes provide a high-performance, flexible mechanism for abstraction and concretion that removes the need to write Java interfaces and classes when programming in Clojure. With protocols and datatypes, you can create new abstractions and new types that implement those abstractions and even extend new abstractions to existing types.

In this chapter, we'll explore Clojure's approach to abstraction using protocols and datatypes. First, we'll implement an abstraction to obtain the time from multiple kinds of time sources with protocols. Then, we'll introduce datatypes and records and put everything together to define musical notes and sequences. Work through these exercises and you'll see the power of Clojure's composable abstractions.

Programming to Abstractions

In most programs dealing with people, managing dates and times makes an appearance. Java has a long history in building date/time abstractions, starting with `java.util.Date`, essentially a wrapper for a long value representing milliseconds since the *epoch* (January 1, 1970, 00:00:00 GMT). `java.util.Calendar` was added to support parsing and formatting dates with time zones. However, both `Date` and `Calendar` suffer from being mutable objects (along with other deficiencies).

Java 8 added a new set of `java.time` packages to the JDK to represent dates, times, instants, durations, timezones, and periods as immutable objects. New code typically uses `java.time`, but many libraries were built on `Date` and `Calendar` (including Clojure's core) and thus both often must be supported for backwards compatibility. In this example, we will be looking at how to build an abstraction to obtain the milliseconds since the epoch for time sources of different types.

All of these examples will take place in the context of the following namespace that imports `Date` and `Instant`:

```
src/examples/instant.clj
(ns examples.instant
  (:import [java.util Date]
           [java.time Instant]))
```

We'll start by writing a basic version of the function that can read from `Instant` only. We'll then refactor the function to explore different approaches to supporting additional datatypes. Working through this will give you a good feel for the usefulness of programming to abstractions in general and the flexibility and power of Clojure's protocols and datatypes in particular.

The `inst->ms` function simply calls the Java interop method to get the epoch milliseconds from an `Instant`:

```
src/examples/instant.clj
(defn inst->ms
  "Given a java.time.Instant, return ms since the epoch"
  [^Instant instant]
  (.toEpochMilli instant))
```

But what if we want to support additional types like `Date`? We would need a more general version that accepted objects of multiple types and chose the right implementation. Let's do so and call it `time->ms`:

```
src/examples/instant.clj
(defn time->ms
  "Given a time source, return ms since the epoch"
  [time-source]
  (cond
    (instance? Instant time-source)
    (.toEpochMilli ^Instant time-source)

    (instance? Date time-source)
    (.getTime ^Date time-source)))
```

Let's try it out with some tests. Rather than see the same long value many times, create a constant for a test value `TEST-MS`:

```
(def TEST-MS 1760323945754)
-> #'examples.instant/TEST-MS
(= TEST-MS (time->ms (Instant/ofEpochMilli TEST-MS)))
-> true
(= TEST-MS (time->ms (Date. TEST-MS)))
-> true
```

It works well—we created a test `Instant` object and a test `Date` object and retrieved the expected long value back from `time->ms`.

The problem with this approach is that it's closed: nobody else can come along and add support for new time sources without rewriting `time->ms` themselves. What we need is an open solution, one where support for new types can be added after the fact and by different parties. In Java, interfaces are the primary tool for abstraction—they let different types implement the same operation in different ways.

Interfaces

Java interfaces define abstract methods. Invocations to the interface methods are dispatched to a specific implementation based on the datatype of the object that implements the interface.

Following are the strengths of interfaces:

- Datatypes can implement multiple interfaces.
- Interfaces provide only a specification, which allows the implementation of multiple interfaces without the problems associated with multiple class inheritance.

The weakness of interfaces is that existing datatypes cannot be extended to implement new interfaces without rewriting them.

In our case, `Instant` and `Date` are part of the JDK, and we don't have the option to modify them. You could consider extending or wrapping these types, but that introduces yet more barriers to extension and use. Historically, Java programmers have solved this issue with the Visitor design pattern¹ which still requires up-front integration into the datatypes and a lot of additional boilerplate code.

The *expression problem*² refers to the design goal of supporting both adding new types and new operations without modifying the existing code. Fortunately, Clojure has a solution.

1. https://en.wikipedia.org/wiki/Visitor_pattern

2. https://en.wikipedia.org/wiki/Expression_problem

Protocols

One piece of Clojure's solution to the expression problem is the protocol. Protocols provide a flexible mechanism for abstraction that leverages the best parts of interfaces by providing only specification, not implementation, and by letting datatypes implement multiple protocols. Additionally, protocols address the key weaknesses of interfaces by allowing nonintrusive extension of existing types to support new protocols.

The following are the strengths of protocols:

- Datatypes can implement multiple protocols.
- Protocols provide only specification, not implementation, which allows the implementation of multiple interfaces without the problems associated with multiple-class inheritance.
- Existing datatypes can be extended to implement new interfaces with no modification to the datatypes.
- Protocol method names are namespaced, so there's no risk of name collision when multiple parties choose to extend the same type.

The `defprotocol` macro takes a name, possible options, and one or more method signatures (no implementations).

```
(defprotocol name & opts+sigs)
```

It's important to note that the protocol's methods are normal namespaced functions in the namespace where the protocol is defined. Callers refer those functions from the protocol namespace just like any other function.

Let's redefine `time->ms` as the `InstantProvider` protocol with a `to-ms` method.

```
src/examples/instant.clj
```

```
(defprotocol InstantProvider
  "An abstraction for instant providers. Use to-ms to resolve
  to ms since the epoch."
  (to-ms [source]
    "Given a time source, return ms since the epoch"))
```

Notice that we can include a document string for the protocol as a whole, as well as for each of its methods. Now, let's extend `Instant` and `Date` to implement our `InstantProvider` protocol.

The `extend` function associates a type to a protocol and provides the required function implementations, usually referred to as *methods* in this context. The parameters to `extend` are the name of the type, the name of the protocol, and

a map of method implementations, where the keys are keywordized versions of the method names.

```
(extend type & proto+mmaps)
```

However, this low-level function is almost never used directly. Instead, two convenience macros `extend-type` (single type, multiple protocols) and `extend-protocol` (single protocol, multiple types) are strongly preferred, so we'll focus on those. If you are doing a single extension to a single protocol, they are essentially the same and either can be used.

Since we have two implementations and one protocol, we'll use `extend-protocol`:

```
(extend-protocol protocol & specs)
```

This takes the name of the protocol, followed by one or more type names with their respective method implementations.

Let's apply that to extend the protocol `InstantProvider` to the types `Date` and `Instant`:

```
src/examples/instant.clj
(extend-protocol InstantProvider
  Instant
    (to-ms [inst]
      (.toEpochMilli ^Instant inst))
  Date
    (to-ms [date]
      (.getTime ^Date date)))
```

Importantly, note that extensions are external to both the definition of the type and the definition of the protocol—this allows them to be added at any time as the code evolves, resolving the expression problem.

Two special cases you will frequently encounter are explicitly handling `nil` (often an error case) and handling the "default" case (no type match). For the former, we simply extend `nil` as the type. For the latter, we will lean on another protocol feature—protocols use the Java class hierarchy to find the closest class match. Thus, we can extend `Object` to catch every type without a hierarchy match in the protocol extensions.

Let's use `extend-protocol` one more time to cover these two cases and treat them both as errors.

```
src/examples/instant.clj
(defn unknown-src
  [time-source]
  (throw
    (ex-info (str "Unknown time source: "
                  (or
                    (some-> time-source class .getName)
                    "nil"))
              {})))

(extend-protocol InstantProvider
  nil
  (to-ms [src]
    (unknown-src src))
  Object
  (to-ms [src]
    (unknown-src src)))
```

Since these extensions do the same thing, the code has been moved to a function `unknown-src`. Let's give that a try:

```
(to-ms "oops")
| Execution error (ExceptionInfo) at examples.instant/unknown-src
| Unknown time source: java.lang.String
(to-ms nil)
| Execution error (ExceptionInfo) at examples.instant/unknown-src
| Unknown time source: nil
```

With those default extensions, we now have better error handling for undefined cases.

Datatypes

We've shown how to extend existing types to implement new abstractions with protocols, but what if we want to create a new type in Clojure? That's where datatypes come in.

A datatype provides the following:

- A unique class, either named or anonymous
- Structure, either explicitly as fields or implicitly as a closure
- Fields that can have type hints and can be primitive
- Immutability on by default
- Unification with maps (via records)
- Optional implementations of abstract methods specified in protocols or interfaces

In the last section, we built support for using both Dates and Instants as time sources in our application. However, imagine we are building a simulation

that runs on a simulated clock with variable granularity. We may want our simulation clock to also act as a time source. We'll look next at creating a custom type that extends multiple protocols.

First, let's create a second protocol defining our simulation behavior (or at least the part of it relevant to advancing time):

```
src/examples/instant.clj
(defprotocol Simulation
  (advance! [_] "Advance the simulation to the next step"))
```

We'll use `deftype` to create the new type:

```
(deftype name [& fields] & opts+specs)
```

It takes the name of the type and a vector of fields contained by the type. The naming convention for datatypes is the same as used by Java classes, CamelCase.

After the fields you can implement any number of protocols or interfaces with the same syntax we've already seen when extending protocols. Implementing these inline is optional—they could be extended externally too, just as we saw in prior examples, but performance is slightly better when inlined.

Let's implement our `SimClock` type, extending both the `Simulation` and `InstantProvider` protocols, as well as a helper constructor function:

```
src/examples/instant.clj
(deftype SimClock
  [start granularity tick]
  Simulation
  (advance! [_]
    (swap! tick inc))

  InstantProvider
  (to-ms [_]
    (+ start (* granularity @tick))))

(defn init-simulation
  [granularity]
  (->SimClock (to-ms (Instant/now)) granularity (atom 0)))
```

`SimClock` is a type with three fields. `start` records the simulation start time in milliseconds. `granularity` is the number of milliseconds per tick. `tick` is stateful—it is an atom recording the number of ticks since start.

`advance!` moves the simulation forward one tick and `to-ms` calculates the current time of the simulation based on all of the fields. Because the `SimClock` requires some configuration at construction, it's helpful to provide a constructor function.

Let's try our simulation.

```
(def sim (init-simulation 1000))
-> #'examples.instant/sim
(dotimes [_ 5]
  (advance! sim)
  (println (to-ms sim)))
| 1760480520359
| 1760480521359
| 1760480522359
| 1760480523359
| 1760480524359
```

As you can see from the output, each time we advance, the reported time advances one second, our granularity.

This type can only be used as a Simulation or InstantProvider, it doesn't have any other visible methods. However, it's common to need a custom type, possibly implementing protocols that expose its fields through standard Clojure map-like interfaces. In the next section, we'll see how to do that with defrecord.

Records

Classes in object-oriented programs tend to fall into two distinct categories: those that represent programming artifacts, such as String, Socket, InputStream, and OutputStream, and those that represent application domain information, such as Employee and PurchaseOrder.

Unfortunately, using classes to model application domain information hides it behind a class-specific micro-language of setters and getters. You can no longer take a generic approach to information processing, and you end up with a proliferation of unnecessary specificity and reduced reusability. See Clojure's documentation on datatypes³ for more information.

This is why Clojure has always encouraged the use of maps for modeling such information. That holds true even with datatypes, which is where records come in. A record is a datatype, like those created with deftype, that also implements PersistentMap and therefore can be used like any other map (mostly); since records are also classes, they support type-based polymorphism through protocols. With records, we have the best of both worlds: maps that can implement protocols.

What could be more natural than using records to play music? So, let's create a record that represents a musical note, with fields for pitch, octave, and duration. We'll use the JDK's built-in MIDI synthesizer to play sequences of these notes.

3. <https://clojure.org/reference/datatypes>

Since records are maps, we'll be able to change the properties of individual notes using the `assoc` and `update-in` functions, and we can create or transform entire sequences of notes using `map` and `reduce`. This gives us access to the entirety of Clojure's collection API.

All examples in the rest of this chapter will be evaluated in terms of this namespace definition:

```
src/examples/note.clj
(ns examples.note
  (:import [javax.sound.midi MidiSystem]))
```

We'll create a `Note` record with the `defrecord` macro, which behaves like `deftype`:

```
(defrecord name [& fields] & opts+specs)
```

A `Note` record has three fields: `pitch`, `octave`, and `duration`. All of the remaining examples in this chapter will be evaluated in the context of:

```
src/examples/note.clj
(defrecord Note [pitch octave duration])
```

The pitch will be represented by a keyword like `:C`, `:C#`, and `:Db`, which represent the notes C, C# (C sharp), and Db (D flat), respectively. Each pitch can be played at different octaves; for instance, middle C is in the fourth octave. Duration indicates the note length; a whole note is represented by 1, a half note by 1/2, a quarter note by 1/4, and a 16th note by 1/16. For example, we can represent a D# half note in the fourth octave with this `Note` record:

```
(->Note :D# 4 1/2)
-> #examples.note.Note{:pitch :D#, :octave 4, :duration 1/2}
```

Records *are* maps:

```
(map? (->Note :D# 4 1/2))
-> true
```

so we can also access their fields using keywords:

```
(:pitch (->Note :D# 4 1/2))
-> :D#
```

We can create modified records with `assoc` and `update-in`:

```
(assoc (->Note :D# 4 1/2) :pitch :Db :duration 1/4)
-> #examples.note.Note{:pitch :Db, :octave 4, :duration 1/4}

(update-in (->Note :D# 4 1/2) [:octave] inc)
-> #examples.note.Note{:pitch :D#, :octave 5, :duration 1/2}
```

Records are open, so we can associate extra fields into a record:

```
(assoc (->Note :D# 4 1/2) :velocity 100)
-> #examples.note.Note{:pitch :D#, :octave 4, :duration 1/2, :velocity 100}
```

Use the optional `:velocity` field to represent the force with which a note is played.

When used on a record, both `assoc` and `update-in` return a new record, but the `dissoc` function works a bit differently; it will return a new record if the field being dissociated is optional, like `velocity` in the previous example, but it will return a plain map if the field is mandated by the `defrecord` specification, like `pitch`, `octave`, or `duration`.

In other words, if you remove a required field from a record of a given type, it's no longer a record of that type, and it simply becomes a map.

```
(dissoc (->Note :D# 4 1/2) :octave)
-> {:pitch :D#, :duration 1/2}
```

Notice that `dissoc` returns a map, not a record. One difference between records and maps is that, unlike maps, records are not functions of keywords.

```
((->Note :D# 4 1/2) :pitch)
-> Execution error (ClassCastException) at user/eval176 (REPL:1).
-> class examples.note.Note cannot be cast to class clojure.lang.IFn
```

`ClassCastException` is thrown because records do not implement the `IFn` interface like maps do. Records are not simply functions of key to value; they have additional functionality and may also implement the `IFn` interface to do something else.

When accessing a collection, you should place the collection first. When accessing a map that's acting (conceptually) as a data record, you should place the keyword first, even if the record is implemented as a plain map. Now that we have our basic `Note` record, let's add some methods so we can play them with the JDK's built-in MIDI synthesizer. We'll start by creating a `MidiNote` protocol with three methods:

```
src/examples/note.clj
(defprotocol MidiNote
  :extend-via-metadata true

  (msec [this tempo] "Note duration in ms")
  (key-number [this] "MIDI key number")
  (velocity [this] "MIDI velocity"))
```

The `:extend-via-metadata` option is a protocol feature that we'll talk more about later in [Metadata Extension, on page 198](#).

To play our note with the MIDI synthesizer, we need a representation of a MIDI note to play, which we capture in the `MidiNote` protocol. Given a note, we need to translate its pitch and octave into a MIDI key number and its duration into milliseconds. Also, the note will be played with some velocity (usually treated as volume).

Before we extend the `Note` record to implement the `MidiNote` protocol, let's create a few helper methods. Creating these smaller utility functions makes them easier to test and the overall code easier to understand.

The `beat->msec` function translates the note's number of beats (whole note = 1, half note = 1/2, and so on), into milliseconds based on the given tempo, which is represented in beats per minute (bpm).

```
src/examples/note.clj
;; Assumes whole note = 4 beats
(defn beats->msec [beats tempo]
  (* (/ beats tempo) 4 60 1000))
```

The `note->key` function maps the keywords used to represent pitch into a number ranging from 0 to 11 and then uses this number along with the given octave to find the corresponding MIDI key-number.

```
src/examples/note.clj
(defn note->key [pitch octave]
  (let [scale {:C 0, :C# 1, :Db 1, :D 2,
               :D# 3, :Eb 3, :E 4, :F 5,
               :F# 6, :Gb 6, :G 7, :G# 8,
               :Ab 8, :A 9, :A# 10, :Bb 10,
               :B 11}]
    (+ (* 12 (inc octave)) (scale pitch))))
```

With that in place, we can now extend `Note` to `MidiNote`.

```
src/examples/note.clj
(extend-type Note
  MidiNote
  (msec [this tempo]
    (beats->msec (:duration this) tempo))
  (key-number [this]
    (note->key (:pitch this) (:octave this)))
  (velocity [this]
    (or (:velocity this) 64)))
```

Now, we need a function that sets up the MIDI synthesizer and plays a sequence of notes.

```
src/examples/note.clj
```

```
(defn perform [notes & {:keys [tempo] :or {tempo 120}}]
  (with-open [synth (doto (MidiSystem/getSynthesizer) .open)]
    (let [channel (aget (.getChannels synth) 0)]
      (doseq [note notes]
        (.noteOn channel (key-number note) (velocity note))
        (Thread/sleep (long (msec note tempo)))))))
```

The `perform` function takes a sequence of notes and an optional tempo value, opens a MIDI synthesizer, gets a channel from it, and then plays the note for the specified MIDI pitch and velocity. The note continues to play while the current thread is asleep, stopping only when the next note is sent to the channel.

All the pieces are in place, so let's make music using a sequence of Note records:

```
(def close-encounters [(->Note :D 3 1/2)
                       (->Note :E 3 1/2)
                       (->Note :C 3 1/2)
                       (->Note :C 2 1/2)
                       (->Note :G 2 1/2)])
-> #'user/close-encounters
```

In this case, our “music” consists of the five notes used to greet the alien ships in the movie *Close Encounters of the Third Kind*. To play it, just pass the sequence to the `perform` function:

```
(perform close-encounters)
-> nil
```

We can also generate sequences of notes dynamically with the `for` macro.

```
(def jaws (for [duration [1/2 1/2 1/4 1/4 1/8 1/8 1/8 1/8]
                pitch [:E :F]]
             (Note. pitch 2 duration)))
-> #'user/jaws

(perform jaws)
-> nil
```

The result is the shark theme from *Jaws*—a sequence of alternating E and F notes progressively speeding up as they move from half notes to quarter notes to eighth notes.

Since notes are records and records are map-like, we can manipulate them with any Clojure function that works on maps. For instance, we can map the `update-in` function across the *Close Encounters* sequence to raise or lower its octave.

```
(perform (map #(update-in % [:octave] inc) close-encounters))
-> nil

(perform (map #(update-in % [:octave] dec) close-encounters))
-> nil
```

Or we can create a sequence of notes that have progressively larger values of the optional `:velocity` field:

```
(perform (for [velocity [64 80 90 100 110 120]]
              (assoc (Note. :D 3 1/2) :velocity velocity)))
-> nil
```

This results in a sequence of increasingly more forceful D notes. Manipulating sequences is a particular strength of Clojure, so there are endless possibilities for programmatically creating and manipulating sequences of Note records.

reify

The `reify` macro lets you create an anonymous instance of a datatype that implements either a protocol or an interface. You do not need to declare fields or methods; the object returned only has the methods of the interface or protocol it implements. This is useful when you need a one-time implementation of an interface or protocol without a reusable type.

```
(reify & opts+specs)
```

`reify`, like `deftype` and `defrecord`, takes the name of one or more protocols, or interfaces, and a series of method bodies. Unlike `deftype` and `defrecord`, it doesn't take a name or a vector of fields; datatype instances produced with `reify` don't have explicit fields, relying instead on closures.

Let's compose some John Cage-style⁴ aleatoric music⁵ or, better yet, create an aleatoric music generator. We'll use `reify` to create an instance of a `MidiNote` that will play a different random note each time its `play` method is called.

```
src/examples/note.clj
```

```
(let [min-duration 250
      min-velocity 64
      rand-note (reify MidiNote
                  (msec [_ tempo] (+ (rand-int 1000) min-duration))
                  (key-number [_] (rand-int 100))
                  (velocity [_] (+ (rand-int 100) min-velocity)))]
  (perform (repeat 15 rand-note)))
```

4. https://en.wikipedia.org/wiki/John_Cage

5. https://en.wikipedia.org/wiki/Aleatoric_music

First, we bind two values, `min-duration` and `min-velocity`, that we will use in the `MidiNote` method implementations. Then, we use `reify` to create an instance of an anonymous type, which implements the `MidiNote` protocol, that will select a random note, duration, and velocity each time its `play` method is called. Finally, we use the `repeat` function to create a sequence of 15 notes, consisting of a single instance of `rand-note`, and perform it. *Voila*, you now have a virtual John Cage!

Metadata Extension

We've now seen how to extend new types to protocols, and how to reify anonymous instances of a protocol. But in some cases, we might need to dynamically treat existing data as an extension of a protocol, without defining that extension for every use of the data.

Clojure protocols have an opt-in extension mechanism, `:extend-via-metadata`, that requires no extension be declared at all. In this case, extension can be supplied on a per-object basis by defining the extension directly in the metadata of the instance. Because this extension is instance-only, no other types or objects are affected.

For example, what if we wanted to combine the MIDI library we already have with a to-do list application to create a customizable sonic representation of your to-do list. Let's start with some existing to-do list data:

```
src/examples/note.clj
;; To do item with estimated times in minutes
(defrecord ToDo [desc est type])

(def todo-list
  [(->ToDo "Grocery shopping" 60 :errand)
   (->ToDo "Mow yard" 30 :house)
   (->ToDo "Brush teeth" 2 :personal)
   (->ToDo "Walk dog" 10 :house)
   (->ToDo "Call mom" 15 :personal)])
```

Add a user theme definition that converts task estimates to note length (minutes per beat) and task types to notes:

```
src/examples/note.clj
(defrecord Theme [mpb pitches volume])

(def default-theme
  (->Theme 60 {:errand :C, :house :E, :personal :G} 64))
```

Because the sonic mapping is customizable per user via themes, we don't want to declare a single static protocol extension that ties together the `ToDo` type and the `MidiNote` protocol. Instead, we can dynamically apply the themed extension in metadata:

`src/examples/note.clj`

```
(defn to-note [todo theme]
  (let [{:keys [est type]} todo
        {:keys [mpb pitches volume]} theme
        octave 4]
    (with-meta todo
      {'msec (fn [_ bpm] (beats->msec (/ est mpb) bpm))
       `key-number (fn [_] (note->key (get pitches type) octave))
       `velocity (constantly volume)}))))
```

The `to-note` function combines a `ToDo` record and a user theme to return a new instance with a metadata map that defines the protocol extension. When using protocol extension via metadata, the keys are fully qualified symbols, and the values are the function implementations.

Backquote



You may have noticed the use of symbols with a leading backquote in the previous example, like ``msec`. We'll talk more about this in [Chapter 11, Macros, on page 239](#), but backquote is a special form of quote that fully resolves symbols in the context of the current namespace. It's being used here as a shorter way to say `examples.note/msec`.

Finally, we can map the `to-note` function over the `to-do` list data to play our custom `to-do` list melody:

`src/examples/note.clj`

```
(defn play-todos [todos theme]
  (perform (map #(to-note % theme) todos)))

(play-todos todo-list default-theme)
```

There is some performance cost to invoking protocol functions that allow `:extend-via-metadata` (they have to check for metadata extension at the call site), which is why this is an opt-in feature of `defprotocol`. However, the ability to dynamically extend protocols per instance is an amazingly flexible tool when you need it.

Wrapping Up

We covered a lot of ground in this chapter, from the general use of abstraction in programming to some (but not all) of the specific abstraction mechanisms Clojure provides. We explored implementing abstractions using protocols in Clojure and made some music in the process!

In the next chapter, we'll explore an even more flexible mechanism for abstraction with multimethods.

Multimethods

In the last chapter, we explored protocols as a means to define abstract behaviors that dispatch to type-specific implementations. Protocols leverage the JVM's polymorphic invocation to choose the implementation method based on the Java type hierarchy of the first argument. However, Clojure provides an even more general form of dispatch: *multimethods*.

Multimethods are not tied to either a single type hierarchy or even to the type of a particular argument at all. Instead, multimethods choose how to dispatch based on the invocation of an arbitrary function of the method's arguments. This is far more flexible and retains the open extension properties of protocols.

In this chapter, you'll develop an appreciation for multimethods by first living without them. Then, you'll build an increasingly complex series of multimethod implementations—first using multimethods to simulate type-based polymorphism and then using multimethods to implement various ad hoc taxonomies.

Multimethods in Clojure are used much less often than polymorphism in object-oriented languages. But where they are used, they're often the key feature in the code. We'll close the chapter by looking at how multimethods are used in several open source Clojure projects and offer guidelines for when to use them in your own programs.

Living Without Multimethods

The best way to appreciate multimethods is to spend a few minutes living without them, so let's do that. Clojure can already print anything with `print/println`. But pretend for a moment that these functions don't exist and that you need to build a generic print mechanism. To get started, create a `my-print` function that can print a string to the standard output stream `*out*`:

```
src/examples/life_without_multi.clj
(defn my-print [ob]
  (.write *out* ob))
```

Next, create a `my-println` that calls `my-print` and then adds a line feed:

```
src/examples/life_without_multi.clj
(defn my-println [ob]
  (my-print ob)
  (.write *out* "\n"))
```

The line feed makes `my-println`'s output easier to read when testing at the REPL. For the rest of this section, you'll make changes to `my-print` and test them by calling `my-println`. Test that `my-println` works with strings:

```
(my-println "hello")
| hello
-> nil
```

That's nice, but `my-println` doesn't work so well with nonstrings such as `nil`:

```
(my-println nil)
| Execution error (NullPointerException) at java.io.Writer/write.
```

It's not a big deal, though. Just use `cond` to add special-case handling for `nil`:

```
src/examples/life_without_multi.clj
(defn my-print [ob]
  (cond
    (nil? ob) (.write *out* "nil")
    (string? ob) (.write *out* ob)))
```

With the conditional in place, you can print `nil` with no trouble:

```
(my-println nil)
| nil
-> nil
```

Of course, there are still all kinds of types that `my-println` can't deal with. If you try to print a vector, neither of the `cond` clauses will match, and the program will print only an empty line:

```
(my-println [1 2 3])
|
-> nil
```

By now, you know the drill. Just add another `cond` clause for the vector case. The implementation here is a little more complex, so you might want to separate the actual printing into a helper function, such as `my-print-vector`:

```
src/examples/life_without_multi.clj
(require '[clojure.string :as str])
(defn my-print-vector [ob]
  (.write *out* "[")
  (.write *out* (str/join " " ob))
  (.write *out* "]"))
(defn my-print [ob]
  (cond
    (vector? ob) (my-print-vector ob)
    (nil? ob) (.write *out* "nil")
    (string? ob) (.write *out* ob)))
```

Make sure that you can now print a vector:

```
(my-println [1 2 3])
| [1 2 3]
-> nil
```

my-println now supports three types: strings, vectors, and nil. And you have a road map for new types: just add new clauses to the cond in my-println. But it's a crummy road map, because it conflates two things: the decision process for selecting an implementation and the specific implementation detail.

You can improve the situation somewhat by pulling out helper functions like my-print-vector. But then you'll have to make two separate changes every time you want to add a new feature to my-println:

- Create a new type-specific helper function.
- Modify my-println to add a new cond invoking the feature-specific helper.

As we saw in [Protocols, on page 188](#), what we really want is a mechanism for extension that is open, allowing us to handle new cases without changing a single existing function every time. Protocols allowed us to do that on the basis of the type of the first argument, but next, we'll see how to achieve the same goal with multimethods.

Defining Multimethods

Multimethods capture the same pattern we explored in the previous section, but support adding new cases without changing the existing code. Multimethods also provide some additional features that we'll look at later on. Multimethods consist of two parts: a *dispatch function* (created with defmulti) and a set of *methods* (created with defmethod).

To define a multimethod, use defmulti:

```
(defmulti name dispatch-fn)
```

name is the name of the new multimethod, and Clojure will invoke `dispatch-fn` against the method arguments to select one particular method (implementation) of the multimethod.

Consider `my-print` from the previous section. It takes a single argument, the thing to be printed, and you want to select a specific implementation based on the type of that argument. So, `dispatch-fn` needs to be a function of one argument that returns the type of that argument. Clojure has a built-in function matching this description, namely, `class`. Use `class` to create a multimethod called `my-print`:

```
src/examples/multimethods.clj
(defmulti my-print class)
```

At this point, you've provided a description of how the multimethod will select a specific method but no actual specific methods. Unsurprisingly, attempts to call `my-print` will fail:

```
(my-println "foo")
| Execution error (IllegalArgumentException) at ...
| No method in multimethod 'my-print' for
|   dispatch value: class java.lang.String
```

To add a specific method implementation to `my-println`, use `defmethod`:

```
(defmethod name dispatch-val & fn-tail)
```

`name` is the name of the multimethod to which an implementation belongs. Clojure matches the result of `defmulti`'s `dispatch` function with `dispatch-val` to select a method, and `fn-tail` contains arguments and body forms just like a normal function.

Create a `my-print` implementation that matches on strings:

```
src/examples/multimethods.clj
(defmethod my-print String [s]
  (.write *out* s))
```

Now, call `my-println` with a string argument:

```
(my-println "stu")
| stu
-> nil
```

Next, create a `my-print` that matches on `nil`:

```
src/examples/multimethods.clj
(defmethod my-print nil [s]
  (.write *out* "nil"))
```

Notice that you've solved the problem raised in the previous section. Instead of being joined in a big cond, each implementation of my-println is separate. Methods of a multimethod can live anywhere in your source, and you can add new ones any time, without having to touch the original code.

Dispatch Is Inheritance-Aware

Multimethod dispatch knows about Java inheritance. To see this, create a my-print that handles Number by printing a number's toString representation:

```
src/examples/multimethods.clj
(defmethod my-print Number [n]
  (.write *out* (.toString n)))
```

Test the Number implementation with an integer:

```
(my-println 42)
| 42
-> nil
```

42 is a Long, not a Number. Multimethod dispatch is smart enough to know that a long is a number and match anyway. Internally, dispatch uses the isa? function:

```
(isa? child parent)
```

isa? knows about Java inheritance, so it knows that a Long is a Number:

```
(isa? Long Number)
-> true
```

isa? is not limited to inheritance. Its behavior can be extended dynamically at runtime, as you will see later in [Creating Ad Hoc Taxonomies, on page 208](#).

Multimethod Defaults

It would be nice if my-print could have a fallback representation that you could use for any type you haven't specifically defined. You can use :default as a dispatch value to handle any methods that don't match anything more specific. Using :default, create a my-println that prints the Java toString value of objects, wrapped in #<>:

```
src/examples/multimethods.clj
(defmethod my-print :default [s]
  (.write *out* "#<")
  (.write *out* (.toString s))
  (.write *out* ">"))
```

Now, test that `my-println` prints random things, using the default method:

```
(my-println (java.sql.Date. 0))
-> #<1969-12-31>

(my-println (java.util.Random.))
-> #<java.util.Random@1c398896>
```

In the unlikely event that `:default` already has some specific meaning in your domain, you can create a multimethod using this alternate signature:

```
(defmulti name dispatch-fn :default default-value)
```

The `default-value` lets you specify a custom default value.

Dispatching a multimethod on the type of the first argument, as you’ve done with `my-print`, is by far the most common kind of dispatch. In many object-oriented languages, in fact, it’s the *only* kind of dynamic dispatch, and it goes by the name *polymorphism*.

Clojure’s multimethod dispatch is much more general. Let’s add a few complexities to `my-print` and move beyond what’s possible with plain ol’ polymorphism.

Moving Beyond Simple Dispatch

Clojure’s `print` function prints various “sequencey” things as lists. If you wanted `my-print` to do something similar, you could add a method that dispatched on a collection interface high in the Java inheritance hierarchy, such as `Collection`:

```
src/examples/multimethods.clj
(require '[clojure.string :as str])
(defmethod my-print java.util.Collection [c]
  (.write *out* "(")
  (.write *out* (str/join " " c))
  (.write *out* "))")
```

Now, try various sequences to see that they get a nice print representation:

```
(my-println (take 6 (cycle [1 2 3])))
| (1 2 3 1 2 3)
-> nil

(my-println [1 2 3])
| (1 2 3)
-> nil
```

Perfectionist that you are, you cannot stand that vectors print with rounded braces, unlike their literal square-brace syntax. So, add yet another `my-print` method, this time to handle vectors. Vectors all implement an `IPersistentVector`, so this should work:

```
src/examples/multimethods.clj
```

```
(defmethod my-print clojure.lang.IPersistentVector [c]
  (.write *out* "[")
  (.write *out* (str/join " " c))
  (.write *out* "]"))
```

But it doesn't work. Instead, printing vectors now throws an exception:

```
(my-println [1 2 3])
| java.lang.IllegalArgumentException: Multiple methods match
| dispatch value: class clojure.lang.LazilyPersistentVector ->
| interface clojure.lang.IPersistentVector and
| interface java.util.Collection, and neither is preferred
```

The problem is that two dispatch values now match for vectors: `Collection` and `IPersistentVector`. Many languages constrain method dispatch to make sure these conflicts never happen, such as by forbidding multiple inheritance. Clojure takes a different approach. You can create conflicts, and you can resolve them with `prefer-method`:

```
(prefer-method multi-name loved-dispatch dissed-dispatch)
```

When you call `prefer-method` for a multimethod, you tell it to prefer the `loved-dispatch` value over the `dissed-dispatch` value whenever there's a conflict. Since you want the vector version of `my-print` to trump the collection version, tell the multimethod what you want:

```
src/examples/multimethods.clj
```

```
(prefer-method
  my-print clojure.lang.IPersistentVector java.util.Collection)
```

Now, you should be able to route both vectors and other sequences to the correct method implementation:

```
(my-println (take 6 (cycle [1 2 3])))
| (1 2 3 1 2 3)
-> nil

(my-println [1 2 3])
| [1 2 3]
-> nil
```

Many languages create complex rules or arbitrary limitations to resolve ambiguities in their systems for dispatching functions. Clojure allows a much simpler approach: just don't worry about it! If there's an ambiguity, use `prefer-method` to resolve it.

Creating Ad Hoc Taxonomies

Multimethods let you create ad hoc taxonomies, which can be helpful when you discover type relationships that are not explicitly declared as such.

For example, consider a financial application that deals with checking and savings accounts. This example will take place in the namespace `examples.multimethods`. We'll model an account using a Clojure map, with a `:tag` key to specify the account type:

```
{:id 1, :tag ::savings, :balance 100M}
```

We'll use `::checking` and `::savings` as the `:tag` values to differentiate. The doubled `::` is the syntax for auto-resolved keywords, which resolves these keywords in the current namespace context. To demonstrate the namespace resolution, compare entering `:checking` and `::checking` at the REPL:

```
:checking
-> :checking

::checking
-> :examples.multimethods/checking
```

Placing keywords in a namespace helps prevent name collisions with other people's code. When you use `::savings` or `::checking` from another namespace, you'll need to fully qualify them:

```
{:id 1, :tag :examples.multimethods/savings, :balance 100M}
```

Or, you can use `:as-alias` in `ns :require` to create the short name `acc`:

```
(ns other.namespace
  (:require [examples.multimethods :as-alias acc]))
```

Create two top-level test objects, a savings account and a checking account:

```
(def test-savings {:id 1, :tag ::savings, :balance 100M})
(def test-checking {:id 2, :tag ::checking, :balance 250M})
```

Note that the trailing `M` creates a `BigDecimal` literal and does not mean you have millions of dollars.

The interest rate is 0% for checking accounts and 5% for savings accounts. Create a multimethod `interest-rate` that dispatches based on `:tag`, like so:

```
src/examples/multimethods.clj
(defmulti interest-rate :tag)
(defmethod interest-rate ::checking [_] 0M)
(defmethod interest-rate ::savings  [_] 0.05M)
```

Check your test-savings and test-checking to make sure that interest-rate works as expected.

```
(interest-rate test-savings)
-> 0.05M

(interest-rate test-checking)
-> 0M
```

Accounts have an annual service charge, with rules as follows:

- Normal checking accounts incur a \$25 fee.
- Normal savings accounts incur a \$10 fee.
- Premium accounts have no fee.
- Checking accounts with a balance of \$5,000 or more are premium.
- Savings accounts with a balance of \$1,000 or more are premium.

In a realistic example, the rules would be more complex. Premium status would be driven by average balance over time, and there would probably be other ways to qualify. But the previous rules are complex enough to demonstrate the point.

You could implement service-charge with a bunch of conditional logic, but premium feels like a type, although there's no explicit premium tag on an account. Create an account-level multimethod that returns `::premium` or `::basic`:

```
src/examples/multimethods.clj
(defmulti account-level :tag)
(defmethod account-level ::checking [acct]
  (if (>= (:balance acct) 5000) ::premium ::basic))
(defmethod account-level ::savings [acct]
  (if (>= (:balance acct) 1000) ::premium ::basic))
```

Test account-level to make sure that checking and savings accounts require different balance levels to reach `::premium` status:

```
(account-level {:id 1, :tag ::savings, :balance 2000M})
-> :examples.multimethods/premium
(account-level {:id 1, :tag ::checking, :balance 2000M})
-> :examples.multimethods/basic
```

Now, you might be tempted to implement service-charge using account-level as a dispatch function:

```
src/examples/multimethods.clj
; bad approach
(defmulti service-charge account-level)
(defmethod service-charge ::basic [acct]
  (if (= (:tag acct) ::checking) 25 10))
(defmethod service-charge ::premium [_] 0)
```

The conditional logic in `service-charge` for `::basic` is exactly the kind of type-driven conditional that multimethods should help us avoid. The problem here is that you're already dispatching by account-level, and now, you need to be dispatching by `:tag` as well. No problem—you can dispatch on *both*. Write a `service-charge2` whose dispatch function calls both account-level and `:tag`, returning the results in a vector:

```
src/examples/multimethods.clj
```

```
(defmulti service-charge2 (fn [acct] [(account-level acct) (:tag acct)]))
(defmethod service-charge2 [::basic ::checking] [_] 25)
(defmethod service-charge2 [::basic ::savings] [_] 10)
(defmethod service-charge2 [::premium ::checking] [_] 0)
(defmethod service-charge2 [::premium ::savings] [_] 0)
```

`service-charge2` dispatches against two different taxonomies: the `:tag` intrinsic to an account and the externally defined `account-level`. Try a few accounts to verify that `service-charge2` works as expected:

```
(service-charge2 {:tag ::checking :balance 1000})
-> 25

(service-charge2 {:tag ::savings :balance 1000})
-> 0
```

There's one further improvement you can make to `service-charge`. Since all premium accounts have the same service charge, it seems redundant to have to define two separate `service-charge` methods for `::savings` and `::checking` accounts. It would be nice to have a parent type `::account` so you could define a multimethod that matches `::premium` for any kind of `::account`. Clojure lets you define arbitrary parent-child relationships with `derive`:

```
(derive child parent)
```

Using `derive`, you can specify that both `::savings` and `::checking` are kinds of `::account`:

```
src/examples/multimethods.clj
```

```
(derive ::savings ::account)
(derive ::checking ::account)
```

When you start to use `derive`, `isa?` comes into its own. In addition to understanding Java inheritance, `isa?` knows all about derived relationships:

```
(isa? ::savings ::account)
-> true
```

Now that Clojure knows that `savings` and `checking` are accounts, you can define a `service-charge3` using a single method to handle `::premium`:

```
src/examples/multimethods.clj
```

```
(defmulti service-charge3 (fn [acct] [(account-level acct) (:tag acct)]))
(defmethod service-charge3 [::basic ::checking] [_] 25)
(defmethod service-charge3 [::basic ::savings] [_] 10)
(defmethod service-charge3 [::premium ::account] [_] 0)
```

At first glance, you may think that `derive` and `isa?` duplicate functionality that's already available to Clojure via Java inheritance. This is not the case. Java inheritance relationships are forever fixed at the moment you define a class. derived relationships can be created when you need them and can be *applied to existing objects without their knowledge or consent*. So, when you discover a useful relationship between existing objects, you can derive that relationship without touching the original objects' source code and without creating tiresome “wrapper” classes.

If the number of different ways you might define a multimethod has your head spinning, don't worry. In practice, most Clojure code uses multimethods sparingly. Let's take a look at some open source Clojure code to get a better idea of how multimethods are used.

When Should I Use Multimethods?

Multimethods are extremely flexible, and with that flexibility comes choices. How should you choose when to use multimethods, as opposed to some other technique? We approach this question from two directions, asking the following:

- Where do Clojure projects use multimethods?
- Where do Clojure projects *eschew* multimethods?

The most striking thing is that multimethods are *rare*—about one per 1,000 lines of code. So, don't worry that you're missing something important if you build a Clojure application with few, or no, multimethods. A Clojure program that defines no multimethods isn't nearly as odd as an object-oriented program with no polymorphism.

Many multimethods dispatch on class. Dispatch-by-class is the easiest kind of dispatch to understand and implement. We already covered it in detail with the `my-print` example, so we'll say no more about it here.

Clojure multimethods that dispatch on something other than class are fairly rare. We can look directly in Clojure for some examples. The `clojure.inspector` and `clojure.test` namespaces use unusual dispatch functions.

The Inspector

Clojure’s inspector namespace uses Swing to create simple views of data. You can use it to get a tree view of your system properties:

```
(require '[clojure.inspector :refer [inspect inspect-tree]])
(inspect-tree (System/getProperties))
```

inspect-tree returns (and displays) a JFrame with a tree view of anything that’s treeish. So, you could also pass a nested map to inspect-tree:

```
(inspect-tree {:clojure {:creator "Rich" :runs-on-jvm true}})
```

Treeish things are made up of nodes that can answer two questions:

- Who are my children?
- Am I a leaf node?

The treeish concepts of “tree,” “node,” and “leaf” all sound like candidates for classes or interfaces in an object-oriented design. But the inspector doesn’t work this way. Instead, it adds a “treeish” type system in an ad hoc way to existing types, using a dispatch function named collection-tag:

```
; from Clojure's clojure/inspector.clj
(defn collection-tag [x]
  (cond
    (map-entry? x) :entry
    (instance? java.util.Map x) :seqable
    (instance? java.util.Set x) :seqable
    (sequential? x) :seq
    (instance? clojure.lang.Seqable x) :seqable
    :else :atom))
```

collection-tag returns one of the keywords :entry, :map, :seqable, :seq, or :atom. These act as the type system for the treeish world. The collection-tag function is then used to dispatch three different multimethods that select specific implementations based on the treeish type system.

```
(defmulti is-leaf collection-tag)

(defmulti get-child
  (fn [parent index] (collection-tag parent)))

(defmulti get-child-count collection-tag)
; method implementations elided for brevity
```

The treeish type system is added around the existing Java type system. Existing objects don’t have to *do* anything to become treeish; the inspector namespace does it for them. Treeish demonstrates a powerful style of reuse.

You can discover new type relationships in existing code and take advantage of these relationships simply, without having to modify the original code.

clojure.test

The `clojure.test` namespace in Clojure lets you write several different kinds of assertions using the `is` macro. You can assert that arbitrary functions are true. For example, 10 is not a string:

```
(require '[clojure.test :refer [is]])
(is (string? 10))

| FAIL in () (NO_SOURCE_FILE:2)
| expected: (string? 10)
| actual: (not (string? 10))
-> false
```

Although you can use an arbitrary function, `is` knows about a few specific functions and provides more detailed error messages. For example, you can check that a string is not an instance? of Collection:

```
(is (instance? java.util.Collection "foo"))

| FAIL in () (NO_SOURCE_FILE:3)
| expected: (instance? java.util.Collection "foo")
| actual: java.lang.String
-> false
```

`is` also knows about `=`. Verify that power does not equal wisdom.

```
(is (= "power" "wisdom"))

| FAIL in () (NO_SOURCE_FILE:4)
| expected: (= "power" "wisdom")
| actual: (not (= "power" "wisdom"))
-> false
```

Internally, `is` uses a multimethod named `assert-expr`, which dispatches not on the type but on the actual *identity* of its first argument:

```
(defmulti assert-expr (fn [form message] (first form)))
```

Since the first argument is a symbol representing what function to check, this amounts to yet another ad hoc type system. This time, there are three types: `=`, `instance?`, and everything else.

The various `assert-expr` methods add specific error messages associated with different functions you might call from `is`. Because multimethods are open-ended, you can add your own `assert-expr` methods with improved error messages for other functions you frequently pass to `is`.

Counterexamples

As you saw in the previous section, you can often use multimethods to hoist branches that are based on type out of the main flow of your functions. To find counterexamples where multimethods should not be used, we looked through Clojure's core to find type branches that had *not* been hoisted to multimethods.

A simple example is Clojure's `class`, which is a null-safe wrapper for the underlying Java `getClass`. Minus comments and metadata, class is as follows:

```
(defn class [x]
  (if (nil? x) x (.getClass x)))
```

You could write a version of `class` as a multimethod by dispatching on `identity`:

```
src/examples/multimethods.clj
(defmulti my-class identity)
(defmethod my-class nil [_] nil)
(defmethod my-class :default [x] (.getClass x))
```

Any `nil`-check could be rewritten this way. But we find the original `class` function easier to read than the multimethod version. This is a nice “exception that proves the rule.” Even though `class` branches on type, the branching version is easier to read.

Use the following general rules when deciding whether to create a function or a multimethod:

- If a function branches based on a type, or multiple types, consider a multimethod.
- Types are whatever you discover them to be. They do not have to be explicit Java classes or data tags.
- You should be able to interpret the dispatch value of a `defmethod` without having to refer to the `defmulti`.
- Don't use multimethods merely to handle optional arguments or recursion.

When in doubt, try writing the function in both styles and pick the one that seems more readable.

Wrapping Up

Multimethods support arbitrary dispatch. Usually, multimethods work based on type relationships. Sometimes these types are formal, as in Java classes. Other times, they are informal and ad hoc and emerge from the properties of objects in the system.

Java Interop

Clojure's Java support is both powerful and lean. It's powerful in that it brings the expressiveness of Lisp syntax, plus some syntactic sugar tailored to Java. It's lean in that it compiles to bytecode without a translation layer and can thus achieve Java-level performance in nearly every case.

Clojure embraces Java and its libraries. Idiomatic Clojure code calls Java libraries directly and doesn't try to wrap everything under the sun to look like Lisp. This surprises many new Clojure developers, but it's very pragmatic. Where Java isn't broken, Clojure doesn't fix it. In this chapter, you'll see how Clojure's access to Java is convenient, elegant, and fast. In addition, you'll see how to flip the script and call Clojure from Java.

Clojure's exception handling is easy to use. Better yet, explicit exception handling is rarely necessary. Clojure's exception primitives are the same as Java's. However, Clojure does not require you to deal with checked exceptions and makes it easy to clean up resources using the *with-open* idiom.

Clojure is *fast*, unlike many other dynamic languages on the JVM. You can use custom support—for type hints, primitives, and arrays—to cause Clojure's compiler to generate the same code that a Java compiler would generate.

Clojure is designed to let you get things done and have fun while doing it. However, an important part of getting things done is being able to use your platform to its full potential. Let's start by seeing how to extend Java interfaces and classes in Clojure.

Creating Java Objects in Clojure

When calling Java libraries or the Java standard library, you'll often need to pass in Java objects that either implement an interface or extend a particular class. Clojure provides solutions for this problem in several ways—direct use

of Java types and interfaces, anonymous interface implementation with reify, and class extension with proxy.

Direct Use of Java Types

Clojure's implementation reuses Java's own concrete types like String, Character, Boolean, the numeric classes, Date, and more. Because these Clojure objects are actually Java objects, they can be passed directly to Java APIs without wrapping or modification.

Additionally, many Clojure objects implement key Java interfaces where applicable, so they play directly with existing Java APIs without additional effort. For example, Clojure functions implement the interfaces Runnable and Callable making them useful in many concurrency and task-oriented APIs. For example, a Thread is started with a Runnable so any Clojure function can be used:

```
(defn say-hi []
  (println "Hello from thread" (.getName (Thread/currentThread))))

(dotimes [_ 3]
  (.start (Thread. say-hi)))
```

The last line will start a new thread three times, running the say-hi function, so you should see (in possibly a different order or interleaved):

```
Hello from thread Thread-1
Hello from thread Thread-2
Hello from thread Thread-3
```

Additionally, the Clojure data structures implement key interfaces from the Java Collections API, such as Collection, List, Map, and Set. This means that any Java interface that takes a collection can be directly passed the collection created in Clojure without wrapping or copying the data.

For example, consider the Java library class `java.util.Collections`, which has helpful utility methods for Java collections. One provided method is `binarySearch`, which takes a sorted list and uses a binary search algorithm to determine whether the list contains an element. If the list also implements the `RandomAccess` interface (which Clojure vectors do), this can be accomplished in $O(\log n)$ element comparisons.

Since the Clojure collections implement the proper interfaces, they can be passed directly to this Java method:

```
(java.util.Collections/binarySearch [1 13 42 1000] 42)
-> 2
```

The `binarySearch` method returns the index in the collection where the element can be found.

Between Clojure's direct use of Java primitives and extensive use of Java interfaces, a lot of Java interop will simply do what you expect when you invoke a method, both in the code as well as in the underlying bytecode. There's no magic here, just Clojure adhering to good Java implementation principles.

However, you'll inevitably encounter cases where you need to invoke a method that takes an instance conforming to a Java interface where you need to provide that instance. In the next section, we'll see some options for how to do this.

Implementing Java Interfaces

When invoking a Java method takes an instance implementing a Java interface, one of the key questions is whether you need to generate that instance just for the purposes of the call, or you need to make and reuse that instance in many places.

In the first case (common when creating callbacks or using event-based APIs), it's easiest to just create an anonymous instance at the point of use with `reify`.

For example, the `java.util.concurrent.ThreadFactory` interface is used to provide Java executors with newly constructed (and properly configured) threads. The `ThreadFactory` class has just one method, `newThread(Runnable r)`.

When creating an executor service, you need a single instance of a properly constructed `ThreadFactory`, and it's easiest to construct one using `reify` to create an anonymous instance implementing the interface.

```
(import [java.util.concurrent ThreadFactory Executors])

(defn create-compute-executor [n]
  (Executors/newFixedThreadPool n
    (reify ThreadFactory
      (newThread [this runnable]
        (doto (Thread. runnable)
          (.setName "Compute")
          (.setDaemon false))))))
```

In this example, `create-compute-executor` uses a helper method on `Executors` to construct a fixed-size thread pool of the specified size. The anonymous `ThreadFactory` instance creates a thread named "Compute" with its daemon status set to false.

The `reify` function takes an interface, which is followed by implementations of each method in that interface. The first argument to each method is always

the anonymous instance itself, which is commonly called this (although there's nothing special about this name). Any number of interfaces can be implemented in a single call to `reify`. Any interface methods that are not specified will be added automatically and will throw an `UnsupportedOperationException`.

In other cases, you'll want to create an object with data fields and a type that implements a Java interface. Both `defrecord` and `deftype` can be used to implement interfaces inline.

For example, a `Counter` instance could have a field `n` for the maximum count. You could create this with `reify`, but you'd be unable to get or modify the data field inside the instance, and you'd have no concrete type for the instances.

Instead, we could use `defrecord` to implement `Runnable` directly:

```
(defrecord Counter [n]
  Runnable
  (run [this] (println (range n))))
-> user.Counter

(def c (->Counter 5))
-> #'user/c

(.start (Thread. c))
-> (0 1 2 3 4)
```

The advantage here is that `c` is not just an opaque anonymous object. It has a concrete type (`user.Counter`), which can be used for polymorphism and a field `n` that can be retrieved and/or updated.

```
(:n c)
-> 5

(def c2 (assoc c :n 8))
-> #'user/c2

(.start (Thread. c2))
-> (0 1 2 3 4 5 6 7)
```

To create stateful objects, we extend this further by using fields that hold reference types like atoms.

So far, we've only looked at implementing interfaces. Next, we'll consider how to deal with the need to extend an actual class.

Converting Clojure Functions to Java Functional Interfaces

Java emulates functions with single-method interfaces marked with the `FunctionalInterface` annotation. When invoking a Java method that takes a func-

tional interface, Clojure functions may be passed directly and will be automatically converted to the intended Java functional interface.

For example, the `java.io.File` class contains a method `list(FilenameFilter filter)` to obtain a list of files in a directory that satisfy the filter. The `FilenameFilter` class has just one method, `accept(File dir, String name)`, and is a functional interface.

Consider building a function `list-files` that takes a directory and a suffix and returns a sequence of all files with that suffix in the directory:

```
(import [java.io File])

(defn list-files [^String dir suffix]
  (seq
    (.list (File. dir)
      (fn [_dir ^String name] (.endsWith name suffix)))))

(list-files "." ".clj")
-> ("project.clj")
```

When we invoke the `list` method, we need to pass a suffix-accepting instance of `FilenameFilter`. Because that is a functional interface, we can instead simply pass a Clojure function that matches the `FilenameFilter` signature, and conversion happens automatically.

Additionally, you can explicitly coerce a Clojure function to a Java functional interface in a `let` binding by adding a type hint to the binding name. For example, here the `suffix-filter` binding is a Clojure function. Applying the `^FilenameFilter` type hint coerces the Clojure function to be an instance of `FunctionFilter`:

```
(import [java.io File FilenameFilter])

(defn list-files [^String dir suffix]
  (let [^FilenameFilter suffix-filter (fn [_dir ^String name]
                                       (.endsWith name suffix))]
    (seq (.list (File. dir) suffix-filter))))
```

Once `suffix-filter` is bound in the `let` it can be used thereafter without incurring a conversion each time.

Extending Classes with Proxies

In addition to interfaces, many Java APIs provide base classes that can be used to ease the implementation of custom instances of an interface. In particular, this is common in places like Swing or XML parsers. Clojure can easily generate one-off proxies or classes on disk when needed.

A good example is parsing XML with a Simple API for XML (SAX) parser. To get ready for this example, go ahead and import the classes listed at the top of the next page. We'll need them all before we're done.

```
(import [org.xml.sax InputSource]
        [org.xml.sax.helpers DefaultHandler]
        [java.io StringReader]
        [javax.xml.parsers SAXParserFactory])
```

To use a SAX parser, you need a callback mechanism. Often, it's easiest to extend the DefaultHandler class. In Clojure, you can extend a class with the proxy function:

```
(proxy class-and-interfaces super-cons-args & fns)
```

As a simple example, use proxy to create a DefaultHandler that prints the details of all calls to startElement:

```
(def print-element-handler
  (proxy [DefaultHandler] []
    (startElement [uri local qname atts]
      (println (format "Saw element: %s" qname)))))
```

proxy generates an instance of a proxy class. The first argument to proxy is [DefaultHandler], a vector of the superclass and superinterfaces. The second argument, [], is a vector of arguments to the base class constructor. In this case, no arguments are needed.

After the proxy setup, comes the implementation code for zero or more proxy methods. The proxy shown earlier has one method. Its name is startElement, and it takes four arguments and prints the name of the qname argument.

Now, you need a parser to pass the handler to. This requires plowing through a pile of Java factory methods and constructors. For a simple exploration at the REPL, you can create a function that parses XML in a string:

```
(defn demo-sax-parse [source handler]
  (.. SAXParserFactory newInstance newSAXParser
    (parse (InputSource. (StringReader. source)) handler)))
```

Now, the parse is easy:

```
(demo-sax-parse "<foo>
  <bar>Body of bar</bar>
</foo>" print-element-handler)
| Saw element: foo
| Saw element: bar
```

This example shows how to create a Clojure proxy to deal with Java's XML interfaces. You can take a similar approach to implementing your own custom

Java interfaces. But if all you're doing is XML processing, the `data.xml` library has terrific XML support and can work with any SAX-compatible Java parser.

For one-off tasks, such as XML and thread callbacks, Clojure's proxies are quick and easy to use. If you need a longer-lived class, you can generate an entirely new class from Clojure using `gen-class`, which is an advanced topic that won't be discussed here.

If you noticed the `..` above, this is a macro similar to the threading macros that rewrites a series of methods into a chain of invocations. We'll examine it in more detail in [Expanding Macros, on page 243](#).

Now that we've seen how to invoke Java APIs from Clojure, let's turn things around and consider how to invoke Clojure from Java.

Calling Clojure from Java

While using Java APIs from Clojure is common due to Clojure's embrace of the host platform, there are times when you'll need to invoke Clojure directly from Java. For these cases, Clojure provides a Java API¹ that can be used to directly access Clojure namespaces, vars, and functions.

The main entry point for the Clojure Java API is the `clojure.java.api.Clojure` class, which provides a few static methods for reading data, finding vars, and invoking functions. With this handful of tools, you can invoke almost any aspect of Clojure.

For example, in a Java program, you can find the `+` function by using the `var` method of the `Clojure` class, which takes the namespace and name of a var and returns its value (in this case, a function). All Clojure function instances implement the `clojure.lang.IFn` interface, which can be invoked with arguments:

```
IFn plus = Clojure.var("clojure.core", "+");
System.out.println(plus.invoke(1, 2, 3));
```

(The Java interfaces for Clojure functions generally take `Object` and return `Object`.)

The `Clojure` class also provides a `read` method that can be used to read literal data into Clojure data structures:

```
Object vector = Clojure.read("[1 2 3]");
```

By default, the Clojure Java API requires `clojure.core`, and thus all core functions are available without needing to load them. When you need to load another namespace, you can do so using `require` through the `var` interface to load namespaces just like you would at the REPL:

1. <https://clojure.github.io/clojure/javadoc>

```
IFn require = Clojure.var("clojure.core", "require");
require.invoke(Clojure.read("clojure.set"));
```

As you can see, the provided tools are just the minimum needed to read, load, and invoke Clojure functions. When designing a Java API into Clojure code, the best approach is to create that API via a set of Java interfaces, implement those interfaces in Clojure via protocols, records, or reify, and then use the Clojure Java API solely to construct the initial interface implementation via a Java factory method.

Next, we'll look at how to handle catching and throwing exceptions in our Clojure code.

Exception Handling

In Java code, exception handling crops up for three reasons:

- Wrapping checked exceptions
- Using a finally block to clean up nonmemory resources, such as file and network handles
- Responding to the problem: ignoring the exception, retrying the operation, converting the exception to a nonexceptional result, and so on

Checked Exceptions

Java's checked exceptions must be explicitly caught or rethrown from every method where they can occur in Java. This seemed like a good idea at first: checked exceptions could use the type system to rigorously document error handling, with compiler enforcement. Most Java programmers now consider checked exceptions to be a failed experiment because the costs in code bloat and maintainability outweigh their advantages. For more on the history of checked exceptions, see Rod Waldhoff's article^a and the accompanying links.

a. <https://radio-weblogs.com/0122027/stories/2003/04/01/javasCheckedExceptionsWereAMistake.html>

In Clojure, things are similar but simpler. The try and throw special forms give you all of the capabilities of Java's try, catch, finally, and throw. But you shouldn't have to use them very often, because Clojure doesn't require you to deal with checked exceptions, and there are helpful macros like with-open to encapsulate resource cleanup.

Let's see what this looks like in practice.

Keeping Exception Handling Simple

Java programs often wrap checked exceptions at abstraction boundaries. A good example is Apache Ant, which tends to wrap low-level exceptions (such as I/O exceptions) with an Ant-level build exception:

```
try {
    newManifest = new Manifest(r);
} catch (IOException e) {
    throw new BuildException(...);
}
```

In Clojure, you're not forced to deal with checked exceptions. You don't have to catch them or declare that you throw them. So, the previous code would translate to the following:

```
(Manifest. r)
```

The absence of exception wrappers makes idiomatic Clojure code easier to read, write, and maintain than idiomatic Java. That said, nothing prevents you from explicitly catching, wrapping, and rethrowing exceptions in Clojure. It simply is not required. You *should* catch exceptions when you plan to respond to them in a meaningful way, and in the next exception, we'll see how Clojure handles this in its data-centric way.

Rethrowing with ex-info

In Java, it's common to create many custom exception subclasses corresponding to all manner of contingencies. Often, these are built in deeply nested exception hierarchies. In practice, the majority of these exception classes add little value beyond their specific class name, which can be caught and handled.

In Clojure, we instead have a single custom exception class provided with the language (`ExceptionInfo`), which carries a map of data where you can place any information that's necessary or useful to handle the error.

For example, a common use for custom exceptions is inside the code that serves a web request. Various error conditions might result in different HTTP status codes. Rather than have dozens of exceptions, we can simply throw a custom exception with a map of data, including the status code that should be returned:

```
(defn load-resource
  [path]
  (try
    (if (forbidden? path)
      (throw (ex-info "Forbidden resource"
                     {:status 403, :resource path})))
    (slurp path)))
```



```
(catch FileNotFoundException e
  (throw (ex-info "Missing resource"
    {:status 404, :resource path})))
(catch IOException e
  (throw (ex-info "Server error"
    {:status 500, :resource path}))))
```

The `load-resource` function first checks to see whether the resource is forbidden. If so, we throw a 403. Otherwise, we try to read and return the resource. We also handle two different types of errors: 1) the case where something is missing, and 2) an unknown IO failure. In all of these cases, we use the `ex-info` function to create a custom exception instance with a message and the map of data.

Higher up the call stack, some other code can catch this exception (with type `clojure.lang.ExceptionInfo`) and retrieve the map of data using `ex-data`. This handler or middleware could then construct the proper HTTP response message to return.

When using external resources, it's important to properly close and dispose of those resources. Often, in Java, this can become a tangle of exception handling code. In the next section, we'll look at some options Clojure provides for easier cleanup.

Cleaning Up Resources

Garbage collection will clean up resources in memory. If you use resources that live outside of garbage-collected memory, such as file handles, you need to make sure that you clean them up, even in the event of an exception. In Java, this is normally handled in a `finally` block or using a `try-with-resources` statement.

If the resource you need to free follows the convention of having a `close` method, you can use Clojure's `with-open` macro:

```
(with-open [name init-form] & body)
```

Internally, `with-open` creates a `try` block, sets `name` to the result of `init-form`, and then runs the forms in `body`. Most important, `with-open` always closes the object bound to `name` in a `finally` block. A good example of `with-open` is the `spit` function in `clojure.string`:

```
(clojure.core/spit file content)
```

`spit` simply writes a string to file. Try it:

```
(spit "hello.out" "hello, world")
-> nil
```

You should now find a file at `hello.out` with the contents `hello, world`.

The implementation of `spit` is simple:

```
; from clojure.core
(defn spit
  "Opposite of slurp. Opens f with writer, writes content, then
  closes f. Options passed to clojure.java.io/writer."
  {:added "1.2"}
  [f content & options]
  (with-open [^java.io.Writer w (apply jio/writer f options)]
    (.write w (str content)))))
```

`spit` creates a `PrintWriter` on `f`, which can be just about anything that is writable: a file, a URL, a URI, or any of Java's various writers or output streams. It then prints content to the writer. Finally, `with-open` guarantees that the writer is closed at the end of `spit`.

If you need to do something other than close in a finally block, the Clojure `try` form looks like this:

```
(try expr* catch-clause* finally-clause?)
; catch-clause -> (catch classname name expr*)
; finally-clause -> (finally expr*)
```

You can use it as follows:

```
(try
  (throw (Exception. "something failed")))
(finally
  (println "we get to clean up")))
| we get to clean up
-> java.lang.Exception: something failed
```

The previous fragment also demonstrates Clojure's `throw` form, which simply throws whatever exception is passed to it.

Responding to an Exception

The most interesting case is when an exception handler attempts to respond to the problem in a `catch` block. As a simple example, write a function to test whether a particular class is available at runtime:

```
src/examples/interop.clj
; not caller-friendly
(defn class-available? [class-name]
  (Class/forName class-name))
```

This approach is not very caller-friendly. The caller just wants a yes/no answer but instead gets an exception:

```
(class-available? "borg.util.Assimilate")
| Execution error (ClassNotFoundException) at ...
| borg.util.Assimilate
```

A friendlier approach uses a catch block to return false:

```
src/examples/interop.clj
(defn class-available? [class-name]
  (try
    (Class/forName class-name) true
    (catch ClassNotFoundException _ false)))
```

The caller experience is much better now:

```
(class-available? "borg.util.Assimilate")
-> false

(class-available? "java.lang.String")
-> true
```

Clojure gives you everything you need to throw and catch exceptions and to cleanly release resources. At the same time, Clojure keeps exceptions in their place. They're important but not so important that your mainline code is dominated by the exceptional.

Optimizing for Performance

In Clojure, it's idiomatic to call Java using the techniques described in [Calling Java, on page 37](#). The resulting code will be fast enough for 90 percent of scenarios. When you need to, though, you can make localized changes to boost performance. These changes will not change how outside callers invoke your code, so you're free to make your code work and then make it fast.

One of the most common ways to make Java interop faster is by adding type hints to remove reflective calls. We'll look at that first, and then we'll consider how to use Java primitives and arrays as a way to optimize memory use and numeric calculation performance.

Adding Type Hints

Clojure supports adding type hints to function parameters, let bindings, variable names, and expressions. These type hints serve three purposes:

- Disambiguate Java interop calls
- Optimize critical performance paths
- Document the expected type

For example, consider the following function, which returns information about a Java class:

```
(defn describe-class [c]
  {:name (.getName c)
   :final (java.lang.reflect.Modifier/isFinal (.getModifiers c))})
```

You can ask Clojure how much type information it can infer by setting the special dynamic variable `*warn-on-reflection*` to true:

```
(set! *warn-on-reflection* true)
-> true
```

The exclamation point on the end of `set!` is an idiomatic indication that `set!` changes mutable state. `set!` is described in detail in [Working with Java Callback APIs, on page 172](#). With `*warn-on-reflection*` set to true, compiling `describe-class` will produce the following warnings:

```
Reflection warning, line: 87
- reference to field getName can't be resolved.

Reflection warning, line: 88
- reference to field getModifiers can't be resolved.
```

These warnings indicate that Clojure has no way of knowing the type of `c` so it falls back to runtime reflection. You can provide a type hint to fix this, using the metadata syntax `^Class`:

```
(defn describe-class [^Class c]
  {:name (.getName c)
   :final (java.lang.reflect.Modifier/isFinal (.getModifiers c))})
```

With the type hint in place, the reflection warnings will disappear. The compiled Clojure code will be exactly the same as the compiled Java code. Further, attempts to call `describe-class` with something other than a `Class` will fail with a `ClassCastException`:

```
(describe-class StringBuffer)
-> {:name "java.lang.StringBuffer", :final true}

(describe-class "foo")
| Execution error (ClassCastException) at user/describe-class (REPL:1).
| java.lang.String cannot be cast to java.lang.Class
```

When you provide a type hint, Clojure will insert an appropriate class cast to avoid making slow, reflective calls to Java methods. But if your function doesn't actually call any Java methods on a hinted object, then Clojure will not insert a cast. Consider this `wants-a-string` function:

```
(defn wants-a-string [^String s] (println s))
-> #'user/wants-a-string
```

You might expect that `wants-a-string` would complain about nonstring arguments. In fact, it'll be perfectly happy:

```
(wants-a-string "foo")
| foo

(wants-a-string 0)
| 0
```

Clojure can tell that `wants-a-string` never actually uses its argument as a string (`println` will happily try to print any kind of argument). Since no string methods need to be called, Clojure doesn't attempt to cast `s` to a string.

If you want to choose a specific method, qualified method invocations can also be adorned with special *param-tags* metadata. This metadata annotation is in the form of a vector specifying the type arguments of the intended method. For example, the `java.util.Arrays` class has a set of `binarySearch` methods overloaded on different array types. If you want to choose a specific one, you can use *param-tags*:

```
(import java.util.Arrays)
-> java.util.Arrays
(def data (long-array [0 10 20 30 99 100]))
-> #'data
(^[long/1 long] Arrays/binarySearch data 99)
-> 4
```

This example creates a long array of data and uses `binarySearch` to find the index of 99 in the array. The *param-tags* adorn the `Arrays/binarySearch` method (only qualified methods support *param-tags*). The first type, `long/1`, uses the syntax for a long array of dimension 1. Parameters that are not relevant to resolve an ambiguous overloaded method can use `_` as a wildcard.

When you need speed, type hints will let Clojure code compile down to the same code Java will produce. But you won't need type hints that often. Make your code work first, and then worry about making it fast.

Integer Math

Clojure provides three different sets of operations for integer types:

- The default operators
- The promoting operators
- The unchecked operators

The following table gives a sampling of these operator types.

Default	Promoting	Unchecked
+	+'	unchecked-add
-	-'	unchecked-subtract
*	*'	unchecked-multiply
inc	inc'	unchecked-inc
dec	dec'	unchecked-dec

The unchecked operators correspond exactly with primitive math in Java. They are fast but dangerous in that they can overflow silently and give incorrect answers. In Clojure, the unchecked operators should be used only in the rare situation that overflow is the desired behavior (like hashing) or to match the behavior of Java exactly.

```
(unchecked-add 9223372036854775807 1)
-> -9223372036854775808
```

The default operators use Java primitives where possible for performance but always make overflow checks and throw an exception.

```
(+ 9223372036854775807 1)
| Execution error (ArithmeticException) at java.lang.Math/addExact.
| long overflow
```

The promoting operators automatically promote from primitives to big numbers on overflow. This makes it possible to handle an arbitrary range but at significant performance cost. Because primitives and big numbers share no common base type, math with the promoting operators precludes the use of primitives as return types.

```
(+' 9223372036854775807 1)
-> 9223372036854775808N
```

Clojure relies on Java's `BigDecimal` class for arbitrary-precision decimal numbers. See the online documentation² for details. `BigDecimal`s provide arbitrary precision but at a price: `BigDecimal` math is significantly slower than Java's floating-point primitives.

Clojure has its own `BigInt` class to handle `BigInteger` conversions. Clojure's `BigInt` has some performance improvements over using Java's `BigInteger` directly. It also wraps some of the rough edges of `BigInteger`. In particular, it properly implements `hashCode`. This makes equality take precedence over representation, which you'll see in almost every abstraction in the language.

2. <https://docs.oracle.com/javase/8/docs/api/java/math/BigDecimal.html>

Under the hood, Clojure uses Java's `BigInteger`. The performance difference comes in how `BigInt` treats its values. A `BigInt` consists of a long part and a `BigInteger` part. When the value passed into a `BigInt` is small enough to be treated as a long, it is. When numerical operations are performed on `BigInts`, if their result is small enough to be treated as a long, it is. This gives the user the ability to add the overflow hint (`N`) without paying the `BigInteger` cost until it's absolutely necessary.

Using Java Arrays

Clojure's collections supplant the Java collections for most purposes. Clojure's collections are concurrency safe, have good performance characteristics, and implement the appropriate Java collection interfaces. So, you should generally prefer Clojure's own collections when you're working in Clojure and even pass them back into Java when convenient. If you do choose to use the Java collections, nothing in Clojure will stop you. From Clojure's perspective, the Java collections are classes like any other, and all of the various Java interop forms will work. But the Java collections are designed for lock-based concurrency. They will not provide the concurrency guarantees that Clojure collections do and won't work well with Clojure's software transactional memory.

One place where you'll need to deal with Java collections is the special case of Java arrays. In Java, arrays have their own syntax and their own bytecode instructions. Java arrays don't implement any Java interface. Clojure collections cannot masquerade as arrays. (Java collections can't either!) The Java platform makes arrays a special case in every way, so Clojure does, too.

Clojure provides `make-array` to create Java arrays:

```
(make-array class length)
(make-array class dim & more-dims)
```

`make-array` takes a class and a variable number of array dimensions. For a one-dimensional array of strings, you might say this:

```
(make-array String 5)
-> #object["[Ljava.lang.String;" 0x6a129a7d "[Ljava.lang.String;@6a129a7d"]
```

The odd output is courtesy of Java's implementation of `toString()` for arrays: `[Ljava.lang.String;` is the JVM specification's encoding for "one-dimensional array of strings." That's not very useful at the REPL, so you can use Clojure's `seq` to wrap any Java array as a Clojure sequence so that the REPL can print the individual array entries:

```
(seq (make-array String 5))
-> (nil nil nil nil nil)
```

Clojure also includes a family of functions with names such as `int-array` for creating arrays of Java primitives. You can issue the following command at the REPL to review the documentation for these and other array functions:

```
(find-doc "-array")
```

Clojure provides a set of low-level operations on Java arrays, including `aset`, `aget`, and `alength`:

```
(aset java-array index value)
(aset java-array index-dim1 index-dim2 ... value)
(aget java-array index)
(aget java-array index-dim1 index-dim2 ...)
(alength java-array)
```

Use `make-array` to create an array and then experiment with using `aset`, `aget`, and `alength` to work with the array:

```
(defn painstakingly-create-array ^String/1 []
  (let [^String/1 arr (make-array String 5)]
    (aset arr 0 "Painstaking")
    (aset arr 1 "to")
    (aset arr 2 "fill")
    (aset arr 3 "in")
    (aset arr 4 "arrays")
    arr))

(aget (painstakingly-create-array) 0)
-> "Painstaking"

(alength (painstakingly-create-array))
-> 5
```

Most of the time, you'll find it simpler to use higher-level functions such as `to-array`, which creates an array directly from any collection:

```
(to-array sequence)
```

`to-array` always creates an Object array:

```
(to-array ["Easier" "array" "creation"])
-> #object["Ljava.lang.Object;" 0x23aa363a "[Ljava.lang.Object;@23aa363a"]
```

`to-array` is also useful for calling Java methods that take a variable argument list, such as `String/format`:

```
; example. prefer clojure.core/format
(String/format "Training Week: %s Mileage: %d"
  (to-array [2 26]))
-> "Training Week: 2 Mileage: 26"
```


to-array's cousin into-array can create an array with a more specific type than Object:

```
(into-array type? seq)
```

You can pass an explicit type as an optional first argument to into-array:

```
(into-array String ["Easier", "array", "creation"])
-> #object["[Ljava.lang.String;" 0x21072f13 "[Ljava.lang.String;@21072f13"]
```

If you omit the type argument, into-array will guess the type based on the first item in the sequence:

```
(into-array ["Easier" "array" "creation"])
-> #object["[Ljava.lang.String;" 0x88821c2 "[Ljava.lang.String;@88821c2"]
```

As you can see, the array contains Strings, not Objects. If you want to transform every element of a Java array without converting to a Clojure sequence, you can use amap:

```
(amap a idx ret expr)
```

amap creates a clone of the array a, binding that clone to the name you specify in ret. It then executes expr once for each element in a, with idx bound to the index of the element. Finally, amap returns the cloned array. You could use amap to uppercase every string in an array of strings:

```
(def strings (into-array ["some" "strings" "here"]))
-> #'user/strings

(seq (amap strings idx _ (.toUpperCase (aget strings idx))))
-> ("SOME" "STRINGS" "HERE")
```

The ret parameter is set to _ to indicate that it's not needed in the map expression, and the wrapping seq is simply for convenience in printing the result at the REPL. Similar to amap is areduce:

```
(areduce a idx ret init expr)
```

Where amap produces a new array, areduce produces anything you want. The ret is initially set to init and later set to the return value of each subsequent invocation of expr. areduce is normally used to write functions that “tally up” a collection in some way. For example, the following call finds the length of the longest string in the strings array:

```
(areduce strings idx ret 0 (max ret (.length (aget strings idx))))
-> 7
```

amap and areduce are special-purpose macros for interoperating with Java arrays.

Handling Java Streams

Java streams represent a sequence of values on which you can perform a series of pipelined functional operations. Java developers use streams and functional lambda expressions to write code similar to Clojure's use of sequences and the functional sequence API. One important difference is that Clojure sequences are immutable while Java streams are stateful and may only be consumed once.

There are two primary paths in Clojure to working with Java streams: invoke them using standard Java interop calls, or transform the data into types that Clojure already handles (collections, sequences, reduce, or transduce).

For example, recall the line-counting example from [Seq-ing a Character Stream, on page 86](#) with this inner loop to count the lines in a file:

```
(with-open [rdr (reader file)]
  (count (filter non-blank? (line-seq rdr))))
```

We can easily rewrite this to use Java streams instead. The reader function returns a `BufferedReader` which has a `lines` method to return a `Stream` of lines from the reader. Java streams have a `filter` method that takes a `Predicate`, which is a Java functional interface of `Object` to `boolean`. Clojure will automatically convert Clojure functions to Java functional interfaces, so we can continue using our existing helper function:

```
(with-open [^BufferedReader rdr (reader file)]
  (.. rdr lines (filter non-blank?) count))
```

The final call in this chain just calls the `Stream` `count` method. While this example looks almost identical to the prior example, note that `filter` and `count` in this example are stream methods, not the Clojure functions from the previous example.

The other approach is to use one of the Clojure functions that transform streams into collections, sequences, or reductions. These functions are structurally identical to `into`, `seq`, `reduce`, or `transduce`:

- `(stream-into! to [xform] stream)` to fill a collection
- `(stream-seq! stream)` to create a lazy `seq`
- `(stream-reduce! f [init] stream)` to reduce over the stream
- `(stream-transduce! xform f [init] stream)` to transduce over the stream

Stream support was not added to the equivalent `seq` functions because the stream is consumed (and modified), and this behavioral difference is important enough to warrant different names.

One reason to use streams directly is to get access to their parallel pipeline capabilities. For large amounts of data, parallel streams can provide a significant boost to performance. With automatic function conversion, you can still write code that feels almost like native Clojure `seq` operations.

A Real-World Example

While it's great to talk about the different interop cases and learn how to eke out some additional performance, you still need to have some practical, hands-on knowledge. In this example, we will build an application to test the availability of websites. The goal here is to check whether the website returns an HTTP 200 OK response. If anything other than our expected response is received, the website should be marked as unavailable.

Let's start by creating a directory to hold our project and switch to it:

```
mkdir pinger
cd pinger
```

By default, source files will be loaded from the `src` directory, according to the namespace of the code. We plan to work in the `pinger.core` namespace, so we need to create the directory structure:

```
mkdir -p src/pinger
```

First, we need to write the code that connects to a URL and captures the response code. We can accomplish this by using Java's `URL` class. We'll create this code in `src/pinger/core.clj`:

```
(ns pinger.core
  (:import [java.net URL HttpURLConnection]))

(defn response-code [address]
  (let [conn ^HttpURLConnection (.openConnection (URL. address))
        code (.getResponseCode conn)]
    (when (< code 400)
      (-> conn .getInputStream .close))
    code))
```

You should load this project in your editor and start a connected REPL as discussed in [Chapter 3, Developing Interactively, on page 49](#). You might want to try your function so far by creating a rich comment block in `pinger.core`, loading the namespace in your REPL, and testing it out by evaluating a call to `response-code` in the connected REPL:

```
(comment
  (response-code "https://google.com")
  ;; -> 200
)
```

Now, let's create a predicate function that uses response-code and decides whether the specified URL is available. We will define available in our context as "returning an HTTP 200 response code."

```
(defn available? [address]
  (= 200 (response-code address)))
```

Again, let's try this in the connected REPL (we won't print the comment wrapper each time):

```
(available? "https://google.com")
-> true

(available? "https://google.com/badurl")
-> false
```

Next, we need a way to start our program and have it check a list of URLs that we care about every so often and report their availability. Let's create a -main function.

```
(defn -main []
  (let [addresses ["https://google.com"
                  "https://clojure.org"
                  "https://google.com/badurl"]]
    (while true
      (doseq [address addresses]
        (println address ":" (available? address)))
      (Thread/sleep (* 1000 60)))))
```

In this example, we create a list of addresses (two good and one bad) and use a simple while loop that never exits to obtain a never-ending program execution. It will continue to check these URLs once a minute until the program is terminated.

It's time to run our program from the command line:

```
clj -M -m pinger.core
https://google.com : true
https://clojure.org : true
https://google.com/badurl : false
```

You should see your program start and continue to run until you press Ctrl-C to stop it.

A while loop that's always true will continue to run until terminated, but it's not really the cleanest way to obtain the result because it doesn't allow for a

clean shutdown. We can use a scheduled thread pool that will start and execute the desired command in a similar fashion as the while loop but with a much greater level of control. Create a file `src/pinger/scheduler.clj` and enter the following code:

```
(ns pinger.scheduler
  (:import [java.util.concurrent Executors ExecutorService
        ScheduledExecutorService
        ScheduledFuture TimeUnit]))

(set! *warn-on-reflection* true)

(defn scheduled-executor
  "Create a scheduled executor."
  ^ScheduledExecutorService [threads]
  (Executors/newScheduledThreadPool threads))

(defn periodically
  "Schedule function f to run on executor e every 'delay'
  milliseconds after a delay of 'initial-delay' Returns
  a ScheduledFuture."
  ^ScheduledFuture
  [^ScheduledExecutorService e f initial-delay delay]
  (.scheduleWithFixedDelay e f initial-delay delay TimeUnit/MILLISECONDS))

(defn shutdown-executor
  "Shutdown an executor."
  [^ExecutorService e]
  (.shutdown e))
```

This namespace provides functions to create and shut down a Java `ScheduledExecutorService`. It also defines a function called `periodically` that will accept an executor, a function, an initial-delay, and a repeated delay. It will execute the function for the first time after the initial delay and then continue to execute the function with the delay specified thereafter. This will continue to run until the thread pool is shut down.

Let's update `pinger.core` to take advantage of the scheduling code and make the `-main` function responsible only for calling a function that starts the loop. Replace the old `-main` with the following functions:

```
(defn check []
  (let [addresses ["https://google.com"
                  "https://clojure.org"
                  "https://google.com/badurl"]]
    (doseq [address addresses]
      (println address ":" (available? address)))))

(def immediately 0)
(def every-minute (* 60 1000))

(defn start
```

```

"REPL helper. Start pinger on executor e."
[e]
(scheduler/periodically e check immediately every-minute))

(defn stop
  "REPL helper. Stop executor e."
  [e]
  (scheduler/shutdown-executor e))

(defn -main []
  (start (scheduler/scheduled-executor 1)))

```

Make sure to update your namespace declaration to include the scheduler code:

```

(ns pinger.core
  (:require [pinger.scheduler :as scheduler])
  (:import [java.net URL HttpURLConnection]))

```

Not everything in the previous sample is necessary, but it makes for more readable code. Adding the start and stop functions makes it easy to work interactively from the REPL, which will be a huge advantage should you choose to extend this example. Give the program another try—everything should function exactly as it did before.

We could easily expand on this example by extending the check for a valid web page, checking its response time, or enhancing how sites are persisted or stored.

Wrapping Up

We just covered a good chunk of how Clojure and Java get along. We even mixed the two up in some interesting ways.

But there's still more. Clojure's macro implementation is easy to learn and use correctly for common tasks and yet powerful enough for complex metaprogramming. In the next chapter, you'll see how Clojure is bringing macros to mainstream programming.

Macros

Macros give Clojure great power. With most programming techniques, you build features *within* the language. When you write macros, it's more accurate to say that you're *adding features to* the language. This is a powerful capability, so you should follow the rules in this chapter until you have enough experience to decide for yourself when to deviate. We'll explore an example of how to use macros to add a new feature to Clojure.

While powerful, macros are not always simple. Clojure works to make macros as simple as is feasible by including conveniences to solve many common problems that occur when writing macros. We'll explain these problems and show how Clojure mitigates them.

Macros are so different from other programming idioms that you may struggle to know when to use them. There's no better guide than the shared experience of the community, so we'll close the chapter by introducing a taxonomy of Clojure macros, based on the macros in Clojure and contrib libraries.

When to Use Macros

In 1996, author Chuck Palahniuk released the novel *Fight Club*, which was later made into a movie. The so-called fight club in the story had a set of rules. The first rule of fight club was, "You don't talk about fight club."

In a similar spirit, we introduce Macro Club. Macro Club has two rules, plus one exception. The first rule of Macro Club is, "Don't write macros." Macros are complex, and they require you to think carefully about the interplay of macro expansion time and compile time. If you can write it as a function, think twice before using a macro.

The second rule of Macro Club is, "Write macros if that is the only way to encapsulate a pattern." All programming languages provide some way to

encapsulate patterns, but without macros these mechanisms are incomplete. In most languages, you sense that incompleteness whenever you say, “My life would be easier if only my language had feature X.” In Clojure, you just implement feature X using a macro.

An exception to the second rule is that you can also write macros if that makes life easier for your callers when compared to the equivalent function. But to understand this exception, you need some practice writing macros and comparing them to functions. So, let’s get started with an example.

Writing a Control Flow Macro

Clojure provides the `if` special form as part of the language:

```
(if (= 1 1) (println "yep, math still works today"))
| yep, math still works today
```

Some languages have an `unless`, which is (almost) the opposite of `if`. `unless` performs a test and then executes its body only if the test is logically false.

Clojure doesn’t have `unless`, but it does have an equivalent macro called `when-not`. For the sake of having a simple example to start with, let’s pretend that `when-not` doesn’t exist and create an implementation of `unless`. To follow the rules of Macro Club, begin by trying to write `unless` as a function:

```
src/examples/macros.clj
; This is doomed to fail...
(defn unless [expr form]
  (if expr nil form))
```

Check that `unless` correctly evaluates its form when its test `expr` is false:

```
(unless false (println "this should print"))
| this should print
```

Things look fine so far. But let’s be diligent and test the true case, too:

```
(unless true (println "this should not print"))
| this should not print
```

Clearly, something has gone wrong. The problem is that Clojure evaluates all of the arguments before passing them to a function, so the `println` is called before `unless` *ever sees it*. In fact, both calls to `unless` earlier call `println` too soon, before entering the `unless` function. To see this, add a `println` inside `unless`:

```
(defn unless [expr form]
  (println "About to test...")
  (if expr nil form))
```


Now, you can clearly see that function arguments are always evaluated before they are passed to `unless`:

```
(unless false (println "this should print"))
| this should print
| About to test...

(unless true (println "this should not print"))
| this should not print
| About to test...
```

Macros solve this problem because they don't evaluate their arguments immediately. Instead, you get to choose when (and if!) the arguments to a macro are evaluated.

When Clojure encounters a macro, it processes it in two steps. First, it expands (executes) the macro and substitutes the result back into the program. This is called *macro expansion time*. Then, it continues with the normal *compile time*.

To write `unless`, you need to write Clojure code to perform the following translation at macro expansion time:

```
(unless expr form) -> (if expr nil form)
```

Then, you need to tell Clojure that your code is a macro by using `defmacro`, which looks almost like `defn`:

```
(defmacro name doc-string? attr-map? [params*] body)
```

Because Clojure code is just Clojure data, you already have all of the tools you need to write `unless`. Macros are essentially functions that take code (Clojure data) and emit new code (Clojure data). Write the `unless` macro using `list` to build the `if` expression:

```
(defmacro unless [expr form]
  (list 'if expr nil form))
```

The body of `unless` executes at macro expansion time, producing an `if` form for compilation. If you enter this expression at the REPL:

```
(unless false (println "this should print"))
```

then Clojure will (invisibly to you) expand the `unless` form into the following:

```
(if false nil (println "this should print"))
```

Then, Clojure compiles and executes the expanded if form. Verify that unless works correctly for both true and false:

```
(unless false (println "this should print"))
| this should print
-> nil

(unless true (println "this should not print"))
-> nil
```

Congratulations, you have written your first macro. unless may seem pretty simple, but consider this: what you have just done is *impossible* in most languages. In languages without macros, special forms get in the way.

Special Forms, Design Patterns, and Macros

Clojure has no special syntax for code. Code is composed of data structures. This is true for normal functions and for special forms and macros.

Consider a language with more syntactic variety, such as Java. In Java, the most flexible mechanism for writing code is the instance method. Imagine that you're writing a Java program. If you discover a recurring pattern in some instance methods, you have the entire Java language at your disposal to encapsulate that recurring pattern.

Good so far. But Java also has lots of “special forms” (although they're not normally called by that name). Unlike Clojure special forms, which are just Clojure data, each Java special form has its own syntax. For example, if is a special form in Java. If you discover a recurring pattern of usage involving if, there's *no way to encapsulate* that pattern. You can't create an unless, so you're stuck simulating unless with an idiomatic usage of if:

```
if (!something) ...
```

This may seem like a relatively minor problem. Java programmers can certainly learn to mentally make the translation from if (!foo) to unless (foo). But the problem is not just with if: *every distinct syntactic form* in the language inhibits your ability to encapsulate recurring patterns involving that form.

As another example, Java new is a special form. Polymorphism is not available for new, so you must simulate polymorphism, for example, with an idiomatic usage of a class method:

```
Widget w = WidgetFactory.makeWidget(...)
```

This idiom is a little bulkier. It introduces a whole new class, WidgetFactory. This class is meaningless in the problem domain and exists only to work around the constructor special form. Unlike the unless idiom, the “polymorphic

instantiation” idiom is complicated enough that there’s more than one way to implement a solution. Thus, the idiom should more properly be called a *design pattern*.

Design patterns became popular in the object-oriented community as a way to capture a set of classes and their relationships such that they could be used together to solve a common problem. Patterns are not a set of explicit reusable classes like a library.

That’s where macros fit in. Macros provide a layer of *indirection* so that you can *automate the common parts of any recurring pattern*. Macros and code-as-data work together, enabling you to reprogram your language on the fly to encapsulate patterns.

Of course, this argument doesn’t go entirely in one direction. Many people would argue that having a bunch of special syntactic forms makes a programming language easier to learn or read. We do not agree, but even if we did, we’d be willing to trade syntactic variety for a powerful macro system. Once you get used to code as data, the ability to automate design patterns is a huge payoff.

Expanding Macros

When you created the `unless` macro, you quoted the symbol `if`:

```
(defmacro unless [expr form]
  (list 'if expr nil form))
```

But you didn’t quote any other symbols. To understand why, you need to think carefully about what happens at macro expansion time:

- By quoting `if`, you prevent Clojure from evaluating `if` at macro expansion time. Instead, evaluation strips off the quote, leaving `if` to be compiled.
- You don’t want to quote `expr` and `form`, because they’re macro arguments. Clojure will substitute them without evaluation at macro expansion time.
- You don’t need to quote `nil`, since `nil` evaluates to itself.

Thinking about what needs to be quoted can get complicated quickly. Fortunately, you don’t have to do this work in your head. Clojure includes diagnostic functions so that you can test macro expansions at the REPL.

The function `macroexpand-1` shows you what happens at macro expansion time:

```
(macroexpand-1 form)
```

Use `macroexpand-1` to prove that `unless` expands to a sensible `if` expression:

```
(macroexpand-1 '(unless false (println "this should print")))
-> (if false nil (println "this should print"))
```

Macros are complicated beasts, and we cannot overstate the importance of testing them with `macroexpand-1`. Let's go back and try some incorrect versions of `unless`. Here's one that incorrectly quotes the `expr`:

```
(defmacro bad-unless [expr form]
  (list 'if 'expr nil form))
```

When you expand `bad-unless`, you'll see that it generates the symbol `expr`, instead of the actual test expression:

```
(macroexpand-1 '(bad-unless false (println "this should print")))
-> (if expr nil (println "this should print"))
```

If you try to actually use the `bad-unless` macro, Clojure will complain that it can't resolve the symbol `expr`:

```
(bad-unless false (println "this should print"))
| Syntax error compiling at (REPL:1:1).
| Unable to resolve symbol: expr in this context
```

Sometimes, macros expand into other macros. When this happens, Clojure will continue to expand all macros until only normal code remains. For example, the `..` macro expands recursively, producing a dot operator call, wrapped in another `..` to handle any arguments that remain. You can see this with the following macro expansion:

```
(macroexpand-1 '(.. arm getHand getFinger))
-> (.. (. arm getHand) getFinger)
```

If you want to see `..` expanded all of the way, use `macroexpand`:

```
(macroexpand form)
```

If you `macroexpand` a call to `..`, it will recursively expand until only dot operators remain:

```
(macroexpand '(.. arm getHand getFinger))
-> (. (. arm getHand) getFinger)
```

(It's not a problem that `arm`, `getHand`, and `getFinger` don't exist. You're only expanding them, not attempting to compile and execute them.)

Another recursive macro is `and`. If you call `and` with more than two arguments, it will expand to include another call to `and`, with one less argument:

```
(macroexpand '(and 1 2 3))
-> (let* [and__3585__auto__ 1]
    (if and__3585__auto__ (clojure.core/and 2 3)
      and__3585__auto__))
```

This time, `macroexpand` does *not* expand all the way. `macroexpand` works only against the top level of the form you give it. Since the expansion of `and` creates a new `and` nested inside the form, `macroexpand` does not expand it.

You might also be wondering about the curious `and__3585__auto__` style symbols in that expansion. We'll learn more about generated names later in [Creating Names in a Macro, on page 249](#).

when and when-not

Your `unless` macro could be improved slightly to execute multiple forms, avoiding this error:

```
(unless false (println "this") (println "and also this"))
-> Execution error (ArityException) at clojure.main/main (main.java:40).
   Wrong number of args (3) passed to: user/unless
```

Think about how you would write the improved `unless`. You'd need to capture a variable argument list and stick a `do` in front of it so that every form executes. Clojure provides exactly this behavior in its `when` and `when-not` macros:

```
(when test & body)
```

```
(when-not test & body)
```

`when-not` is the improved `unless` you're looking for:

```
(when-not false (println "this") (println "and also this"))
| this
| and also this
-> nil
```

Given your practice writing `unless`, you should now have no trouble reading the source for `when-not`:

```
; from Clojure core
(defmacro when-not [test & body]
  (list 'if test nil (cons 'do body)))
```

And, of course, you can use `macroexpand-1` to see how `when-not` works:

```
(macroexpand-1 '(when-not false (print "1") (print "2")))
-> (if false nil (do (print "1") (print "2")))
```

when is the opposite of when-not and executes its forms only when its test is true. Note that when differs from if in two ways:

- if allows an else clause, and when does not. This reflects English usage, because nobody says “when ... else.”
- Since when does not have to use its second argument as an else clause, it’s free to take a variable argument list and execute all of the arguments inside a do.

You don’t really need an unless macro. Just use Clojure’s when-not. Always check to see whether somebody else has written the macro you need.

Making Macros Simpler

The unless macro is a great, simple example, but most macros are more complex. In this section, we’ll build a set of increasingly complex macros, introducing Clojure features as we go. For your reference, the following table summarizes the features introduced.

Form	Description
foo#	Auto-gensym: Inside a syntax-quoted section, create a unique name prefixed with foo.
(gensym prefix?)	Create a unique name, with optional prefix.
(macroexpand form)	Expand form with macroexpand-1 repeatedly until the returned form is no longer a macro.
(macroexpand-1 form)	Show how Clojure will expand form.
(... ~@form ...)	Splicing unquote: Use inside a syntax quote to splice an unquoted list into a template.
`form	Syntax quote: Quote form, but allow internal unquoting so that form acts as a template. Symbols inside form are resolved to help prevent inadvertent symbol capture.
~form	Unquote: Use inside a syntax quote to substitute an unquoted value.

First, let’s build a replica of Clojure’s .. macro. We’ll call it chain, since it chains a series of method calls. Here are some sample expansions of chain:

Macro Call	Expansion
(chain arm getHand)	(. arm getHand)
(chain arm getHand getFinger)	(. (. arm getHand) getFinger)

Begin by implementing the simple case where the chain calls only one method. The macro needs only to make a simple list:

```
src/examples/macros/chain_1.clj
; chain reimplements Clojure's .. macro
(defmacro chain [x form]
  (list '. x form))
```

chain needs to support any number of arguments, so the rest of the implementation should define a recursion. The list manipulation becomes more complex, since you need to build two lists and concat them together:

```
src/examples/macros/chain_2.clj
(defmacro chain
  ([x form] (list '. x form))
  ([x form & more] (concat (list 'chain (list '. x form)) more)))
```

Test chain using macroexpand to make sure it generates the correct expansions:

```
(macroexpand '(chain arm getHand))
-> (. arm getHand)

(macroexpand '(chain arm getHand getFinger))
-> (. (. arm getHand) getFinger)
```

The chain macro works fine as written, but it's difficult to read the expression that handles more than one argument:

```
(concat (list 'chain (list '. x form)) more)))
```

The definition of chain oscillates between macro code and the body to be generated. The intermingling of the two makes the entire thing hard to read. And this is just a baby of a form, only one line in length. As macro forms grow more complex, assembly functions such as list and concat quickly obscure the meaning of the macro.

One solution to this kind of problem is a templating language. If macros were created from templates, you could take a “fill-in-the-blanks” approach to creating them. The definition of chain might look like this:

```
; hypothetical templating language
(defmacro chain
  ([x form] (. ${x} ${form}))
  ([x form & more] (chain (. ${x} ${form}) ${more})))
```

In this hypothetical templating language, the `${}` lets you substitute arguments into the macro expansion.

Notice how much easier the definition is to read and how it clearly shows what the expansion will look like.

Syntax Quote, Unquote, and Splicing Unquote

Clojure macros support templating without introducing a separate language. The *syntax quote* character, which is a backquote (```), works almost like normal quoting. But inside a syntax-quoted list, the *unquote character* (`~`, a tilde) turns quoting off again. The overall effect is templates that look like this:

```
src/examples/macros/chain_3.clj
(defmacro chain [x form]
  `( . ~x ~form))
```

Test that this new version of chain can correctly generate a single method call:

```
(macroexpand '(chain arm getHand))
-> ( . arm getHand)
```

Unfortunately, the syntax quote/unquote approach won't quite work for the multiple-argument variant of chain:

```
src/examples/macros/chain_4.clj
; Does not quite work
(defmacro chain
  ([x form] `( . ~x ~form))
  ([x form & more] `(chain ( . ~x ~form) ~more)))
```

When you expand this chain, the parentheses aren't quite right:

```
(macroexpand '(chain arm getHand getFinger))
-> ( . ( . arm getHand) (getFinger))
```

The last argument to chain is a list of more arguments. When you drop more into the macro “template,” it has parentheses because it's a list. But you don't want these parentheses; you want more to be *spliced* into the list. This comes up often enough that there is a reader macro for it: *splicing unquote* (`~@`). Rewrite chain using splicing unquote to splice in more:

```
src/examples/macros/chain_5.clj
(defmacro chain
  ([x form] `( . ~x ~form))
  ([x form & more] `(chain ( . ~x ~form) ~@more)))
```

Now, the expansion should be spot on:

```
(macroexpand '(chain arm getHand getFinger))
-> ( . ( . arm getHand) getFinger)
```

Many macros follow the pattern of chain, aka Clojure's `..` function.

1. Begin the macro body with a syntax quote (```) to treat the entire thing as a template.

2. Insert individual arguments with an unquote (~).
3. Splice in more arguments with splicing unquote (~@).

The macros we've built so far have been simple enough to avoid creating any bindings with `let` or binding. Let's create such a macro next.

Creating Names in a Macro

Clojure has a time macro that times an expression, writing the elapsed time to the console:

```
(time (str "a" "b"))
| "Elapsed time: 0.06 msecs"
-> "ab"
```

Let's build a variant of time called `bench` that is designed to collect data across many runs. Instead of writing to the console, `bench` will return a map that includes both the return value of the original expression and the elapsed time.

The best way to begin writing a macro is to write its desired expansion by hand. `bench` should expand like this:

```
; (bench (str "a" "b"))
; should expand to
(let [start (System/nanoTime)
      result (str "a" "b")]
  {:result result :elapsed (- (System/nanoTime) start)})
-> {:elapsed 61000, :result "ab"}
```

The `let` binds `start` to the start time and then executes the expression to be benched, binding it to `result`. Finally, the form returns a map including the result and the elapsed time since start.

With the expansion in hand, you can now work backward and write the macro to generate the expansion. Using the technique from the previous section, try writing `bench` using syntax quoting and unquoting:

```
src/examples/macros/bench_1.clj
; This won't work
(defmacro bench [expr]
  `(let [start (System/nanoTime)
        result ~expr]
     {:result result :elapsed (- (System/nanoTime) start)}))
```

If you try to call this version of `bench`, Clojure will complain:

```
(bench (str "a" "b"))
| Syntax error macroexpanding clojure.core/let at (REPL:1:1).
| ...
```

Clojure is accusing you of trying to let a qualified name, which is illegal. Calling `macroexpand-1` confirms the problem:

```
(macroexpand-1 '(bench (str "a" "b")))
-> (clojure.core/let [examples.macros/start (System/nanoTime)
  examples.macros/result (str "a" "b")]
  {:elapsed (clojure.core/- (System/nanoTime) examples.macros/start)
   :result examples.macros/result})
```

When a syntax-quoted form encounters a symbol, it resolves the symbol to a fully qualified name. At the moment, this seems like an irritant, because you *want* to create local names, specifically `start` and `result`. But Clojure's approach protects you from a nasty macro bug called *symbol capture*.

What would happen if macro expansion *did* allow the unqualified symbols `start` and `result`, and then `bench` was later used in a scope where those names were already bound to something else? The macro would *capture* the names and bind them to different values, with bizarre results. If `bench` captured its symbols, it would appear to work fine most of the time. Adding 1 and 2 gives you 3:

```
(let [a 1 b 2]
  (bench (+ a b)))
-> {:result 3, :elapsed 39000}
```

... until the unlucky day that you picked a local name like `start`, which collided with a name inside `bench`:

```
(let [start 1 end 2]
  (bench (+ start end)))
-> {:result 1228277342451783002, :elapsed 39000}
```

`bench` captures the symbol `start` and binds it to `(System/nanoTime)`. All of a sudden, “1 plus 2” seems to equal 1228277342451783002.

Clojure's insistence on resolving names in macros helps protect you from symbol capture, but you still don't have a working `bench`. You need some way to introduce local names, ideally *unique* ones that can't collide with any names used by the caller.

Clojure provides a reader form for creating unique local names. Inside a syntax-quoted form, you can append an octothorpe (`#`) to an unqualified name, and Clojure will create an autogenerated symbol, or *auto-gensym*: a symbol based on the name plus an underscore and a unique ID. Try it at the REPL:

```
`foo#
foo__138__auto__
```

With automatically generated symbols at your disposal, it's easy to implement `bench` correctly:

```
(defmacro bench [expr]
  `(let [start# (System/nanoTime)
        result# ~expr]
    {:result result# :elapsed (- (System/nanoTime) start#)}))
```

Test it at the REPL:

```
(bench (str "a" "b"))
-> {:elapsed 63000, :result "ab"}
```

Clojure makes it easy to generate unique names, but if you're determined, you can still force symbol capture. The sample code for the book includes an `evil-bench` that shows a combination of syntax quoting, quoting, and unquoting that leads to symbol capture. Don't use symbol capture unless you have a thorough understanding of macros.

Taxonomy of Macros

Now that you've written several macros, we can restate the rules of Macro Club with more supporting detail.

The first rule of Macro Club is, "Don't write macros." Macros are complex. If you can avoid that complexity, you should.

The second rule of Macro Club is, "Write macros if that is the only way to encapsulate a pattern." As you've seen, the patterns that resist encapsulation tend to arise around special forms, which are irregularities in a language. So the second rule can also be called the Special Form Rule.

Special forms have special powers that you, the programmer, do not have:

- Special forms provide the most basic flow control structures, such as `if` and `recur`. All flow control macros must eventually call a special form.
- Special forms provide direct access to Java. When you call Java from Clojure, you're going through at least one special form, such as the `.` (`dot`) or `new`.
- Names are created and bound through special forms, whether defining a var with `def`, creating a lexical binding with `let`, or creating a dynamic binding with `binding`.

As powerful as they are, special forms are not functions. They can't do some things that functions can do. You cannot apply a special form, store a special form in a var, or use a special form as a filter with the sequence functions. In short, special forms are not first-class citizens of the language.

The specialness of special forms could be a major problem and lead to repetitive, unmaintainable patterns in your code. But macros neatly solve the problem, because you can use macros to generate special forms. In a practical sense, *all language features are first-class features at macro expansion time*.

Macros that generate special forms are often the most difficult to write but also the most rewarding. As if by magic, such macros seem to *add new features to the language*.

The exception to the Macro Club rules is caller convenience: *you can write any macro that makes life easier for your callers when compared with an equivalent function*. Because macros don't evaluate their arguments, callers can pass raw code to a macro, instead of wrapping the code in an anonymous function. Or, callers can pass unescaped names, instead of quoted symbols or strings.

We have reviewed the macros in Clojure and contrib libraries, and almost all of them follow the rules of Macro Club. Also, they fit into one or more of the categories in the following table, which shows the taxonomy of Clojure macros.

Justification	Category	Examples
Special form	Conditional evaluation	when, when-not, and, or, comment
Special form	Defining vars	defn, defmacro, defmulti, defstruct, declare
Special form	Java interop	.., doto, import-static
Caller convenience	Postponing evaluation	lazy-cat, lazy-seq, delay
Caller convenience	Wrapping evaluation	with-open, dosync, with-out-str, time, assert
Caller convenience	Avoiding a lambda	(Same as “Wrapping evaluation”)
Caller convenience	Syntax rewriting	->, ->>, ..

Let's examine each of the categories in turn.

Conditional Evaluation

Because macros do not immediately evaluate their arguments, they can be used to create custom control structures. You've already seen this with the unless example in [Writing a Control Flow Macro, on page 240](#).

Macros that do conditional evaluation tend to be fairly simple to read and write. They follow a common form: evaluate some argument (the condition); then, based on that evaluation, pick which other arguments to evaluate, if any. A good example is Clojure's and:

```

Line 1 (defmacro and
2      ([] true)
3      ([x] x)
4      ([x & rest]
5        `(let [and# ~x]
6            (if and# (and ~@rest) and#))))

```

and is defined recursively. The zero- and one-argument bodies set up base cases:

- For no arguments, return true.
- For one argument, return that argument.

For two or more arguments, and uses the first argument as its condition, evaluating it on line 5. Then, if the condition is true, and proceeds to evaluate the remaining arguments by recursively anding the rest (line 6).

To short-circuit evaluation after the first non-true value is encountered, and must be a macro. Unsurprisingly, and has a close cousin macro, or. Their signatures are the same:

```
(and & exprs)
```

```
(or & exprs)
```

The difference is that and stops on the first logical false, while or stops on the first logical true:

```
(and 1 0 nil false)
-> nil
```

```
(or 1 0 nil false)
-> 1
```

The all-time, short-circuit evaluation champion is the comment macro:

```
(comment & exprs)
```

comment never evaluates *any* of its arguments and is sometimes used at the end of a source code file to demonstrate the usage of an API, as discussed in [Loading and Evaluating Code, on page 59](#).

Creating Vars

Clojure vars are created by the def special form. Anything else that creates a var must eventually call def. So, for example, defn, defmacro, and defmulti are all themselves macros.

Some macros can condense many lines into a single form. Consider the issue of forward declarations. You're writing a program that needs forward references to vars `a`, `b`, `c`, and `d`. You can call `def` with no arguments to define the var names without an initial binding:

```
(def a)
(def b)
(def c)
(def d)
```

But this is tedious and wastes a lot of vertical space. The `declare` macro takes a variable list of names and `defs` each name for you:

```
(declare & names)
```

Now, you can declare all of the names in a single compact form:

```
(declare a b c d)
-> #'user/d
```

The implementation of `declare` is built into Clojure:

```
(defmacro declare
  [& names] `(do ~@(map #(list 'def %) names)))
```

Let's analyze `declare` from the inside out. The anonymous function `map` `list` `def` `%` is responsible for generating a single `def`. Test this form alone at the REPL:

```
(#(list 'def %) 'a)
-> (def a)
```

The `map` invokes the inner function once for each symbol passed in. Again, you can test this form at the REPL:

```
(map #(list 'def %) '[a b c d])
-> ((def a) (def b) (def c) (def d))
```

The leading `do` makes the entire expansion into a single legal Clojure form:

```
`(do ~@(map #(list 'def %) '[a b c d]))
-> (do (def a) (def b) (def c) (def d))
```

Substituting `'[a b c d]` in the previous form is the manual equivalent of testing the entire macro with `macroexpand-1`:

```
(macroexpand-1 '(declare a b c d))
-> (do (def a) (def b) (def c) (def d))
```

Many of the most interesting parts of Clojure are macros that expand into special forms involving `def`. We've explored a few here, but you can read the source of any of them from the REPL.

Java Interop

Clojure programs call into Java via the `.` (dot), `new`, and `set!` special forms. However, idiomatic Clojure code often uses macros such as `..` (threaded member access) and `doto` to simplify forms that call Java.

You (or anyone else) can extend how Clojure calls Java by writing a macro. Consider the following scenario. You're writing code that uses several of the constants in `java.lang.Math`:

```
Math/PI
-> 3.141592653589793
(Math/pow 10 3)
-> 1000.0
```

In a longer segment of code, the `Math/` prefix would quickly become distracting, so it would be nice if you could say simply `PI` and `pow`. Clojure doesn't provide a direct way to do this, but you could define a bunch of vars by hand:

```
(def PI Math/PI)
-> #'user/PI
(defn pow [b e] (Math/pow b e))
-> #'user/pow
```

Alessandra Sierra¹ automated the boilerplate with the `import-static` macro:

```
(examples.import-static/import-static class & members)
```

`import-static` imports static members of a Java class as names in the local namespace. Use `import-static` to import the members you want from `Math`.

```
(require '[examples.import-static :refer [import-static]])
(import-static java.lang.Math PI pow)
-> nil

PI
-> 3.141592653589793

(pow 10 3)
-> 1000.0
```

Note: Clojure now provides a standard math namespace, `clojure.math`, that you should prefer for the constants and static methods in `java.lang.Math` instead.

1. <https://www.lambdasierra.com>

Postponing Evaluation

Most sequences in Clojure are lazy. When you're building a lazy sequence, you often want to combine several forms whose evaluation is postponed until the sequence is forced. Since evaluation is not immediate, a macro is required.

You've already seen such a macro in [Lazy and Infinite Sequences, on page 80](#): lazy-seq. Another example is delay:

```
(delay & exprs)
```

When you create a delay, it holds on to its exprs and does nothing with them until it's forced to. Try creating a delay that simulates a long calculation by sleeping:

```
(def slow-calc (delay (Thread/sleep 5000) "done!"))
-> #'user/slow-calc
```

To actually execute the delay, you must force it:

```
(force x)
```

Try forcing your slow-calc a few times:

```
(force slow-calc)
-> "done!"
(force slow-calc)
-> "done!"
```

The first time you force a delay, it executes its expressions and caches the result. Subsequent forces simply return the cached value.

The macros that implement lazy and delayed evaluation all call Java code in clojure.jar. In your own code, you should not call such Java APIs directly. Treat the lazy/delayed evaluation macros as the public API, and treat the Java classes as an implementation detail that's subject to change.

Wrapping Evaluation

Many macros wrap the evaluation of a set of forms, adding some special semantics before and/or after the forms are evaluated. You've already seen several examples of this kind of macro:

- time starts a timer, evaluates forms, and then reports how long they took to execute.
- let and binding establish bindings, evaluate some forms, and then tear down the bindings.

- with-open takes an open file (or other resource), executes some forms, and then makes sure the resource is closed in a finally block.
- dosync executes forms within a transaction.

Another example of a wrapper macro is with-out-str:

```
(with-out-str & exprs)
```

with-out-str temporarily binds *out* to a new StringWriter, evaluates its exprs, and then returns the string written to *out*. with-out-str makes it easy to use print and println to build strings on the fly:

```
(with-out-str (print "hello, ") (print "world"))
-> "hello, world"
```

The implementation of with-out-str has a simple structure that can act as a template for writing similar macros:

```
Line 1 (defmacro with-out-str
2   [& body]
3   `(let [s# (new java.io.StringWriter)]
4       (binding [*out* s#]
5         ~@body
6         (str s#))))
```

Wrapper macros usually take a variable number of arguments (line 2), which are the forms to be evaluated. They then proceed in three steps:

1. *Setup*: Create some special context for evaluation, introducing bindings with let (line 3) and bindings (line 4) as necessary.
2. *Evaluation*: Evaluate the forms (line 5). Since there is typically a variable number of forms, insert them via a splicing unquote: ~@.
3. *Teardown*: Reset the execution context to normal and return a value as appropriate (line 6).

When writing a wrapper macro, always ask yourself whether you need a finally block to implement the teardown step correctly. For with-out-str, the answer is No, because both let and binding take care of their own cleanup. If, however, you're setting some global or thread-local state via a Java API, you'll need a finally block to reset this state.

This talk of mutable state leads to another observation. Any code whose behavior changes when executed inside a wrapper macro is obviously *not* a pure function. print and println behave differently based on the value of *out*

and so are not pure functions. Macros that set a binding, such as `with-out-str`, do so to alter the behavior of an impure function somewhere.

Not all wrappers change the behavior of the functions they wrap. You've already seen `time`, which times a function's execution. Another example is `assert`:

```
(assert expr)
```

`assert` tests an expression and raises an exception if it's not logically true:

```
(assert (= 1 1))
-> nil

(assert (= 1 2))
| Execution error (AssertionError) at user/eval159 (REPL:1).
| Assert failed: (= 1 2)
```

Macros like `assert` and `time` violate the first rule of Macro Club to avoid unnecessary lambdas.

Avoiding Lambdas

For historical reasons, anonymous functions are often called *lambdas*. Sometimes a macro can be replaced by a function call, with the arguments wrapped in a lambda. For example, the `bench` macro from [Syntax Quote, Unquote, and Splicing Unquote, on page 248](#) does not need to be a macro. You can write it as a function:

```
(defn bench-fn [f]
  (let [start (System/nanoTime)
        result (f)]
    {:result result :elapsed (- (System/nanoTime) start)}))
```

However, if you want to call `bench-fn`, you must pass it a function that wraps the form you want to execute. The following code shows the difference:

```
; macro
(bench (+ 1 2))
-> {:elapsed 44000, :result 3}

; function
(bench-fn (fn [] (+ 1 2)))
-> {:elapsed 53000, :result 3}
```

For things like `bench`, macros and anonymous functions are near substitutes. Both prevent immediate execution of a form. However, the anonymous function approach requires more work on the part of the caller, so it's OK to break the first rule and write a macro instead of a function.

Another reason to prefer a macro for `bench` is that `bench-fn` is not a perfect substitute; it adds the overhead of an anonymous function call at runtime. Since `bench`'s purpose is to time things, you should avoid this overhead.

Syntax Rewriting

One of the great things about macros is that they allow us to rewrite the code before it is evaluated. Some macros exist purely to allow us to express ourselves in a simpler way. We first encountered the `thread-first` (`->`) and `thread-last` (`->>`) macros in [Threading Functions, on page 79](#). These macros do no additional work; they merely allow us to take a nested form and write it instead as a left-to-right pipeline of functions for readability.

We also saw another example with `..` earlier in this chapter. These are important programmer conveniences that let us rearrange the world to say what we intend more clearly. It is sometimes tempting to push this power to extremes, inventing extensive domain-specific languages. Experience has shown that these kinds of macros should be used tastefully, only when the payoff is worth creating what is effectively new syntax for the user.

Wrapping Up

Clojure macros let you automate patterns in your code. Because they transform source code at macro expansion time, you can use macros to abstract away *any* kind of pattern in your code. You're not limited to working within Clojure. With macros, you can *extend* Clojure into your problem domain. If you want a much deeper dive into this subject, check out [Mastering Clojure Macros \[Jon14\]](#).

In the final part of the book, we will put our Clojure to work, examining how to use the broad range of Clojure tools and build and test an actual application.

Part IV

Clojure in Practice

Bring your journey full circle by learning how to build, test, and release real software with Clojure's modern tooling. Put your knowledge into practice by building a complete application.

Project Tooling

Until now, we've been working with code largely in small units, exploring individual features of the language. However, most of your time as a developer is in the context of a *project*. Projects are not an explicit feature of Clojure, but any JVM-based program has a scope implicitly defined by its *classpath*, which specifies all of the source folders and dependencies available, and effectively delineates a project scope. Projects may correspond to a single source control repository (especially for libraries), or multiple projects may be combined into a single repository.

In this chapter, we'll explore the tools provided by Clojure for working with dependencies and tools in the scope of a project and everything you need to package your code for release. We won't go deep on any of the tools discussed here; we are instead demonstrating just the basics that you need for working in a project. Let's start by looking at how the JVM finds and loads your code.

Project Sources and Dependencies

A Clojure project consists broadly of two sources of code—your project code in source directories, and other people's code in external dependencies. Clojure, as a JVM language, expects both kinds of code to be on the project classpath. In this section, we will explain how the JVM classpath works and how to use tools to declare the location of your source files and the dependencies you wish to use.

The JVM Classpath

The JVM classpath is a series of source file *roots*. Classpath roots can either be directories (common for your source files) or JAR (Java ARchive) files (common for external dependencies). JAR files are merely ZIP files containing their own set of files and directories.

The classpath is a string separating roots with a path separator (: on Unix, ; on Windows), for example, “src:test”. When Clojure needs to load the namespace `your.project.core`, it looks for a source file at the path `your/project/core.clj` in each classpath root, in order. In this example, it will look for `src/your/project/core.clj` first, and then `test/your/project/core.clj`. The search stops when an existing file is found, and that file is loaded and compiled.

The .cljc Extension

In Clojure 1.7, the `.cljc` file extension was defined for *common* source files suitable for loading in multiple dialects of Clojure, with support for conditional code depending on the platform. We won’t cover the details in this book, but any place we talk about `.clj` files, `.cljc` files are also allowed.

Clojure is a *source-first* language. Importantly, this means that Clojure code does not need to be compiled into a binary form for distribution, making it easy to distribute Clojure as source. However, Clojure code is always compiled to JVM class files eventually, and applications may benefit from doing this ahead of time to make loading faster. We’ll cover how to do this in [Compiling Applications, on page 276](#).

When Clojure loads a namespace, it will also look to see if a compiled `.class` file exists in addition to checking for the `.clj` source file, and will load whichever is newer.

Declaring Source Roots in `deps.edn`

Over Clojure’s history, many project management tools have come and gone—but ultimately, they all provide a way to declare the project source roots and the external dependencies so they can be combined into a classpath for the purposes of running your program. The two most common tools in use today are Leiningen¹ (a community tool) and the Clojure CLI² (created by the Clojure core team). In this book, we will focus on the latter.

The CLI (command line interface) uses a data file named `deps.edn`³ at the project root to declare project sources and dependencies. This file is in `edn`⁴ (Extensible Data Notation) format, which is essentially a data-oriented subset of

1. <https://leiningen.org/>
2. https://clojure.org/reference/clojure_cli
3. https://clojure.org/reference/deps_edn
4. <https://github.com/edn-format/edn>

Clojure’s syntax. The `deps.edn` file is a map with a few well-known keys. For the purposes of declaring your source paths, this is a sufficient starting point:

```
{:paths ["src"]}
```

The `:paths` key is a vector of source roots to put on the classpath. Conventionally, most Clojure projects use `src` as the primary source root directory. If you wanted to add an additional path for resource files, you could just add a second directory path in your `deps.edn`:

```
{:paths ["src" "resources"]}
```

Next, we will consider how to include other libraries in your project.

Maven Dependencies

Clojure has the benefit of working with either source files or class files that are either in the file system or packaged in JARs, yielding a lot of flexibility. When consuming external dependencies, the most common way to do so is by downloading a JAR of source files (for Clojure dependencies) or of class files (for Java dependencies) or possibly a mix.

For over two decades, Maven repositories have been the primary technology for Java developers to publish and consume open source and commercial libraries. From the consumption side, this is usually done via tools like `mvn` or `gradle` that take requests for a library, interact with Maven repositories, download and cache the library JAR file, then make it available to applications. We use the Clojure CLI to download and manage dependencies.

Maven artifacts are identified by coordinates made up of several parts:

- `groupId`—the “organization” providing the library, typically a DNS-reversed domain name or trademarked org name
- `artifactId`—the library name (scoped to the `groupId`)
- `version`—a string representing a version, most commonly in the form “major.minor.patch”
- `classifier`—an optional string representing a library variant or related artifact (not common)

The `deps.edn` format assembles these together into two parts:

- **Library name**—a symbol combining the `groupId`, `artifactId`, and `classifier` with the form `groupId/artifactId$classifier` (the `classifier` is optional)
- **Coordinate**—a map describing information about how to find the library version: `{:mvn/version "..."}`

Putting this all together, you can specify dependencies in the `deps.edn` file using the `:deps` key:

```
{:paths ["src"]
 :deps {
   org.clojure/clojure {:mvn/version "1.12.0"}
   org.clojure/data.json {:mvn/version "2.5.1"}
 }
```

This `deps.edn` file retains the `:paths` key from the prior example and adds dependencies on two libraries—Clojure itself and the `data.json` library.

The Clojure CLI always includes a dependency on Clojure itself via the built-in root `deps.edn`, so including it here is not strictly necessary, but it can be a good practice if you want to define a minimum Clojure version this project depends on.

Any time you execute the Clojure CLI, it computes the classpath from the paths and deps in the `deps.edn`. Computing the deps does not just involve the dependencies listed in the `deps.edn`, it also includes any transitive dependencies needed by the top-level dependencies, recursively. We won't go into any more detail in this book, but know that there are many ways to modify how the full classpath is computed from the top-level deps. Once the classpath has been computed, the Clojure CLI ensures every lib is downloaded and caches them in the Maven cache (usually at `~/.m2/repository`).

While dependencies in the JVM ecosystem are usually released as JAR artifacts, we mentioned previously that Clojure can work directly from source, and no JAR is required. Let's look at how to use Clojure dependencies directly from a Git repository.

Git Dependencies

Git has become the de facto source control software used today, especially in open source. Providers like GitHub and GitLab have made public hosting of open source accessible to all. The majority of the libraries in the Clojure ecosystem are available in one of these open source Git hosts.

Git repositories at a specific commit or tag are effectively nothing more than a set of files. Because Clojure works from source, a public Git repository has everything we need to consume it as a Clojure library. There is no need for the author to compile source to classes, assemble class files into a JAR, publish them to a Maven repository, download the jar from the repository, and put the JAR on the classpath. Instead, a Clojure library author can

simply publish their open source project, and the consumer can refer to it directly as a dependency.

In `deps.edn`, the library name and coordinate can be configured differently to pull in a Git dependency. The library name no longer refers to a Maven groupId and artifactId; instead, the library name has these parts:

- Git host (reversed URL name)
- Organization/user
- Project

Many popular Git hosts have default mappings from a `deps.edn` lib name:

- GitHub: `[io,com].github.ORG/PROJECT`
- GitLab: `[io,com].gitlab.ORG/PROJECT`
- BitBucket: `[io,org].bitbucket.ORG/PROJECT`
- Beanstalk: `[io,com].beanstalkapp.ORG/PROJECT`
- Sourcehut: `ht.sr.ORG/PROJECT`

For the coordinate, we want a dependency to refer to a specific commit. In Git, every object is identified by a unique 40-character hexadecimal string known as the *SHA*, a hash value created with the SHA-1 algorithm (Secure Hash Algorithm 1). For convenience, Git also supports the use of short SHA prefixes if they are unique in the context of the repository. A `deps.edn` Git dep must specify either a full commit SHA, or a short SHA and a tag name that both resolve to the same full commit SHA.

For example, the following Git dep refers to the `cognitect-labs.test-runner` tool, which we'll use later in the chapter:

```
io.github.cognitect-labs/test-runner
{:git/tag "v0.5.1" :git/sha "dfb30dd"}
```

The first line is the library name, which auto-resolves to the GitHub url `https://github.com/cognitect-labs/test-runner`. The coordinate indicates a commit by specifying a version tag (for human understanding) and a matching short SHA for verification. Alternatively, a full SHA could have been used instead:

```
io.github.cognitect-labs/test-runner
{:git/sha "dfb30dd6605cb6c0efc275e1df1736f6e90d4d73"}
```

Specifying a full SHA means the commit could be anything, including a non-tagged version, or even from a branch or a fork, not just a tag on the main branch. This is a great feature for library authors because they can provide a tentative fix to a user from a development branch for testing without doing an official release. But usually, it's best to stick to a tag and version—the tag

provides semantic meaning, and the version provides a check that the tag hasn't been moved (which is unusual, but possible).

Git libs are also downloaded to a Git repository cache, by default in `~/.gitlibs`, so the repository object cache is updated to the necessary SHA, and a checkout of the repository at the specific SHA is separately cached. In addition to the Maven local repository and Git lib cache, computed classpaths are also cached in the project `.cpcache` directory. In all of these cases, the data being cached is immutable, and keeping it on disk allows the CLI to minimize network use and improve performance. In the common case, everything is in a cache, and your program starts as quickly as possible.

Next, let's look at how to use the Clojure CLI to start a REPL, probably the mode you'll use the most.

Running a REPL

The Clojure CLI has several different execution modes for running different kinds of programs. The simplest of those is starting a REPL with the current project classpath. The CLI has two commands, `clojure` and `clj`—the latter is merely a wrapper for the former, which adds command-line editing support via the `rlwrap` utility. Because of this, you should always use the `clj` to start a REPL, and no arguments are needed:

```
$ clj
Clojure 1.12.0
user=>
```

If you have not downloaded any of the dependencies before, you might also see downloads happening before the REPL starts:

```
Downloading: org/clojure/clojure/1.12.0/clojure-1.12.0.pom from central
Downloading: org/clojure/data.json/2.5.1/data.json-2.5.1.pom from central
...etc
```

These files are downloaded to the local Maven cache, and subsequent executions will use them without needing to re-download the files.

Let's move on to defining and running tests.

Defining Tests

There are many good test frameworks in Clojure, but here we will only briefly cover the `clojure.test` library included in Clojure core. For a larger treatment of testing approaches, see [Clojure Applied \[VM15\]](#).

By convention, Clojure source code is stored in the `src/` directory and unit tests are stored in the `test/` directory. Because both source directories will be included on the classpath during testing, their namespaces should not overlap. Usually, this is done by taking the source namespace and adding `-test` to the end (which becomes `_test` in the file name).

For example, a project with `example.core` namespace and `example.core-test` namespace will look like this:

```
src/
  example/
    core.clj
test/
  example
    core_test.clj
```

The `example.core` namespace might have a function in it:

```
(ns example.core)

(defn within? [min max val]
  (<= min val max))
```

And the `example.core-test` namespace will have some tests for the code:

```
(ns example.core-test
  (:require
    [clojure.test :refer [deftest is]]
    [example.core :as core]))

(deftest test-within?
  (doseq [x (range 0 10)]
    (is (true? (core/within? 0 10 x))))
  (is (false? (core/within? 0 10 -1)))
  (is (false? (core/within? 0 10 11))))
```

The `deftest` macro defines a test var `test-within?` and uses `is` to test assertions in the scope of the test.

Next, we need to declare a `deps.edn` file that adds the `test/` directory to the project paths (we'll see a better way to do this soon, but this will work for now):

```
{:paths ["src" "test"]}
```

Then, you can run these tests by starting the REPL with `clj`, loading the test namespace, and calling `run-tests`:

```
(require 'example.core-test 'clojure.test)
-> nil
(clojure.test/run-tests 'example.core-test)
|
| Testing example.core-test
```

```
|
| Ran 1 tests containing 12 assertions.
| 0 failures, 0 errors.
-> {:test 1, :pass 12, :fail 0, :error 0, :type :summary}
```

Once you start to accumulate more than one test namespace, this quickly gets cumbersome. Most Clojure developers rely on helper tooling in their editor environments to run all of the tests in a namespace or project and project tools to run all tests from the command line or in continuous integration.

Next, we'll look at how to set up a command-line test runner in `deps.edn` using the `test-runner` tool.

Running Tests

The Clojure CLI does not come with a built-in test runner and instead provides generic support for build tools, including test runners. One of the most commonly used `clojure.test` test runners is `cognitect-labs.test-runner`.⁵

To set up the test runner, we need to introduce another feature of `deps.edn`, *aliases*. Aliases associate a name (the alias) with an arbitrary map of `edn` data. The Clojure CLI defines a special set of alias keys that can modify the behavior of the CLI itself.

We declare the alias in the `deps.edn` like this:

```
{:paths ["src"]
 :aliases {
   :test {:extra-paths ["test"]
          :extra-deps {io.github.cognitect-labs/test-runner
                       {:git/tag "v0.5.1" :git/sha "dfb30dd"}}
          :exec-fn cognitect.test-runner.api/test}
 }
}
```

Note that we're not including the `"test"` directory in `:paths`—instead, everything we use for testing will be isolated in the `:test` alias. The new keys here are:

- `:extra-paths`—paths to add to `:paths` when this alias is used
- `:extra-deps`—deps to add to `:deps` when this alias is used, here as a Git dep (check the project to find the newest version available)
- `:exec-fn`—the default tool function to invoke (some tools have multiple functions)

5. <https://github.com/cognitect-labs/test-runner>

To run the test-runner, we also need to introduce a new “mode” of the Clojure CLI, *function execution mode*, triggered with `-X`. The alias (or aliases) to use are appended to the `-X` option like this:

```
$ clj -X:test
Running tests in #{"test"}
Testing example.core-test
Ran 1 tests containing 12 assertions.
0 failures, 0 errors.
```

Passing `-X` executes a function, usually in a tool, but potentially also in the project itself. Appending `:test` specifies the alias data to activate in `deps.edn`, which modifies the classpath by adding the test source path and test-runner dependency. The `:exec-fn` specifies the test-runner function to run. The default behavior of test-runner is to look for namespaces ending in `-test` and run all of the tests it finds. The example above finds and runs the only test namespace `example.core-test`.

The test-runner tool has other options to run specific namespaces, subsets of marked namespaces, specific vars, and more, but we won’t cover those here.

Now that you know how to set up your project paths and deps, start a REPL, and run tests, let’s look at how to build and release a library for others to use.

Building and Releasing Libraries

Clojure libraries can be released to either Maven or Git (or both), and there are some tradeoffs to consider. We’ll discuss Git deployment first, which is easier but has some usage limitations, and then cover Maven deployment, which is more complicated but usable by more Clojure and Java tools.

Git Deployment

If you are developing an open source Clojure library, you are likely already managing it in a Git host like GitHub. If so, you are almost done deploying it! Because no compilation or artifact assembly is required, this mostly requires a few steps, which you may already be doing:

- Create and publish a public Git repo in your account.
- Include a `deps.edn` file at the project root (there is also support for nested projects, but we’ll assume here the project root is also the repository root).
- Tag the repo to create a version—while this is optional, it’s a good practice.

You can create a tag for your project and find its short SHA with Git commands:

```
# Create an annotated tag with a message
# Using "vX.Y.Z" is a common practice for version tags
git tag -a 'v0.1.0' -m 'v0.1.0'

# Push the tags to the upstream repository
git push tags

# Obtain the short SHA version
git rev-parse --short v0.1.0^{commit}
```

The last line uses `^{commit}` to unpeel the tag to its related commit SHA, which should match what you see in the Git host user interface.

You now have all of the information you need to publish your Git lib and coordinate for users to use—the Git library name is based on the Git host, user, and project, and the coordinate is the tag and short SHA.

With a bit of additional work, you can easily automate this into something like a GitHub action and make tagging quick and easy.

The major limitation with Git libraries is that this feature is only available in the Clojure CLI, so all downstream users must also be using the Clojure CLI. As mentioned previously, Clojure developers may use other tools like Leiningen, and publishing only a Git library limits their access to the library. Also, if you want other language communities to make use of the library, tools like Maven and Gradle in Java or sbt in Scala won't understand how to use this library.

For these reasons, most Clojure developers use Git libraries primarily when publishing Clojure CLI tools (like the test-runner we saw earlier) or when producing libraries mostly for their own use.

To be maximally available, library authors can create a JAR and publish to a Maven repository. All JVM-based dependency management tools understand how to download and use JARs in Maven repositories.

Maven Repositories

The Maven ecosystem has been around for over two decades and is its own sprawling and complex system. Maven repositories can be created and supported in many ways, and there are dozens available for niche parts of the library ecosystem. However, as Clojure developers, there are really only two that are important—Maven Central and Clojars.

Maven Central is, as its name implies, a “central” repository and the default repository used by every Maven-based tool. Maven Central was started by the same authors as Maven itself and is the oldest and largest Maven repository, with a huge number of libraries, both open source and commercial. Clojure itself and all of the Clojure contrib libraries are published on Maven central. Maven Central requires uploading a variety of auxiliary artifacts, signing all artifacts using public-private key cryptography, and an account authenticating that you own or control the groupId you plan to use. In other words, deployment is fairly complicated.

Clojars is a Clojure-centric Maven repository started in the early days of Clojure and is still run by the community. The rules for artifact and groupId construction are somewhat looser, and the process is, in general, much easier to manage. For this book, we are going to focus on deploying to Clojars, but deploying to Maven Central is similar—it just requires a few extra steps.

Artifact Building

We will produce our project artifact with `tools.build`, a library for writing build programs. Once we can build the artifact, we will deploy it to Clojars. Clojars will only allow the deployment of artifacts under names we control (which affects how we build the artifact in the first part), so let’s take a brief detour to create an account on Clojars.

Go to <https://clojars.org/> and click Register. Create an account and password to use as your Clojars login. You will use your Clojars user name in the library name below.

Next, let’s look at how to create a `build.clj` build program using `tools.build`. The main steps in building a library artifact are:

- Load the project basis (data representing the project classpath).
- Copy the Clojure source and resources into a working directory.
- Write a Maven pom file to the working directory (this file tells Maven what this artifact depends on).
- Zip the working directory into a JAR file.

The full `build.clj` script is shown at the top of the next page.

You should also include the `:scm` properties linking your artifact to your code if it is open source. For brevity, we have not included these here. Refer to the docs⁶ for these and other optional Maven properties.

6. <https://clojure.github.io/tools.build/clojure.tools.build.api.html#var-write-pom>

```
(ns build
  (:require [clojure.tools.build.api :as b]))

(def class-dir "target/classes")
(def lib 'org.clojars.USER/LIB) ;; Insert your own Clojars user and lib name
(def license-data
  [[:licenses
    [:license
      [:name "Eclipse Public License 1.0"]
      [:url "https://opensource.org/license/epl-1-0/"]]]]])

(defn jar [{:keys [version]}]
  (b/delete {:path "target"})
  (b/write-pom {:class-dir class-dir, :src-dirs ["src"],
               :lib lib, :version version,
               :basis (b/create-basis {:project "deps.edn"})
               :pom-data license-data})
  (b/copy-dir {:src-dirs ["src"], :target-dir class-dir})
  (b/jar {:class-dir class-dir, :jar-file "target/lib.jar"}))
```

The build.clj script is placed at the root of the project, and then deps.edn needs a new alias to run this build script with tools.build:

```
{:paths ["src"]
 :aliases {
   :test {:extra-paths ["test"]
          :extra-deps {io.github.cognitect-labs/test-runner
                       {:git/tag "v0.5.1" :git/sha "dfb30dd"}}
          :exec-fn cognitect.test-runner.api/test}
   :build {:deps {io.github.clojure/tools.build {:mvn/version "0.10.9"}}
           :ns-default build}
 }
}
```

Here, we need to introduce a new Clojure CLI execution mode for running *tools*. Tools are useful programs for your project that have their own classpath and don't include the project classpath. In this case, the program we are running is the build.clj itself, which uses tools.build as a library.

To execute the build as a tool, we use `-T` instead of `-X`:

```
clj -T:build jar :version "0.1.0"
```

Note that we specify both the alias (`:build`) and the function to run in the build.clj script (`jar`). The build script might contain other build functions as well. Also, we need to pass the version to build as an argument, here, version "0.1.0".

The build leaves the JAR file in `target/lib.jar`. It contains the manifest files defining a Maven JAR file and the Clojure source files from the project.

Once we've created the archive, we need to deploy it to Clojars.

Clojars Deployment

It's now time to add one more alias to our `deps.edn` to deploy the built jar using another tool, `slipset/deps-deploy`. Here is the `deps.edn` with that added alias, `:deploy`:

```
{:paths ["src"]
 :aliases {
   :test {:extra-paths ["test"]
          :extra-deps {io.github.cognitect-labs/test-runner
                       {:git/tag "v0.5.1" :git/sha "dfb30dd"}}
          :exec-fn cognitect.test-runner.api/test}
   :build {:deps {io.github.clojure/tools.build {:mvn/version "0.10.9"}}
           :ns-default build}
   :deploy {:extra-deps {slipset/deps-deploy {:mvn/version "0.2.2"}}
            :exec-fn deps-deploy.deps-deploy/deploy
            :exec-args
              {:installer :remote
               :sign-releases? false ;; skip GPG signing for now
               :artifact "target/lib.jar"
               :pom-file
                "target/classes/META-INF/maven/org/clojars/USER/LIB/pom.xml"}}}
 }
```

The alias refers to the jar and pom file that were created by the build—these contain sufficient info about the library and version to deploy to the Clojars repository. Note that the `:pom-file` directory will depend on your Clojars user and lib name.

If you have not yet created a deploy token in Clojars, log in to the site and go to “Deploy Tokens” in the menu to create a user token—make sure to save this as it will only be shown once.

Now, we're ready to deploy the artifact to Clojars. Export environment variables for the user and password (use the token), then invoke the `:deploy` tool:

```
$ export CLOJARS_USERNAME=username
$ export CLOJARS_PASSWORD=clojars-token
$ clj -X:deploy
```

If everything goes smoothly, you should quickly see your library appear in the Clojars site. At that point, other users can consume the jar using the Maven coordinates from Clojars.

In general, Clojure libraries are usually distributed as source. Compiling Clojure library code makes some early decisions that are generally better left to the application, namely, which Clojure version and JDK version to compile against. The Clojure version commits to a specific version of the compiler, which may not match the Clojure version used by an application. Clojure

compiled code is generally forward-compatible, but it's not necessarily backward-compatible, so you have narrowed the applicability of your library. Similarly, the JDK version available at compile time affects Java interop choices and may make different method resolution choices than would be made on a different JDK version.

We've now completed the final step for a library: deployment of the source jar to the Clojars Maven repository. Next, we'll look at a place where compilation does make sense—in an application.

Compiling Applications

While compilation of Clojure libraries is unusual, compilation of Clojure applications is often a great choice that moves compile costs to build time so that the application can start more quickly at runtime. There are no issues with Clojure or JDK version; you've already made those choices at the application level and won't need to use them in other contexts.

To compile an application, we will imagine a new project with a different `build.clj`. In this modified build script, we need to introduce a new build task to compile the Clojure source. Clojure compilation is transitive and will also compile any code transitively loaded by your application, including library code. One of the critical choices is specifying which namespaces should be compiled, and these should include any main entry points to the application.

First, make sure that your application's main entry point contains a `(:gen-class)` clause and a `-main` function so that it will be compiled to a Java class with a main suitable for direct invocation:

```
(ns example.main
  (:gen-class))

;; This must be named -main
(defn -main [& args]
  (do-stuff))
```

Here's what the modified `build.clj` will look like:

```
(ns build
  (:require [clojure.tools.build.api :as b]))

(def class-dir "target/classes")
(def basis (b/create-basis {:project "deps.edn"}))

(defn app [_]
  (b/delete {:path "target"})
  (b/copy-dir {:src-dirs ["src"], :target-dir class-dir})
  (b/compile-clj {:basis basis, :ns-compile '[example.main],
                  :class-dir class-dir}))
```

```
(b/uber {:class-dir class-dir, :uber-file "target/app.jar"
        :basis basis, :main 'example.main}))
```

Notice that some of the things we did for the library build are no longer needed. We don't need a pom file if we're not deploying to Maven, and we don't need to build a library jar file. Instead, we add two new tasks: `compile-clj` to compile the `example.main` namespace into the `class-dir`, and the `uber` task to build a single jar containing all of the dependencies plus the compiled source into a single `target/app.jar`.

We will run it just like we ran the build before:

```
clj -T:build app
```

When this completes, the self-contained application will be in `target/app.jar` and you can run it like this:

```
java -jar target/app.jar
```

Once you've constructed this “uber” jar, you can run it on your own machine or on a physical or virtual host. You will only need Java and your application uber jar. It's even possible to use the Java `jpackage` utility to create a standalone application or the GraalVM JDK to create a native application (particularly good for fast-starting applications). The tradeoff between these approaches is generally extra time and effort for building the application artifact vs. improved size and startup time of the application. Most Clojure users run directly from an uber jar, so we will not cover the `jpackage` or GraalVM approaches further here.

Wrapping Up

Whew! That was a whirlwind tour through basic project tooling, including an overview of how to declare source paths and dependencies, run the REPL, write and run tests, build and deploy libraries to Clojars, and compile and build application uber jars.

Next, we will build an application and put everything we've learned so far to use!

Building an Application

Now that you've learned the basics of the Clojure language, it's time to begin using Clojure in your own projects. But when you start to work on your own killer Clojure app, you'll quickly discover that knowledge of the language is only part of what you need to work effectively. You also need to consider questions of workflow, data structures, polymorphism, testing, and more.

As our sample application, we'll implement a version of the game Hangman where a player uncovers a word by guessing a sequence of letters. We'll show you all of the code as we go, but this chapter is not really about code. It's about how we approach solving problems in Clojure by focusing on how to represent our problem in data, then building functional algorithms around that data.

We'll also look at how we can use specs to describe both our data and the functions used in the program, then test those functions with generative testing.

Getting Started

Before you start writing code, it's good to initialize a project directory on your file system and set up your editor with a connected REPL in that context. In some cases, creating a new project or new namespaces may be easier in your editor, but we will do the setup in the terminal, which works with any editor.

First, create a `hangman` directory and switch into it:

```
$ mkdir hangman
$ cd hangman
```

While a `deps.edn` file is not required to use the Clojure CLI, it's a good practice to create one and add some initial dependencies.

Create a `deps.edn` file in the `hangman` directory with the following setup:

```
{:paths ["src"]
 :deps
 [org.clojure/clojure {:mvn/version "1.12.0"}
  org.clojure/test.check {:mvn/version "1.1.1"}]}
```

Clojure applications depend on Clojure, of course, but we will also include the `test.check` library, which we'll use for testing later.

The paths above include our source code path, typically named `src` in Clojure projects:

```
$ mkdir src
```

In Clojure (as in Java), single-segment namespaces are frowned upon because they do not provide a sufficient project or domain prefix. In this project, we'll use `hangman` as our first namespace segment.

Commonly, Clojure projects use `core` as a “main” namespace, so let's shift over to our editor, load the project in the `hangman` directory, and create the `hangman.core` namespace source file `src/hangman/core.clj`. Start with a few requirements you'll need later for string handling and I/O:

```
(ns hangman.core
  (:require [clojure.java.io :as jio]
            [clojure.string :as str]))
```

You should also ensure that you have started a connected REPL for the project and loaded the `hangman.core` namespace. Now, we're ready to start working on the application.

Developing the Game Loop

There are many paths to developing successful applications with Clojure (or any language) and many ways this application could be written. However, writing any program involves starting from high-level goals and breaking the problem down into smaller problems until it's easy to solve each subproblem with a single (usually small) function.

Our application is a game with a single player where the player guesses letters to slowly reveal a word. The goal for the player is to guess all of the letters in the word with as few guesses as possible.

When you don't know where to start, it's always useful to think about the problem and ask what inputs the code must take and what it will return. That's enough to give you the shape of a function. Then, break that function down into smaller problems and repeat.

The inputs to our game are the word the player will guess and a player who can make guesses, so those will be your initial arguments. The return value will be the score the player received.

```
(defn game [word player] ...)
```

This game (and most games) centers around a repeated series of actions where the player makes a choice, the game state is updated, and the player makes another choice. This game loop gives us the overall framework for the game.

The game loop also serves as a guide for what code you need to write next. As you break the loop into steps, these help you identify both the functions that are the subproblems and the data structures passed between the functions. Repeat this process for each function until you hit the bottom, where you'll find functions that are self-contained.

Within each iteration, the game needs to ask the player for the next guess and update the progress made on the word. If the word has been guessed, the game is complete, and the game should exit with the number of guesses that were made.

In almost every use of a loop, you'll see a check for termination. If the game is complete, the loop should terminate with the final score. Otherwise, a recur should take the game back to the top of the loop, ready for the next iteration.

```
(defn game
  [word player]
  (loop [progress (new-progress word), guesses 1]
    (let [guess (next-guess player progress)
          progress' (update-progress progress word guess)]
      (if (complete? progress' word)
          guesses
          (recur progress' (inc guesses))))))
```

This code maps very closely to the textual description of the game, with a lot of missing details. There are two pieces of state carried by the loop across iterations: the progress in guessing the word and the score.

The game function invokes next-guess to obtain the next guess from a player, and we'll consider this later in the chapter when you implement the players.

The other functions we used but didn't define all relate to creating, updating, or checking the progress of the game: new-progress, update-progress, and complete?. We haven't yet defined the data structure of the word progress, but it's clearly a central part of the implementation.

At this point, it's good to create empty functions for all of the functions we invented while writing the game loop. These empty functions serve as a road map of work you still need to complete. They also allow you to successfully start loading and running the code in the REPL.

```
(defn next-guess [player progress])
(defn new-progress [word])
(defn update-progress [progress word guess])
(defn complete? [progress word])
```

You can't write any of the functions to create, update, or check the game progress without knowing what the progress data structure looks like, so that's where you should focus next.

Representing Progress

The word the player is guessing is a string, made up of characters. Two of the most important kinds of data structures we deal with in Clojure are maps (keyed by an index) and sequences (which rely on sequential traversal). We have a number of choices for representing the progress in the game, but choosing between an indexed or sequential view is the critical decision.

So let's think, at least at a high level, about the operations that use this data structure in the code. The update-progress function seems like the one we care about the most. Given the word and the guessed letter, you need to check whether each letter in the word matches the guess, and if so, update the progress, which keeps track of all letters guessed so far in the word.

Looking back at that description of the update operation, you need to check whether “each letter in the word” has some property. This indicates that a sequential traversal based on the original word would be a good match (considering all values). You then need to update the progress data structure in the corresponding location if there is a match.

If the progress is an indexed data structure, you need to traverse the original word and keep track of the index as you do so, to know which index to update in the progress. The Clojure sequence functions include `keep-indexed` and `map-indexed`, which helps with this kind of thing.

If you instead treat the progress as a sequential data structure and update it at the same time that you traverse the original word, you can avoid tracking or using the indexes at all. When the progress is a sequential data structure where each element might need to be updated, you should strongly consider `map`, which transforms every element. Additionally, `map` is one of the only sequence functions that can traverse multiple sequences at the same time.

So, let's proceed on the assumption that a sequential structure with each element corresponding to a letter in the original word is the working model. You could use some kind of flag to indicate which letters have been guessed: (true true false false true). Or you could use the actual letters and a known “blank” character: (\h \e _ _ \o). (Recall that literal characters are represented with a leading \ in Clojure.) The latter representation gives you a built-in human-readable representation of our progress, so it might be slightly more useful, but in truth, either would work.

We've spent enough time thinking about our data structure at this point. While that took a while, it was time well spent because you can proceed with your functions with a clear sense of the needs and constraints of the data.

Start with new-progress. Given a word (a string), you need a sequence of blank characters of the same length. Here, you can reach into the sequence function bag of tricks and pull out repeat, which creates a sequence of the same element of a specific length, here, the count of the word:

```
hangman/src/hangman/core.clj
(defn new-progress [word]
  (repeat (count word) \_))
```

Now for the update. We already established that you want to map over both the word (a string, automatically treated as a sequence of characters) and the progress (a sequence of characters, either the actual character or a blank). The map function will take two inputs—a letter from the word and the corresponding letter from the progress. It needs to output the new letter in the updated progress. You also have available from the outer function the letter that was the guess.

Putting all that together is actually pretty easy—you just need to check whether the guessed letter matches the word character, and if so, include it in the updated progress. If not, then use the original progress character to remember the progress so far.

```
hangman/src/hangman/core.clj
(defn update-progress [progress word guess]
  (map #(if (= %1 guess) guess %2) word progress))
```

Finally, checking whether the player is “done” is just a check of whether the characters in the original word match the progress. Because the word is a string, explicitly convert it to a sequence for comparison purposes.

```
hangman/src/hangman/core.clj
(defn complete? [progress word]
  (= progress (seq word)))
```

And that's the core of our game loop. Now, you need a player.

Implementing Players

Your interaction with the player so far has been represented by a single function, `next-guess`. It's time to fill out that idea a bit more. There are many potential ways to implement either a human or a computer player. Any time you determine a function is open for extension, you should strongly consider using a multimethod or protocol to make that possible.

In this case, you only have a single function, so either choice is viable. There might be a need for players to keep state (remembering what they've guessed), so protocols are a bit easier to use by encapsulating that state in a record, which extends the protocol.

So, replace that function with a protocol:

```
hangman/src/hangman/core.clj
(defprotocol Player
  (next-guess [player progress]))
```

Given this protocol, consider a player who just makes random guesses without regard to previous guesses.

To start with the random player, you'll need a pool of letters to draw from. All characters have an integer mapping, and you can leverage this, along with standard core functions like `range`, to build a vector of all legal letters:

```
hangman/src/hangman/core.clj
(def letters (mapv char (range (int \a) (inc (int \z)))))
```

You can then build a function that generates a random letter by using `'rand-nth'`:

```
hangman/src/hangman/core.clj
(defn rand-letter []
  (rand-nth letters))
```

And finally, you have everything you need to build your first player. Because you don't need any state for a random player, you can use `reify` to create an anonymous implementation of the random player:

```
hangman/src/hangman/core.clj
(def random-player
  (reify Player
    (next-guess [_ progress] (rand-letter))))
```

Now that you have a player, you can actually run the game and see how the random player does.

```
(game "hello" random-player)
-> 92
```

Not so good. Given only 26 letters in the alphabet, any score over 26 is pretty bad. If you can give your player some memory, it could at least avoid guessing the same (bad) letters over and over again.

In fact, keeping in mind that the maximum number of guesses you should need to make is 26, it's reasonable to create a player who is simply given the choices to make in order.

Memory requires state, so you'll need to use one of the stateful Clojure constructs to store that memory. Since each player will only be used in a single thread and does not require state coordination or asynchronous updating, an atom is sufficient.

The choices-player will use an atom containing the sequence of choices to make. The next-guess implementation then simply takes the first choice and updates the atom to retain the rest of the choices for the next call.

The player could be implemented by closing over the atom and using `reify` as you did with `random-player`, but instead, use a record to hold the state and extend it to the protocol:

```
hangman/src/hangman/core.clj
(defrecord ChoicesPlayer [choices]
  Player
  (next-guess [_ progress]
    (let [guess (first @choices)]
      (swap! choices rest)
      guess)))

(defn choices-player [choices]
  (->ChoicesPlayer (atom choices)))
```

You can then create a shuffled-player that guesses each letter in a random order by just shuffling the letters vector:

```
hangman/src/hangman/core.clj
(defn shuffled-player []
  (choices-player (shuffle letters)))
```

Run a few games and try it:

```
(game "hello" (shuffled-player))
-> 24

(game "hello" (shuffled-player))
-> 19

(game "hello" (shuffled-player))
-> 21
```

That's certainly better than 92, but it still doesn't seem very good. You could instead implement a player who picks the letters in alphabetical order instead of shuffled order by just not shuffling:

```
hangman/src/hangman/core.clj
(defn alpha-player []
  (choices-player letters))
```

You only need to run this test once, as there's no random element:

```
(game "hello" (alpha-player))
-> 15
```

That's a better score for this word, but it would be worse for others—you can actually predict the score, as it will be the index of the latest letter in the alphabet (here “o”, which is 15th).

Over a wide range of words, you would expect the frequency of letters in English words¹ to be a good ordering. You can just hard-code that into a freq-player:

```
hangman/src/hangman/core.clj
(defn freq-player []
  (choices-player (seq "etaoinshrdlcumwfgypbvjkxqz")))
```

Give it a try:

```
(game "hello" (freq-player))
-> 11
```

Just letting the computer play isn't very fun, though. Next, let's add an interactive player so you can play, too.

Interactive Play

As you consider interactive play, you'll need to make a few additions to the program. Right now, the game only returns the final score, but an interactive game should report progress as the game progresses. Also, you're choosing the word to guess, but we really need to let the program choose a random word so it's a mystery to a human player.

Let's tackle the random word first. Included in the source download for the hangman project is a file words.txt, which contains about 4000 words that you can use as a word bank. Copy it to the root of your project and read those words into a data structure.

1. https://en.wikipedia.org/wiki/Letter_frequency#Relative_frequencies_of_letters_in_the_English_language

```
hangman/src/hangman/core.clj
(defn valid-letter? [c]
  (<= (int \a) (int c) (int \z)))

(defonce available-words
  (with-open [r (jio/reader "words.txt")]
    (->> (line-seq r)
          (filter #(every? valid-letter? %))
          vec)))
```

The `clojure.java.io` namespace (aliased here to `jio`) has a number of helpful functions for interacting with the Java I/O library. Java has several I/O abstractions for different purposes. For example, *streams* represent binary streams of data and *readers* and *writers* are used for reading and writing character-based data. The `jio/reader` function will coerce many input sources into a Java reader.

Once you have the reader, you can break it into lines with `line-seq`, `filter` to keep only those that contain valid letters (omitting those with punctuation), and finally leave the final result in a vector. This is a typical sequence processing pipeline, tied together with the `->>` thread-last operator.

Note that `defonce` is used here. `defonce` is a special wrapper for `def` that will prevent re-execution if this namespace is reloaded. This change avoids re-reading the word file, which is expensive. This mostly helps during development.

Now that you have a vector of valid words, you can easily pick a random one with `rand-nth`:

```
hangman/src/hangman/core.clj
(defn rand-word []
  (rand-nth available-words))
```

Try it out:

```
(repeatedly 5 rand-word)
-> ("sophisticated" "humor" "proclaim" "threshold" "obtain")
```

Next, let's revisit our game function and add some printing to reveal the game progress. It's common to add optional keyword arguments to the end of an invocation, so define a new `:verbose` option. Clojure supports destructuring the varargs sequential arguments as if they were a map for this purpose. It's also good practice to declare defaults using the `:or` destructuring syntax.

Within the game loop, add a call to report progress when the verbose flag is set:

```
hangman/src/hangman/core.clj
(defn game
  [word player & {:keys [verbose] :or {verbose false}}]
  (when verbose
    (println "You are guessing a word with" (count word) "letters"))
```

```
(loop [progress (new-progress word), guesses 1]
  (let [guess (next-guess player progress)
        progress' (update-progress progress word guess)]
    (when verbose (report progress guess progress'))
    (if (complete? progress' word)
        guesses
        (recur progress' (inc guesses))))))
```

Calling out to a function here keeps the reporting out of the main loop and makes the core loop code easier to read. The progress reporting looks like this:

hangman/src/hangman/core.clj

```
(defn report [begin-progress guess end-progress]
  (println)
  (println "You guessed:" guess)
  (if (= begin-progress end-progress)
      (if (some #{guess} end-progress)
          (println "Sorry, you already guessed:" guess)
          (println "Sorry, the word does not contain:" guess))
      (println "The letter" guess "is in the word!"))
  (println "Progress so far:" (apply str end-progress)))
```

And finally, you need to create the interactive player, which will accept guesses interactively from the player:

hangman/src/hangman/core.clj

```
(def interactive-player
  (reify Player
    (next-guess [_ progress] (take-guess))))
```

Like the random-player, no state is being used here, so you can define a single interactive-player to use that does nothing more than defer to a function take-guess that interacts with the console.

Clojure provides access to the stdin input stream via the **in** dynamic variable, which will be an instance of `java.io.Reader`. Here's the full code:

hangman/src/hangman/core.clj

```
(defn take-guess []
  (println)
  (print "Enter a letter: ")
  (flush)
  (let [input (.readLine *in*)
        line (str/trim input)]
    (cond
      (str/blank? line) (recur)
      (valid-letter? (first line)) (first line)
      :else (do
        (println "That is not a valid letter!")
        (recur)))))
```

Start by printing the instructions to the user using `print`, which will not print a newline character at the end but will instead wait for the user's response. Next, call `flush` to force the buffered output stream to print so the user will see it. Then, you're ready to read the user's input from the input stream—just call `readLine` via Java interop.

Once the user hits enter, you can consider their response. If the line is blank, you can recur back to the top of the function (remember that functions serve as implicit loop targets) to try again. If the response starts with a valid letter, return that. And if the letter is invalid, notify the user and try again. Now, you can put all this together and play a game yourself.

```
(game (rand-word) interactive-player :verbose true)
```

```
You are guessing a word with 4 letters
```

```
Enter a letter: a
```

```
The player guessed: a
```

```
The letter a is in the word!
```

```
Progress so far: _a__
```

```
Enter a letter: e
```

```
The player guessed: e
```

```
The letter e is in the word!
```

```
Progress so far: ea__
```

```
Enter a letter: c
```

```
The player guessed: c
```

```
Sorry, the word does not contain: c
```

```
Progress so far: ea__
```

```
Enter a letter: s
```

```
The player guessed: s
```

```
The letter s is in the word!
```

```
Progress so far: eas_
```

```
Enter a letter: t
```

```
The player guessed: t
```

```
Sorry, the word does not contain: t
```

```
Progress so far: eas_
```

```
Enter a letter: y
```

```
The player guessed: y
```

```
The letter y is in the word!
```

```
Progress so far: easy
```

```
-> 6
```

It seems like the interactive player works. Next, let's consider how we can use specs to document and test the program.

Documenting and Testing Your Game

You have a working game at this point, but you also need to consider the poor developer who's going to pick this up six months from now (particularly if it's you). You worked hard to pick a good data structure and implement your functions, but you need to communicate those data structures for future use.

If you write some specs, you can describe the key data structures, annotate the functions, and even generate some automated tests that check whether everything works (especially when you start making changes at some future date).

Create a new namespace `hangman.specs` (in `src/hangman/specs.clj`), start it with some requires you'll need later, and load it in your connected REPL:

```
hangman/src/hangman/specs.clj
(ns hangman.specs
  (:require [clojure.spec.alpha :as s]
            [clojure.spec.gen.alpha :as gen]
            [clojure.spec.test.alpha :as stest]
            [hangman.core :as h :refer
             [letters valid-letter? Player
              random-player shuffled-player alpha-player freq-player]]))
```

When you started working on the implementation earlier, you quickly honed in on the progress data structure and the trio of internal functions (`new-progress`, `update-progress`, and `complete?`) that dealt with creating, updating, and checking that data structure. Because that data structure is critical to the internals of the game, it's also a good place to start writing specs.

The signature for `new-progress` takes a word and returns the initial progress value. Our spec defines the structure of the arguments and the return value of that function.

```
(s/def hangman.core/new-progress
  :args (s/cat :word ::word)
  :ret ::progress)
```

You haven't defined a `::word` or `::progress` spec yet, but that's ok—these show us what you need to do next.

Words are made up of letters, and you've already made some definitions in your code for letters that you can use. This is a common case when writing specs—often, the implementation and the specs use the same predicates and talk in the same “language,” which is why specs feel so expressive.

```
hangman/src/hangman/specs.clj
(s/def ::letter (set letters))
```

A `::letter` spec is the set of valid letters, which you already defined in the random player. We also could have used the predicate `valid-letter?`; however, we want to have our specs act as generators as well. The `valid-letter?` predicate doesn't have an automatically created generator, whereas these are provided for sets.

Now, you can create a spec for a word, which is just a string that consists of at least one valid letter:

```
(s/def ::word
  (s/and string?
    #(pos? (count %))
    #(every? valid-letter? (seq %))))
```

Clojure spec will attempt to create an automatic generator from this spec, but it uses a wide range of characters, certainly more than our narrow `::letter` spec will allow. The automatic generator will produce strings of this broader character set, then filter to just strings of the allowed characters, which requires much more work than seems necessary (and it may not work at all). Instead, you should supply your own generator, tailored to the character set.

The `s/with-gen` macro wraps a spec and attaches a custom generator. In this case, we want to generate a collection of one or more valid letters, then create a string from those letters. In `clojure.spec.gen.alpha`, the `fmap` function takes a source generator and applies an arbitrary function to each sample, defining a new generator:

```
hangman/src/hangman/specs.clj
(s/def ::word
  (s/with-gen
    (s/and string?
      #(pos? (count %))
      #(every? valid-letter? (seq %)))
    #(gen/fmap
      (fn [letters] (apply str letters))
      (s/gen (s/coll-of ::letter :min-count 1)))))
```

You can test it out directly at the REPL:

```
(gen/sample (s/gen ::word))
-> ("hilridqg"
   "ipllomgodmzh"
   "xbsllzg"
   "etdjwdtvquuswpox"
   "adrgrhntbuzewbdvfa"
   ... )
```

Those look appropriately word-like for our purposes. You now have a spec for words, so let's think about the `::progress` spec. We decided that progress

would be represented by a sequence of either letters or `\_` to indicate an unguessed character. Since we added an additional character, we'll need a new spec `::progress-letter` that expands `::letter` to include `\_`. The `::progress` spec is then a collection of at least one of that expanded letter set:

```
hangman/src/hangman/specs.clj
(s/def ::progress-letter
  (conj (set letters) \_))

(s/def ::progress
  (s/coll-of ::progress-letter :min-count 1))
```

That's enough specs to start testing `new-progress`. You can use `stest/check` for that and summarize what happened with `summarize-results`:

```
(stest/summarize-results (stest/check 'hangman.core/new-progress))
{:sym hangman.core/new-progress}
-> {:total 1, :check-passed 1}
```

You tested one function and it passed. However, you aren't really testing as much as you could in this function. If you look again at the `args` (the word) and the return value (the progress value), there's another constraint—both values should be the same length. You can encode this in the `:fn` spec of the `new-progress` spec, which takes a map of the conformed `:args` and `:ret` spec values. Constraints that include both the `args` and the `ret` value are always recorded in the `:fn` spec.

```
hangman/src/hangman/specs.clj
(defn- letters-left
  [progress]
  (-> progress (keep #{\_} count)))

(s/fdef hangman.core/new-progress
  :args (s/cat :word :word)
  :ret ::progress
  :fn (fn [{:keys [args ret]}]
        (= (count (:word args)) (count ret) (letters-left ret))))
```

First, create a helper function `letters-left` to compute the number of unguessed letters in a progress data structure. You can then check that the number of letters in the input word, the number of letters in the progress, and the number of unguessed letters are all the same. Rerunning the test reveals no unseen problems, but it's good to have the extra constraint for future changes.

Next, you can handle the `update-progress` function using specs you've already defined for `::progress`, `::word`, and `::letter`. Again, you can add a useful `:fn` constraint to verify that the number of letters left unguessed is less than or equal to the number unguessed at the beginning.

```
hangman/src/hangman/specs.clj
```

```
(s/fdef hangman.core/update-progress
  :args (s/cat :progress ::progress :word ::word :guess ::letter)
  :ret ::progress
  :fn (fn [{:keys [args ret]]}
        (>= (-> args :progress letters-left)
          (-> ret letters-left))))
```

And finally, the spec for complete? is straightforward:

```
hangman/src/hangman/specs.clj
```

```
(s/fdef hangman.core/complete?
  :args (s/cat :progress ::progress :word ::word)
  :ret boolean?)
```

Now that you've described the progress functions, you can turn to the main game function itself. The game function takes a word, a player, an optional verbose tag, and returns a score.

We've not yet considered how to spec a player, but you can check the protocol in a predicate. In the ::player spec, we want the generator to work and produce a player, so let's have it choose a random one of the players you've defined:

```
hangman/src/hangman/specs.clj
```

```
(defn player? [p]
  (satisfies? Player p))

(s/def ::player
  (s/with-gen player?
    #(s/gen #{random-player
              shuffled-player
              alpha-player
              freq-player})))
```

While you could spec the verbose flag and score in-line with the game spec, pulling these out as independent specs creates better, more concrete names, which are potentially reusable. For the verbose flag, you want your tests to be quiet, so the generator is hardcoded to always return false.

```
hangman/src/hangman/specs.clj
```

```
(s/def ::verbose (s/with-gen boolean? #(s/gen false?)))
(s/def ::score pos-int?)

(s/fdef hangman.core/game
  :args (s/cat :word ::word
              :player ::player
              :opts (s/keys* :opt-un [::verbose]))
  :ret ::score)
```

If you test this out by running check, you'll see that everything looks good:

```
(stest/check 'hangman.core/game)
```

```
-> ({:spec #object[...],
      :clojure.spec.test.check/ret
      {:result true,
       :pass? true,
       :num-tests 1000,
       :time-elapsed-ms 155,
       :seed 1761243873033},
      :sym hangman.core/game})
```

check runs 1000 games with a random player and word. It's also useful to verify all of the function specs you have while running these tests. You can do that by instrumenting all of the specs before running check:

```
(stest/instrument (stest/enumerate-namespace 'hangman.core))
=> [hangman.core/update-progress hangman.core/new-progress
    hangman.core/game hangman.core/complete?]
```

Any time you run stest/instrument, it's good to verify that the return value states all of the instrumented functions you expect to see as a test of reasonability.

If you rerun the check, you still see no issues, but now all of the arg specs are being verified as well, giving you greater confidence in the correctness of the code. You can then do a final check that runs check on all spec'ed functions you defined:

```
(-> 'hangman.core
    stest/enumerate-namespace
      stest/check
      stest/summarize-results)
| {:sym hangman.core/update-progress}
| {:sym hangman.core/new-progress}
| {:sym hangman.core/game}
| {:sym hangman.core/complete?}
-> {:total 4, :check-passed 4}
```

You could go further with testing some of the details of the individual players, but this should give you a taste for testing with spec.

Farewell

Congratulations. You have come a long way in a short time. You have learned the many ideas that combine to make Clojure great: Lisp, Java, functional programming, and explicit concurrency. And in this chapter, you saw one (of a great many) possible workflows for developing a full application in Clojure.

We've only scratched the surface of Clojure's great potential, and we hope you'll take the next step and become an active part of the Clojure community.

Developer Tools

This appendix provides a categorized list of commonly used tools to aid developers while they are writing, testing, and debugging code. If you are seeking a resource for a specific purpose, find the category and evaluate some of the options listed here. This list is not intended to be exhaustive, but rather to provide the most common starting points.

Because developer tools change more frequently than books are published, you might find some of the resources here have fallen out of active development or use (although Clojure libraries are remarkably resilient due to the stability of the language). In that case, you still might be able to use these tools to find something new that serves the same purpose.

Testing

Testing and quality are an essential part of any development project. Clojure includes the `clojure.test` framework, which is limited but sufficient enough that many developers do not venture beyond it. If you do choose to go further, consider other test frameworks like these:

- `kaocha`:¹ newer test framework with more features
- `lazy-test`:² behavior driven development test framework
- `test.check`:³ property-based testing framework
- `Cloverage`:⁴ unit test coverage

1. <https://github.com/lambdaisland/kaocha>
2. <https://github.com/NoahTheDuke/lazytest>
3. <https://github.com/clojure/test.check>
4. <https://github.com/cloverage/cloverage>

Code Style and Linting

Clojure has several well-written code analyzers that provide linting, navigation, and formatting capabilities. Some of these will likely already be integrated into your editor; others may need to be installed.

- `clj-kondo`:⁵ static analyzer and linter
- `clojure-lsp`:⁶ language server for editors, providing static analysis features
- `splint`:⁷ linter focused on code shape
- `cljfmt`:⁸ Clojure code formatter (whitespace-focused)
- `cljstyle`:⁹ Clojure style enforcer (structured rewriting)
- `zprint`:¹⁰ formatter with extensive configurability

Data Browsing and Visualization

Clojure’s focus on the REPL and immutable data makes it easy (and essential) to build tools to view and inspect both data and the runtime process while you are developing, and there are many useful tools available. Clojure 1.10 added the `datafy` and `nav` protocols for representing and traversing arbitrary graphs of data, including Java data, and these tools provide browsing and introspection, mostly based on `datafy` and `tap>`.

- `Morse`:¹¹ graphical interactive tool for browsing Clojure data
- `Portal`:¹² visual inspector in your browser and VS Code extension
- `Reveal`:¹³ interactive data viewer and debugger written with JavaFX
- `Clerk`:¹⁴ notebook-style editing and visualization in the browser

Debugging

Clojure’s focus on interactive development at the REPL encourages the use of tools that enhance that workflow with more information, taking the place of “print”-style debugging.

-
5. <https://github.com/clj-kondo/clj-kondo>
 6. <https://clojure-lsp.io/>
 7. <https://github.com/noahtheduke/splint>
 8. <https://github.com/weavejester/cljfmt>
 9. <https://github.com/greglook/cljstyle>
 10. <https://github.com/kkinnear/zprint>
 11. <https://github.com/nubank/morse>
 12. <https://github.com/djblue/portal>
 13. <https://vlaaad.github.io/reveal>
 14. <https://clerk.vision/>

- `tap>`: built-in Clojure core function that allows you to send arbitrary data to a queue for further manipulation
- `FlowStorm`:¹⁵ a full-featured tool to record, debug, and replay without breakpoints or code modification
- `tools.trace`:¹⁶ a simple code-tracing framework
- `scope-capture`:¹⁷ lets you run code in your REPL with the context of local bindings
- `snitch`:¹⁸ tool for injecting inline defs into your functions for debugging
- `hashp`:¹⁹ a tool to improve print-style debugging when you need it

Profiling

Performance is always important, and ultimately, you will need tools to identify bottlenecks and performance issues.

- `clj-async-profiler`:²⁰ high-precision profiler that creates flamegraphs
- `clj-java-decompiler`:²¹ a disassembler (bytecode) and decompiler (Clojure bytecode to Java) is essential to understand how Clojure is compiled
- `memory-meter`:²² for understanding how much memory Clojure collections and other objects consume
- `Criterion`:²³ a benchmarking library for Clojure

Dependency Management

Managing dependencies sometimes seems like a full-time job on any project; use these tools to assist you.

- `antq`:²⁴ find outdated dependencies and update them
- `clj-watson`:²⁵ rule-based vulnerability checking
- `nvd-clojure`:²⁶ dependency vulnerability checking

15. <https://www.flow-storm.org/>

16. <https://github.com/clojure/tools.trace>

17. <https://github.com/vvvvalvalval/scope-capture>

18. <https://github.com/AbhinavOmprakash/snitch>

19. <https://github.com/weavejester/hashp>

20. <https://github.com/clojure-goes-fast/clj-async-profiler>

21. <https://github.com/clojure-goes-fast/clj-java-decompiler>

22. <https://github.com/clojure-goes-fast/clj-memory-meter>

23. <https://github.com/hugoduncan/criterion>

24. <https://github.com/liquidz/antq>

25. <https://github.com/clj-holmes/clj-watson>

26. <https://github.com/rm-hull/nvd-clojure>

Bibliography

- [Goe06] Brian Goetz. *Java Concurrency in Practice*. Addison-Wesley, Boston, MA, 2006.
- [Hof99] Douglas R. Hofstadter. *Gödel, Escher, Bach: An Eternal Golden Braid*. Basic Books, New York, NY, 20th Anniv, 1999.
- [Jon14] Colin Jones. *Mastering Clojure Macros*. The Pragmatic Bookshelf, Dallas, TX, 2014.
- [VM15] Ben Vandgrift and Alex Miller. *Clojure Applied*. The Pragmatic Bookshelf, Dallas, TX, 2015.

Thank you!

We hope you enjoyed this book and that you're already thinking about what you want to learn next. To help make that decision easier, we're offering you this gift.

Head on over to <https://pragprog.com> right now, and use the coupon code BUYANOTHER2026 to save 30% on your next ebook. Offer is void where prohibited or restricted. This offer does not apply to any edition of *The Pragmatic Programmer* ebook.

And if you'd like to share your own expertise with the world, why not propose a writing idea to us? After all, many of our best authors started off as our readers, just like you. With up to a 50% royalty, world-class editorial services, and a name you trust, there's nothing to lose. Visit <https://pragprog.com/become-an-author/> today to learn more and to get started.

Thank you for your continued support. We hope to hear from you again soon!

The Pragmatic Bookshelf

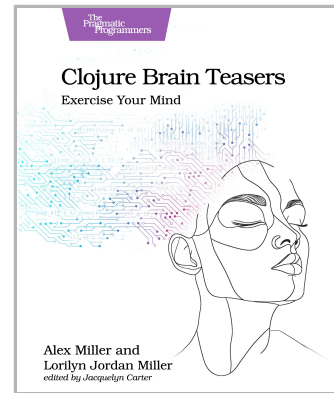


SAVE 30%
with coupon code
BUYANOTHER2026
when you checkout at pragprog.com

Clojure Brain Teasers

Challenge your knowledge of Clojure with 25 short Clojure teasers, sometimes with surprising results! Inspired by years of developer questions and feedback, these teasers are handpicked to clarify common points of confusion. Each code challenge illustrates Clojure's elegant design, explaining how and why it works. Enjoy these simple exercises solo or with friends to find gaps in your knowledge, challenge assumptions, and gain valuable insights. Tackle the most common points of confusion Clojure developers encounter, become more efficient when writing and debugging, and better predict the outcomes of Clojure code. Regardless of your Clojure experience, you're certain to learn something new.

Alex Miller and Lorilyn Jordan Miller
(130 pages) ISBN: 9798888651292. \$32.95
<https://pragprog.com/book/mmclobrain>



Web Development with Clojure, Third Edition

Today, developers are increasingly adopting Clojure as a web-development platform. See for yourself what makes Clojure so desirable as you create a series of web apps of growing complexity, exploring the full process of web development using a modern functional language. This fully updated third edition reveals the changes in the rapidly evolving Clojure ecosystem and provides a practical, complete walkthrough of the Clojure web stack.

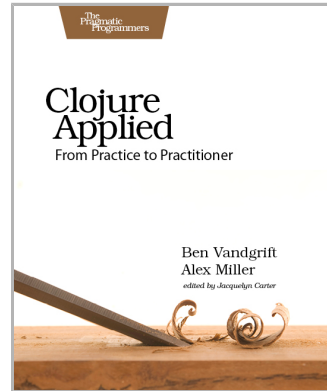
Dmitri Sotnikov and Scot Brown
(468 pages) ISBN: 9781680506822. \$47.95
<https://pragprog.com/book/dswdcloj3>



Clojure Applied

Think in the Clojure way! Once you're familiar with Clojure, take the next step with extended lessons on the best practices and most critical decisions you'll need to make while developing. Learn how to model your domain with data, transform it with pure functions, manage state, spread your work across cores, and structure apps with components. Discover how to use Clojure in the real world, and unlock the speed and power of this beautiful language on the Java Virtual Machine.

Ben Vandgrift and Alex Miller
(238 pages) ISBN: 9781680500745. \$38
<https://pragprog.com/book/vmclojeco>

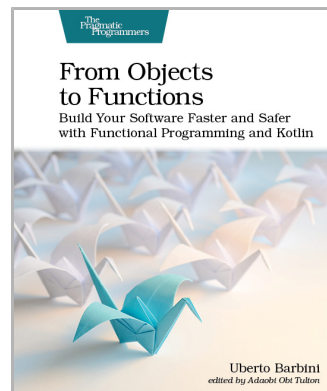


From Objects to Functions

Build applications quicker and with less effort using functional programming and Kotlin. Learn by building a complete application, from gathering requirements to delivering a microservice architecture following functional programming principles. Learn how to implement CQRS and EventSourcing in a functional way to map the domain into code better and to keep the cost of change low for the whole application life cycle.

If you're curious about functional programming or you are struggling with how to put it into practice, this guide will help you increase your productivity composing small functions together instead of creating fat objects.

Uberto Barbini
(468 pages) ISBN: 9781680508451. \$47.95
<https://pragprog.com/book/uboop>



The Pragmatic Bookshelf

The Pragmatic Bookshelf features books written by professional developers for professional developers. The titles continue the well-known Pragmatic Programmer style and continue to garner awards and rave reviews. As development gets more and more difficult, the Pragmatic Programmers will be there with more titles and products to help you stay on top of your game.

Visit Us Online

This Book's Home Page

<https://pragprog.com/book/shcloy4>

Source code from this book, errata, and other resources. Come give us feedback, too!

Keep Up-to-Date

<https://pragprog.com>

Join our announcement mailing list (low volume) or follow us on Twitter @pragprog for new titles, sales, coupons, hot tips, and more.

New and Noteworthy

<https://pragprog.com/news>

Check out the latest Pragmatic developments, new titles, and other offerings.

Buy the Book

If you liked this ebook, perhaps you'd like to have a paper copy of the book. Paperbacks are available from your local independent bookstore and wherever fine books are sold.

Contact Us

Online Orders: <https://pragprog.com/catalog>

Customer Service: support@pragprog.com

International Rights: translations@pragprog.com

Academic Use: academic@pragprog.com

Write for Us: <http://write-for-us.pragprog.com>